unooria
plataforma educacional B2G

DOCUMENTAÇÃO TÉCNICA DE ARQUITETURA DE SOFTWARE, ARQUITETURA DE NUVEM E CÓDIGO

Plataforma Unooria

Hub multi-tenant e 48 experiências imersivas

Monorepo de uma plataforma de educação básica para redes públicas de ensino: um *hub* em Next.js que embeda experiências estáticas versionadas em CDN, uma API NestJS com isolamento lógico por escola em *Row-Level Security*, telemetria em dois canais e uma camada de IA generativa com quota por perfil — uma *stack* de nuvem por contrato, cem escolas por *stack*.

DOCUMENTO	**PU-DOC-001 — Documentação Técnica Consolidada**
VERSÃO	**1.0**
DATA	**27 de agosto de 2026**
AUTOR	**Lucas Salomão**
REPOSITÓRIO	**`plataforma-unooria` (*monorepo*, tronco `main`)**
CLASSIFICAÇÃO	**Documento técnico de projeto — uso interno**

Escopo do documento: contexto de negócio, requisitos, casos de uso, arquitetura de *software* nos quatro níveis do modelo C4, arquitetura de nuvem e implantação em GCP, modelo de *multi-tenancy* e segurança, modelo de dados, contratos de integração, telemetria, camada de IA, catálogo de experiências, especificação da API, *design system*, engenharia de entrega, qualidade, registro de decisões arquiteturais, riscos e evolução. Diagramas vetoriais em TikZ; fontes Mermaid equivalentes no Apêndice A.

Ficha técnica do documento

Identificação

Campo	Conteúdo
Título	Plataforma Unooria — Documentação Técnica Consolidada de Arquitetura de *Software*, Arquitetura de Nuvem e Código
Identificador	PU-DOC-001
Versão	1.0
Data de emissão	27 de agosto de 2026
Autor	Lucas Salomão (<lucas.salomao@unooria.com>)
Repositório	`plataforma-unooria` — *monorepo* pnpm + Turborepo
Tronco	`main` (histórico linear; uma *branch* por experiência)
Linguagem principal	TypeScript 5.7 (*strict*), Node.js 20
Prazo de produção	novembro de 2026
Estado	Em desenvolvimento — M0–M6 concluídos; *stack* de *dev* em GCP operacional
Repositório de infra	Separado (Terraform). Contrato em `docs/INFRA.md`

Escopo e propósito

Este documento consolida, em fonte única, o que estava distribuído entre `CLAUDE.md` (as convenções e regras invioláveis do repositório), `README.md`, `docs/ARQUITETURA.md`, `docs/DECISOES.md`, `docs/INFRA.md`, `docs/INFRA-IDENTITY.md` e os comentários de decisão que vivem no próprio código-fonte — que, neste projeto, é onde as decisões mais finas estão registradas.

Acrescenta o material que não existia em forma escrita: especificação de requisitos funcionais e não funcionais rastreáveis, catálogo de casos de uso, modelo C4 completo nos quatro níveis para o *software* e para a nuvem, diagramas de sequência dos contratos de integração, dicionário de dados, especificação da API rota por rota e o registro formal das decisões arquiteturais (ADR) com suas consequências.

> **NOTA**
>
> Este documento é **normativo** para o projeto: quando ele divergir de uma suposição, ele vence. Quando divergir do código, o código vence e o documento está desatualizado — e atualizá-lo faz parte da *Definition of Done* de qualquer tarefa (Capítulo 16).

Histórico de revisões

Versão	Data	Autor	Alterações
0.1	22/08/2026	L. Salomão	Estrutura do *monorepo*; arquitetura e ADRs iniciais em `docs/`.
0.2	25/08/2026	L. Salomão	Migração para o *monorepo* mergeada na `main` (26 *commits*, *fast-forward*); *tenancy* com RLS, exp-01 e exp-02.
0.3	26/08/2026	L. Salomão	exp-70 (Vila do Lume) e exp-91 (bem-estar); decisão de retenção sem prazo declarado.
1.0	**27/08/2026**	**L. Salomão**	**Documentação técnica consolidada: requisitos, casos de uso, modelo C4 de *software* e de nuvem, *multi-tenancy*, modelo de dados, contratos, telemetria particionada, IA, API, *design system*, entrega, qualidade, ADRs e bibliografia em ABNT.**

Documentos de referência do projeto

Artefato	Conteúdo
`CLAUDE.md`	Convenções, regras invioláveis de *tenancy* e segurança, contratos de integração e *Definition of Done*. É o documento de governança do repositório.
`README.md`	Instalação, os três modos de execução local e *smoke* visual.
`docs/ARQUITETURA.md`	Visão arquitetural resumida por contrato.
`docs/DECISOES.md`	Registro resumido de decisões (base do Capítulo 17).
`docs/INFRA.md`	Contrato entre este repositório e o de Terraform: nomes de recursos, variáveis e o que o CI publica.
`docs/INFRA-IDENTITY.md`	Contrato do gerenciador central de identidade (WSO2 IS), que é único para vários contratos.
`apps/api/prisma/schema.prisma`	Modelo de dados anotado, com o porquê de cada decisão de esquema.
`packages/sdk/src/`	Contrato *postMessage* e cliente de telemetria — a fronteira entre hub e experiência.

`.env.example`	Catálogo completo de variáveis de ambiente (Apêndice B).

Sumário

Ficha técnica do documento i

Lista de figuras xvi

Lista de tabelas xviii

Lista de siglas e abreviaturas xix

1 Introdução 1

1.1 Propósito deste documento . . . 1

1.2 O problema . . . 1

1.3 A solução em uma página . . . 2

1.4 Como ler este documento . . . 3

1.4.1 Estrutura . . . 3

1.4.2 Convenções tipográficas . . . 4

1.5 O que este documento não cobre . . . 5

2 Contexto de negócio e restrições 6

2.1 Modelo de negócio . . . 6

2.2 Escala alvo . . . 6

2.3 Restrições de projeto . . . 7

2.4 Os *drivers* arquiteturais . . . 8

2.5 Interessados . . . 9

2.6 As sete regras invioláveis . . . 10

2.7 Fora de escopo, deliberadamente . . . 10

3 Requisitos 12

3.1 Requisitos funcionais 12

3.1.1 Identidade, papéis e acesso 12

3.1.2 Experiências e *embed* 13

3.1.3 Telemetria 14

3.1.4 Inteligência artificial 14

3.1.5 Pedagógico e relatórios 15

3.2 Requisitos não funcionais 17

3.3 Rastreabilidade 18

4 Casos de uso 20

4.1 Atores 20

4.2 Diagrama de casos de uso 21

4.3 Especificação dos casos de uso 21

4.3.1 UC-01 — Autenticar-se 22

4.3.2 UC-03 — Realizar experiência 22

4.3.3 UC-10 — Preparar atividade 23

4.3.4 UC-13 — Ler resultados da turma 24

4.3.5 UC-14 — Gerar plano de aula 25

4.3.6 UC-15 — Fazer *check-in* de bem-estar 25

4.3.7 UC-23 — Publicar manifesto de experiência 26

4.4 Rastreabilidade de casos de uso 27

5 Arquitetura de *software* 28

5.1 Visões arquiteturais 28

5.2 Estilo arquitetural e princípios 28

5.3 C4 nível 1 — Contexto do sistema 30

5.4 C4 nível 2 — Contêineres 30

5.4.1 Catálogo de contêineres e artefatos 31

5.5 C4 nível 3 — Componentes 32

5.5.1 Componentes da API 32

5.5.2 Componentes do hub 34
5.6 C4 nível 4 — Código 34
5.6.1 O contrato da ponte e o *guard* de *tenancy* 35
5.6.2 O gabarito de um módulo de domínio 35
5.7 Estrutura do *monorepo* 36
5.8 Fluxos principais 37
5.8.1 Curadoria de um bloco de *feed* (exp-01) 37
5.8.2 Estados de uma geração assíncrona 38

6 Arquitetura de nuvem e implantação 39
6.1 Princípios de nuvem 39
6.2 C4 nível 1 de implantação — o contrato 40
6.3 Diagrama de implantação detalhado 40
6.4 Catálogo de recursos por contrato 42
6.5 Caminhos de tráfego 43
6.6 Identidade de carga de trabalho e segredos 44
6.7 Dimensionamento 45
6.7.1 Volumes medidos 45
6.8 Ambientes 46
6.9 O provedor de identidade fora da *stack* 47
6.10 Observabilidade, continuidade e reversão 47

7 *Multi-tenancy* e segurança 49
7.1 Os três níveis de isolamento 49
7.2 Nível 2: *Row-Level Security* no PostgreSQL 49
7.2.1 Como a política é armada 50
7.2.2 A porta única: `comTenancy` 50
7.2.3 Como a escola chega ao contexto 51
7.2.4 A exceção: `admin-contrato` 51
7.3 Nível 3: autorização por papel 52

7.4 Segurança do *embed* 52
7.5 Segredos e o que nunca vai ao cliente 53
7.6 Privacidade e dados de menores 54
7.6.1 Retenção: uma decisão explícita de não declarar prazo 54
7.6.2 Localidade de processamento 55
7.7 Verificação 55
7.8 Ameaças consideradas 56

8 Modelo de dados 58

8.1 A convenção de nomes 58
8.1.1 A migração de referência, e duas armadilhas registradas 59
8.2 Modelo da plataforma 59
8.3 Telemetria particionada 60
8.4 Modelo da exp-01 (Instanooria) 61
8.5 Modelo das experiências 02, 70, 90 e 91 64
8.6 Decisões de esquema por experiência 65
8.6.1 exp-70 — as decisões que custam informação de propósito 65
8.6.2 exp-90 — conteúdo fora do banco 66
8.6.3 exp-91 — ausência como decisão de esquema 66
8.7 Migrações 66

9 Contratos de integração 68

9.1 Protocolo `postMessage` hub ↔ experiência 68
9.1.1 Especificação das mensagens (protocolo v1) 68
9.1.2 O *handshake*, passo a passo 69
9.1.3 Regras invioláveis do protocolo 69
9.1.4 Política de evolução 70
9.2 Manifesto de experiências 71
9.2.1 Como o hub resolve a URL do `iframe` 72
9.2.2 Publicação e reversão 72
9.3 Contrato de telemetria 73

9.4 Contrato do *provider* de IA 74

9.5 Contrato entre duas experiências 75

10 Telemetria 76

10.1 Por que dois canais 76

10.2 O canal analítico, ponta a ponta 78

10.3 A rota de ingestão e sua tranca 78

10.4 Destino: PostgreSQL, não BigQuery 79

10.4.1 `INSERT` multi-linha 79

10.4.2 Partição mensal 80

10.5 Privacidade da telemetria 80

10.6 Comportamento em desenvolvimento 81

11 Camada de inteligência artificial 82

11.1 Onde a IA é usada 82

11.2 A camada única 83

11.3 Quota por perfil 83

11.4 Credencial: ADC, nunca chave de API 84

11.5 Modelo, região e o custo de escolher 85

11.5.1 Imagem: proporção pedida ao modelo, não recortada depois 85

11.6 Raciocínio conta como saída 86

11.7 *Guardrails* do *check-in* de bem-estar 86

11.8 Variáveis de configuração da IA 88

12 Catálogo de experiências 89

12.1 Visão do catálogo 89

12.2 exp-01 — Instanooria 90

12.3 exp-02 — UnooTube 91

12.4 exp-70 — Vila do Lume 91

12.4.1 Duas leituras de uma partida, e elas não se tocam 92

12.4.2 O servidor recalcula; nunca aceita o perfil do cliente 92

12.4.3 `grau` é invariante de API, não regra de tela 92
12.4.4 A bolha é do servidor 93
12.4.5 Telas do professor 93
12.4.6 A exceção de ADR: o jogo em JavaScript clássico 94
12.5 exp-90 — Gerador de plano de aula 94
12.6 exp-91 — Check-in de bem-estar 95
12.7 Como criar uma experiência nova 95

13 Especificação da API 97
13.1 Convenções transversais 97
13.1.1 As três rotas sem `OidcGuard` 97
13.2 Mapa de rotas por área 99
13.3 Superfície que a SDK consome 99
13.4 Grupos de rotas em detalhe 100
13.4.1 Motor de *feed* (exp-01) 100
13.4.2 Planos de aula (exp-90) 101
13.4.3 Bem-estar (exp-91) 101
13.4.4 Análise 102
13.5 Sanidade e prontidão 102

14 *Design system* e camada de interface 104
14.1 Duas famílias no mesmo pacote 104
14.2 A linha que separa as seções é a REVELAÇÃO 105
14.3 Consumo: sempre pela ponte 105
14.4 CSS: uma vez, no ponto de entrada 106
14.4.1 No hub, o CSS não basta: as fontes também 106
14.5 Frames: dois níveis, não empilhar 107
14.6 Tema 107
14.7 Versão: atualizar é ato deliberado 108
14.8 Compensações locais 108

14.8.1 Lacunas abertas, verificadas na 0.1.14 108
14.8.2 Avisos que ficaram obsoletos 109
14.9 Cor da plataforma: classe escrita à mão 109
14.10 Exceção registrada 110

15 Engenharia de entrega 111

15.1 O *monorepo* e o *build* incremental 111
15.2 Integração contínua 112
15.3 Entrega contínua 113
15.3.1 Nada de nome de recurso no *workflow* 113
15.3.2 Concorrência e reexecução 114
15.4 Ambientes locais 114
15.5 O provedor de identidade sobe à parte 115
15.6 Convenções de *git* 115

16 Qualidade 117

16.1 A *Definition of Done* 117
16.2 Estratégia de testes 118
16.3 O *smoke* visual, em detalhe 118
16.3.1 O que só o *smoke* visual pega 119
16.4 Análise estática 120
16.5 Armadilhas de ferramenta registradas 120
16.6 Cobertura: onde está e onde não está 121

17 Registro de decisões arquiteturais 122

17.1 Índice 122
17.2 ADR-01 — Experiências como *bundles* estáticos 123
17.3 ADR-02 — *Monorepo* único 124
17.4 ADR-03 — Autenticação por OIDC genérico 124
17.5 ADR-04 — Uma *stack* por contrato; escolas por RLS 124
17.6 ADR-05 — Cloud SQL HA, não AlloyDB 125

17.7 ADR-06 — Pub/Sub para telemetria desde o início 125
17.8 ADR-07 — IA atrás de um *gateway* na própria API 126
17.9 ADR-08 — RLS forçada e uma porta única de banco 126
17.10 ADR-09 — Domínio por experiência na API; gabarito *server-side* 126
17.11 ADR-10 — Bem-estar: instrumento fixo, IA só no entorno, sem gestor 127
17.12 ADR-11 — Telemetria sem PII e vocabulário fechado 128
17.13 ADR-12 — `grau` como invariante de API 128
17.14 ADR-13 — Retenção sem prazo declarado 128
17.15 ADR-14 — Telemetria no PostgreSQL particionado 129
17.16 ADR-15 — O hub assina os *tokens* de experiência 130
17.17 ADR-16 — ADC como credencial; modelo *versus* região 130
17.18 ADR-17 — *Claim* atômico em vez de fila externa 131
17.19 ADR-18 — Exceções registradas 131
17.20 ADR-19 — *Design system* externo como fonte única de interface 132
17.21 ADR-20 — Um banco, um *schema*; prefixo `exp_NN_` 132
17.22 ADR-21 — *Push subscription* para a própria API 132

18 Riscos e evolução 134

18.1 Registro de riscos 134
18.2 Estado de implementação 136
18.3 Evolução planejada 137
18.3.1 Antes de novembro de 2026 (bloqueante) 137
18.3.2 Depois, por ordem de retorno 137
18.4 O que continuará fora de escopo 138

19 Conclusão 139

19.1 O que foi construído 139
19.2 As decisões que mais definiram o resultado 139
19.3 O que este projeto ensinou sobre documentação 140
19.4 Estado de prontidão para produção 140

19.5 Encerramento 141

A Fontes Mermaid dos diagramas 142

A.1 Figura 1.1 — Visão geral da plataforma 142

A.2 Figura 4.1 — Casos de uso 143

A.3 Figura 5.1 — C4 nível 1: contexto 144

A.4 Figura 5.2 — C4 nível 2: contêineres 144

A.5 Figura 5.3 — C4 nível 3: componentes da API 145

A.6 Figura 5.4 — C4 nível 3: componentes do hub 146

A.7 Figura 5.5 — C4 nível 4: tipos que carregam a regra 146

A.8 Figura 5.6 — Grafo de dependências do *monorepo* 147

A.9 Figura 5.7 — Curadoria de um bloco de *feed* 147

A.10 Figura 5.8 — Estados de uma geração assíncrona 148

A.11 Figura 6.1 — Implantação de um contrato 148

A.12 Figura 6.2 — Implantação detalhada 149

A.13 Figura 6.3 — Identidade de carga de trabalho 150

A.14 Figura 7.1 — Os três níveis de isolamento 150

A.15 Figura 7.2 — Propagação da escola 151

A.16 Figura 8.1 — Modelo de dados da plataforma 151

A.17 Figuras 8.2 e 8.3 — Modelo da exp-01 152

A.18 Figuras 8.4 e 8.5 — Demais experiências 153

A.19 Figura 9.1 — *Handshake* hub-experiência 154

A.20 Figura 9.2 — Resolução da URL do *embed* 155

A.21 Figura 9.3 — Contrato entre duas experiências 155

A.22 Figura 10.1 — Canal analítico 155

A.23 Figura 11.1 — Camada de IA 156

A.24 Figura 11.2 — *Guardrails* do bem-estar 157

A.25 Figura 12.1 — Os quatro graus de medida 157

A.26 Figura 13.1 — Mapa de rotas da API 157

A.27 Figura 14.1 — A linha da revelação 158

A.28 Figura 15.1 — *Pipeline* de entrega 158

A.29 Figura 16.1 — Estratégia de testes 159

A.30 Figura 18.1 — Matriz de riscos 159

B Variáveis de ambiente 160

B.1 API — `apps/api/.env` 160

B.1.1 Banco e identidade 160

B.1.2 Inteligência artificial 161

B.1.3 Armazenamento 161

B.1.4 Telemetria e chaves de serviço 162

B.2 Hub — `apps/hub/.env.local` 162

B.3 Experiências — variáveis de *build* do Vite 163

B.4 `vars` do repositório (CI/CD) 163

B.5 Configuração mínima para subir sem GCP 164

C Catálogo de *endpoints* 165

C.1 Sanidade 165

C.2 Autenticação 165

C.3 Usuários 165

C.4 Manifesto de experiências 166

C.5 Progresso genérico (SDK) 166

C.6 Telemetria 166

C.7 IA 167

C.8 exp-01 — atividades (professor) 167

C.9 exp-01 — geração de conteúdo (professor) 167

C.10 exp-01 — perfis e largada 168

C.11 exp-01 — motor de *feed* (aluno) 168

C.12 exp-02 — UnooTube 169

C.13 exp-70 — Vila do Lume 170

C.14 exp-90 — planos de aula 170

C.15 exp-91 — bem-estar 171

C.16 Análise 171

C.17 Rotas do hub (Next.js) 172

D *Payloads* e esquemas de validação 173

D.1 Protocolo `postMessage` (v1) 173

D.2 *Token* de experiência 174

D.3 Telemetria — canal analítico 174

D.4 exp-70 — o *payload* de uma partida 175

D.4.1 As decisões que cada campo carrega 176

D.4.2 As duas travas estruturais 177

D.4.3 O cap de eventos, e por que ele caiu de 2000 para 200 178

D.5 Manifesto de experiência 178

D.6 Validação de variáveis de ambiente 179

D.7 Importação de usuários por CSV 180

E Dicionário de dados 181

E.1 Índice de tabelas 181

E.2 Tabelas da plataforma 182

E.2.1 `escolas` 182

E.2.2 `turmas` 182

E.2.3 `users` 183

E.2.4 `experiencias` (catálogo) 183

E.2.5 `progresso` 184

E.2.6 `ia_uso_diario` 184

E.2.7 `telemetria_eventos` (particionada) 184

E.3 Tabelas da exp-01 184

E.3.1 `exp_01_activities` 185

E.3.2 `exp_01_posts` 185

E.3.3 `exp_01_runs` 186

E.3.4 `exp_01_events` — o canal operacional 187
E.3.5 `exp_01_exp_70_perfis_importados` 187
E.4 Tabelas das demais experiências 187
E.4.1 `exp_02_sessoes` 187
E.4.2 `exp_70_sessoes` 188
E.4.3 `exp_70_perfis` 189
E.4.4 `exp_70_bolhas` 189
E.4.5 `exp_90_planos` e `exp_90_referencias` 190
E.4.6 `exp_91_ciclos` e `exp_91_checkins` 190
E.5 O que o Prisma não modela 191
F Operação 192
F.1 Comandos de desenvolvimento 192
F.2 *Smoke* visual 193
F.3 *Design system* 193
F.4 Publicação e reversão 194
F.5 Provisionamento de um contrato novo 194
F.6 Diagnóstico: sintoma → causa 195
F.7 *Checklist* de operação 197
F.8 Compilar esta documentação 198
Referências 199

Lista de figuras

1.1 Visão geral da plataforma 3

4.1 Casos de uso da plataforma 21

5.1 C4 nível 1 — contexto 30
5.2 C4 nível 2 — contêineres 31
5.3 C4 nível 3 — componentes da API 33
5.4 C4 nível 3 — componentes do hub 34
5.5 C4 nível 4 — tipos que carregam a regra 35
5.6 Grafo de dependências do monorepo 36
5.7 Sequência: curadoria de um bloco de feed 37
5.8 Estados de uma geração assíncrona 38

6.1 Implantação de um contrato 40
6.2 Implantação detalhada de um contrato 41
6.3 Identidade de carga de trabalho 44

7.1 Os três níveis de isolamento 49
7.2 Propagação da escola pela requisição 51

8.1 Modelo de dados da plataforma 59
8.2 Modelo de dados da exp-01: atividade e conteúdo 61
8.3 Modelo de dados da exp-01: sessão do aluno 62
8.4 Modelo de dados das exps 02 e 70 64
8.5 Modelo de dados das exps 90 e 91 65

9.1 Sequência do handshake hub-experiência 69

9.2 Resolução da URL do embed 72
9.3 Contrato entre duas experiências 75

10.1 Canal analítico de telemetria 78

11.1 Camada de IA 83
11.2 Guardrails do check-in de bem-estar 87

12.1 Os quatro graus de medida 92

13.1 Mapa de rotas da API 99

14.1 A linha da revelação 105

15.1 Pipeline de entrega 113

16.1 Estratégia de testes 118

18.1 Matriz de riscos 136

Lista de tabelas

Lista de siglas e abreviaturas

Siglas técnicas

Sigla	Significado
ADC	*Application Default Credentials* — cadeia de resolução de credencial do Google Cloud
ADR	*Architecture Decision Record* — registro de decisão arquitetural
API	*Application Programming Interface*
ANPD	Autoridade Nacional de Proteção de Dados
BFF	*Backend for Frontend* — camada de *proxy* do hub para a API
BNCC	Base Nacional Comum Curricular
C4	Modelo de visualização arquitetural em quatro níveis (Contexto, Contêiner, Componente, Código)
CDN	*Content Delivery Network*
CI/CD	*Continuous Integration / Continuous Delivery*
CORS	*Cross-Origin Resource Sharing*
CSP	*Content Security Policy*
CSV	*Comma-Separated Values*
DASS-21	*Depression, Anxiety and Stress Scales* — versão de 21 itens
DoD	*Definition of Done* — critério de conclusão de tarefa
DPA	*Data Processing Agreement* — acordo de tratamento de dados
DS	*Design System*
EF	Ensino Fundamental (séries `1EF`–`9EF`)
EM	Ensino Médio (séries `1EM`–`3EM`)
ER	Entidade-Relacionamento (modelo de dados)
GCP	*Google Cloud Platform*
HA	*High Availability* — alta disponibilidade
HTTP(S)	*HyperText Transfer Protocol* (*Secure*)
IaC	*Infrastructure as Code* — infraestrutura como código
IdP	*Identity Provider* — provedor de identidade

Sigla	Significado
JSON	*JavaScript Object Notation*
JWK/JWKS	*JSON Web Key* / *JSON Web Key Set*
JWT	*JSON Web Token*
LB	*Load Balancer*
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei 13.709/2018)
LLM	*Large Language Model* — modelo de linguagem de grande escala
OIDC	*OpenID Connect*
ORM	*Object-Relational Mapping*
PII	*Personally Identifiable Information* — dado pessoal identificável
PITR	*Point-in-Time Recovery* — recuperação a um instante no tempo
PK/FK	*Primary Key* / *Foreign Key*
REST	*Representational State Transfer*
RF	Requisito Funcional
RLS	*Row-Level Security* — segurança em nível de linha do PostgreSQL
RNF	Requisito Não Funcional
RS256	RSASSA-PKCS1-v1_5 com SHA-256 — algoritmo de assinatura de JWT
SA	*Service Account* — conta de serviço do GCP
SDK	*Software Development Kit*
SLA/SLO	*Service Level Agreement* / *Objective*
SQL	*Structured Query Language*
SSR	*Server-Side Rendering*
TLS	*Transport Layer Security*
TTL	*Time To Live* — tempo de validade
UC	Caso de Uso (*Use Case*)
UML	*Unified Modeling Language*
VPC	*Virtual Private Cloud*
WIF	*Workload Identity Federation* — federação de identidade de carga de trabalho
WSO2 IS	WSO2 *Identity Server* — o gerenciador central de identidade do produto
xAPI	*Experience API* — especificação de telemetria de aprendizagem

Termos do domínio

Os nomes de domínio deste projeto são em português brasileiro por convenção explícita (`CLAUDE.md`, seção "Convenções de código"); termos técnicos permanecem em inglês. O glossário abaixo fixa o significado de cada um para todo o documento.

Termo	Significado neste projeto
`contrato`	**Tenant de nível 1**: uma rede pública (estado ou município) que contratou a plataforma. Um contrato equivale a uma *stack* de infraestrutura GCP inteira e separada.
`escola`	**Tenant de nível 2**: uma unidade escolar dentro de um contrato. Cem ou mais escolas compartilham a mesma *stack*, isoladas logicamente por `escola_id` + RLS.
`turma`	Agrupamento de alunos de uma mesma série dentro de uma escola. Chave de balanceamento de vários instrumentos (bolhas, distribuição de lados).
`aluno`	Usuário-estudante. É o único papel cujos dados são de menores, o que governa a maior parte das decisões de retenção e de log.
`professor`	Usuário que prepara e conduz a aula; consome relatórios de turma e usa as ferramentas de professor (exp-90, exp-91).
`gestor`	Usuário administrativo da escola: importa usuários, redefine senhas. Não vê dado de bem-estar de professor.
`admin-contrato`	Único papel autorizado a atravessar escolas, e apenas em rotas explicitamente marcadas. Publica manifestos de experiência.
`experiência`	Aplicação imersiva independente (*bundle* Vite estático) que o hub embeda em `iframe`. Nunca é um serviço.
`hub`	Aplicação Next.js que autentica, roteia por papel, embeda as experiências e desenha todo o *chrome* institucional.
`aparelho`	Metáfora de interface: a casca de dispositivo (`DispositivoFrame`) que o hub desenha em volta do `iframe` quando a experiência declara `viewport`.
`clone`	Superfície de interface que imita um aplicativo real (Instagram, YouTube, WhatsApp, ChatGPT, TikTok) para que a experiência possa medir comportamento em contexto reconhecível.
`revelação`	Momento pedagógico em que a experiência sai da ficção do clone e volta à linguagem institucional. É a linha que separa as seções do *design system*.
`missão`	Unidade pedagógica: uma experiência com objetivo, guia do professor e leitura de resultado.
`manifesto`	Linha da tabela `experiencias` (ou o `public/experiencia.json` do *bundle*) que descreve id, nome, versão ativa, séries, *capabilities*, `viewport`, público e permissões.
`versaoAtiva`	Versão do *bundle* que o hub serve. *Rollback* é a troca deste campo — não um novo *deploy*.

Termo	Significado neste projeto
`bolha`	Nível de dificuldade do instrumento da exp-70 (A, B ou C), atribuído pelo servidor e estável por aluno.
`grau`	Quanto uma partida sustenta a medida de um valor: `scored`, `indicative`, `zero` ou `none`. A diferença entre `zero` e `none` é invariante de API.
`run`	Sessão de um aluno numa atividade da exp-01: carrega o estado do motor de feed adaptativo.
`comTenancy`	Porta única de acesso ao banco na API: abre transação e arma a RLS com a escola do *token*.

1

Introdução

1.1 Propósito deste documento

Este documento descreve a arquitetura, o escopo e a implementação da **Plataforma Unooria**: uma plataforma de educação básica destinada a redes públicas de ensino, composta por um *hub* web multi-tenant e um catálogo alvo de 48 experiências imersivas que cobrem do 1º ano do Ensino Fundamental ao 3º ano do Ensino Médio.

A descrição arquitetural segue a norma ISO/IEC/IEEE 42010 (International Organization for Standardization, 2022): identifica os interessados, enuncia os *drivers* que restringem o desenho, apresenta visões complementares do sistema e registra as decisões com suas consequências. A notação de visualização é o modelo C4 (BROWN, 2026; BROWN, 2024), aplicado *duas vezes* — uma para a arquitetura de *software* (Capítulo 5) e outra para a arquitetura de nuvem e implantação (Capítulo 6) — porque neste projeto os dois recortes não coincidem: há contêineres lógicos que não são serviços em execução, e há serviços em execução que não são contêineres lógicos novos.

> **NOTA**
>
> Todo diagrama deste documento é vetorial, desenhado nativamente em TikZ. Não há imagem rasterizada de diagrama, nem dependência de ferramenta externa de renderização. Para quem precisa editar, revisar ou versionar o diagrama fora do LaTeX, o Apêndice A traz a fonte Mermaid equivalente de cada figura, referenciando o mesmo número.

1.2 O problema

Uma plataforma educacional para rede pública tem três exigências que, juntas, determinam quase toda a arquitetura — e que raramente aparecem juntas em produto educacional comercial:

1. **Isolamento contratual forte, isolamento escolar barato.** Cada contrato é uma relação jurídica com um ente federativo. Vazamento de dado entre *contratos* é inaceitável; por isso

cada contrato recebe uma *stack* de infraestrutura própria. Já as cem escolas de um mesmo contrato não podem custar cem infraestruturas: elas compartilham a *stack* e são isoladas logicamente.

2. **Pico de carga previsível e brutal.** A rede toda entra na plataforma às oito da manhã. O alvo de projeto é de aproximadamente dez mil alunos por contrato com dois a quatro mil simultâneos nesse pico — uma curva de carga que não se parece com a de um SaaS de escritório.
3. **Dados de menores.** A maior parte dos titulares dos dados tratados é criança ou adolescente. Isso não é um detalhe de conformidade tratado no fim: é um *driver* arquitetural que decide o que se registra, onde se registra, quem lê e o que o relatório está *proibido* de afirmar (Brasil, 2018; Autoridade Nacional de Proteção de Dados, 2024).

A quarta restrição não é do domínio, é do projeto, e é a mais severa: **uma pessoa desenvolve, com prazo de produção em novembro de 2026**. Isso transforma "simplicidade" de preferência estética em requisito de viabilidade. Cada peça de infraestrutura nova é uma peça que uma pessoa vai operar de plantão; cada abstração a mais é código que ninguém revisa em par.

> **DECISÃO DE PROJETO**
>
> A regra de projeto que decorre disso está escrita no documento de governança do repositório e é aplicada em todo este documento: **simplicidade** > **sofisticação**. Nenhum serviço, fila, cache ou biblioteca de estado entra sem *gargalo medido*. As duas exceções são decisões tomadas de antemão e justificadas: o Pub/Sub da telemetria (**ADR-06**) e o *provider* plugável de IA (**ADR-07**).

1.3 A solução em uma página

A plataforma é um *monorepo* com quatro tipos de artefato, e a distinção entre eles é a decisão arquitetural mais consequente do projeto:

Artefato	Tecnologia	Como é entregue
`apps/hub`	Next.js (App Router)	Contêiner no Cloud Run
`apps/api`	NestJS + PostgreSQL	Contêiner no Cloud Run
`apps/experiencias/*`	Vite + React + `@edu/sdk`	***Bundle* estático** em Cloud Storage atrás de CDN
`packages/*`	TypeScript puro	Consumido por *workspace*; nunca implantado sozinho

> **REGRA INVIOLÁVEL**
>
> **Experiências nunca viram serviços.** Uma experiência é um conjunto de arquivos estáticos em `gs://{contrato}-exp/{expId}/{versao}/`. Não há processo, não há porta, não há *scaling*. Isso vale para as 48: o custo marginal de publicar a quadragésima oi-

> tava experiência é o custo de copiar arquivos para um *bucket*. *Rollback* é o hub passar a apontar para a versão anterior — uma linha de `UPDATE`, não um *deploy*.

A Figura 1.1 resume o caminho de uma sessão de aluno, do *login* à coleta de telemetria.

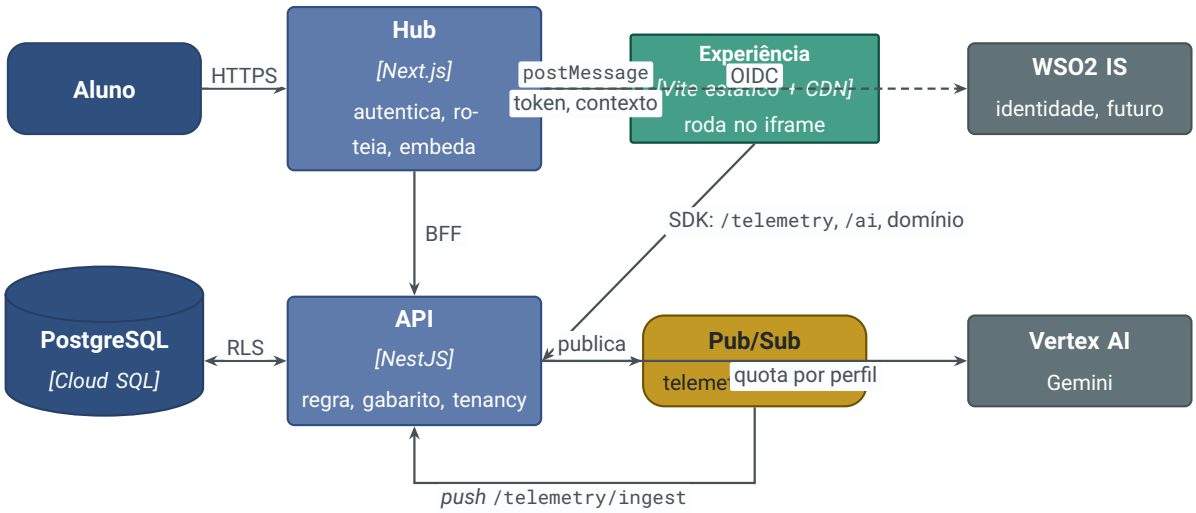


Figura 1.1 – Visão geral: o hub é a única porta do usuário; a experiência é estática e recebe o contexto por `postMessage`; a API concentra regra, gabarito e *tenancy*; a telemetria analítica sai pelo Pub/Sub e volta à própria API por *push*.

Legenda: pessoa · contêiner da plataforma · artefato estático · banco de dados · fila · sistema externo

1.4 Como ler este documento

1.4.1 Estrutura

Cap.	Título	Responde a
1–2	Introdução, contexto de negócio	Por que a plataforma existe e sob que restrições
3–4	Requisitos, casos de uso	O que ela deve fazer, e para quem
5	Arquitetura de *software* (C4)	Como o *software* está organizado
6	Arquitetura de nuvem (C4 + implantação)	Onde ele roda e quanto custa operá-lo
7	*Multi-tenancy* e segurança	Como um dado de escola não vaza para outra
8	Modelo de dados	Que dados existem e por quê
9–11	Contratos, telemetria, IA	As três fronteiras que mais custam se quebrarem
12–14	Experiências, API, *design system*	O catálogo, a superfície HTTP e a interface

15–16	Entrega e qualidade	Como o código chega à escola e o que o impede de chegar errado
17–19	ADRs, riscos, conclusão	As decisões, o que pode dar errado e o estado atual
A–F	Apêndices	Mermaid, variáveis, *endpoints*, *payloads*, dicionário de dados, operação

1.4.2 Convenções tipográficas

Marca	Significado
`identificador`	Identificador de código: campo, função, tipo, coluna, valor.
`caminho/arquivo.ts`	Caminho de arquivo ou diretório no repositório.
`GET /rota/:id`	*Endpoint* HTTP da API.
`VARIAVEL`	Variável de ambiente.
RF-01, **RNF-01**	Requisito funcional / não funcional (Capítulo 3).
UC-01	Caso de uso (Capítulo 4).
ADR-01	Decisão arquitetural registrada (Capítulo 17).
DR-01	*Driver* arquitetural (Seção 2.4).

Os blocos destacados têm significado fixo ao longo do documento:

> **NOTA**
>
> Informação de contexto que ajuda a entender uma escolha, mas cuja ausência não quebra nada.

> **DECISÃO DE PROJETO**
>
> Decisão de projeto tomada deliberadamente, com a alternativa descartada e o motivo. Toda decisão deste tipo tem uma entrada correspondente no Capítulo 17.

> **REGRA INVIOLÁVEL**
>
> Invariante que não pode ser violada por nenhuma implementação futura. As sete regras invioláveis da plataforma estão consolidadas na Seção 2.6.

> **ATENÇÃO**
>
> Armadilha conhecida: algo que já falhou, ou que falha silenciosamente. Quase todos estes blocos registram um erro efetivamente cometido e medido no projeto.

LIMITAÇÃO CONHECIDA

Limitação assumida do estado atual, com o caminho de saída. Não é bug: é escopo que ficou de fora, de propósito.

1.5 O que este documento não cobre

- **O código Terraform.** Ele vive em repositório separado. O *contrato* entre os dois repositórios — nomes de recursos esperados, variáveis consumidas, artefatos publicados — está no Capítulo 6 e em `docs/INFRA.md`.
- **O conteúdo pedagógico.** O catálogo de missões, os textos e a trilha por ano são produto, não arquitetura. O Capítulo 12 descreve a *mecânica* de cada experiência apenas até onde ela impõe exigências ao *software*.
- **O *design system*.** Os componentes vivem em repositório próprio (`storybook-unooria`) e chegam aqui como pacote npm publicado. O Capítulo 14 documenta as *regras de consumo*, não o catálogo de componentes.
- **As 43 experiências ainda não escritas.** O Capítulo 12 documenta as cinco existentes e o *gabarito* que as próximas devem seguir.

2

Contexto de negócio e restrições

2.1 Modelo de negócio

A Plataforma Unooria é vendida **B2G**: o cliente é um ente federativo — uma secretaria estadual ou municipal de educação — e não a escola nem a família. A consequência arquitetural imediata é que a *unidade comercial* e a *unidade de uso* são diferentes, e o desenho precisa acomodar as duas.

Nível	Entidade	Isolamento
1	`contrato` (rede)	**Físico**: *stack* GCP inteira e separada, provisionada por um *module* Terraform por contrato. Projeto, *buckets*, banco, *Load Balancer* e contas de serviço próprios.
2	`escola`	**Lógico**: `escola_id` obrigatório em toda tabela de dado de aluno, mais *Row-Level Security* no PostgreSQL. Cem ou mais escolas por *stack*.
3	`turma` / `aluno`	**Autorização**: *claims* do JWT (`turma_id`, `perfil`) verificados na API por `RolesGuard` e por filtro explícito nas *queries*.

> **DECISÃO DE PROJETO**
>
> **Uma *stack* por contrato, escolas por RLS** (ADR-04). A alternativa "uma *stack* por escola" daria isolamento físico perfeito e transformaria a empresa em operadora de cem infraestruturas por contrato — inviável para uma pessoa. A alternativa oposta, uma *stack* única para todos os contratos, tornaria um incidente em um município um incidente em todos, e não sobreviveria à primeira análise jurídica de um edital. O ponto escolhido separa fisicamente o que é contratual e logicamente o que é operacional.

2.2 Escala alvo

Grandeza	Alvo por contrato	Consequência

Alunos cadastrados	~10.000	Cabe folgadamente numa instância pequena de Cloud SQL.
Simultâneos no pico (8h)	2.000–4.000	`min-instances` no Cloud Run para eliminar *cold start*; *pooler* de conexões obrigatório.
Escolas por *stack*	100+	RLS e `escola_id` em todo lugar; nenhuma *query* cruza escolas fora de `admin-contrato`.
Experiências no catálogo alvo	48	Custo marginal de publicação *deve* ser próximo de zero ⇒ *bundle* estático.
Faixa etária	1º EF ao 3º EM	Instrumentos e interface variam por série; regras de instrumento mudam nos anos iniciais.

O perfil de carga é o dado mais importante desta tabela e merece ser dito por extenso: **o tráfego não é distribuído, é uma parede**. Entre 7h50 e 8h10 de um dia letivo, uma fração grande da rede faz *login* e abre uma experiência quase ao mesmo tempo. O desenho responde a isso com três medidas independentes:

1. O grosso dos *bytes* do pico — os *bundles* das experiências e as imagens geradas — não passa pela aplicação. Sai da CDN, com o caminho versionado e, portanto, imutável e cacheável indefinidamente (**ADR-01**).
2. Os serviços têm instâncias mínimas provisionadas, então o pico não paga inicialização de contêiner nem *warm-up* de *pool* do Prisma.
3. A telemetria de alto volume não é síncrona: vai para o Pub/Sub e é gravada em lote, fora do caminho da requisição do aluno (Capítulo 10).

2.3 Restrições de projeto

ID	Restrição	Origem e consequência
RE-01	Uma pessoa desenvolve e opera	Restrição de recurso. Nenhuma peça de infraestrutura sem gargalo medido; preferência sistemática por absorver capacidade na API existente em vez de criar serviço novo.
RE-02	Produção em novembro de 2026	Restrição de prazo. Ordem de implementação por milestones (M0–M6); o que não bloqueia a primeira rede fica documentado como evolução (Capítulo 18).
RE-03	Identidade centralizada em WSO2 IS	Exigência do cliente e da estratégia multi-contrato. A aplicação nunca conhece o WSO2: conhece OIDC (**ADR-03**).
RE-04	Google Cloud Platform	Decisão de plataforma. Serviços gerenciados preferidos a orquestração própria; Cloud Run em vez de GKE.

RE-05	Interface exclusivamente do *design system*	Decisão de produto e de marca. Nenhum componente de UI nasce neste repositório; o que falta nasce no `storybook-unooria` (Capítulo 14).
RE-06	Dados de menores	LGPD art. 14 e orientação da ANPD (Brasil, 2018; Autoridade Nacional de Proteção de Dados, 2024). Governa telemetria, log, retenção e *o que o relatório pode afirmar*.
RE-07	Dado de saúde de professor	LGPD art. 11 (dado sensível). Determina a ausência deliberada de visão de gestor na exp-91.

2.4 Os *drivers* arquiteturais

Um *driver* arquitetural é uma exigência que, se ignorada, invalida a solução — não a piora, invalida. Foram sete.

ID	Driver	Consequência arquitetural
DR-01	Isolamento verificável entre escolas	`escola_id NOT NULL` + `FORCE ROW LEVEL SECURITY` em toda tabela de dado de aluno; acesso ao banco por uma *única* porta (`PrismaService.comTenancy`) que arma a política antes de qualquer trabalho. *Teste* de *tenancy* com *role* sem *ownership* (`tests/tenancy.spec.ts`).
DR-02	Custo marginal quase nulo por experiência	Experiência é *bundle* estático versionado em *bucket*, nunca um serviço. *Rollback* por ponteiro (`versaoAtiva`), não por *deploy* (**ADR-01**).
DR-03	Correção e pontuação não podem estar no cliente	Gabarito, correção e qualquer nota são *server-side*, num módulo de domínio por experiência na API. O *bundle* nunca recebe a resposta certa (**ADR-09**).
DR-04	Troca de provedor de identidade sem tocar código	Validação de *token* por OIDC genérico: `OIDC_ISSUER` e `OIDC_JWKS_URI`. Hoje o emissor é o próprio hub; amanhã é o WSO2, e o `OidcGuard` não muda (**ADR-03**).
DR-05	IA com custo previsível e sem segredo no cliente	Toda chamada de IA passa por `apps/api/src/ai`, com quota diária persistida *por perfil* verificada antes da chamada. Nenhuma chave, URL interna ou *prompt* de sistema em *bundle* (**ADR-07**).
DR-06	O pico da manhã não pode derrubar o transacional	Dois canais de telemetria com garantias diferentes: o operacional é síncrono e transacional; o analítico é assíncrono via Pub/Sub, com *insert* multi-linha e tabela particionada por mês (**ADR-06**, **ADR-14**).
DR-07	O relatório não pode produzir rótulo sobre uma criança	Invariantes *de tipo*, não de tela: valor com uma única fonte de medida não recebe número; "não teve oportunidade" nunca é lido como "escolheu não"; listagem de turma não exibe valor psicométrico (**ADR-12**).

> **NOTA**
>
> **DR-07** é incomum como *driver* arquitetural, porque parece uma questão de produto. Ele está aqui porque foi implementado como *estrutura*: a API devolve uma união discriminada em que só a variante `scored` tem `numero: number`; as outras têm `null` *por tipo*. O hub fica estruturalmente incapaz de imprimir "Poder 100" a partir de um único sinal — ele nunca recebe esse número, e o compilador nem deixa procurá-lo. Prosa num comentário não é controle; um tipo é.

2.5 Interessados

Interessado	O que precisa	Onde é atendido
Aluno	Entrar, escolher a missão, jogar sem atrito, ver o próprio resultado	Hub do aluno (`/aluno`), `/exp/*`, dossiê da experiência
Professor	Preparar a aula, conduzir o momento coletivo, ler a turma, gerar plano de aula, cuidar do próprio bem-estar	`/professor/*`, exp-90, exp-91, relatórios por turma
Gestor escolar	Cadastrar e manter usuários da escola	`/admin/usuarios`, importação CSV
Secretaria (rede)	Visão agregada da rede, sem identificar criança	`/analise/*` (papel `admin-contrato`)
Encarregado de dados (DPO)	Saber o que é coletado, por quanto tempo, quem lê, e onde o dado é processado	Capítulos 7 e 10; **ADR-11**, **ADR-13**
Desenvolvedor	Adicionar experiência sem reaprender a plataforma	SDK, `_template`, *scaffolding* (`pnpm new:exp`), Capítulo 12
Operação	Subir, observar e reverter sem adivinhar	CI/CD (Capítulo 15), Apêndice F

2.6 As sete regras invioláveis

Estas regras estão registradas em `CLAUDE.md` e são normativas: uma implementação que as viole está errada, independentemente de passar nos testes. Cada uma tem o capítulo onde é detalhada e verificada.

#	Regra	Detalhe em
1	Toda *query* SQL é escopada por `escola_id` (e `aluno_id` quando aplicável). Tabelas de dado de aluno têm `escola_id NOT NULL` + RLS. Nenhuma *query* atravessa escolas, exceto em rotas `admin-contrato`.	Cap. 7
2	Nenhum segredo em *bundle* de cliente: sem chave de API, sem *prompt* de sistema, sem URL interna. IA só pela API.	Cap. 11
3	Resposta correta, correção e pontuação que vale nota são *server-side*. O cliente envia a resposta do aluno; a API corrige.	Cap. 12
4	O *token* é obtido pelo hub via OIDC e repassado à experiência por `postMessage` com `targetOrigin` explícito — nunca `"*"`. A experiência mantém o *token só em memória*.	Cap. 9
5	IA tem quota diária por usuário *e por perfil*, e *rate limit*. Nenhuma chamada de IA sem passar pelo `QuotaService`: `temSaldo` antes, `consumir` depois.	Cap. 11
6	Dados de menores: nenhuma PII em log em texto claro. A telemetria carrega `aluno_id` opaco, nunca nome ou *e-mail*.	Cap. 10
7	Nenhuma migração remove `escola_id` ou afrouxa RLS. Renomeações usam `RENAME`, nunca `DROP`/`CREATE`.	Cap. 8

2.7 Fora de escopo, deliberadamente

O que *não* entra é parte do desenho. As exclusões abaixo são decisões, não omissões, e cada uma tem um gatilho explícito de reavaliação (Capítulo 18).

Descartado	Motivo e gatilho de reavaliação
Kubernetes (GKE)	Cloud Run entrega *autoscaling*, TLS e revisões sem *control plane* para operar. Gatilho: necessidade de *workload* de longa duração ou topologia de rede que o Cloud Run não expresse.
AlloyDB	A robustez vem de HA + PITR + *pooler*, não do *engine*. Ambos são PostgreSQL; migrar depois é trivial (**ADR-05**).
Microsserviço por experiência	Seriam 48 serviços para operar, cada um com *deploy*, log e escala próprios — para servir arquivos estáticos (**ADR-01**).
Redis como *buffer* de telemetria	Quem amortece o pico já é o Pub/Sub, e ele é *durável*: um *buffer* em memória perde os eventos do processo que morre antes do *flush*. O gargalo medido era o laço de `INSERT`, e ele foi corrigido (**ADR-14**).

BigQuery para o canal analítico	Nenhum relatório atual lê `telemetria_eventos`; seria uma segunda plataforma de dados para operar sem resolver problema existente (**ADR-14**).
Multi-região	Uma rede é geograficamente concentrada. Gatilho: exigência contratual de continuidade regional.
Worker dedicado de telemetria	Substituído por *push subscription* para a própria API — uma peça a menos para operar. Gatilho: volume que justifique isolar o consumo.

3

Requisitos

Este capítulo especifica os requisitos da plataforma de forma rastreável: cada requisito recebe identificador estável, prioridade, estado e o ponto do sistema onde é verificado. Requisito sem ponto de verificação é intenção, não requisito.

A prioridade segue MoSCoW: **M** (*must*) bloqueia a entrada em produção; **S** (*should*) é esperado na primeira rede; **C** (*could*) é evolução planejada. O estado reflete o repositório em 27 de agosto de 2026.

3.1 Requisitos funcionais

3.1.1 Identidade, papéis e acesso

ID	Requisito	Pri	Estado	Verificação
RF-01	Autenticar usuário por credencial da rede e emitir sessão.	M	Implementado	`apps/hub/lib/auth.ts`; `POST /auth/login`
RF-02	Emitir *token* de experiência assinado em RS256, com TTL de 15 minutos e *claims* `{sub, escola_id, turma_id?, perfil}`, audiência `plataforma-api`.	M	Implementado	`lib/chave-token.ts`; `GET /.well-known/jwks.json`
RF-03	Validar *token* na API por JWKS do emissor configurado, sem conhecer o produto de identidade.	M	Implementado	`src/auth/oidc.guard.ts`
RF-04	Restringir rota por papel (`aluno`, `professor`, `gestor`, `admin-contrato`).	M	Implementado	`src/auth/roles.ts`

ID	Requisito	Pri	Estado	Verificação
RF-05	Redirecionar usuário que acessa área de outro papel para a home do próprio papel, sem expor 403.	S	Implementado	`apps/hub/middleware.ts`; `lib/papeis.test.ts`
RF-06	Importar usuários em massa por CSV (`nome,usuario,senha,papel,tu`	M	Implementado	`POST /usuarios/importar-csv`; `tests/usuarios.spec.ts`
RF-07	Redefinir senha de usuário da escola.	S	Implementado	`POST /usuarios/:id/reset-senha`
RF-08	Trocar o emissor de identidade para o WSO2 IS alterando apenas configuração.	M	Preparado	`OIDC_ISSUER`, `OIDC_JWKS_URI`; `wso2/`

3.1.2 Experiências e *embed*

ID	Requisito	Pri	Estado	Verificação
RF-10	Manter manifesto de versões por experiência e servir a versão ativa.	M	Implementado	Tabela `experiencias`; `PUT /experiencias/:expId`
RF-11	Reverter versão de experiência trocando `versaoAtiva`, sem *build* nem *deploy*.	M	Implementado	`scripts/publicar-manifesto.mjs`
RF-12	Embutir a experiência em `iframe` com *sandbox* mínimo, concedendo capacidade extra apenas a quem a declara no manifesto.	M	Implementado	`components/ExpFrame.tsx`; `scripts/probe-pointerlock.mjs`
RF-13	Entregar *token* e contexto à experiência por `postMessage` com `targetOrigin` explícito.	M	Implementado	`packages/sdk/src/bridge.ts`; `bridge.test.ts`
RF-14	Renovar *token* da experiência sob demanda, sem recarregar o `iframe`.	M	Implementado	`exp:token-expirando` → `GET /api/exp-token` → `hub:token`
RF-15	Pausar e retomar a experiência quando a página do hub perde ou recupera visibilidade/foco.	S	Implementado	`hub:pause` / `hub:resume`
RF-16	Desenhar casca de aparelho quando a experiência declara `viewport`, lendo o manifesto *ou* o `experiencia.json` do *bundle*.	S	Implementado	`lib/embed-experiencia.ts`; `components/DispositivoFrame.tsx`

ID	Requisito	Pri	Estado	Verificação
RF-17	Persistir e recuperar progresso genérico por (`escola`, `aluno`, `experiência`).	M	Implementado	`GET/POST /progresso/:expId`
RF-18	Expor uma bancada local de *embed* para desenvolvimento (`/dev-embed?src=`).	C	Implementado	`app/dev-embed`

3.1.3 Telemetria

ID	Requisito	Pri	Estado	Verificação
RF-20	Coletar evento analítico no padrão xAPI simplificado, em lote de até 50 eventos com *flush* a cada 10 segundos.	M	Implementado	`packages/sdk/src/telemetry.ts`
RF-21	Enviar o último lote no `pagehide` por `sendBeacon`, com o *token* no corpo.	M	Implementado	`POST /telemetry/beacon`
RF-22	Derivar identidade do evento do *token*, nunca do corpo enviado pelo cliente.	M	Implementado	`src/telemetry/telemetry.controller.ts`
RF-23	Gravar o canal analítico de forma idempotente sob entrega *at least once*.	M	Implementado	`ON CONFLICT (event_id, ts) DO NOTHING`
RF-24	Autenticar o consumo do *push* do Pub/Sub, falhando fechado na ausência de configuração.	M	Implementado	`src/telemetry/pubsub-push.guard.ts`; `tests/telemetria-push.spec.ts`
RF-25	Manter o canal operacional (motor de feed) síncrono e transacional, fora do Pub/Sub.	M	Implementado	`POST /runs/:id/eventos`, `GET /runs/:id/feed`

3.1.4 Inteligência artificial

ID	Requisito	Pri	Estado	Verificação
RF-30	Concentrar toda chamada a provedor de IA num único módulo com interface neutra de *vendor*.	M	Implementado	`src/ai/provider.ts`

ID	Requisito	Pri	Estado	Verificação
RF-31	Verificar quota diária por usuário *e por perfil* antes da chamada, e registrar consumo depois.	M	Implementado	`src/ai/quota.service.ts`; tabela `ia_uso_diario`
RF-32	Autenticar no provedor por ADC, sem chave de API em variável de ambiente.	M	Implementado	`src/ai/vertex.provider.ts`; `tests/vertex-provider.spec.ts`
RF-33	Oferecer *provider* de teste (`mock`) que permita subir sem GCP.	M	Implementado	`AI_PROVIDER=mock`
RF-34	Detectar corte de saída do modelo (`finishReason` = `MAX_TOKENS`) e tratá-lo como erro, não como conteúdo.	M	Implementado	`src/planos/planos.service.ts`
RF-35	Gerar plano de aula anual de forma assíncrona, com *claim* atômico e consulta por *polling*.	M	Implementado	`POST /planos` (202), `GET /planos/:id`
RF-36	Guardar conteúdo autoral do professor em armazenamento privado, fora do *bucket* público.	M	Implementado	`{contrato}-planos`; `src/object-storage`
RF-37	Validar todo texto gerado por IA que chegue ao professor no contexto de bem-estar (sem vocabulário clínico, ordem, promessa ou *link*), com *fallback* fixo.	M	Implementado	`src/bemestar/regras.ts`; `tests/bemestar-regras.spec.ts`

3.1.5 Pedagógico e relatórios

ID	Requisito	Pri	Estado	Verificação
RF-40	Corrigir e pontuar no servidor; nunca enviar gabarito ao *bundle*.	M	Implementado	`src/unootube/`, `src/motor/`, `src/vila/`
RF-41	Curar o feed adaptativo por blocos, de forma determinística dado um gerador de aleatoriedade.	M	Implementado	`src/motor/curadoria.ts`; `tests/motor-pesos.spec.ts`
RF-42	Serializar a curadoria por `run` com *lock* consultivo, para que dois pedidos simultâneos não descartem um bloco.	M	Implementado	`pg_advisory_xact_lock`; `tests/motor-concorrencia.spec.ts`

ID	Requisito	Pri	Estado	Verificação
RF-43	Atribuir bolha (A/B/C) balanceada dentro da turma, no servidor, e mantê-la estável entre partidas.	M	Implementado	`exp_70_bolhas`; `tests/vila-regras.spec.ts`
RF-44	Não servir a bolha B nos anos iniciais (1EF–5EF).	M	Implementado	`bolhasElegiveis`
RF-45	Recalcular o perfil de valores no servidor a partir das medidas brutas, recusando *payload* que traga o perfil.	M	Implementado	`CHAVES_DE_PERFIL` em `src/vila/contrato.ts`
RF-46	Permitir reprocessar o histórico quando a versão do mapeamento subir.	M	Implementado	`exp_70_sessoes.payload`; `versaoMapeamento`
RF-47	Devolver leitura em que só a variante `scored` tem número; as outras têm `null` *por tipo*.	M	Implementado	`src/vila/leitura.ts`
RF-48	Abrir o relatório individual pelo *nível* do instrumento antes do desempenho.	M	Implementado	`/professor/entre-redes/vila-do-lume/alu`
RF-49	Distinguir “cartões servidos” de “cartões que apareceram”, e nunca afirmar omissão a partir de ausência.	M	Implementado	`cartoesServidos`
RF-50	Zerar os dados da turma de uma experiência, apagando também a atribuição de bolha, e *sem* tocar dados de outra experiência.	S	Implementado	`DELETE /vila/professor/turmas/:turmaId/dados`
RF-51	Importar o perfil da Vila como largada do Instanooria, preservando redistribuição manual do professor.	S	Implementado	`POST /atividades/:id/perfis/da-vila`; `tests/vila-ato1.spec.ts`
RF-52	Exportar análise por turma e por rede em CSV.	S	Implementado	`GET /atividades/:id/analise/csv`, `GET /analise/csv`
RF-53	Não expor visão de gestor nem agregado para o dado de bem-estar do professor.	M	Implementado (por ausência)	`src/bemestar/bemestar.controller.ts`
RF-54	Permitir ao professor apagar integralmente os próprios dados de bem-estar.	M	Implementado	`DELETE /bemestar/dados`

3.2 Requisitos não funcionais

Organizados pelas características de qualidade de produto da ISO/IEC 25010 (International Organization for Standardization, 2023).

ID	Característica	Requisito e critério	Estado
RNF-01	Desempenho — capacidade	Suportar 2.000–4.000 sessões simultâneas por contrato no pico, sem *cold start*: `min-instances` $\geq$ 1 na API e no hub, concorrência 80 na API.	Configurado; teste de carga pendente
RNF-02	Desempenho — vazão de ingestão	O caminho de gravação da telemetria analítica deve custar $O(\text{lotes})$ e não $O(\text{eventos})$ em *round-trips* ao banco.	Implementado (`INSERT` multi-linha)
RNF-03	Desempenho — borda	*Bundles* e mídia servidos por CDN com caminho imutável; só o `index.html` tem *cache* curto (60 s).	Implementado
RNF-04	Confiabilidade	Banco com HA e PITR; *pooler* de conexões obrigatório; *deploy* por revisões reversíveis.	Provisionado
RNF-05	Confiabilidade — entrega	Perda de evento analítico tolerada apenas quando o processo morre antes do *flush*; a partir do Pub/Sub, a entrega é durável e *at least once* com gravação idempotente.	Implementado
RNF-06	Segurança — isolamento	Nenhuma *query* de dado de aluno pode retornar linha de outra escola, mesmo com erro de aplicação. Verificado por RLS forçada e por teste com *role* sem *ownership*.	Implementado
RNF-07	Segurança — segredo	Zero segredo em artefato de cliente. Segredos em Secret Manager; credencial de IA por ADC.	Implementado
RNF-08	Segurança — superfície	*Sandbox* de `iframe` mínimo; `targetOrigin` explícito; toda mensagem validada por esquema nos dois lados; CORS por lista explícita.	Implementado
RNF-09	Segurança — *fail closed*	Rota cuja proteção depende de configuração responde 401/503 na ausência dela, nunca fica aberta.	Implementado
RNF-10	Privacidade	Telemetria sem PII; texto livre sensível nunca vai a log nem à telemetria; IA não recebe nome, escola, pontuação nem faixa clínica.	Implementado
RNF-11	Manutenibilidade	TypeScript *strict* sem `any` (uma exceção registrada em ADR); validação de todo *input* externo por Zod, inclusive variáveis de ambiente, no *boot*.	Implementado
RNF-12	Manutenibilidade — acoplamento	Nenhum módulo fora de `src/ai` conhece o *vendor* de IA; nenhum componente fora de `packages/ui` importa o *design system* diretamente.	Implementado

ID	Característica	Requisito e critério	Estado
RNF-13	Portabilidade	Provedor de identidade e provedor de IA substituíveis por configuração ou por arquivo novo, sem alterar chamadores.	Implementado
RNF-14	Testabilidade	A plataforma sobe inteira sem GCP (`AI_PROVIDER=mock`, `ASSETS_DRIVER=local`, `PLANOS_DRIVER=local`, `TELEMETRIA_TOPIC` vazio).	Implementado
RNF-15	Observabilidade	*Smoke* visual obrigatório para mudança de tela — *typecheck* não detecta ícone virando texto nem `iframe` vazio.	Implementado
RNF-16	Custo	Custo marginal de experiência publicada ≈ armazenamento + tráfego de CDN. Nenhum serviço por experiência.	Implementado
RNF-17	Conformidade	Localidade de processamento de IA declarada e auditável; ausência de prazo de retenção é decisão registrada, não omissão.	Documentado

LIMITAÇÃO CONHECIDA

RNF-01 está *configurado* mas não *verificado*: o teste de carga com 4.000 *logins* e aberturas de experiência em dez minutos é obrigatório antes de novembro de 2026 e ainda não foi executado. É o maior risco técnico aberto do projeto (Capítulo 18, `R-01`).

3.3 Rastreabilidade

A matriz abaixo liga *drivers*, requisitos-chave e as decisões que os realizam. Ela é a resposta à pergunta “por que o sistema é assim” em uma única página.

Driver	Requisitos	Decisões (ADR)
DR-01	**RNF-06**, **RF-04**	**ADR-04** (*stack* por contrato), **ADR-08** (RLS forçada e porta única)
DR-02	**RF-10**, **RF-11**, **RNF-03**, **RNF-16**	**ADR-01** (*bundle* estático), **ADR-02** (*monorepo*)
DR-03	**RF-40**, **RF-45**	**ADR-09** (domínio por experiência na API)
DR-04	**RF-03**, **RF-08**, **RNF-13**	**ADR-03** (OIDC genérico), **ADR-15** (hub como emissor provisório)
DR-05	**RF-30**–**RF-34**, **RNF-07**	**ADR-07** (*gateway* de IA na API), **ADR-16** (ADC e modelo *versus* região)
DR-06	**RF-20**–**RF-25**, **RNF-02**	**ADR-06** (Pub/Sub), **ADR-14** (Postgres particionado, sem Redis nem BigQuery)

DR-07	**RF-47**–**RF-49**, **RF-53**	**ADR-12** (`grau` como invariante de API), **ADR-10** (bem-estar sem visão de gestor)

4

Casos de uso

Os casos de uso descrevem o sistema pela ótica de quem o usa. Cada um recebe identificador estável, ator primário, pré-condição, fluxo principal, fluxos alternativos e a regra de negócio que o governa. A rastreabilidade para requisitos está na Seção 4.4.

4.1 Atores

Ator	Tipo	Descrição
Aluno	Primário	Estudante da rede. Escolhe a missão no hub próprio e entra na experiência. É titular de dado de menor.
Professor	Primário	Prepara e conduz a aula, lê a turma, gera plano de aula e faz o próprio *check-in* de bem-estar.
Gestor escolar	Primário	Administra usuários da escola.
Admin do contrato	Primário	Único papel autorizado a atravessar escolas. Publica manifestos e lê o agregado da rede.
Provedor de identidade	Secundário	WSO2 Identity Server (futuro). Hoje o papel de emissor é exercido pelo próprio hub.
Vertex AI (Gemini)	Secundário	Geração de texto e de imagem, sempre intermediada pela API.
Pub/Sub	Secundário	Transporte durável do canal analítico de telemetria.
CI/CD (GitHub Actions)	Secundário	Publica *bundles* e faz *upsert* do manifesto. É um ator: age no sistema por API autenticada.

4.2 Diagrama de casos de uso

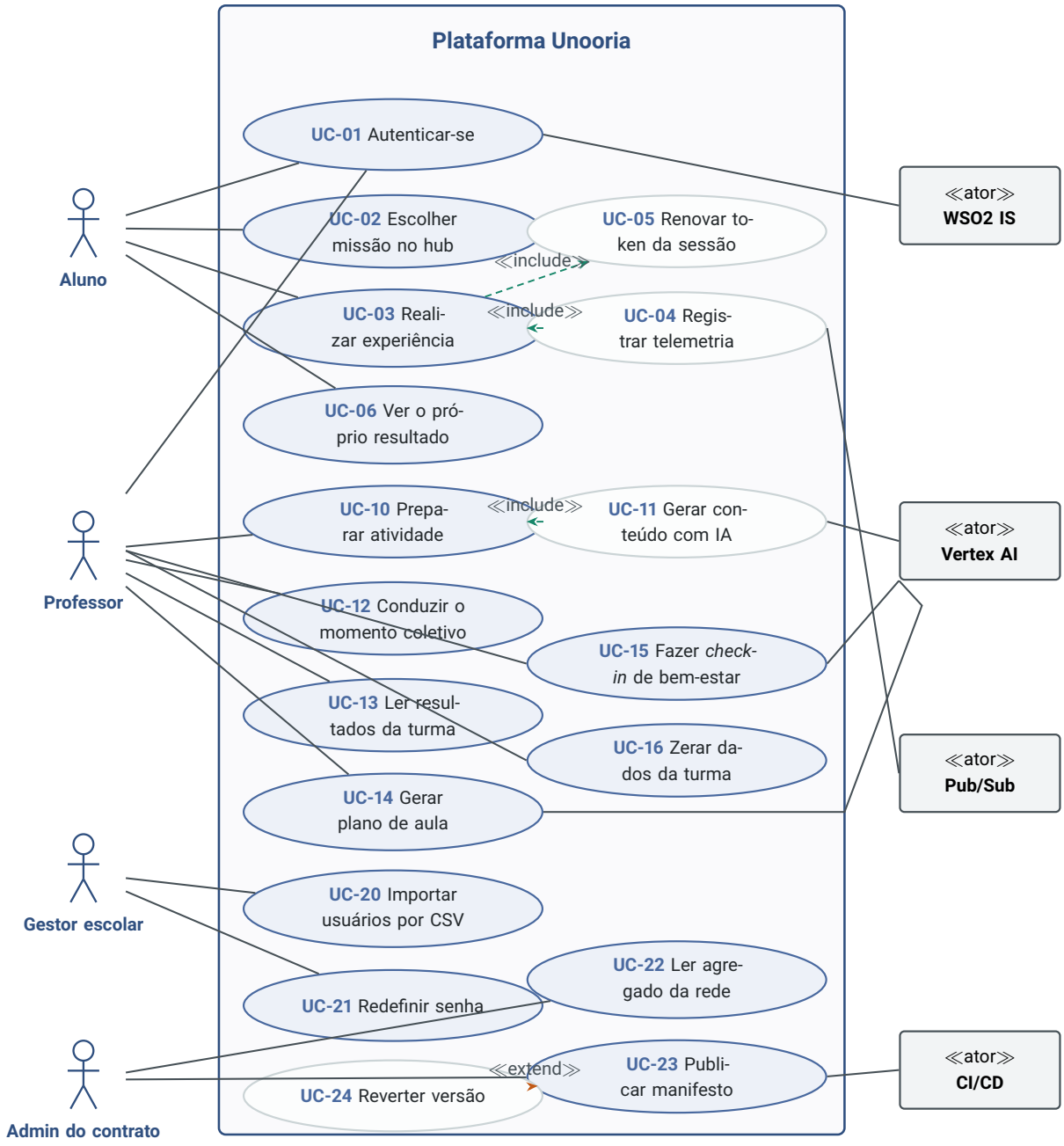


Figura 4.1 – Casos de uso da plataforma. As elipses em tom claro são casos de uso de *suporte*, incluídos por outros e nunca iniciados diretamente por um ator humano. Os quatro papéis são mutuamente exclusivos: o *middleware* do hub redireciona quem entra na área de outro papel.

Legenda: caso de uso primário · caso de uso de suporte · - - - ≪include≫ · - - - ≪extend≫

4.3 Especificação dos casos de uso

4.3.1 UC-01 — Autenticar-se

> **UC-01 Autenticar-se**
>
> | | |
> |---|---|
> | **Ator primário** | Qualquer usuário |
> | **Pré-condição** | Usuário existe na escola (criado por **UC-20**) |
> | **Pós-condição** | Sessão do hub estabelecida; usuário na *home* do próprio papel |
> | **Requisitos** | **RF-01**, **RF-02**, **RF-05** |

Fluxo principal

1. O usuário informa credencial em `/login`.
2. O hub valida a credencial e cria a sessão, com `papel`, `escolaId`, `turmaId` e `serie`.
3. O *middleware* redireciona para a *home* do papel: `/aluno`, `/professor` ou `/admin/usuarios`.

Fluxos alternativos

- **A1 — credencial inválida:** o formulário permanece com mensagem; nenhuma sessão é criada.
- **A2 — usuário já autenticado acessa área de outro papel:** redirecionado para a *home* do próprio papel. *Não* recebe 403 — um aluno que digita `/professor` volta ao lugar dele sem mensagem de erro, o que é decisão de produto, não descuido de segurança (a autorização real é verificada de novo na API).
- **A3 — WSO2 como emissor:** o passo 2 passa a ser um *redirect* OIDC. Nada muda nos passos 1 e 3, nem na API.

4.3.2 UC-03 — Realizar experiência

Este é o caso de uso central da plataforma e o que mais determina a arquitetura.

> **UC-03 Realizar experiência**
>
> | | |
> |---|---|
> | **Ator primário** | Aluno (ou Professor, nas ferramentas de professor) |
> | **Pré-condição** | Sessão válida; experiência disponível para o papel |
> | **Pós-condição** | Progresso e telemetria registrados; conclusão notificada ao hub |
> | **Inclui** | **UC-04** (telemetria), **UC-05** (renovação de *token*) |
> | **Requisitos** | **RF-12**–**RF-17**, **RF-40** |

Fluxo principal

1. O aluno escolhe a missão em `/aluno` e o hub navega para `/exp/{expId}`.

2. O hub monta o *embed*: assina o *token* de experiência (RS256, 15 min), consulta o manifesto e resolve a URL versionada do *bundle*.
3. Se o manifesto declara `viewport`, o hub desenha a casca de aparelho em volta do `iframe`.
4. O *bundle* carrega, a SDK emite `exp:ready` e o hub responde `hub:init` com *token*, dados do aluno, `apiUrl` e `config`.
5. A experiência opera: chama rotas de domínio da API, registra telemetria em lote e salva progresso.
6. Ao terminar, emite `exp:concluir`; o hub registra progresso e libera a tela seguinte.

Fluxos alternativos

- **A1 — *token* perto de expirar:** a experiência emite `exp:token-expirando`; o hub busca `GET /api/exp-token` e responde `hub:token`. O `iframe` não recarrega e o estado não se perde.
- **A2 — versão de protocolo incompatível:** os dois lados verificam a versão *maior*. Sem `hub:init`, o *handshake* expira e a experiência mostra erro. Falha alta, nunca silenciosa.
- **A3 — sem manifesto (ambiente novo ou desenvolvimento):** o hub lê o `experiencia.json` servido ao lado do `index.html` do *bundle* para obter `viewport` e `permissoes`.
- **A4 — página do hub perde foco:** o hub emite `hub:pause`; a experiência congela cronômetros. Ao voltar, `hub:resume`.
- **A5 — experiência de professor acessada por `/exp/*`:** a rota responde 404. Ferramenta de professor não é missão de aluno.

> **ATENÇÃO**
>
> O passo 4 falhava de forma intermitente numa versão anterior da SDK: com os *assets* em *cache* (um F5), a experiência ficava pronta *antes* de o hub hidratar o *listener*, e o único `exp:ready` se perdia — o *handshake* expirava sem culpa de ninguém. A correção foi *re-enviar* `exp:ready` a cada 400 ms até o hub responder, e tratar `hub:init` repetido como evento normal, não como ruído.

4.3.3 UC-10 — Preparar atividade

> **UC-10 Preparar atividade (exp-01)**
>
> | | |
> |---|---|
> | **Ator primário** | Professor |
> | **Pré-condição** | Sessão de professor válida |
> | **Pós-condição** | Atividade `ativa`, com *feed* gerado e revisado |
> | **Inclui** | UC-11 (geração com IA) |
> | **Requisitos** | RF-30, RF-31, RF-41 |

Fluxo principal

1. O professor cria a atividade com tema, recorte e contexto de aula.
2. A API destila o antagonismo (dois lados) e propõe a pergunta-chave; o professor *aprova ou reescreve* — a origem fica registrada em `pergunta_origem`.
3. O professor dispara a geração. O servidor planeja os lotes (o *mix* é decisão do servidor; a IA apenas preenche) e o *worker* executa em segundo plano.
4. O professor acompanha o progresso por *polling*, revisa os posts, reprova ou regenera itens e ativa a atividade.

Fluxos alternativos

- **A1 — sem saldo de quota:** a geração é recusada *antes* da chamada ao provedor. Gastar *token* e depois recusar seria pagar pelo que não se entrega.
- **A2 — geração órfã (instância morta no meio):** o *polling* do painel detecta a fila órfã e redispara; itens `rodando` há tempo demais voltam à fila por `updatedAt`.
- **A3 — falha de geração de imagem:** o post recebe arte procedural de *fallback* e `image_status = "fallback"`. A atividade não trava.

4.3.4 UC-13 — Ler resultados da turma

> **UC-13 Ler resultados da turma**
>
> | | |
> |---|---|
> | **Ator primário** | Professor |
> | **Pré-condição** | Turma com pelo menos uma sessão registrada |
> | **Pós-condição** | Nenhuma — é leitura |
> | **Requisitos** | **RF-47**–**RF-49**, **RF-52** |

Regras de negócio que governam este caso de uso — e que são a parte mais importante dele:

1. **A listagem da turma não mostra valor psicométrico.** Listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez.
2. **O relatório individual abre pelo nível do instrumento.** "Caiu em 2 de 3" sem dizer que era o nível C faz o adulto concluir que a criança é ingênua — exatamente o rótulo que o projeto existe para não produzir.
3. **Ausência de oportunidade nunca é lida como omissão.** Zero cartões aparecidos produz "as mensagens não chegaram nesta partida", nunca "ela ignorou tudo".
4. **Valor com uma única fonte de medida não recebe número.** A API não devolve o número; o hub não pode imprimi-lo.
5. **O agregado é de gestor; a linha de uma criança, não.** São *controllers* separados, e não um `@Roles` misto com exceção por método.

4.3.5 UC-14 — Gerar plano de aula

UC-14 Gerar plano de aula (exp-90)

Ator primário	Professor
Pré-condição	Referências curriculares do contrato importadas
Pós-condição	Plano `pronto` em armazenamento privado, editável
Requisitos	**RF-35**, **RF-36**, **RF-34**

Fluxo principal

1. O professor escolhe disciplina e série; a experiência chama `POST /planos`, que responde **202** imediatamente.
2. A API faz *claim* atômico da linha (`pendente` → `gerando`) e chama o modelo de saída longa com o material real — currículo da rede e as quatro unidades do ano, algo em torno de 190.000 *tokens* só de *prompt*.
3. O Markdown gerado é gravado imutável em `planos/{escolaId}/{id}/gerado.md`; a conversão para HTML alimenta o editor.
4. O professor edita e salva; o HTML é sanitizado por *allowlist* e gravado em `atual.html`. O versionamento de objetos do *bucket* dá histórico de cada salvamento sem uma linha de código.

Fluxos alternativos

- **A1 — saída cortada:** `finishReason = MAX_TOKENS` é *erro*, não conteúdo. O plano vai para `erro` com mensagem, em vez de entregar meio plano.
- **A2 — *restart* no meio da geração:** a linha fica `gerando` sem *worker*; após 12 minutos é considerada órfã e retomada, no máximo duas vezes.
- **A3 — restaurar o original:** o professor descarta a edição e volta ao `gerado.md`, que nunca foi sobrescrito.

4.3.6 UC-15 — Fazer *check-in* de bem-estar

UC-15 *Check-in* de bem-estar (exp-91)

Ator primário	Professor (e somente ele)
Pré-condição	Sessão de professor válida
Pós-condição	*Check-in* do dia registrado; ao sétimo, ciclo pontuado e classificado
Requisitos	**RF-37**, **RF-53**, **RF-54**

Fluxo principal

1. `GET /bemestar/hoje` abre o *check-in* do dia com três itens do DASS-21 — *verbatim*, na versão brasileira validada — e os textos de entorno gerados pela IA.
2. O professor responde os três itens (0–3) e, se quiser, escreve texto livre.
3. `POST /bemestar/checkins/:id/respostas` grava. No sétimo *check-in* a API soma por subescala ($\times 2$) e classifica pelas faixas publicadas.

Fluxos alternativos

- **A1 — texto de IA barrado, sem saldo ou sem *provider*:** entra o texto fixo do catálogo. O *check-in* nunca deixa de abrir.
- **A2 — sinal de apoio** (item 17 ou 21 ≥ 2, ou termo de risco no texto livre): mensagem *fixa* de apoio com CVV 188, SAMU 192 e CAPS/UBS, e a IA *não é chamada*.
- **A3 — ciclo expirado:** 21 dias sem fechar encerra o ciclo como `expirado`, sem pontuação.
- **A4 — apagar tudo:** `DELETE /bemestar/dados` remove ciclos e *check-ins* do professor.

> **REGRA INVIOLÁVEL**
>
> O instrumento é **fixo e *verbatim***. A IA nunca reescreve um item do DASS-21: reformular quebra a validação psicométrica e invalida as faixas de corte. O que a IA gera é o entorno — abertura do dia, pergunta aberta, acolhimento — e cada texto passa por validação antes de chegar ao professor. A IA nunca recebe nome, escola, pontuação ou faixa.

4.3.7 UC-23 — Publicar manifesto de experiência

> **UC-23 Publicar manifesto**
>
> | | |
> |---|---|
> | **Ator primário** | CI/CD (em nome de `admin-contrato`) |
> | **Pré-condição** | *Bundle* publicado em `gs://{contrato}-exp/{expId}/{versao}/` |
> | **Pós-condição** | Linha da tabela `experiencias` atualizada; hub passa a servir a nova versão |
> | **Estendido por** | UC-24 (reverter versão) |
> | **Requisitos** | RF-10, RF-11 |

Fluxo principal

1. O CI sincroniza `dist/` para o caminho versionado do *bucket* e marca o `index.html` com `Cache-Control: max-age=60`.
2. O CI assina um *token* de cinco minutos com a chave de assinatura e chama `PUT /experiencias/{expId}` com o conteúdo do `public/experiencia.json`.
3. A API valida o *token* contra o JWKS que o *hub* publica e faz o *upsert*.

> **NOTA**
>
> A ordem importa e está codificada no *workflow*: as experiências são publicadas *depois* do hub, porque o *token* do passo 2 é validado contra o JWKS servido pelo hub. Hub fora do ar significa manifesto rejeitado — e um manifesto rejeitado é preferível a um manifesto aceito apontando para um hub que não subiu.

UC-24 — Reverter versão: o mesmo *script* com `--versao <anterior>`. Não há *build*, não há imagem, não há *deploy*: é um `UPDATE` de `versao_ativa`, e o próximo aluno que abrir a experiência recebe o *bundle* anterior. É a operação de emergência mais rápida da plataforma, e existe por desenho (**ADR-01**).

4.4 Rastreabilidade de casos de uso

UC	Requisitos	Superfície	Módulo da API
UC-01	RF-01, RF-02, RF-05	`/login`	`auth`
UC-02	RF-10	`/aluno`	`experiencias`
UC-03	RF-12–RF-17	`/exp/:expId`	`motor`, `unootube`, `vila`, `progress`
UC-04	RF-20–RF-25	SDK	`telemetry`
UC-05	RF-14	`/api/exp-token`	`auth`
UC-06	RF-40	dossiê da experiência	`dossie`
UC-10	RF-41	`/professor/atividades/ai`	`atividades`, `geracao`
UC-11	RF-30–RF-33	—	`ai`, `geracao`
UC-12	RF-41	painel de etapas	`atividades`, `motor`
UC-13	RF-47–RF-52	`/professor/entre-re`	`analise`, `vila`, `unootube`
UC-14	RF-34–RF-36	`/professor/plano-de-aula`	`planos`
UC-15	RF-37, RF-53	`/professor/bem-esta`	`bemestar`
UC-16	RF-50	botão de zerar turma	`vila`
UC-20	RF-06	`/admin/usuarios`	`usuarios`
UC-21	RF-07	`/admin/usuarios`	`usuarios`
UC-22	RF-52	`/analise`	`analise`
UC-23	RF-10	CI/CD	`experiencias`
UC-24	RF-11	*script*	`experiencias`

5

Arquitetura de *software*

Este capítulo descreve a organização interna do *software*: que peças existem, o que cada uma faz, como conversam e por que a fronteira está onde está. A arquitetura de *nuvem* — onde essas peças rodam — é o Capítulo 6.

5.1 Visões arquiteturais

O sistema é descrito por cinco visões complementares, no espírito do modelo 4+1 (KRUCHTEN, 1995). Nenhuma delas é suficiente isoladamente.

Visão	Notação	Onde está
Lógica	C4 níveis 1–3; diagrama de classes	Seções 5.3 a 5.6
Processos	Diagramas de sequência e de estado	Seção 5.8; Cap. 9
Desenvolvimento	Estrutura do *monorepo* e grafo de dependências	Seção 5.7
Física / implantação	Diagramas de implantação e de rede	Cap. 6
Cenários	Casos de uso	Cap. 4

5.2 Estilo arquitetural e princípios

A plataforma é, deliberadamente, um **monólito modular** de *backend* (uma API NestJS com módulos de domínio bem separados) servindo uma **constelação de artefatos estáticos** (as experiências). Não é microsserviços, e não é por acaso.

DECISÃO DE PROJETO

Um serviço de *backend*, muitos artefatos de *frontend* (**ADR-01**, **ADR-02**). Microsserviços resolvem escala organizacional — times independentes que precisam implantar sem coordenação. Aqui há uma pessoa. O que precisa escalar independentemente é a *entrega* das experiências, e isso foi resolvido tirando-as do plano de execução: *bundles* estáticos numa CDN escalam sem que exista um serviço. A modularidade que os microsserviços dariam por processo é obtida por *módulo NestJS* e por convenção de prefixo de tabela — com a vantagem de um `JOIN` continuar sendo um `JOIN`, e não uma chamada de rede.

Os princípios que atravessam todo o código:

Validação na fronteira, sempre.
Todo *input* externo passa por Zod (MCDONNELL, 2026): corpo, *query*, *postMessage* recebido e as variáveis de ambiente — estas no *boot*, de modo que configuração errada derruba o processo na subida, não no meio de uma requisição de aluno.

Uma porta por recurso crítico.
O banco só é acessado por `PrismaService.comTenancy`; a IA só por `src/ai`; o *design system* só pela ponte `packages/ui`. Recurso com duas portas tem duas chances de esquecer a tranca.

Regra de domínio pura, casca fina.
Cada módulo de experiência separa `catalogo.ts` (dados), `regras.ts` (funções puras, testáveis sem banco) e um *service* que persiste. O *controller* é só a casca HTTP.

Falhar alto, ou falhar fechado.
Incompatibilidade de protocolo falha alto e visível; rota cuja proteção depende de configuração ausente responde 401, nunca fica aberta.

O tipo carrega a regra.
Quando uma regra pode ser expressa no sistema de tipos, ela é — porque comentário não é controle (Seção 5.6).

5.3 C4 nível 1 — Contexto do sistema

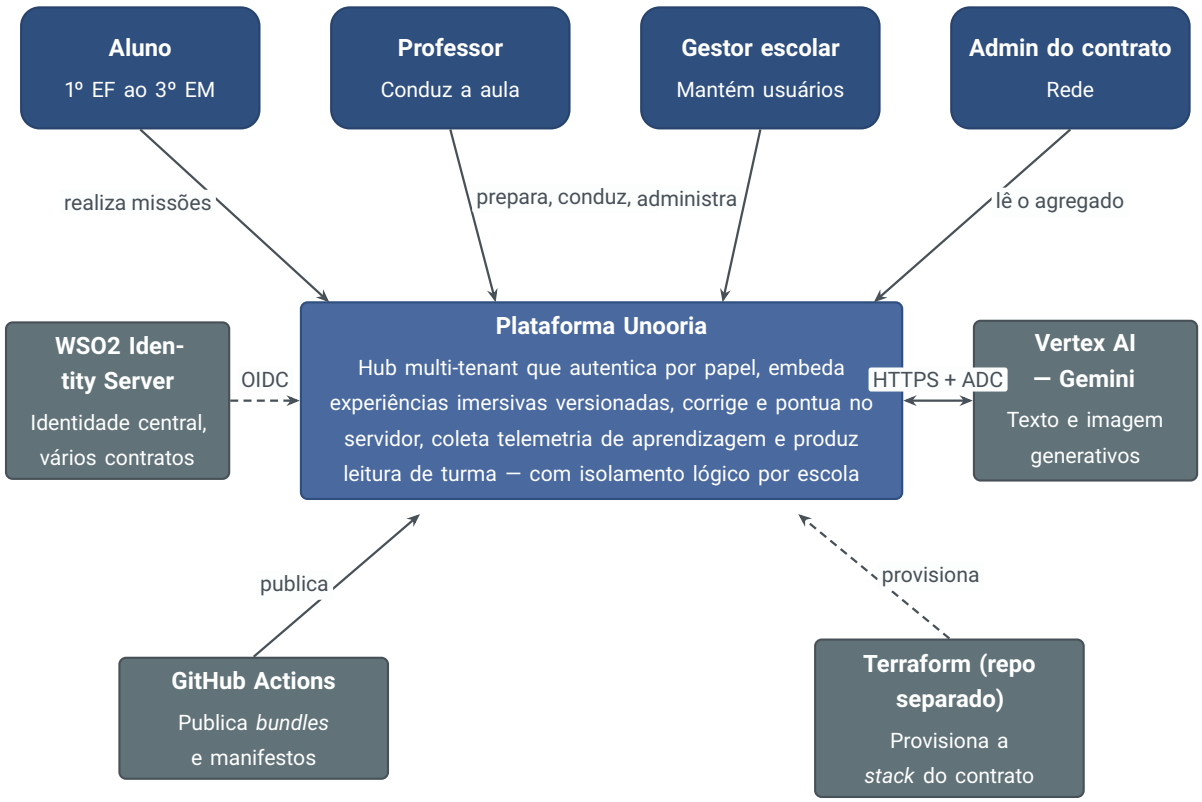


Figura 5.1 – C4 nível 1: contexto. Quatro papéis humanos mutuamente exclusivos, um provedor de identidade compartilhado entre contratos, um único serviço externo na dependência de IA, e dois atores de automação. O Terraform aparece porque a fronteira entre os dois repositórios é um contrato explícito, não um detalhe de operação.

> **NOTA**
>
> A dependência externa *não substituível em tempo de execução* é o Vertex AI, e mesmo assim sua indisponibilidade não derruba a plataforma: as experiências que não usam IA (exp-02, exp-70) seguem íntegras, o *feed* da exp-01 continua sendo curado a partir de conteúdo já gerado, e as ferramentas de professor degradam para texto fixo (exp-91) ou recusam a geração com mensagem clara (exp-90). Essa contenção de acoplamento é deliberada (**RNF-12**).

5.4 C4 nível 2 — Contêineres

O nível de contêineres é onde a assimetria da arquitetura aparece. Há *quatro* contêineres de execução e um número crescente de *artefatos* que não são contêineres — e tratá-los como se fossem seria o erro que este desenho existe para evitar.

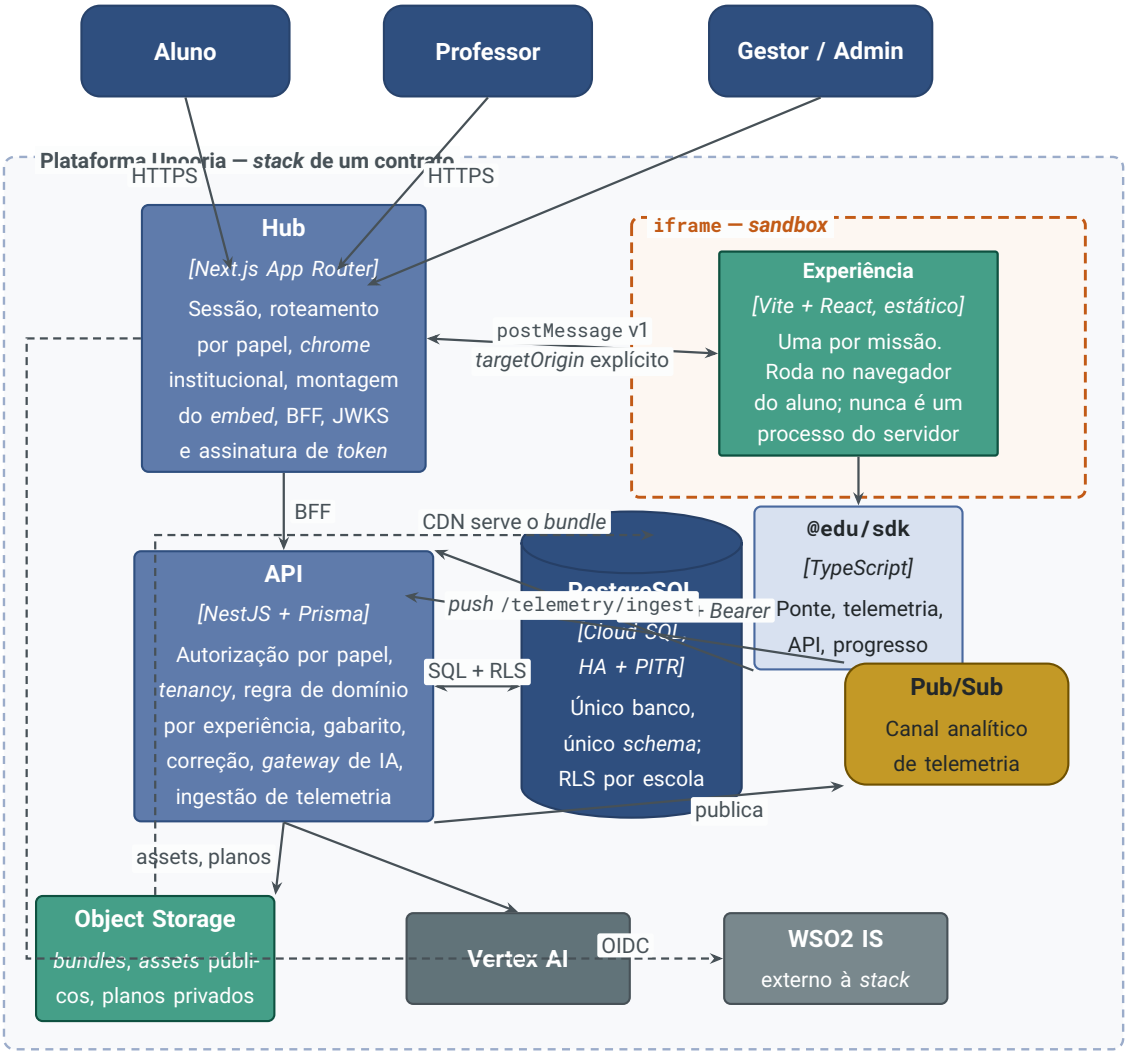


Figura 5.2 – C4 nível 2: contêineres. Só o hub, a API, o banco e o Pub/Sub são peças em execução. A experiência é um *artefato* servido pela CDN e executado no navegador; a SDK é biblioteca dentro dele. A linha tracejada laranja é a fronteira do *sandbox* do `iframe` — a fronteira de segurança mais importante do desenho de *frontend*.

Legenda: contêiner em execução · artefato estático · biblioteca · banco · fila · armazenamento de objetos · externo

5.4.1 Catálogo de contêineres e artefatos

Peça	Tipo	Tecnologia	Responsabilidade e limite
`apps/hub`	Contêiner	Next.js 16, App Router	Autenticar, roteirizar por papel, desenhar *todo* o *chrome* institucional, montar o *embed*, assinar *token* de experiência e publicar JWKS. **Não** contém regra de domínio de experiência.
`apps/api`	Contêiner	NestJS, Prisma	Autorizar, garantir *tenancy*, aplicar regra de domínio, guardar gabarito, corrigir, intermediar IA, ingerir telemetria. É o único que fala com o banco.

Peça	Tipo	Tecnologia	Responsabilidade e limite
`apps/experiencias/*`	Artefato	Vite, React	Interface e interação de uma missão. **Nunca** guarda gabarito, nem calcula nota que valha, nem persiste *token*.
`packages/sdk`	Biblioteca	TypeScript puro, Zod	Contrato `postMessage`, cliente HTTP autenticado, telemetria em lote, progresso. É a *única* forma legítima de uma experiência falar com a plataforma.
`packages/ui`	Biblioteca	TypeScript, React	Ponte para o *design system*: marca `'use client'`, desfaz colisão de nomes entre seções e preserva *tree-shaking*.
PostgreSQL	Contêiner gerenciado	Cloud SQL	Único banco, único *schema* lógico. Guarda tudo, com RLS forçada.
Pub/Sub	Contêiner gerenciado	Pub/Sub	Amortecedor *durável* do pico de telemetria. Entrega *at least once* por *push* autenticado.
Object Storage	Artefato	Cloud Storage	Três *buckets* com regimes diferentes: *bundles* (público, imutável), *assets* (público, CDN), planos (**privado**, versionado).

REGRA INVIOLÁVEL

Um banco, um *schema*. Nem banco por experiência — elas não são isoladas (o perfil da Vila alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos; além disso, um *pool* do Prisma por experiência estouraria `max_connections`) — nem *schema* por experiência: a única coisa que um *schema* daria além do prefixo é fronteira de `GRANT`, e existe *uma role* de aplicação. A separação é por **prefixo de tabela**: `exp_NN_` para tabela de experiência, sem prefixo para tabela de plataforma (Capítulo 8).

5.5 C4 nível 3 — Componentes

5.5.1 Componentes da API

A API é organizada em módulos NestJS. A Figura 5.3 separa-os em quatro faixas, e a faixa a que um módulo pertence determina o que ele pode conhecer.

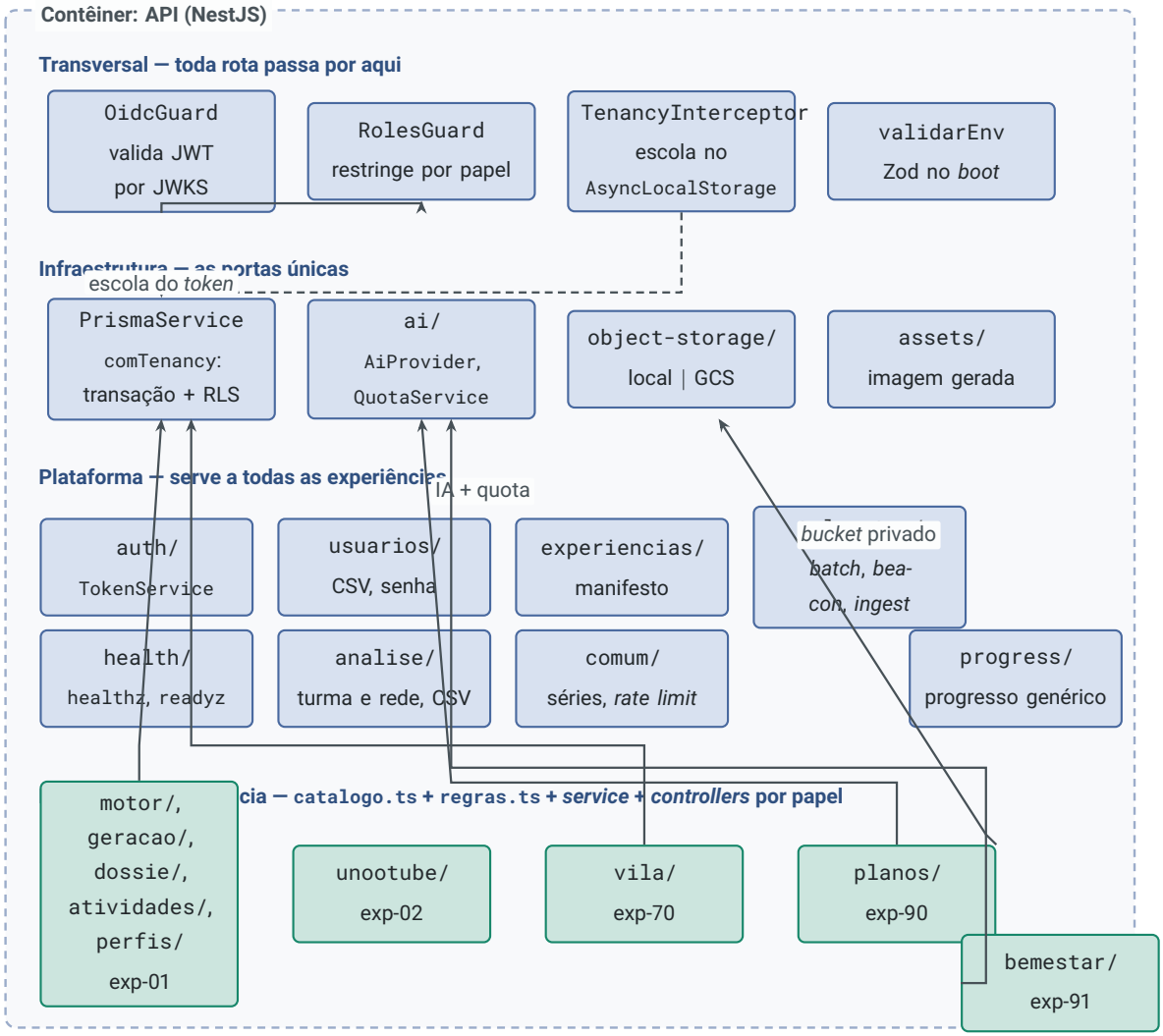


Figura 5.3 – C4 nível 3: componentes da API. A faixa transversal roda em toda rota; a faixa de infraestrutura contém as portas únicas (banco, IA, armazenamento); a faixa de plataforma serve a todas as experiências; a faixa de domínio tem um módulo por experiência, e nenhum módulo dessa faixa conhece outro.

REGRA INVIOLÁVEL

Módulo de domínio não conhece módulo de domínio. `vila/` não importa `unootube/`. Quando duas experiências precisam se comunicar — o perfil da Vila alimentando a largada do Instanooria — a comunicação é por *tabela de contrato* explícita (`exp_01_exp_70_perfis_importados`, que carrega os dois números no nome) e por um *endpoint* de fronteira (`POST /atividades/:id/perfis/da-vila`). Chamar a tabela por um número só mentiria sobre quem depende dela.

5.5.2 Componentes do hub

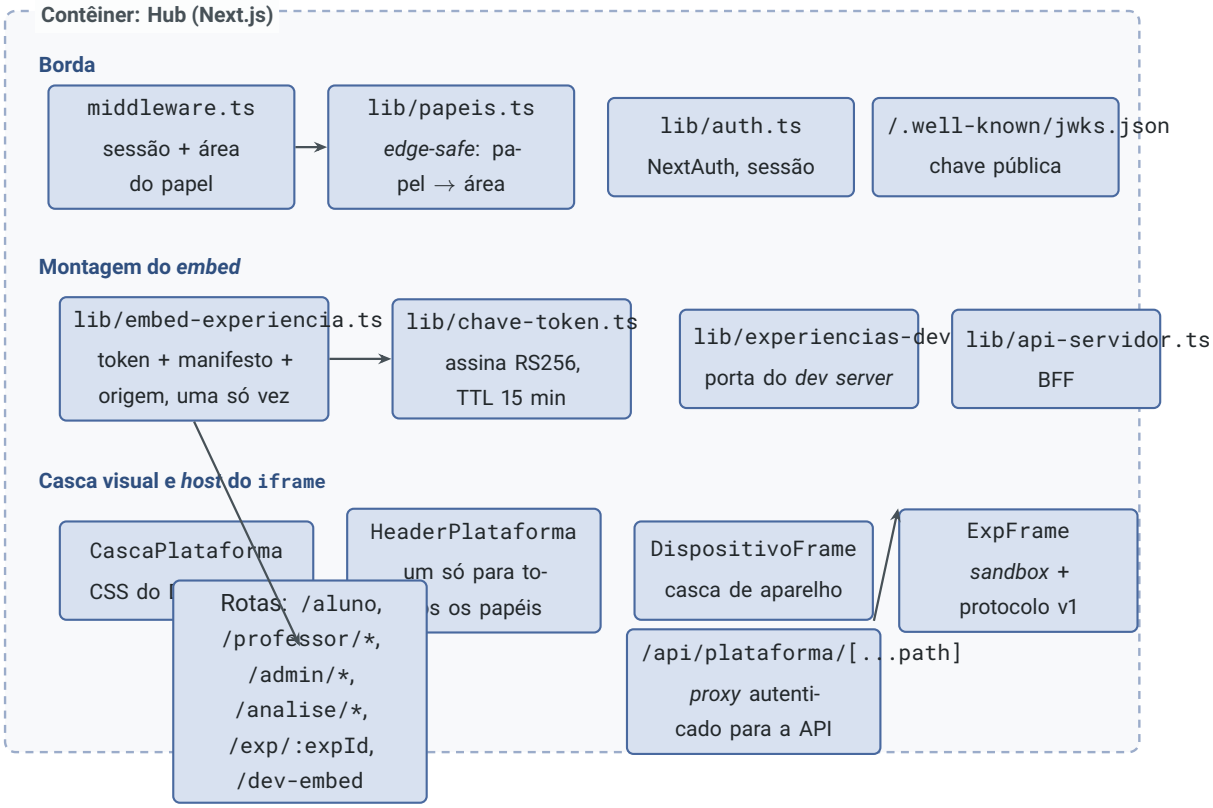


Figura 5.4 – C4 nível 3: componentes do hub. A montagem do *embed* é *uma só função* usada pelas quatro superfícies que embedam (`/exp/[expId]`, `/dev-embed`, `/professor/plano-de-aula` e `/professor/bem-estar`) — duplicá-la era o caminho mais curto para uma delas passar a assinar o *token* com *claims* diferentes.

DECISÃO DE PROJETO

Um único *header* para todos os papéis, e uma única casca visual. Antes, cada área copiava o próprio *header* e a experiência não tinha nenhum. Hoje `HeaderPlataforma` serve `/aluno`, `/professor`, `/admin`, `/analise` e `/exp/*`, e a casca visual saiu do *layout* do grupo de rotas para `CascaPlataforma`, porque `/exp/*` e `/dev-embed` estão fora do grupo e também precisam dela. A regra de ouro passou a valer um nível abaixo: o *header* é *chrome* do hub, desenhado *acima* do aparelho — dentro do `iframe` nada da marca institucional entra.

5.6 C4 nível 4 — Código

O nível 4 do C4 raramente vale a pena desenhar por inteiro. Vale aqui para *três* lugares, porque neles o desenho de tipos *é* a arquitetura.

5.6.1 O contrato da ponte e o *guard* de *tenancy*

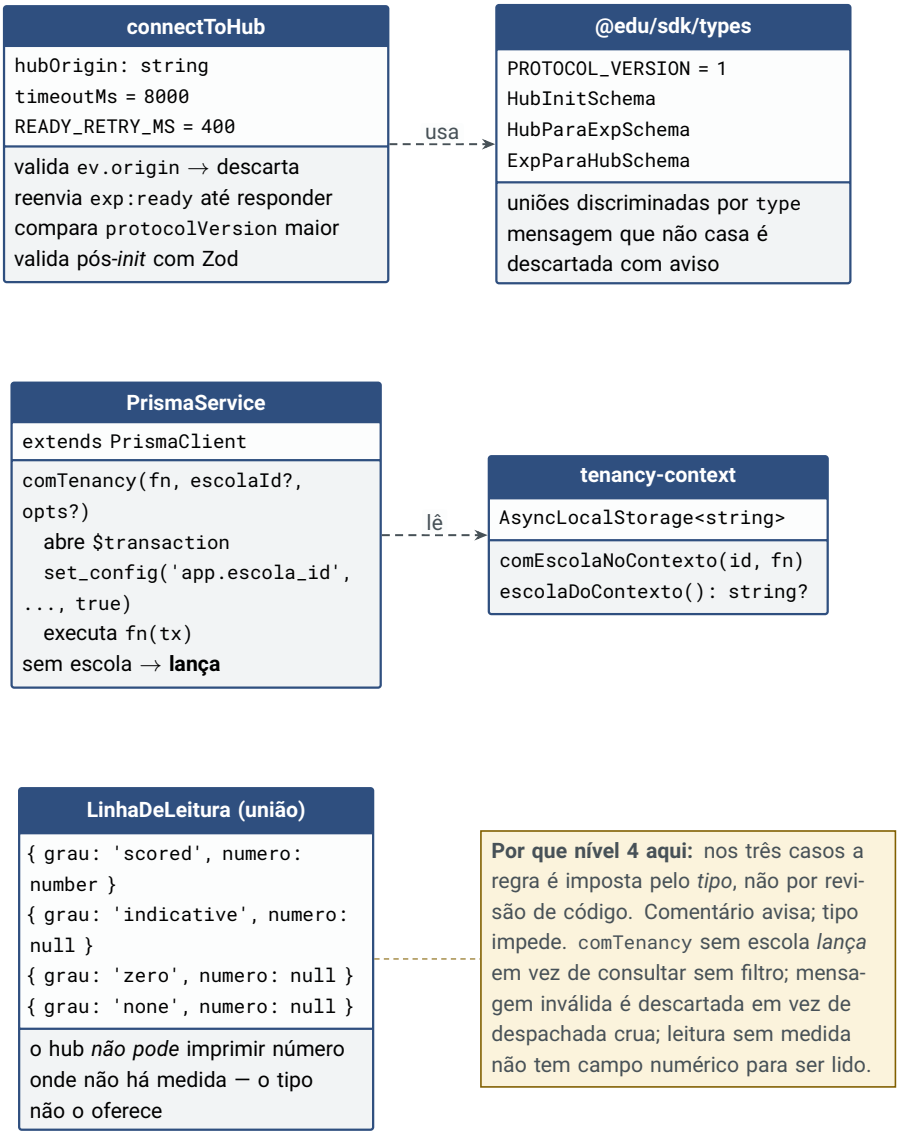


Figura 5.5 – C4 nível 4: os três pontos em que a regra arquitetural está no sistema de tipos. `comTenancy` é a única porta do banco e falha alto sem escola; o contrato da ponte descarta o que não valida; a leitura de valores não oferece número onde não há medida.

5.6.2 O gabarito de um módulo de domínio

Todo módulo de experiência segue a mesma forma. Não é convenção estética: é o que permite testar a regra sem banco e trocar a persistência sem tocar a regra.

Listing 5.1 – Forma canônica de um módulo de domínio de experiência

```
apps/api/src/<slug>/
  catalogo.ts     // DADOS: itens, textos, gabarito. Constantes puras.
  regras.ts       // REGRA: funcoes puras. Sem Prisma, sem fetch, sem Date.
  <slug>.service.ts   // PERSISTENCIA: le com lock, aplica a regra, grava.
                      // Toda query dentro de comTenancy e filtrando escola_id.
  <slug>.controller.ts        // casca HTTP do papel "aluno"
  <slug>-professor.controller.ts  // casca HTTP do papel "professor"
```

```
  <slug>.module.ts

// A separacao de controllers POR PAPEL e deliberada: um @Roles misto com
// excecao por metodo e o caminho mais curto para um GET de professor herdar
// silenciosamente a permissao de um POST de aluno.
```

NOTA

A consequência prática dessa forma aparece nos testes: `vila-regras.spec.ts`, `unootube-regras.spec.ts` e `bemestar-regras.spec.ts` rodam *milissegundos* sem PostgreSQL, porque exercitam `regras.ts` puro. Os testes que precisam de banco (`*-postgres.spec.ts`, `tenancy.spec.ts`) são separados e cobrem exatamente o que não dá para afirmar sem ele: *tenancy*, *lock* e concorrência.

5.7 Estrutura do *monorepo*

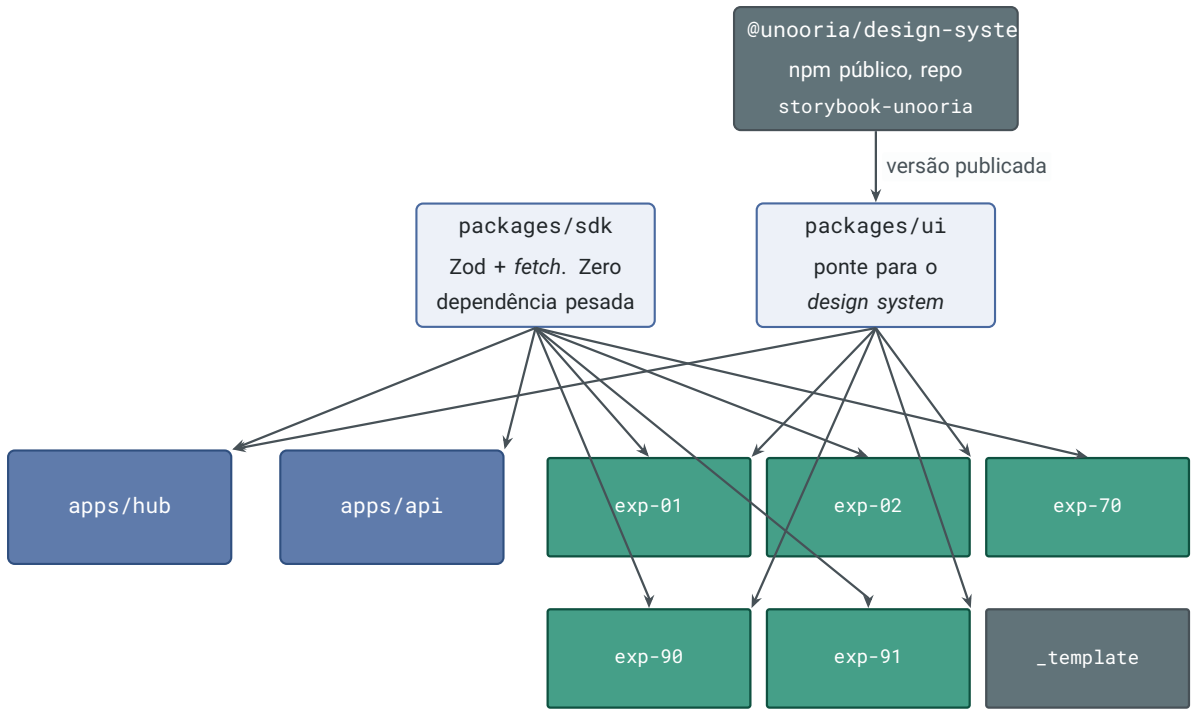


Figura 5.6 – Grafo de dependências do *monorepo*. A SDK é consumida por tudo — é por isso que uma mudança nela precisa testar tudo no mesmo *pull request*, e por isso o *monorepo* (**ADR-02**). A exp-70 não depende de `packages/ui`: é jogo em *canvas* 3D, sem DOM de interface, e é a exceção registrada em ADR à regra do *design system*.

Diretório	Conteúdo
`apps/hub`	Aplicação Next.js: rotas por papel, *embed*, BFF, JWKS.
`apps/api`	Aplicação NestJS: 22 módulos, Prisma, 9 migrações.
`apps/experiencias/*`	Uma pasta por experiência, mais `_template`.

`packages/sdk`	Contrato da ponte e telemetria.
`packages/ui`	Ponte do *design system* e CSS da plataforma.
`docs/`	Contratos com infraestrutura e registro de decisões.
`scripts/`	*Scaffolding*, chave de assinatura, importação de referências, *walkthroughs* de *smoke* visual, *probe* de *pointer lock*.
`wso2/`	*Compose* próprio do provedor de identidade, com ciclo de vida separado.
`documentacao-latex/`	Este documento.

A orquestração é Turborepo (Vercel, 2026), com *pipeline* declarado em `turbo.json`: `build` depende do `build` das dependências; `typecheck` e `test` também. É isso que permite ao CI rodar apenas o que mudou:

Listing 5.2 – O comando que o CI executa

```
pnpm turbo typecheck lint test build --filter='...[HEAD^]'
```

5.8 Fluxos principais

5.8.1 Curadoria de um bloco de *feed* (exp-01)

Este é o fluxo com mais concorrência da plataforma e o que mais exige do banco.

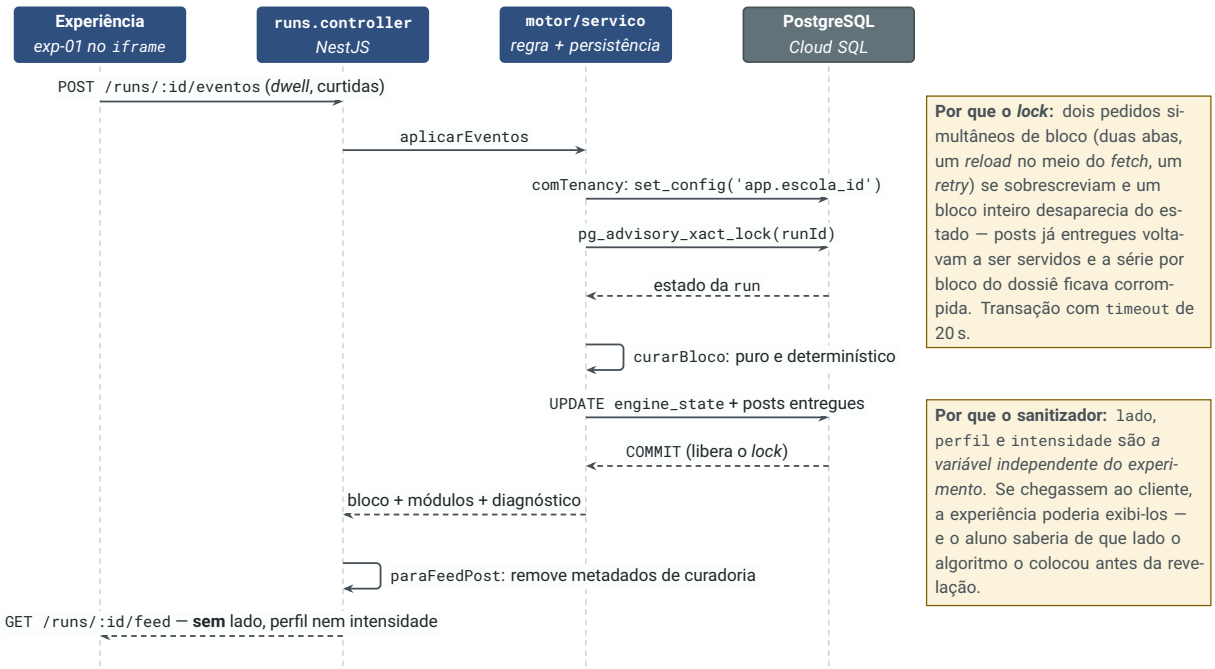
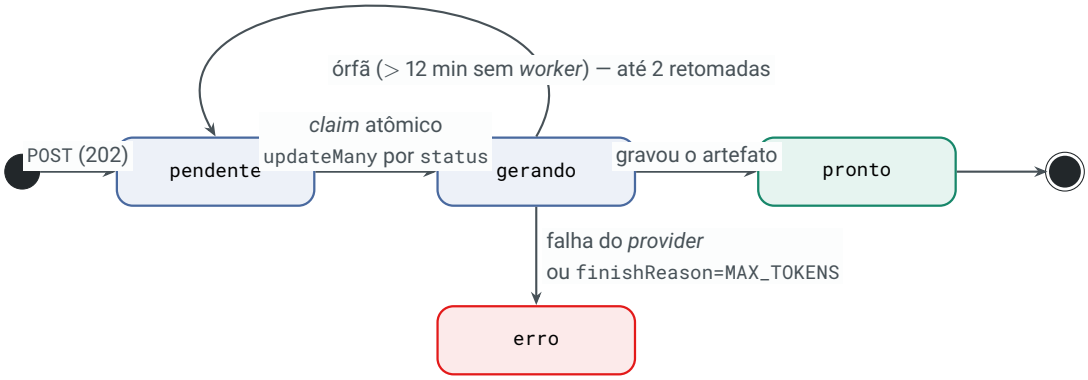


Figura 5.7 – Sequência da curadoria de um bloco de *feed*. A regra (`curarBloco`) é pura; o *service* lê com *lock*, aplica e grava; o *controller* sanitiza. As três responsabilidades ficam em arquivos diferentes de propósito.

5.8.2 Estados de uma geração assíncrona

Duas rotinas da plataforma são assíncronas e longas: a geração de *feed* da exp-01 e a geração de plano de aula da exp-90. Ambas seguem a mesma máquina de estados, com *claim* atômico em vez de fila externa.



O *claim* atômico (`UPDATE ... WHERE status = 'pendente'` com contagem de linhas afetadas) substitui uma fila externa: duas instâncias do Cloud Run podem tentar, e só uma ganha a linha. É a menor peça possível que resolve o problema — e não exige operar um serviço a mais (**ADR-17**).

Figura 5.8 – Máquina de estados das gerações assíncronas (exp-01 e exp-90). O *claim* atômico dispensa fila externa; a retomada de órfã é disparada pelo *polling* do próprio painel, não por um *scheduler*.

LIMITAÇÃO CONHECIDA

Não há retomada no *boot*. Se todas as instâncias morrerem e ninguém abrir o painel, uma geração órfã fica parada até que alguém abra. É aceito porque o *polling* do painel é justamente o momento em que alguém se importa com aquela geração — e porque a alternativa é um *scheduler*, que é uma peça nova para operar. O gatilho de reavaliação é a primeira reclamação real de geração parada.

6

Arquitetura de nuvem e implantação

Este capítulo descreve *onde* o *software* do Capítulo 5 roda: quais recursos gerenciados existem, como o tráfego atravessa a plataforma, como a identidade de carga de trabalho substitui chaves e onde estão as fronteiras de rede.

A infraestrutura é provisionada por Terraform (BRIKMAN, 2022) num **repositório separado**. Este documento descreve o *contrato* entre os dois repositórios — nomes de recursos esperados, variáveis consumidas, artefatos publicados — e não o código Terraform.

> **REGRA INVIOLÁVEL**
>
> **Este repositório nunca cria recurso GCP imperativamente.** Nenhum *script* de `scripts/` chama `gcloud ... create`. O CI apenas *usa* recursos que já existem: envia imagem para um Artifact Registry existente, executa um Cloud Run Job existente, sincroniza um *bucket* existente. Recurso criado por *script* é recurso que o Terraform não sabe que existe — e que ninguém destrói quando o contrato termina.

6.1 Princípios de nuvem

Uma *stack* por contrato.
Todos os nomes de recurso derivam de `{contrato} = {app_name}-{env}`. No piloto, `plataforma-unooria-dev`. Replicar um contrato é instanciar o *module* Terraform com outro prefixo.

Serviço gerenciado antes de orquestração.
Cloud Run em vez de GKE; Cloud SQL em vez de PostgreSQL autogerido; Pub/Sub em vez de *broker* próprio. O critério não é moda: é quantas peças uma pessoa consegue operar de plantão às oito da manhã.

Sem chave de longa duração.
O CI autentica por Workload Identity Federation; a API autentica no Vertex por ADC. Nenhum arquivo JSON de conta de serviço existe em nenhum lugar do fluxo.

Reversão por ponteiro.
Três níveis: revisão do Cloud Run (hub e API), `versaoAtiva` no manifesto (experiências) e versionamento de objetos (planos de aula). Nenhum deles exige *build*.

Estático na borda, dinâmico no mínimo.
O que pode ser servido pela CDN é servido pela CDN. O que sobra para o Cloud Run é regra de negócio.

6.2 C4 nível 1 de implantação — o contrato

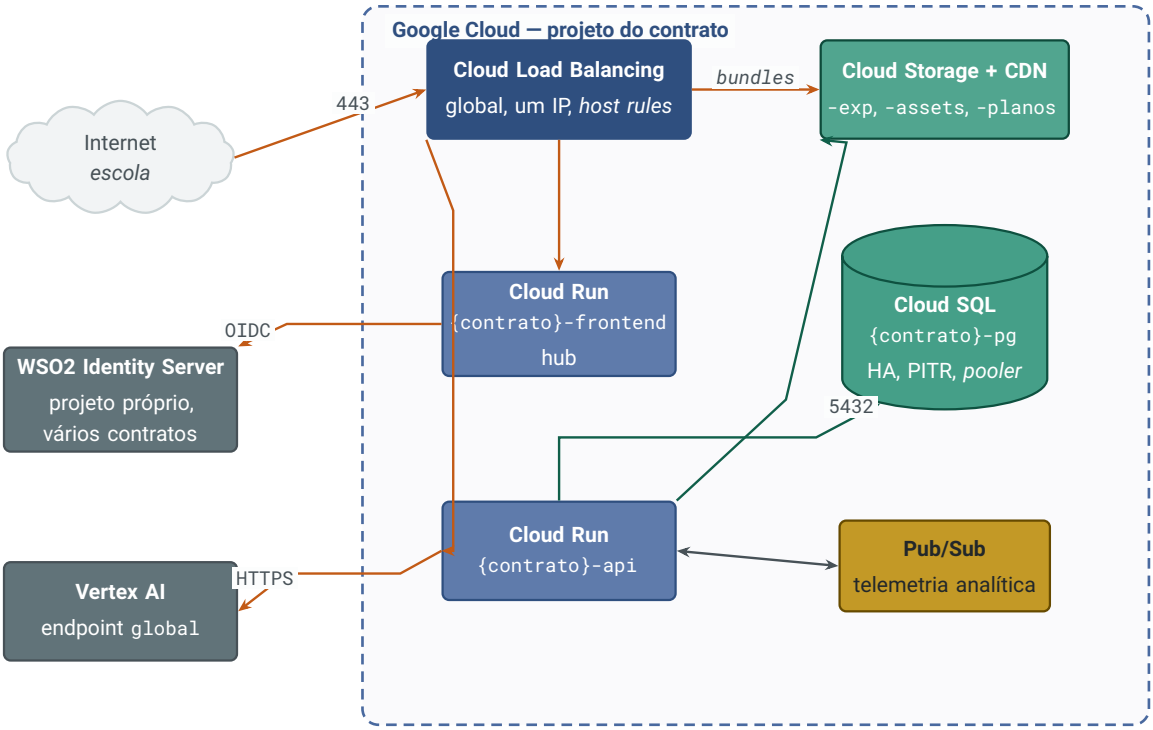


Figura 6.1 – Implantação de um contrato. Um único *Load Balancer* global recebe todo o tráfego HTTPS e distribui por *host rules*: hub, API, *bundles* e *assets* respondem em domínios diferentes com o *mesmo* IP. O Cloud SQL só tem IP privado — daí a existência do Cloud Run Job de migração dentro da VPC.

Legenda: tráfego público (TLS) tráfego privado (VPC) serviço de borda serviço de aplicação banco mensageria fora da *stack*

6.3 Diagrama de implantação detalhado

A Figura 6.2 abre o mesmo desenho no nível em que ele é operado: com os *buckets* separados por regime de acesso, os segredos, o registro de imagens e os dois caminhos de identidade de máquina.

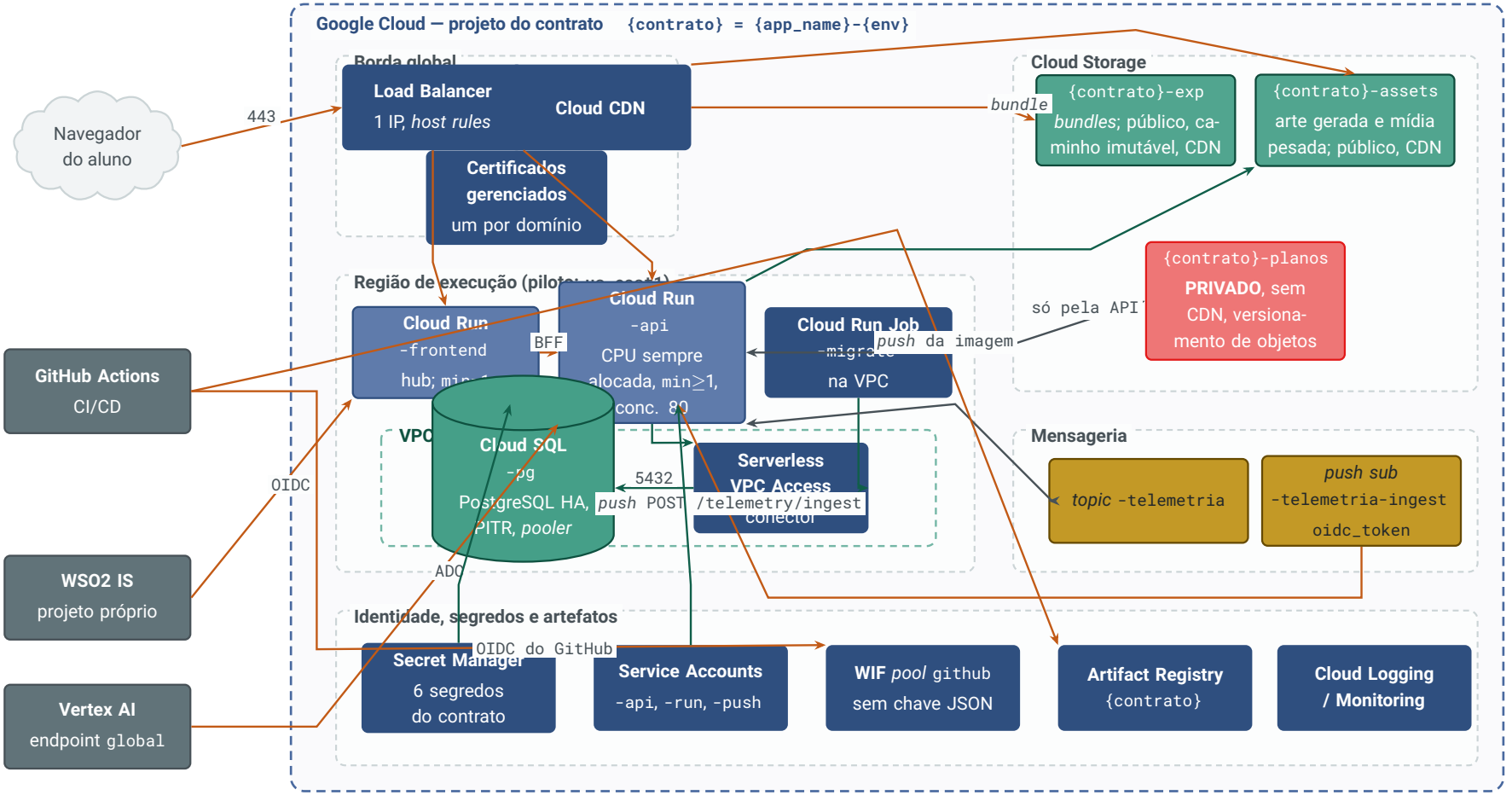


Figura 6.2 – Implantação detalhada. Os três *buckets* têm regimes de acesso deliberadamente diferentes: o de *bundles* e o de *assets* são públicos atrás da CDN (caminho versionado, portanto imutável); o de planos de aula é **privado**, sem CDN, com versionamento de objetos, e só é lido pela API depois de verificar papel e posse. O Cloud SQL não tem IP público: é isso que obriga a migração a rodar como *Job* dentro da VPC.

6.4 Catálogo de recursos por contrato

Este é o contrato formal com o repositório de Terraform. Nome divergente significa CI quebrado.

Recurso	Nome esperado	Consumido por
Cloud Run (serviço)	`{contrato}-frontend` (hub), `{contrato}-api`	`deploy.yml`
Cloud Run (*job*)	`{contrato}-migrate` — na VPC, roda `prisma migrate deploy`	`deploy.yml`
Artifact Registry	`{contrato}` (mesma região do Cloud Run)	`deploy.yml`
Bucket de experiências	`{contrato}-exp` + Cloud CDN	`deploy.yml`; hub (URL do `iframe`)
Bucket de *assets*	`{contrato}-assets` + Cloud CDN, leitura pública	API (`ASSETS_BUCKET`); mídia pesada das experiências
Bucket de planos	`{contrato}-planos` — **privado**, sem CDN, versionamento de objetos ligado	API (`PLANOS_BUCKET`)
Cloud SQL	`{contrato}-pg` — HA, PITR, *pooler* ativo	API (`DATABASE_URL`)
Pub/Sub	*topic* `{contrato}-telemetria` + *push sub* `{contrato}-telemetria-in` → `POST {api}/telemetry/ingest`	API
Secret Manager	`{contrato}-database-url`, `{contrato}-db-password`, `{contrato}-token-signing-key`, `{contrato}-auth-secret`, `{contrato}-perfil-import-token`, `{contrato}-planos-import-token`	*env* do Cloud Run
Service Accounts	`{contrato}-api` (Vertex, *publisher* do Pub/Sub, `objectAdmin` nos *buckets* de *assets* e de planos), `{contrato}-run` (hub), `{contrato}-push` (assina o *push* da telemetria)	Cloud Run, Pub/Sub

Recurso	Nome esperado	Consumido por
Load Balancer	Um só, global: hub em `{hub_domain}`, API em `{api_domain}`, *bundles* em `{exp_domain}`, *assets* em `{assets_domain}` — mesmo IP, *host rules*	DNS
WIF para GitHub Actions	*pool* `github`	`deploy.yml` (sem chave JSON)

DECISÃO DE PROJETO

Uma *role* de banco, não duas. O desenho original previa duas *roles*: uma dona do *schema* para as migrações e uma `app_runtime` sem *ownership* para a aplicação. Na prática ficou **uma**, porque as tabelas usam `FORCE ROW LEVEL SECURITY` — que sujeita o próprio dono às políticas e torna a separação desnecessária. O que *continua* obrigatório é a *role* não ter `BYPASSRLS` nem `SUPERUSER`; há SQL de verificação no *runbook* e um teste local (`tests/tenancy.spec.ts`) que cobre o mesmo contrato com uma *role* sem *ownership*.

6.5 Caminhos de tráfego

Caminho	Rede	Observação
Navegador → LB → hub	Pública, TLS gerenciado	Sessão do hub; SSR e BFF.
Navegador → LB → CDN → *bucket* de *bundles*	Pública, TLS	O grosso dos *bytes* do pico. Caminho versionado, portanto imutável; só o `index.html` tem `max-age=60`.
Navegador (dentro do `iframe`) → LB → API	Pública, TLS + *Bearer*	CORS por lista explícita (`CORS_ORIGINS`).
hub → API	Pública, TLS	`API_INTERNAL_URL`; o hub repassa o *token* do usuário.
API → Cloud SQL	**Privada**, VPC	O banco não tem IP público.
Cloud Run Job → Cloud SQL	**Privada**, VPC	Único caminho para rodar migração.
API → Vertex AI	Pública, TLS + ADC	Endpoint `global`.
API → Pub/Sub *topic*	Pública, TLS + SA	Publicação do lote analítico.
Pub/Sub → API (`/telemetry/ingest`)	Pública, TLS + *ID token*	Verificado por `PubSubPushGuard`: emissor Google, `aud` e `email` da SA de *push*.

hub → WSO2 IS	Pública, TLS	OIDC. Fora da *stack* do contrato.

ATENÇÃO

O *Load Balancer* exige **dois** certificados gerenciados quando o hub e a API estão em domínios distintos — um por domínio. Provisionar apenas o do hub faz a API responder com erro de TLS que *parece* problema de aplicação: o navegador não completa o *handshake* e a experiência mostra falha de rede sem nenhuma linha no log da API.

6.6 Identidade de carga de trabalho e segredos

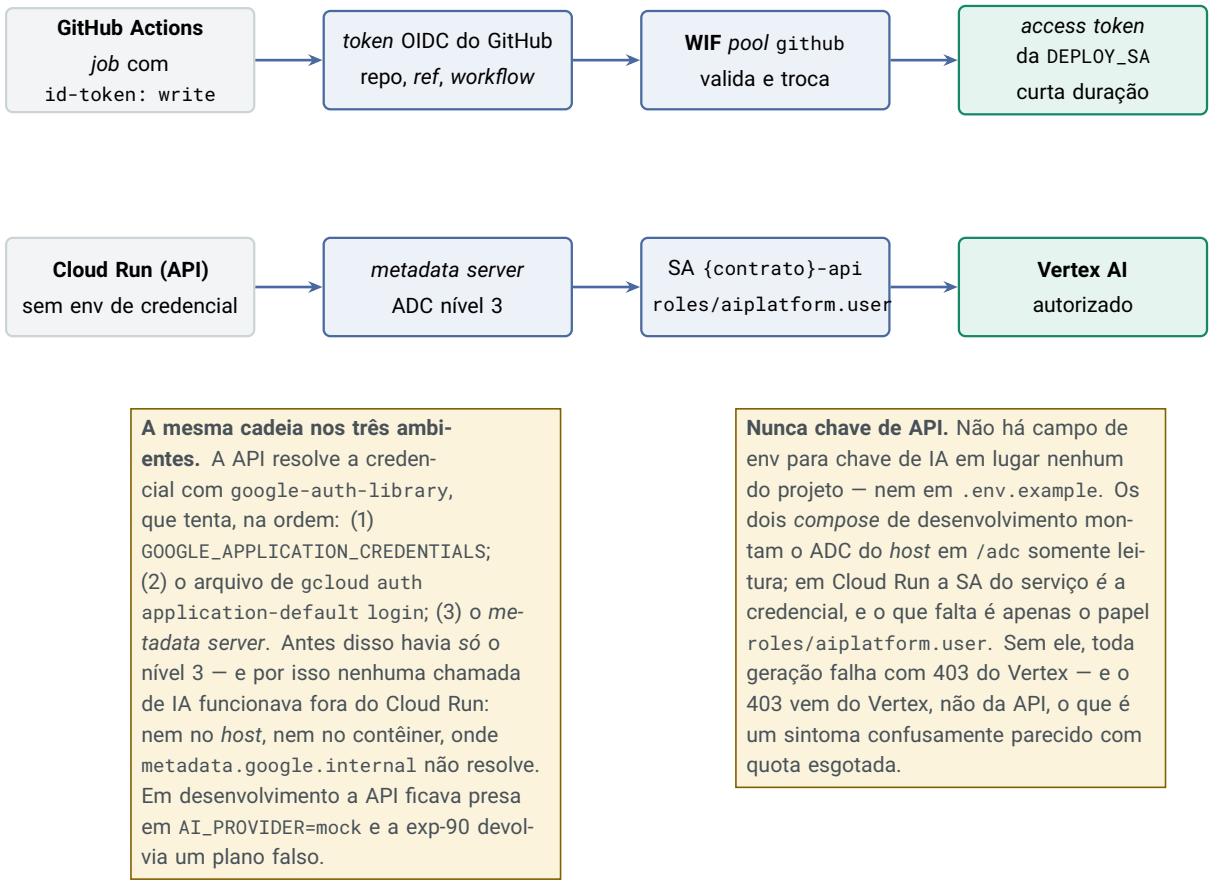


Figura 6.3 – Os dois caminhos de identidade de máquina, ambos sem chave de longa duração: WIF para o CI e ADC para a aplicação.

Segredo / variável	Onde vive	Por quê
`{contrato}-database-url`, `{contrato}-db-password`	Secret Manager	Credencial de banco.
`{contrato}-token-signing-k`	Secret Manager	Chave privada RS256 que assina o *token* de experiência. Também usada pelo CI para assinar o *token* de cinco minutos do *upsert* de manifesto.

`{contrato}-auth-secret`	Secret Manager	Segredo de sessão do NextAuth.
`{contrato}-perfil-import-` `{contrato}-planos-import-`	Secret Manager	Chaves de serviço de duas rotas de importação. **Ausentes = rota desligada** (503), nunca aberta.
`OIDC_ISSUER`, `OIDC_JWKS_URI`	*env* em texto plano	**Não são segredo**: são URLs públicas do hub. Ficam em texto plano derivadas de `hub_domain` — é isso que garante que o `iss` do *token* e o esperado pela API *não podem* divergir.
`TELEMETRIA_PUSH_SA`, `TELEMETRIA_PUSH_AUDIENCE`	*env* em texto plano	Identificam quem pode chamar `/telemetry/ingest`. Ausentes = rota 401.

6.7 Dimensionamento

Recurso	Configuração	Motivo
Cloud Run — API	CPU sempre alocada, `min-instances` ≥ 1, concorrência 80	Há trabalho *in-process* fora do ciclo de requisição: o *worker* de geração de conteúdo da exp-01 e a geração de plano de aula da exp-90. Com CPU liberada entre requisições, a geração morreria no meio.
Cloud Run — hub	`min-instances` ≥ 1	Elimina *cold start* no pico de *login*.
Cloud SQL	Instância pequena, HA, PITR, *pooler*	10.000 alunos e o volume medido de telemetria cabem folgadamente. O *pooler* é obrigatório porque o *autoscaling* do Cloud Run multiplica *pools* do Prisma e estoura `max_connections`.
CDN	*cache* longo no caminho versionado; `max-age=60` no `index.html`	O caminho carrega a versão, logo o conteúdo é imutável. O `index.html` é o ponteiro: *cache* longo nele atrasaria um *rollback*.
Pub/Sub	*Push subscription* com `oidc_token`	Amortecedor durável do pico, e uma peça a menos que um *worker* dedicado.

6.7.1 Volumes medidos

Grandeza	Valor medido	Como se chegou a ele
Payload de uma partida da exp-70	1,1 kB de JSON (1,5 kB em JSONB)	Medido em 2026-08-26. O jogo tem sete pontos de log, todos atos discretos com guarda própria: 8 Chamas, 4 aldeões, 1 Borda + 3 Destaques, 4 cartões, 1 primeira conversa — teto real de ~22 eventos por partida. Nada por quadro, nada por metro andado.

Armazenamento da exp-70 por contrato	**~30** MB	10.000 alunos jogando uma vez. A estimativa *anterior* — centenas de MB a 2,4 GB — partia do teto do contrato (2000 eventos) e errava por duas ordens de grandeza. O teto do contrato caiu de 2000 para 200, que é o que de fato limita a linha se um cliente adulterado mandar lixo.
Custo de gravação da telemetria	~16 `INSERT`/s no pico (era ~800)	Um lote de 50 eventos custava 50 *round-trips*; em `INSERT` multi-linha custa um. No pico de ~4000 simultâneos a diferença é entre saturar e não notar.
Prompt de um plano de aula anual	~190.000 *tokens*	Currículo da rede + as quatro unidades do ano. É o que justifica a quota separada de professor (2.000.000/dia ≈ 6 a 9 planos).
Mídia da exp-02	52 MB (21 arquivos MP4)	Fora do *git* e fora do *bundle*: vai para o *bucket* de *assets*, referenciada por `VITE_VIDEOS_BASE_URL`.

6.8 Ambientes

Ambiente	Como sobe	Diferenças relevantes
Host (desenvolvimento)	`docker compose up -d pg` + `pnpm dev`	Tudo no *host*; só o banco em contêiner. `AI_PROVIDER` pode ser `mock`; *drivers* de armazenamento locais; sem Pub/Sub.
Contêiner (desenvolvimento)	`pnpm docker:dev`	A *stack* inteira em contêiner — banco, API, hub e as cinco experiências — com o código do *host* por *bind mount*: salvar recompila lá dentro. Mesmas portas do `pnpm dev`, então é um *ou* o outro.
Imagens congeladas	`docker compose up -d --build`	Aproxima-se da produção: imagens construídas, migração e *seed* automáticos no primeiro `up`.
`dev` em GCP	CI no *push* para `main`	*Stack* real do piloto (`plataforma-unooria-dev`), com todos os recursos da Figura 6.2.
Produção	Mesmo CI, outras `vars` de repositório	O *workflow* não conhece o nome de nenhum recurso: tudo entra por `vars`, cujos valores saem de `terraform output`.

> **NOTA**
>
> Nos ambientes de contêiner, a leitura *server-side* do `experiencia.json` exige `EXP_INTERNAL_HOST` — que troca apenas o *host* e **preserva a porta**. `EXP_INTERNAL_URL`, que é um endereço fixo, só serve quando existe *uma* origem de *bundles* (a CDN, em produção): em desenvolvimento, cada experiência tem um *dev server* numa porta diferente, e um endereço fixo faria todas lerem o manifesto de uma só. O sintoma dessa configu-

ração errada é específico e memorável: o UnooTube, que não declara `viewport`, aparece dentro da casca de celular da Instanooria.

ATENÇÃO

Os *dev servers* das experiências declaram `allowedHosts: ["host.docker.internal"]`. O Vite 5.4+ recusa cabeçalho `Host` que não conheça, e sem essa declaração o hub em contêiner não lê o manifesto — e a experiência aparece esticada na janela em vez de dentro do aparelho.

6.9 O provedor de identidade fora da *stack*

O WSO2 Identity Server foge do padrão `{contrato}-*` por desenho: é **único para vários contratos** e tem ciclo de vida próprio. Vive em projeto GCP próprio e, em desenvolvimento, num *compose* separado (`wso2/docker-compose.yml`, IS em `:9443` e banco próprio em `:5433`).

DECISÃO DE PROJETO

O hub assina os *tokens* de experiência enquanto o IdP central não entra (**ADR-15**). O hub emite RS256 com TTL de 15 minutos e publica o JWKS em `/.well-known/jwks.json`; a API valida por `OIDC_ISSUER` e `OIDC_JWKS_URI`. Trocar para o WSO2 é trocar essas duas variáveis — o `OidcGuard` não muda uma linha. A alternativa (esperar o IdP para começar a integrar) faria toda a plataforma depender de um servidor Java em *bootstrap* para que qualquer experiência pudesse ser testada.

ATENÇÃO

O `issuer` do WSO2 é *tenant-qualified* (`.../t/{contrato}/oauth2/token`) e deve ser copiado **literalmente** do *discovery*, nunca montado por concatenação. Um `iss` montado à mão que difere por uma barra faz toda validação falhar com "*issuer* inválido" — e a mensagem não diz qual dos dois lados está errado.

6.10 Observabilidade, continuidade e reversão

Aspecto	Como é atendido hoje
Sanidade de processo	`GET /healthz` (processo vivo) e `GET /readyz` (dependências prontas). São rotas sem *guard*, de propósito.
Log	Cloud Logging pelo *stdout* do Cloud Run. **Nenhuma PII em texto claro**; texto livre de bem-estar nunca vai a log (só contagens).
Métricas	Métricas nativas do Cloud Run e do Cloud SQL. Não há métrica de aplicação customizada — é lacuna conhecida (Cap. 18).

Reversão do hub e da API	Revisões do Cloud Run. `gcloud run services update-traffic` devolve o tráfego à revisão anterior.
Reversão de experiência	`node apps/hub/scripts/publicar-manifesto.mjs <exp> --versao <anterior>`. Sem *build*, sem imagem.
Reversão de conteúdo de professor	Versionamento de objetos do *bucket* de planos: cada “Salvar” é uma versão, sem uma linha de código de histórico.
Recuperação do banco	HA + PITR do Cloud SQL.
Migração de *schema*	Cloud Run Job dentro da VPC, executado **antes** do *deploy* do serviço, com `--wait` para que a falha interrompa o *pipeline*.

LIMITAÇÃO CONHECIDA

Não há *alerting* configurado, nem painel. Há log e métrica nativa, e há sanidade por `/readyz` — o que basta para diagnosticar, mas não para ser avisado. Configurar alerta de taxa de erro 5xx e de saturação de conexões do Cloud SQL é item obrigatório antes de novembro de 2026, e está registrado como risco `R-05`.

7

Multi-tenancy e segurança

Este é o capítulo que, se estiver errado, invalida o produto. Uma plataforma que serve cem escolas de uma rede não pode, sob nenhuma circunstância — inclusive sob erro de programação — devolver o dado de uma escola a outra.

7.1 Os três níveis de isolamento

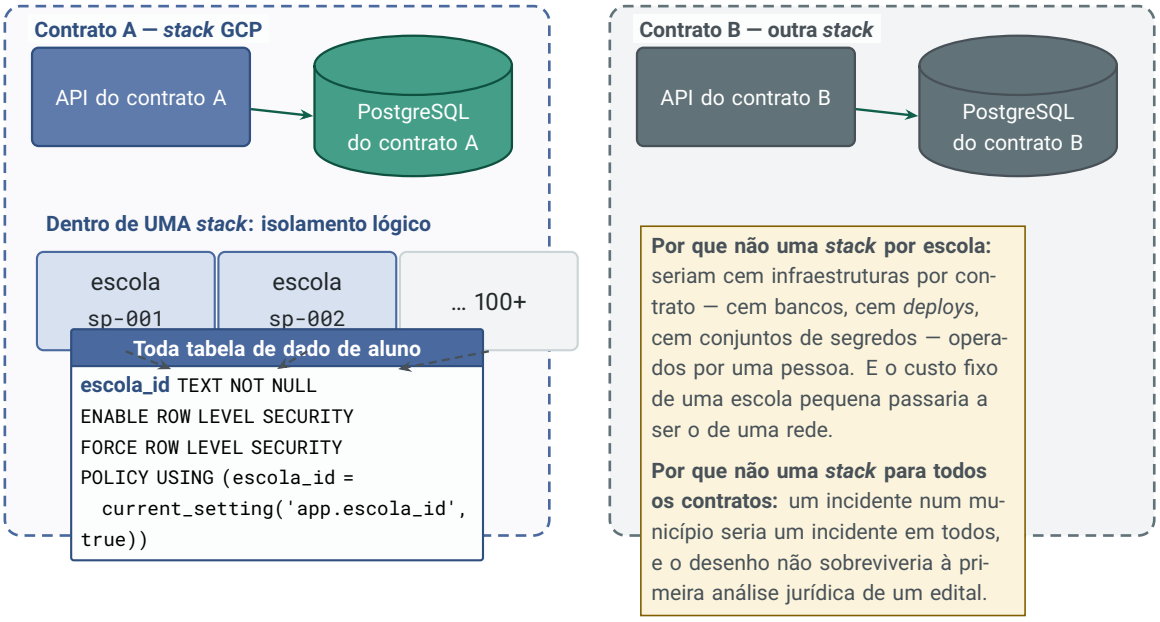


Figura 7.1 – Isolamento em três níveis: físico entre contratos, lógico entre escolas de um contrato, e por autorização entre turmas e alunos de uma escola.

7.2 Nível 2: *Row-Level Security* no PostgreSQL

7.2.1 Como a política é armada

Toda tabela de dado de aluno tem `escola_id TEXT NOT NULL` desnormalizado e uma política de RLS (The PostgreSQL Global Development Group, 2026a). A migração inicial aplica isso em laço sobre a lista de tabelas:

Listing 7.1 – Aplicação da RLS na migração inicial

```
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY[ /* ... tabelas de dado de aluno ... */ ] LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
    EXECUTE format(
      'CREATE POLICY %I ON %I USING (escola_id = current_setting(''app.escola_id'', true))',
      t || '_por_escola', t
    );
  END LOOP;
END $$;
```

> **DECISÃO DE PROJETO**
>
> **`FORCE ROW LEVEL SECURITY`, não apenas `ENABLE`.** `ENABLE` isenta o *dono* da tabela das políticas. Com `FORCE`, nem o dono escapa — e é isso que dispensou a segunda *role* de banco que o desenho original previa. O que continua obrigatório é a *role* da aplicação não ter `BYPASSRLS` nem `SUPERUSER`, porque esses atributos derrotam até o `FORCE`.

7.2.2 A porta única: `comTenancy`

Listing 7.2 – A única porta de acesso ao banco (`src/prisma/prisma.service.ts`)

```
async comTenancy<T>(
  fn: (tx: Prisma.TransactionClient) => Promise<T>,
  escolaId?: string,
  opts?: { timeout?: number },
): Promise<T> {
  const escola = escolaId ?? escolaDoContexto();
  if (!escola) {
    // Falha ALTA. A alternativa — consultar sem filtro — e exatamente o
    // acidente que este metodo existe para tornar impossivel.
    throw new Error(
      "comTenancy sem escola: rota sem OidcGuard/TenancyInterceptor ou " +
      "chamada fora de requisicao sem escolaId explicito",
    );
  }
  return this.$transaction(async (tx) => {
    // `true` = local a TRANSACAO. Um SET de sessao vazaria a escola para a
    // proxima requisicao que reusasse a conexao do pool.
    await tx.$executeRaw`SELECT set_config('app.escola_id', ${escola}, true)`;
    return fn(tx);
  }, opts);
}
```

Três detalhes desse método carregam a segurança inteira:

1. **O terceiro argumento de `set_config` é `true`**, ou seja, o valor é local à transação. Um `SET` de sessão sobreviveria ao `COMMIT` e a conexão voltaria ao *pool contaminada* — a próxima requisição, de outra escola, herdaria a política errada. Esse é o bug clássico de RLS com *pool* de conexões, e é evitado por um booleano.
2. **Sem escola, lança.** Não consulta "sem filtro", não devolve lista vazia: lança. Rota nova que esqueceu o `OidcGuard` quebra no primeiro teste, em vez de funcionar devolvendo tudo.
3. **A escola vem do contexto, não do parâmetro.** O parâmetro explícito existe só para código fora de requisição — *seed*, *worker*, testes.

7.2.3 Como a escola chega ao contexto

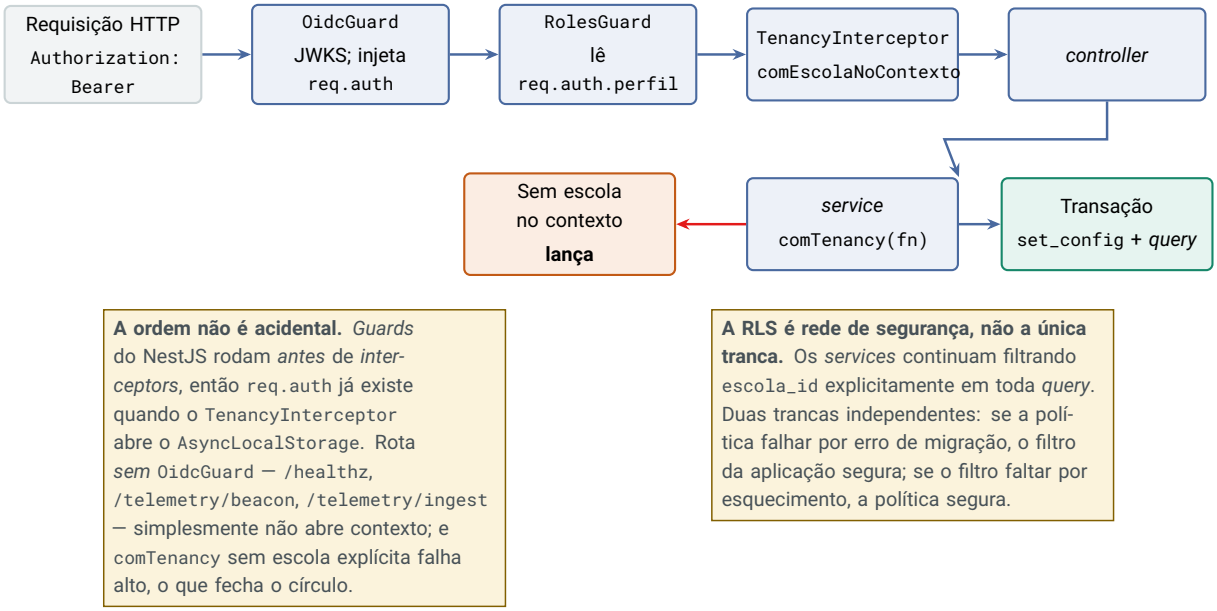


Figura 7.2 – Como a escola do *token* chega à política de RLS. Duas trancas independentes protegem o mesmo invariante.

7.2.4 A exceção: `admin-contrato`

REGRA INVIOLÁVEL

`admin-contrato` é o **único** papel autorizado a atravessar escolas, e só em rotas explicitamente marcadas com `@Roles("admin-contrato")`. Para esse papel, `escolaId` nas *claims* pode ser vazio — e o *CHECK* da migração permite `escola_id NULL` em `users` apenas quando `role = 'admin-contrato'`. Nenhuma outra combinação é aceita pelo banco.

As rotas cross-escola vivem em *controllers* separados (`src/analise/analise.controller.ts`, o *controller* de `/analise/*`) e não num `@Roles` misto com exceção por método — porque um

`@Roles` misto é o caminho mais curto para um `GET` novo herdar silenciosamente a permissão do vizinho.

7.3 Nível 3: autorização por papel

Papel	Escopo de dados	Pode
`aluno`	Os próprios dados, na própria escola	Realizar experiência, salvar progresso, ver o próprio resultado. **Nunca** vê gabarito, nem dado de colega.
`professor`	A própria escola; turmas dela	Preparar e conduzir atividade, ler turma e aluno individual, gerar plano de aula, fazer o próprio *check-in*. **Não** vê bem-estar de outro professor.
`gestor`	A própria escola	Importar usuários, redefinir senha, ler agregado da escola. **Não** vê a linha de uma criança nos instrumentos de valores, nem qualquer dado de bem-estar.
`admin-contrato`	Toda a rede	Ler agregado da rede, publicar manifesto. **Não** tem rota para a linha individual de uma criança.

> **DECISÃO DE PROJETO**
>
> **Agregado e linha individual são *controllers* diferentes.** Em `src/vila/` há `VilaProfessorController` com dois `@Controller` distintos: `/vila/agregado/*` abre para `professor` e `gestor`; `/vila/professor/*`, que devolve a linha de uma criança, só para `professor`. A separação física é o que garante que a próxima rota adicionada ao arquivo herde a permissão *certa*.

7.4 Segurança do *embed*

A fronteira mais exposta da plataforma não é a API: é o `iframe`. Dentro dele roda código que, em produção, vem de outra *origin* (a CDN) e que recebe um *token* do usuário.

Medida	Implementação e por quê
`targetOrigin` explícito	Nunca `"*"`, nos dois sentidos. O hub usa a *origin* derivada da *mesma* URL do `iframe`; a SDK usa o `hubOrigin` recebido por *query string*. `"*"` entregaria o *token* a qualquer documento que conseguisse virar o pai do *frame* (Mozilla Developer Network, 2026b).
Validação de `ev.origin`	Os dois lados descartam mensagem cuja *origin* não bate, *antes* de olhar o conteúdo.

Validação por esquema	Toda mensagem pós-*init* passa por Zod nos dois lados; o que não casa é descartado com aviso, **nunca** despachado cru aos *listeners*.
Verificação de versão maior	Hub e experiência comparam `protocolVersion`. Incompatível falha alto: sem `hub:init`, o *handshake* expira e a experiência mostra erro.
Token só em memória	A experiência nunca escreve o *token* em `localStorage` nem em *cookie*. TTL de 15 minutos, renovado por mensagem.
Sandbox mínimo	`allow-scripts allow-same-origin allow-modals`, mais `allow-pointer-lock` **apenas** para quem declara `pointer-lock` no manifesto (Mozilla Developer Network, 2026a).
`allow` do *frame*	`autoplay; fullscreen; clipboard-write`. A *Permissions-Policy* tem *default* `'self'`: sem delegar `autoplay`, um `<video muted autoplay>` cross-origin rejeita `play()` em silêncio.

NOTA

Cada token do *sandbox* está lá por um motivo medido: `allow-same-origin` porque, sem ele, a *origin* do `iframe` cross-origin vira `"null"` e a validação de *origin* quebra dos dois lados; `allow-modals` porque, sem ele, o navegador *ignora* `confirm()` do `iframe` silenciosamente (devolve `false`), e a exp-90 confirma exclusão assim; `allow-pointer-lock` porque, num `iframe` com *sandbox*, `requestPointerLock()` morre com `SecurityError` sem ele — verificado em `scripts/probe-pointerlock.mjs`, que precisa do Chrome do sistema porque o Chromium do Playwright não concede *pointer lock* nem no topo da página.

DECISÃO DE PROJETO

Permissão de `iframe` é por experiência, não *default* do *frame*. Prender o ponteiro do mouse de um aluno é capacidade de quem a declarou — e concedê-la a todas as experiências significaria que qualquer *bundle* publicado pode capturar o mouse. Já `fullscreen` *não* entra no manifesto: o `allow` do *frame* já o delega a todas, e campo de manifesto que não muda nada é pior que campo nenhum, porque quem lê acredita ter declarado algo.

7.5 Segredos e o que nunca vai ao cliente

REGRA INVIOLÁVEL

Nenhum segredo em *bundle* de cliente. Sem chave de API, sem *prompt* de sistema, sem URL interna. Consequências concretas:

- Chamada de IA só existe como `POST /ai/gerar` na API. O *prompt* de sistema mora em `apps/api/src/ai/prompts.ts` e nunca é devolvido.
- O gabarito da exp-02 (`factual`) não é persistido no estado do aluno nem devolvido em nenhuma rota de aluno.

- Os textos dos cartões da exp-70 chegam ao jogo **sem gabarito**, e apenas os da bolha daquela criança.
- `lado`, `perfil` e `intensidade` dos posts da exp-01 são removidos pelo sanitizador antes de o *feed* sair da API — são a variável independente do experimento.

7.6 Privacidade e dados de menores

Prática	Como é garantida
Telemetria sem PII	O evento carrega `aluno_id` opaco (UUID), nunca nome ou *e-mail*. A identidade é derivada do *token* no servidor, não do corpo enviado pelo cliente — o que é, ao mesmo tempo, anti-*spoof* e garantia de que o cliente não *pode* enviar PII no lugar do id.
Sem canal de texto arbitrário	Em `exp_70_sessoes.payload`, o campo `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**. O campo `texto` livre que existia era o único canal de texto arbitrário do *payload* — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração. É trava estrutural, não lista de proibidos.
Texto sensível fora de log	O texto livre do *check-in* de bem-estar nunca vai a log nem à telemetria; só contagens.
IA sem contexto identificável	O *prompt* de bem-estar recebe apenas dia da semana, mês, número do *check-in* e o texto livre *delimitado como conteúdo*. Nunca nome, escola, pontuação ou faixa.
Ranking sem menor real	O ranking da exp-02 usa colegas **fictícios**, com distribuição desenhada para a lição. Não há pontuação de criança real exposta a outra criança.
Dado de saúde com posse	`exp_91_ciclos` e `exp_91_checkins` têm RLS *e* filtro por `professor_id` em toda *query*, e não existe rota de gestor nem agregado. O próprio titular apaga tudo.

7.6.1 Retenção: uma decisão explícita de não declarar prazo

DECISÃO DE PROJETO

Não há prazo de retenção declarado para `exp_70_sessoes` nem para `telemetria_eventos` (decisão de 2026-08-26, **ADR-13**).
Havia uma coluna `expires_at` de 90 dias no *schema* e *nenhuma rotina que a cumprisse*. Coluna de prazo que ninguém lê é pior que prazo nenhum, porque quem lê o *schema* acredita nela — é a mesma armadilha de um campo `consentimento_em` que existia no aplicativo de origem: a regra saiu do código, a coluna ficou, e todo mundo continuou acreditando. A coluna foi removida na migração `20260826170000_vila_sem_prazo`.

O que se ganha em troca é o **reprocessamento**, que é requisito real deste instrumento: `payload` é a única fonte para recomputar o perfil quando a versão do mapeamento subir, e o mapeamento é hipótese de *design* que vai mudar. 90 dias matavam exatamente a comparação entre duas gerações do mapeamento.
Se um contrato exigir prazo, ele volta **junto com a rotina que o cumpre** — nunca só a coluna. Com a tabela particionada por mês, expurgo passou a ser `DROP TABLE particoes.<mês>` em vez de um `DELETE` que trava a tabela.

LIMITAÇÃO CONHECIDA

A pergunta "qual é o prazo de retenção do dado comportamental de menor" está **aberta** e é decisão de produto e de contrato, não de módulo: ela vale para a plataforma inteira, porque `telemetria_eventos` guarda evento por `aluno_id` de *todas* as experiências. Enquanto não for respondida, o que existe é esta seção — que é melhor que uma coluna mentindo.

7.6.2 Localidade de processamento

ATENÇÃO

O modelo de IA em uso (`gemini-3.6-flash`) só é publicado no endpoint `global`: medido no projeto do contrato, ele responde 404 em `southamerica-east1`, `us-east1` e `us-central1`. A consequência é explícita e foi aceita: **o tráfego de IA sai de São Paulo, inclusive o do aluno**, e isso precisa constar do contrato e do DPA da rede. Voltar à região é possível a qualquer momento, mas exige trocar *também* o modelo pelo que a região publica (lá, a família 2.5) — as duas variáveis mudam juntas ou a geração para de funcionar.

7.7 Verificação

Segurança não documentada por teste é intenção. O que é efetivamente verificado:

Teste	O que afirma
`tests/tenancy.spec.ts`	Com uma *role* sem *ownership* e a política ativa, uma *query* da escola A não retorna linha da escola B — nem quando o filtro da aplicação é omitido.
`tests/planos-tenancy.s`	Um professor não lê nem edita plano de outro professor, mesmo na mesma escola (RLS + posse).
`tests/telemetria-push.spec.ts`	`POST /telemetry/ingest` rejeita *token* sem a `aud` correta, sem a SA correta, e **rejeita tudo** quando as variáveis estão ausentes.
`packages/sdk/src/bridg`	Mensagem de *origin* errada é ignorada; mensagem que não valida é descartada; versão maior divergente rejeita a conexão.

`tests/motor-concorrencia.spec`	Dois pedidos simultâneos de bloco não descartam um bloco (*lock* consultivo).
`tests/vila-regras.spec`	*Payload* que traga chave de perfil é recusado; bolha B não é servida nos anos iniciais; `zero` e `none` não se confundem.
`scripts/probe-pointerlock.js`	Sem o token de *sandbox*, `requestPointerLock()` falha; com ele, funciona.

7.8 Ameaças consideradas

Análise no espírito do OWASP Top 10 (OWASP Foundation, 2021), restrita ao que é específico desta plataforma.

Ameaça	Categoria	Mitigação
Aluno lê dado de outra escola	*Broken access control*	RLS forçada + filtro explícito + *claims* do *token*. Duas trancas independentes, uma delas no banco.
Aluno lê o gabarito	Exposição de dado sensível ao negócio	Correção *server-side*; gabarito não persistido no estado do aluno nem devolvido; sanitizador remove metadados de curadoria.
Cliente adulterado envia perfil pronto	Integridade de medida	Servidor *recalcula* e recusa *payload* que traga chave de perfil; cap de eventos por partida.
Injeção de telemetria falsa	Autenticação ausente em *endpoint* de serviço	`PubSubPushGuard` verifica emissor, audiência e SA; *fail closed* sem configuração.
Roubo de *token* do `iframe`	Exposição de credencial	TTL de 15 min, escopo mínimo, só em memória, `targetOrigin` explícito, HTTPS. E a correção é *server-side* de qualquer forma, então o *token* não compra nota.
Esgotamento de quota de IA por abuso	Consumo de recurso	Quota diária persistida por (escola, usuário, dia), com incremento atômico, verificada *antes* da chamada; *rate limit* por usuário.
Vazamento de PII por log	Privacidade	Proibição explícita; telemetria com id opaco; vocabulário fechado no *payload*; texto sensível nunca logado.
Chave de serviço vazada	Gestão de segredo	Não existe chave de longa duração: WIF no CI, ADC na aplicação.
Rota nova esquecendo a *tenancy*	Erro de programação	`comTenancy` *lança* sem escola; não há caminho alternativo ao banco.

LIMITAÇÃO CONHECIDA

Não foi feita revisão de segurança externa nem teste de intrusão. Para uma plataforma que trata dado de menor numa rede pública, isso é item de contratação — e está registrado como risco `R-06` (Capítulo 18).

8

Modelo de dados

Um banco, um *schema* lógico, 27 tabelas em `public` e um *schema* auxiliar (`particoes`) fora do alcance do ORM. Este capítulo explica a convenção de nomes, o modelo por área e as decisões de esquema que não são óbvias — que, neste projeto, são a maioria.

O dicionário de dados campo a campo está no Apêndice E.

8.1 A convenção de nomes

REGRA INVIOLÁVEL

Tabela de experiência leva o número dela: `exp_NN_`. Assim: `exp_70_sessoes`, `exp_02_sessoes`, `exp_91_checkins`.
Tabela da plataforma — que serve a todas — fica sem prefixo: `escolas`, `turmas`, `users`, `experiencias`, `progresso`, `telemetria_eventos`, `ia_uso_diario`.
Tabela que é contrato entre duas experiências carrega os dois números: `exp_01_exp_70_perfis_importados` — a exp-70 escreve, a exp-01 lê. Chamá-la de uma só mentiria sobre quem depende dela.

DECISÃO DE PROJETO

Um banco, um *schema* — nem banco por experiência, nem *schema* por experiência.
Banco por experiência não funciona porque *as experiências não são isoladas*: o perfil de valores medido na Vila do Lume alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos. Além disso, um *pool* do Prisma por experiência estouraria `max_connections` com o *autoscaling* do Cloud Run.
Schema por experiência não funciona porque a única coisa que um *schema* daria além do prefixo é fronteira de `GRANT` — e existe **uma** *role* de aplicação. Seriam 48 *schemas* para obter exatamente o que um prefixo de nome já dá.

8.1.1 A migração de referência, e duas armadilhas registradas

A renomeação para o padrão `exp_NN_` está em `20260827090000_tabelas_por_experiencia`, e é a migração de referência para qualquer renomeação futura: `RENAME`, nunca `DROP`/`CREATE`, e renomeando *constraints*, índices e políticas de RLS junto.

ATENÇÃO

Duas armadilhas registradas naquela migração, ambas produzindo o mesmo sintoma — nomes duplicados como `exp_01_exp_01_..._pkey`:

1. `FOR ... IN SELECT` sobre o catálogo do sistema **relê o que o próprio laço alterou**. A lista de tabelas a renomear tem de ser materializada antes do laço.
2. `RENAME CONSTRAINT` renomeia **também** o índice que a sustenta. Renomear os dois em sequência aplica o prefixo duas vezes.

8.2 Modelo da plataforma

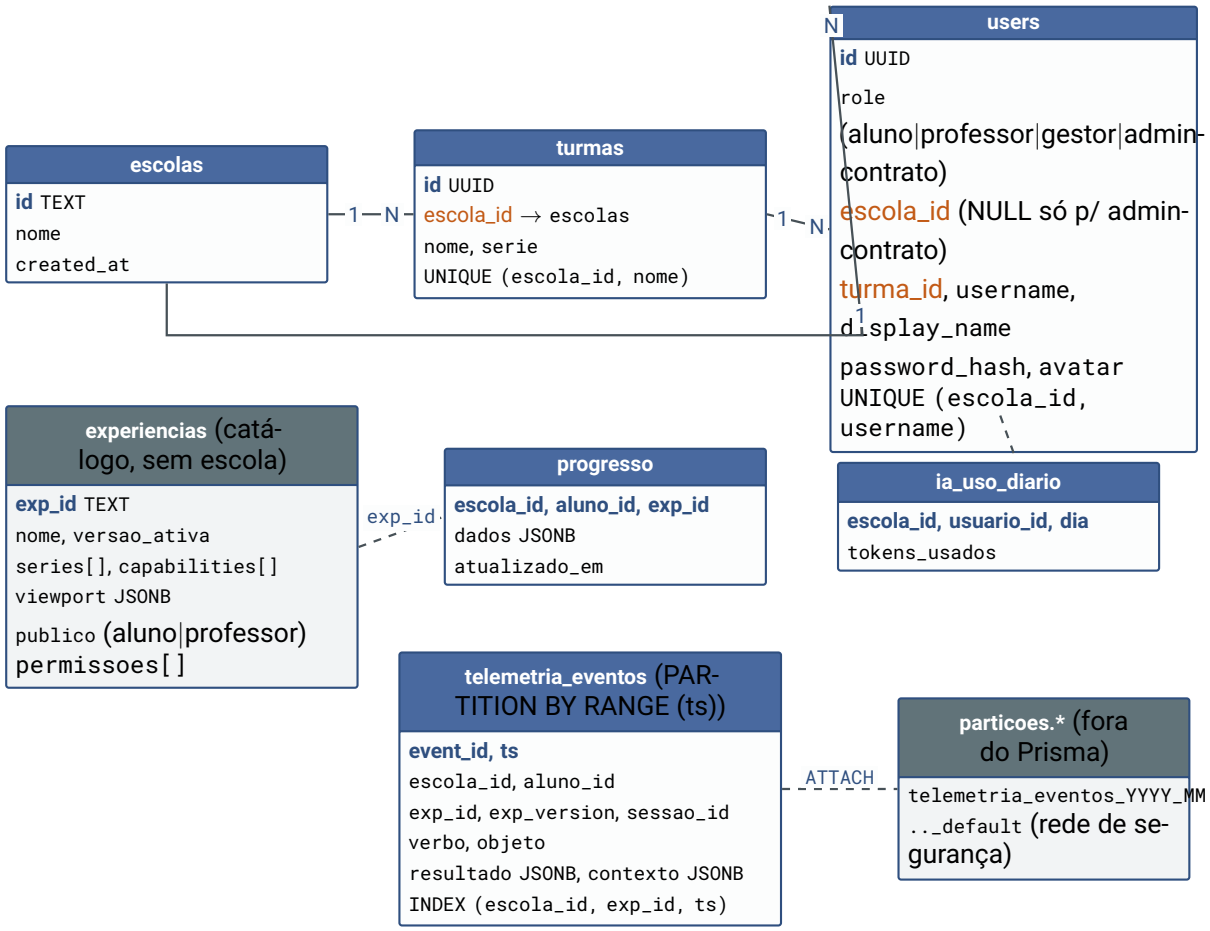


Figura 8.1 – Modelo de dados da plataforma. As caixas em azul são tabelas de *tenant* (têm `escola_id` e RLS); as cinzas são **catálogo do contrato** — `experiencias` e as partições de telemetria não pertencem a uma escola e ficam fora da RLS de propósito.

Tabela	Regime	Nota de esquema
`escolas`	*Tenant* raiz	O `id` é **texto**, e é o *mesmo* valor que circula no *claim* `escola_id` do JWT e no `set_config` da RLS. Um id legível (`sp-001`) vale mais que um UUID quando o valor aparece em três lugares diferentes de um diagnóstico.
`turmas`	*Tenant*	`UNIQUE (escola_id, nome)`: duas escolas podem ter "8A".
`users`	*Tenant*	`username` é único **por escola**, não globalmente — uma rede tem muitos `joao.silva`. *CHECK*: `escola_id NOT NULL` exceto quando `role = 'admin-contrato'`.
`experiencias`	Catálogo	Sem `escola_id`: o manifesto é do contrato, não da escola. Escrita só por `admin-contrato`.
`progresso`	*Tenant*	PK composta `(escola_id, aluno_id, exp_id)`. É o progresso *genérico* da SDK; estado rico de experiência vai para a tabela da experiência.
`ia_uso_diario`	*Tenant*	Contador de quota, uma linha por (escola, usuário, dia). Incremento atômico por `ON CONFLICT DO UPDATE ... RETURNING`.
`telemetria_eventos`	*Tenant*, particionada	Ver Seção 8.3.

8.3 Telemetria particionada

> **DECISÃO DE PROJETO**
>
> `telemetria_eventos` é **particionada por mês** (`PARTITION BY RANGE (ts)`), na migração `20260827100000_telemetria_particionada` (The PostgreSQL Global Development Group, 2026b). As partições vivem no *schema* `particoes`, **fora do alcance do Prisma** — em `public` cada partição viraria *drift* de *schema* a cada `prisma migrate dev`.

Três consequências do particionamento merecem registro:

1. **A chave primária é composta: `(event_id, ts)`.** O PostgreSQL exige a chave de partição em todo índice único. A *dedupe* do Pub/Sub segue correta, porque a reentrega da mesma mensagem traz o *mesmo* `ts` — então `ON CONFLICT (event_id, ts) DO NOTHING` continua sendo idempotência real, não aproximação.
2. **Há uma partição `DEFAULT`, como rede de segurança.** Sem ela, um evento com `ts` fora das faixas seria **recusado** — e telemetria perdida em silêncio é pior que telemetria mal guardada. As faixas declaradas vão até 2028-12.

3. **Expurgo passou a ser `DROP TABLE`.** Se um contrato exigir prazo de retenção, ele é cumprido por `DROP TABLE particoes.<mês>` — instantâneo — em vez de um `DELETE` que trava a tabela.

> **ATENÇÃO**
>
> A partição `DEFAULT` encher é **sinal de que faltou criar partição**, não funcionamento normal. Não há *job* agendado nesta plataforma — de propósito, porque seria a única peça de *scheduling* do sistema — e as faixas vão até 2028-12. Verificar a contagem da partição `DEFAULT` é item do *checklist* de operação (Apêndice F).

8.4 Modelo da exp-01 (Instanooria)

A exp-01 é a experiência mais rica em dados: um motor de *feed* adaptativo com geração de conteúdo por IA, sessão por aluno e dossiê de fechamento. São doze tabelas `exp_01_` mais a tabela de contrato com a exp-70.

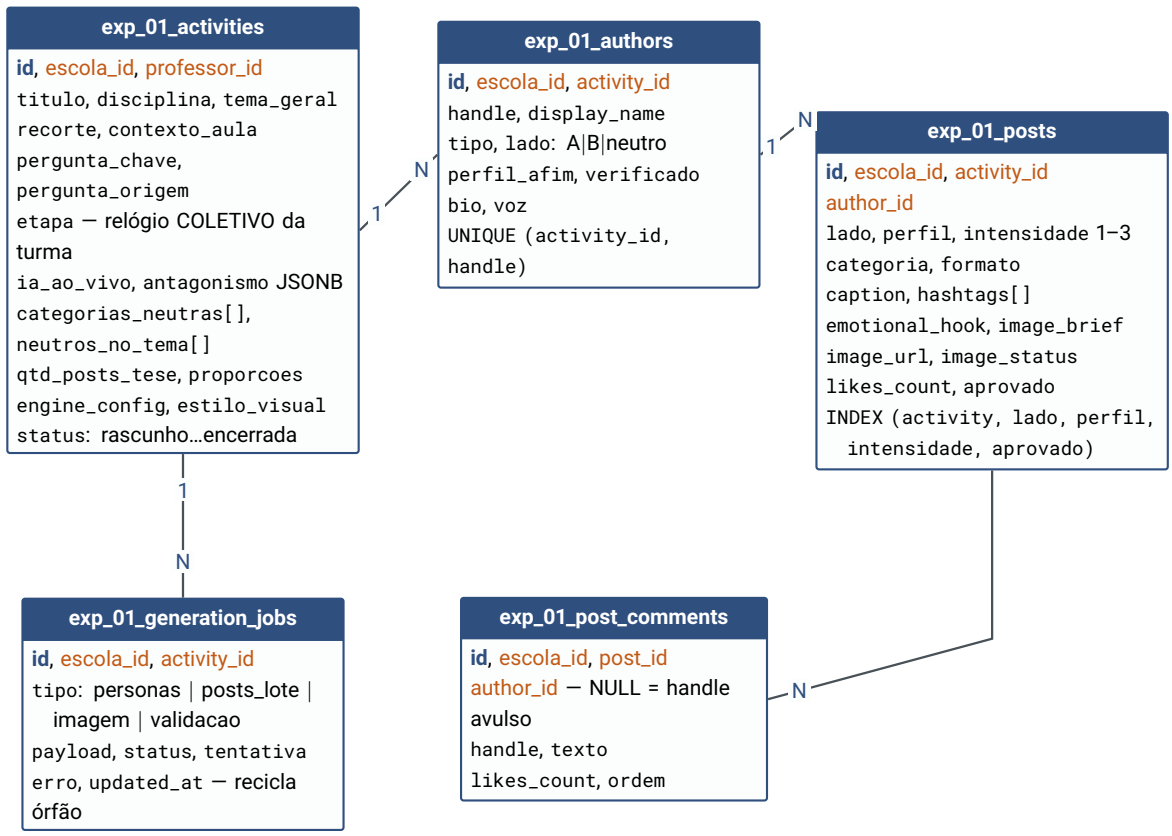


Figura 8.2 – Modelo de dados da exp-01, parte 1: a atividade preparada pelo professor e o conteúdo gerado. `lado`, `perfil` e `intensidade` em `exp_01_posts` são a variável independente do experimento — estão no banco, e o sanitizador da API os remove antes de o *feed* sair (Seção 5.8).

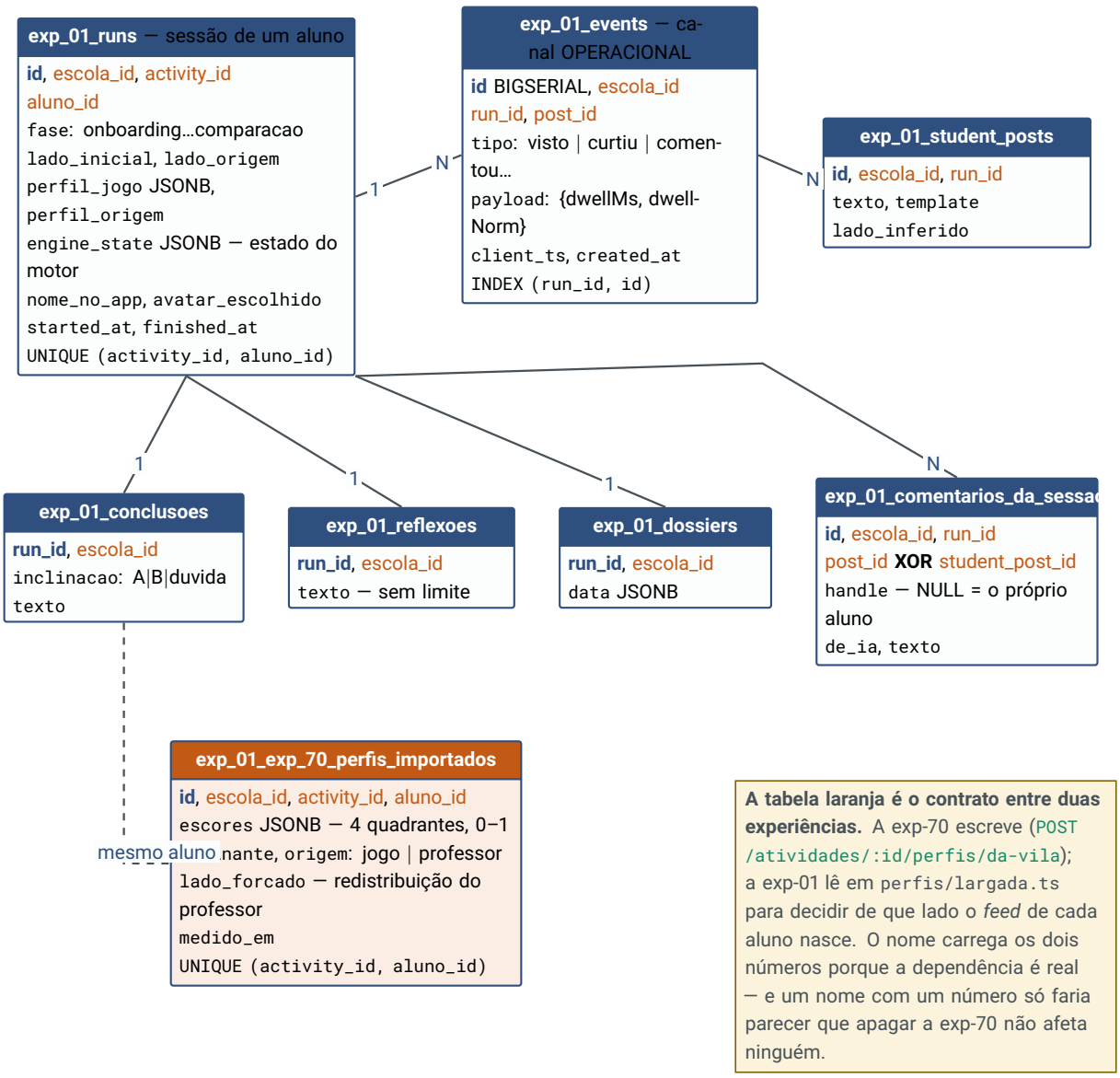


Figura 8.3 – Modelo de dados da exp-01, parte 2: a sessão do aluno. A `run` é o centro — carrega o estado do motor em JSONB e é a unidade de *lock* da curadoria. `exp_01_events` é o canal *operacional* de telemetria, síncrono e transacional, e não deve ser confundido com `telemetria_eventos`, o canal analítico (Capítulo 10).

Decisão de esquema	Por quê
`engine_state` em JSONB, não em tabela	O estado do motor é um agregado lido e escrito *inteiro*, sempre dentro do mesmo *lock*. Normalizá-lo daria *joins* sem nenhum ganho de consulta.
`nome_no_app` e `avatar_escolhido` na `run`, não em `users`	A conta é **do aparelho**: outra turma reusa o mesmo *login*. O nome que a criança digitou pertence àquela sessão.
`pergunta_origem`	Distingue pergunta gerada por IA de pergunta escrita ou editada pelo professor. É auditoria de autoria pedagógica, e custa uma coluna.

`lado_origem` na `run`	`perfil`, `sorteio` ou `redistribuido`: auditoria de *por que* aquele aluno caiu naquele lado. Sem isso, uma reclamação de professor não tem resposta.
`post_id` **XOR** `student_post_id` em `comentarios_da_sessao`	O comentário ancora num post do banco *ou* no post que o aluno publicou. Nunca nos dois — e o *CHECK* está na migração.
`de_ia`	Permite ao professor distinguir, na auditoria, o comentário escrito pela persona (modelo) do escrito pelo aluno.

8.5 Modelo das experiências 02, 70, 90 e 91

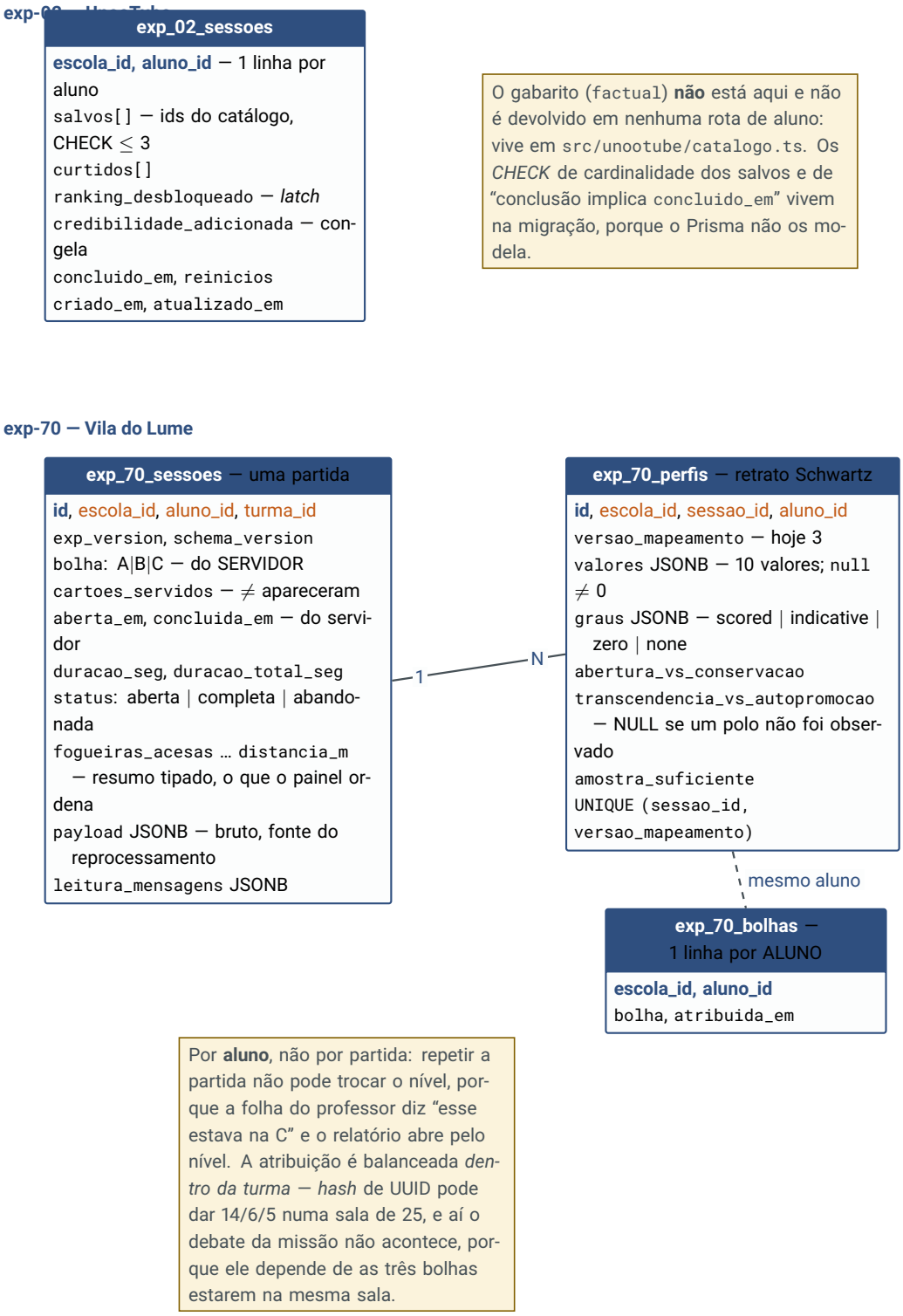


Figura 8.4 – Modelo de dados da exp-02 e da exp-70. Em `exp_70_perfis`, `null` **não é zero**: é valor que não recebeu número. `graus` diz por quê, e é essa distinção que impede um relatório de afirmar recusa onde houve ausência de oportunidade.

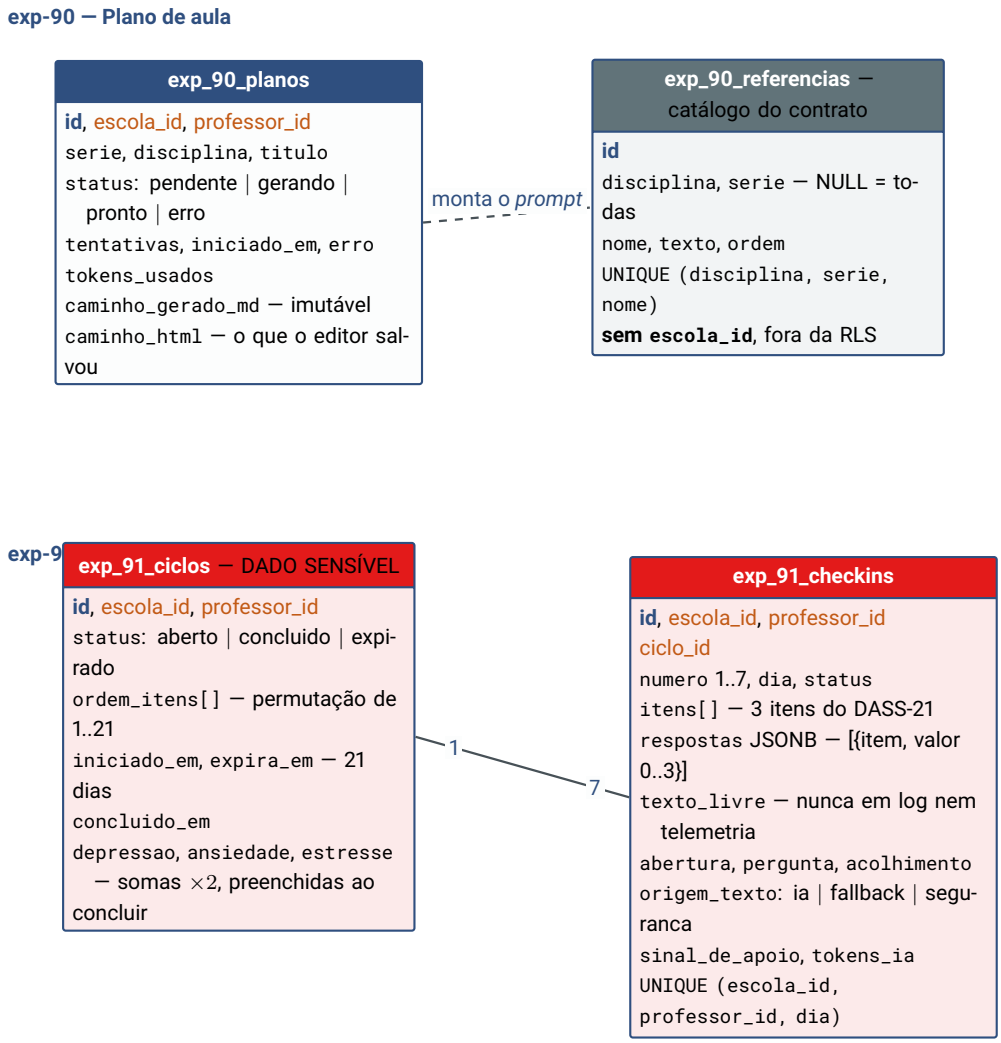


Figura 8.5 – Modelo de dados da exp-90 e da exp-91. As caixas vermelhas são dado *sensível* de saúde (LGPD art. 11): RLS *e* posse por `professor_id` em toda *query*, sem nenhuma rota de gestor ou agregado. `exp_90_referencias` é catálogo do contrato e fica **fora** da RLS, como `experiencias`.

8.6 Decisões de esquema por experiência

8.6.1 exp-70 — as decisões que custam informação de propósito

Decisão	Por quê
`turma_id` desnormalizado na sessão	O aluno troca de turma entre anos letivos, e o resultado precisa continuar *na turma em que foi medido*.
Datas cravadas pelo **servidor**	O aplicativo de origem derivava do relógio do cliente, e documentava que a hora numa máquina de escola costuma estar errada.
O `id` da sessão é emitido pela API	No aplicativo de origem o cliente escolhia a própria chave de idempotência, e duas crianças na mesma máquina colidiam — a segunda partida voltava "duplicada" e desaparecia.

`valores` com `null`, e `graus` separados	`null` **não é zero**: é valor que não recebeu número, porque tem uma fonte de medida só ou porque a oportunidade não chegou. `graus` diz *por quê*, e é essa distinção que impede um relatório de afirmar recusa onde houve ausência.
Eixos do circumplexo anuláveis	Eixo com polo não observado não é zero: é *indefinido*. Colocar zero produziria um ponto no meio do gráfico, que se lê como "equilibrado".
`amostra_suficiente` como coluna	Falso ⇒ o painel abre avisando que **não sabe**. Não é rigor estatístico: é impedir que uma partida sem escolha vire uma frase sobre quem a criança é.
Nova linha por `versao_mapeamento`	O mapeamento é hipótese de *design* e vai mudar. Guardar só o resultado tornaria todo o passado incomparável no dia da revisão. `UNIQUE (sessao_id, versao_mapeamento)`.
Sem tabela de evento fino	O bruto vive em `payload` (fonte do reprocessamento) e o canal analítico da SDK já grava em `telemetria_eventos`. Duas tabelas de log seriam duas histórias de retenção para o mesmo dado.

8.6.2 exp-90 — conteúdo fora do banco

> **DECISÃO DE PROJETO**
>
> O **conteúdo** do plano de aula não fica no PostgreSQL: fica em armazenamento de objetos *privado* (`planos/{escolaId}/{planoId}/gerado.md` imutável + `atual.html` do editor). A tabela guarda metadados, estado da geração e *caminhos*.
> O ganho não é performance: é que o versionamento de objetos do *bucket* dá histórico de cada "Salvar" **sem uma linha de código de histórico**. Um professor que sobrescreveu o plano por engano tem recuperação, e ninguém escreveu uma tabela de versões.

8.6.3 exp-91 — ausência como decisão de esquema

> **DECISÃO DE PROJETO**
>
> Não existe tabela de agregado de bem-estar, e não existe *view* por escola. Isso não é lacuna: é a decisão. Dado de saúde (LGPD art. 11) tem RLS e posse por `professor_id` em toda *query*; o titular apaga tudo com `DELETE /bemestar/dados`. Uma visão agregada e anonimizada para a rede é decisão *futura*, com DPA e *k*-anonimato — e não sai de um `GROUP BY`.

8.7 Migrações

Migração	Conteúdo

`20260824145301_init`	Esquema inicial: *tenancy* (`escolas`, `turmas`, `users`), tabelas da exp-01, tabelas de plataforma, e o laço que aplica `ENABLE`/`FORCE ROW LEVEL SECURITY` + política por escola.
`20260825011950_unootube_sessoe`	Estado *server-side* da exp-02, com *CHECK* de cardinalidade dos salvos.
`20260825120000_planos_de_aula`	Planos e referências curriculares.
`20260825225016_bem_estar`	Ciclos e *check-ins* do DASS-21, com RLS e `GRANT`.
`20260826120000_permissoes_de_if...`	Coluna `permissoes` no manifesto, com *CHECK* do vocabulário aceito.
`20260826140000_vila_do_lume`	Sessões, perfis e bolhas da exp-70.
`20260826170000_vila_sem_prazo`	Remoção da coluna `expires_at` (**ADR-13**).
`20260827090000_tabelas_por_exp`	Renomeação para o padrão `exp_NN_`: `RENAME` de tabelas, *constraints*, índices e políticas. É a migração de referência.
`20260827100000_telemetria_partic...`	Particionamento mensal de `telemetria_eventos` no *schema* `particoes`, com partição `DEFAULT` e faixas até 2028-12.

REGRA INVIOLÁVEL

Nenhuma migração remove `escola_id` ou afrouxa RLS. Renomeação usa `RENAME`; nada de `DROP`/`CREATE` em tabela com dado. Em produção, a migração roda como Cloud Run Job dentro da VPC **antes** do *deploy* do serviço, com `--wait` — porque o *schema* novo tem de estar no banco antes de a revisão nova receber tráfego.

ATENÇÃO

O que o Prisma **não** modela e por isso vive na migração: políticas de RLS, *CHECK constraints*, partições, `GRANT`. Ler apenas `schema.prisma` dá uma visão incompleta do banco — é por isso que os campos correspondentes são `TEXT`/`SMALLINT` no *schema* e a validação de aplicação é feita com Zod.

9

Contratos de integração

Há quatro fronteiras nesta plataforma cuja quebra é caríssima, porque atinge código que já está publicado e não pode ser atualizado em conjunto: o protocolo `postMessage` entre hub e experiência, o formato do evento de telemetria, o manifesto de experiências e a interface do *provider* de IA. Este capítulo especifica os quatro e a política de versionamento de cada um.

> **REGRA INVIOLÁVEL**
>
> **Mudança em contrato exige cuidado extremo e atualização deste documento.** Um *bundle* publicado há três meses no *bucket* de um contrato continua sendo servido a alunos: ele não é reconstruído quando a API muda. Toda alteração de contrato é, por definição, uma alteração *retrocompatível* ou um *bump* de versão maior com suporte à anterior.

9.1 Protocolo `postMessage` hub ↔ experiência

9.1.1 Especificação das mensagens (protocolo v1)

Sentido	Mensagem	Carga e semântica
exp → hub	`exp:ready`	`{ protocolVersion }`. Anúncio de que o *bundle* carregou. É **reenviado** a cada 400 ms até o hub responder.
hub → exp	`hub:init`	`{ protocolVersion, token, aluno{id,nome,serie,turmaId,escolaId}, apiUrl, config }`. Resposta ao `exp:ready`; só a primeira conta.
exp → hub	`exp:concluir`	`{ payload: { score?, dados? } }`. Conclusão da missão. O hub registra progresso.
exp → hub	`exp:token-expirando`	Sem carga. Pedido de renovação; o hub responde `hub:token`.
hub → exp	`hub:token`	`{ token }`. Novo *token*, sem recarregar o `iframe`.
exp → hub	`exp:estado`	`{ estado: {…} }`. Estado de alto nível para o *chrome* opcional do hub. **Aditivo** ao v1; o hub *não* interpreta domínio, só repassa a quem plugou um *callback*.

hub → exp	`hub:pause` / `hub:resume`	Sem carga. Emitidos por visibilidade e foco da página do hub.

9.1.2 O *handshake*, passo a passo

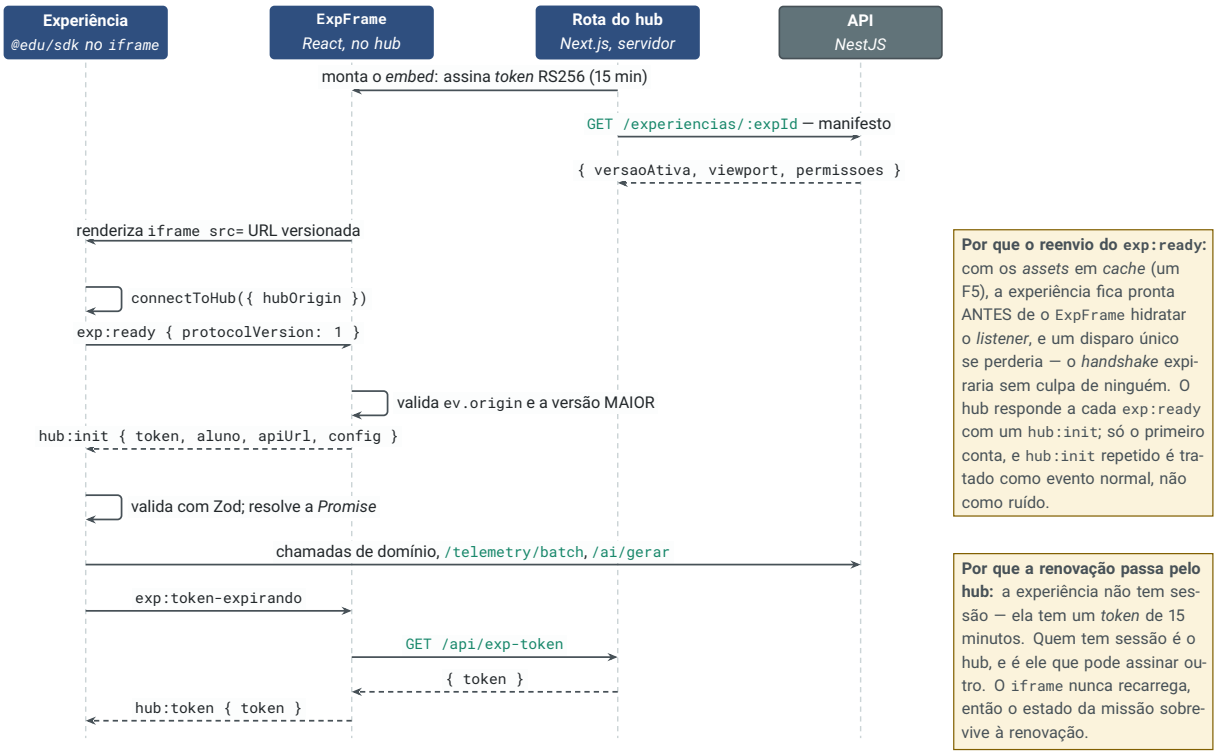


Figura 9.1 – Sequência do *handshake* e da renovação de *token*. O `targetOrigin` é explícito nos dois sentidos, e as duas pontas descartam mensagem de *origin* divergente antes de olhar o conteúdo.

9.1.3 Regras invioláveis do protocolo

REGRA INVIOLÁVEL

1. **`targetOrigin` explícito, nunca `"*"`.** O hub usa a *origin* derivada da *mesma* URL que colocou no `iframe`; a SDK usa o `hubOrigin` que recebeu por *query string*. Com `"*"`, o *token* iria para qualquer documento que conseguisse virar o pai do *frame*.
2. **Toda mensagem pós-*init* é validada com Zod nos DOIS lados.** Mensagem inválida é descartada com aviso, nunca despachada crua aos *listeners*.
3. **Versão MAIOR verificada nos dois lados; incompatível falha alto.** O hub não envia `hub:init` e registra erro no console; a experiência expira o *handshake* e mostra a própria mensagem de erro.
4. **O *token* vive só em memória** na experiência. Nunca `localStorage`, nunca *cookie*.
5. **Nunca quebrar compatibilidade sem *bump* de versão maior e suporte à anterior.**

Listing 9.1 – O núcleo da validação, em `packages/sdk/src/bridge.ts`

```
function handler(ev: MessageEvent) {
  if (ev.origin !== hubOrigin) return;           // regra 1: valida origin

  if (!conectado) {
    const init = HubInitSchema.safeParse(ev.data);
    if (!init.success) return;                   // antes do init, tudo mais e ruido
    // Versao MAIOR precisa bater — hub e experiencia de geracoes diferentes
    // falham ALTO, nao silenciosamente.
    if (Math.trunc(init.data.protocolVersion) !== PROTOCOL_VERSION) {
      reject(new Error(`Protocolo incompativel: hub v${init.data.protocolVersion}, ` +
                       `experiencia v${PROTOCOL_VERSION}`));
      return;
    }
    conectado = true;
    resolve({ init: init.data, send, onMessage });
    return;
  }

  // hub:init repetido (resposta a um exp:ready reenviado) nao e ruido nem erro.
  if ((ev.data as { type?: unknown })?.type === "hub:init") return;

  const msg = HubParaExpSchema.safeParse(ev.data);
  if (!msg.success) {
    console.warn("[sdk] mensagem do hub descartada (schema):", ev.data?.type);
    return;                                      // regra 2: descarta, nao despacha
  }
  listeners.forEach((cb) => cb(msg.data));
}
```

9.1.4 Política de evolução

Tipo de mudança	Versão	Regra
Nova mensagem opcional (como `exp:estado`)	v1, aditivo	Permitido sem *bump*: hub antigo ignora mensagem que não conhece, e experiência antiga nunca recebe a nova sem pedir.
Novo campo opcional em `hub:init`	v1, aditivo	Permitido. O esquema Zod tem `.default()` ou `.optional()`, e a experiência antiga simplesmente não o lê.
Campo existente muda de tipo ou semântica	**v2**	Exige *bump* de versão maior *e* que o hub continue respondendo v1 enquanto houver *bundle* v1 publicado em algum contrato.
Remoção de mensagem ou de campo	**v2**	Mesma regra. A remoção só é efetiva quando o último *bundle* v1 tiver sido republicado.

> **ATENÇÃO**
>
> `hub:pause` e `hub:resume` existem porque **esconder um `iframe` não dispara `visibilitychange` nele**. Sem essas mensagens, o cronômetro de *dwell* da experiência correria com o aluno de costas — e *dwell* é a medida central do motor de *feed*

da exp-01. O hub escuta `visibilitychange` e `blur`/`focus` da janela, porque trocar de janela ou de aplicativo não muda `visibilityState`.

9.2 Manifesto de experiências

O manifesto é o contrato entre o CI, a API e o hub. Ele existe em duas cópias que devem concordar: o arquivo `public/experiencia.json` dentro do *bundle*, e a linha da tabela `experiencias` na API — que o CI escreve lendo o arquivo.

Listing 9.2 – `apps/experiencias/exp-70-vila-do-lume/public/experiencia.json`

```
{
  "id": "exp-70-vila-do-lume",
  "nome": "Vila do Lume — A Noite Longa",
  "versao": "1.0.0",
  "series": [],
  "capabilities": ["telemetria"],
  "permissoes": ["pointer-lock"],
  "verbos": {
    "attempted":  "entrou na vila (partida aberta)",
    "interacted": "acendeu fogueira, gastou tocha, gravou na Pedra dos Nomes ou respondeu a uma mensagem",
    "progressed": "acendeu mais uma das 8 fogueiras",
    "completed":  "acendeu as 8 fogueiras e reacendeu o Fogo Sagrado"
  }
}
```

Campo	Tipo	Semântica
`id`	*string*	Slug da experiência; é a chave primária da tabela e o primeiro segmento do caminho no *bucket*.
`nome`	*string*	Nome de exibição.
`versao`	*semver*	Vem do `package.json` da experiência. É o segundo segmento do caminho no *bucket*, o que torna o *bundle* imutável.
`series`	*string[]*	Séries em que a experiência se aplica. **Vazio significa TODAS as séries**, não nenhuma — ver o alerta abaixo.
`capabilities`	*string[]*	`telemetria`, `progresso`, `ia`, `avaliacao`, `tempo-real`. Declara o que a experiência usa da plataforma.
`viewport`	`{largura, altura}?`	Presente ⇒ o hub embeda dentro do `DispositivoFrame` (casca de aparelho com escala). A exp-01 é 375×812; a exp-02 *não* tem `viewport`, porque foi desenhada para *desktop*.
`publico`	`aluno|professor`	`professor` ⇒ ferramenta do professor, e `/exp/*` responde **404** para ela.
`permissoes`	*string[]*	Permissões que o *iframe* daquela experiência precisa. Hoje só `pointer-lock`. Vazio (o *default*) = nenhuma.

`verbos`	*objeto*	Documentação dos verbos de telemetria que a experiência emite. Não é lido por código; é contrato de leitura para quem analisa os dados.

REGRA INVIOLÁVEL

`series: []` significa **todas as séries**, não nenhuma: é assim que uma experiência *transversal* se declara (é o caso da exp-70, que não pertence a ano nenhum). Quando o filtro por série voltar a valer no hub do aluno, vazio tem de continuar querendo dizer "todas" — o contrário faz a experiência desaparecer da tela sozinha, sem erro em lugar nenhum.

9.2.1 Como o hub resolve a URL do `iframe`

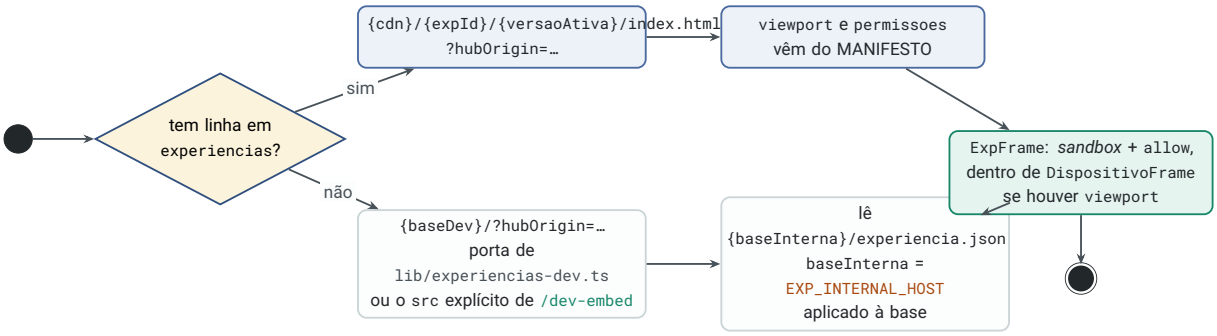


Figura 9.2 – Resolução da URL do *embed* e da casca de aparelho. Os dois caminhos convergem: mesmo sem manifesto, o hub obtém `viewport` e `permissoes` lendo o `experiencia.json` servido ao lado do `index.html` do *bundle*.

ATENÇÃO

A leitura do `experiencia.json` do *bundle* **não é só de desenvolvimento**. Antes era, e uma *stack* com a tabela `experiencias` vazia — *compose* local, ou ambiente novo antes do primeiro *publish* do CI — servia a exp-01 *esticada na janela* em vez de dentro do aparelho de 375×812. Pelo mesmo motivo, a exp-70 abriria sem `allow-pointer-lock`: o jogo monta, a câmera não anda, e nada no console diz por quê.

NOTA

Essa leitura é *server-side*, e `NEXT_PUBLIC_CDN_EXP_URL` é o endereço do **navegador** — de dentro de um contêiner, `localhost:5173` é o próprio contêiner. Daí as duas variáveis: `EXP_INTERNAL_HOST` troca só o *host* e **preserva a porta** (é o que serve em desenvolvimento, onde cada experiência tem um *dev server* numa porta diferente), e `EXP_INTERNAL_URL` é um endereço fixo, que só serve quando há *uma* origem de *bundles* — a CDN, em produção.

9.2.2 Publicação e reversão

Operação	Comando
Publicar (feito pelo CI)	`gcloud storage rsync -r {dir}/dist gs://{bucket}/{exp}/{ver}/` + `node apps/hub/scripts/publicar-manifesto.mjs {dir}`
Reverter	`node apps/hub/scripts/publicar-manifesto.mjs <exp> --versao x.y.z`

O *script* assina um *token* de cinco minutos com a chave de assinatura e chama `PUT /experiencias/:expId` como `admin-contrato`. A API valida esse *token* contra o JWKS que o *hub* publica — o que é a razão de o *workflow* publicar as experiências *depois* do hub.

9.3 Contrato de telemetria

O formato do evento é inspirado em xAPI (Advanced Distributed Learning Initiative, 2023) e simplificado ao que a plataforma efetivamente usa.

Listing 9.3 – Evento de telemetria (canal analítico)

```
{
  "eventId":    "uuid",          // gerado pela SDK; chave de dedupe
  "ts":         "ISO-8601",      // relogio do cliente
  "alunoId":    "...",           // SOBRESCRITO pelo servidor, a partir do token
  "escolaId":   "...",           // SOBRESCRITO pelo servidor
  "expId":      "exp-01",
  "expVersion": "1.2.0",
  "sessaoId":   "uuid",
  "verbo":      "completed | attempted | interacted | scored | progressed | ...",
  "objeto":     "fase-3",
  "resultado":  { },             // livre, por experiencia
  "contexto":   { }              // livre, por experiencia
}
```

Cláusula	Regra
Quem preenche o quê	A SDK preenche tudo **exceto** `verbo`, `objeto` e `resultado` — esses são a única coisa que a experiência informa.
Identidade	`alunoId` e `escolaId` enviados pelo cliente são **descartados**: o servidor os deriva do *token*. É anti-*spoof* e, ao mesmo tempo, garantia de que o cliente não *pode* colocar PII no lugar do id.
Lote	Até 50 eventos por requisição, *flush* a cada 10 s ou quando o *buffer* enche.
Fechamento de página	No `pagehide`, `POST /telemetry/beacon` com o *token* **no corpo** — `sendBeacon` não envia cabeçalho.
Idempotência	`ON CONFLICT (event_id, ts) DO NOTHING`. O Pub/Sub entrega *at least once*, então reentrega é rotina, não exceção.

Documentação dos verbos	Cada experiência declara seus `verbos` no manifesto e no próprio README. Verbo não documentado é dado que ninguém consegue interpretar seis meses depois.

O detalhamento dos dois canais — por que são dois, e o que acontece em cada um — é o Capítulo 10.

9.4 Contrato do *provider* de IA

Listing 9.4 – `apps/api/src/ai/provider.ts` — a interface neutra de *vendor*

```ts
export interface AiRequest {
  system: string;
  user: string;
  maxTokens?: number;
  /** Familia de modelo, NEUTRA de vendor. `padrao` e o modelo do dia a dia;
   *  `longo` e o de saida grande (planos anuais). Quem chama nao sabe qual e. */
  modelo?: "padrao" | "longo";
  temperatura?: number;
}

export interface AiResult {
  texto: string;
  tokensUsados: number;
  /** `STOP` (natural) ou outro motivo do vendor (`MAX_TOKENS`, `SAFETY`...).
   *  Aditivo: quem nao liga, ignora. Quem gera texto longo TEM de olhar. */
  finishReason?: string;
}

export interface AiProvider {
  generate(req: AiRequest): Promise<AiResult>;
  /** Saida JSON estruturada, com retry e reforco do erro anterior no prompt. */
  generateJson<T>(req: AiJsonRequest<T>): Promise<T>;
  /** Bytes originais (jpeg/png). Falha => quem chama cai no fallback procedural. */
  generateImage(prompt: string): Promise<Buffer>;
}

export const AI_PROVIDER = "AI_PROVIDER";   // token de injecao do Nest
```

REGRA INVIOLÁVEL

Nenhum módulo fora de `src/ai` conhece o *vendor*. Trocar de provedor é criar um arquivo novo (`src/ai/<vendor>.provider.ts`) e registrá-lo no módulo por `AI_PROVIDER`. Implementações atuais: `vertex.provider.ts` (*default*) e `mock.provider.ts` (testes e desenvolvimento sem GCP).

NOTA

`AiJsonSchema` é um subconjunto deliberadamente pequeno de JSON Schema (`object`, `array`, `string`, `number`, `boolean`, `enum`), e cada *provider* o traduz para o formato do seu modelo. O campo `validar` do `AiJsonRequest` recebe a saída bruta e *lança* quando ela

não serve — o que faz a tentativa ser regerada com o erro anterior reforçado no *prompt*. Validação como parte do contrato, não como verificação posterior.

9.5 Contrato entre duas experiências

Quando uma experiência precisa do resultado de outra, a comunicação nunca é direta — módulos de domínio não se conhecem (Seção 5.5). O contrato é uma tabela nomeada com os dois números e um *endpoint* de fronteira.

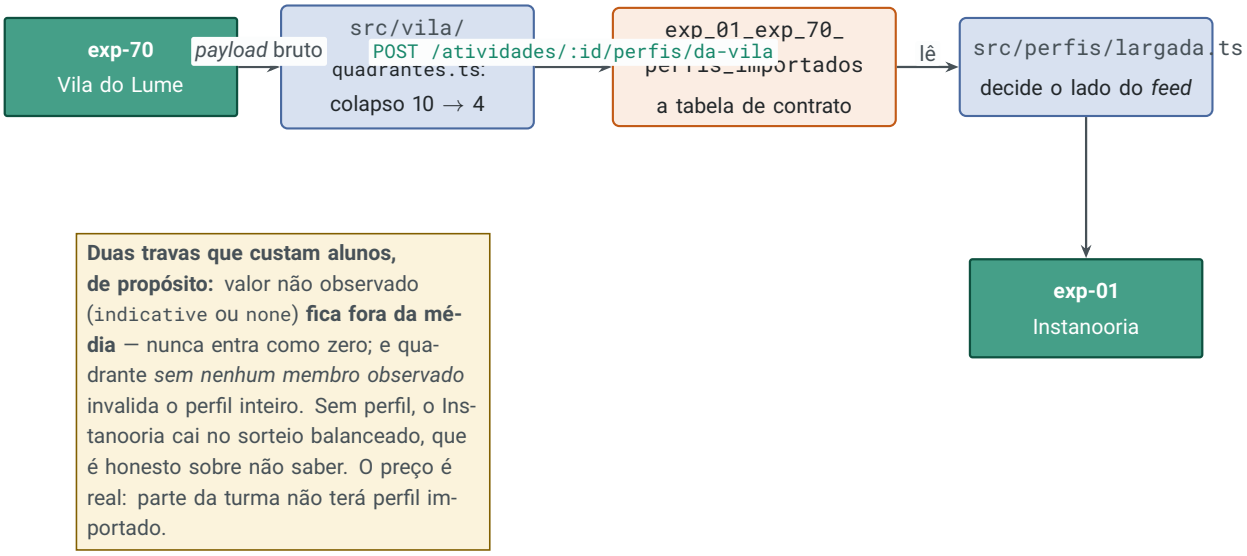


Figura 9.3 – O único acoplamento entre experiências, e como ele é explícito: uma tabela cujo nome carrega os dois números, escrita por um *endpoint* de fronteira acionado pelo professor.

REGRA INVIOLÁVEL

O disparo é do **professor**, e `lado_forcado` é **preservado** no *upsert*. Importar sozinho — ou levar o objeto inteiro no `update` — desfaria em silêncio uma redistribuição de turma que o professor fez antes da aula. `PERFIL_IMPORT_TOKEN` e `POST /perfis/importar` continuam vivos apenas como compatibilidade com a ponte externa original.

10

Telemetria

DECISÃO DE PROJETO

Dois canais de telemetria, não um. A plataforma tem dois fluxos de evento com garantias, latências e destinos diferentes, e confundi-los quebra um dos dois:

- o canal **operacional** (motor), *síncrono e transacional*, lido em tempo quase real para curar o próximo bloco de *feed*;
- o canal **analítico** (SDK), *assíncrono e em lote*, para análise posterior.

10.1 Por que dois canais

Aspecto	Operacional (motor)	Analítico (SDK)
Rota	`POST /runs/:id/eventos`, `GET /runs/:id/feed`	`POST /telemetry/batch`, `/beacon`, `/ingest`
Transporte	HTTP direto, dentro da transação	HTTP → Pub/Sub → *push* → HTTP
Latência	Milissegundos; a próxima tela depende dela	Segundos a minutos; ninguém espera
Ordenação	**Garantida** pela transação e pelo *lock* da `run`	Sem garantia
Destino	`exp_01_events`	`telemetria_eventos` (particionada)
Entrega	Exatamente uma vez (transacional)	*At least once*; *dedupe* por `(event_id, ts)`

Quem lê	O motor de curadoria, o painel do professor, o dossiê	Ninguém, hoje — é dado de análise futura

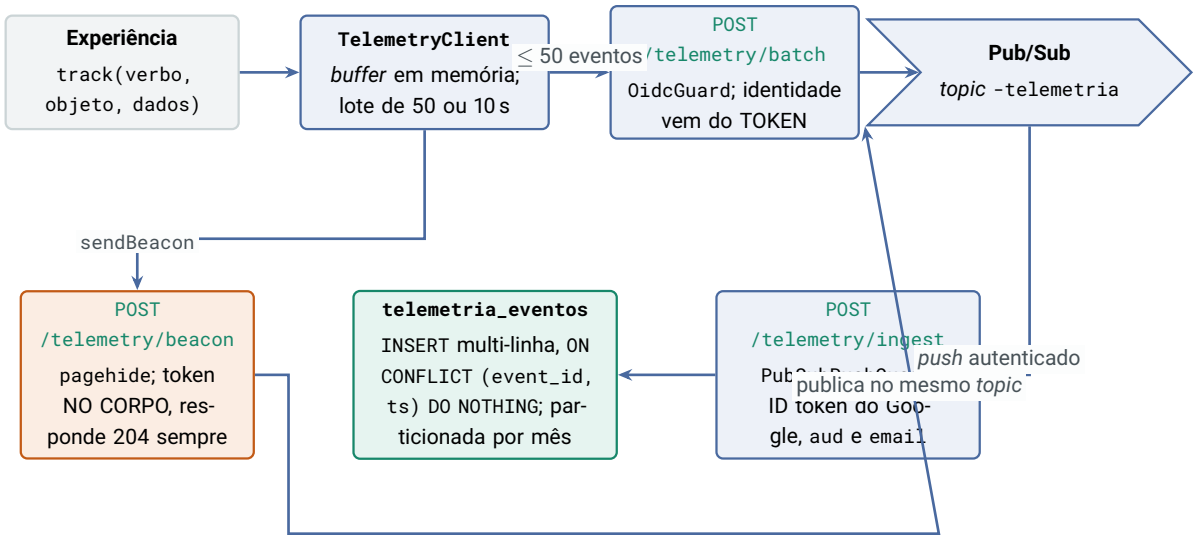
REGRA INVIOLÁVEL

***Dwell* NUNCA passa pelo Pub/Sub.** A latência e a falta de ordenação quebrariam a adaptação do *feed* e o *lock* por `run`: o motor precisa saber, no instante em que cura o próximo bloco, exatamente quais posts o aluno viu e por quanto tempo. Um evento que chega dois minutos depois, fora de ordem, produziria uma curadoria baseada em passado errado — e o aluno não teria como perceber.

NOTA

A recíproca também vale, e é o que justifica o segundo canal: mandar *tudo* pelo caminho transacional colocaria, no pico de quatro mil simultâneos, um `INSERT` sincronizado por interação de aluno dentro da mesma transação que serve a tela. O canal analítico existe para absorver volume sem competir com o que o aluno está esperando.

10.2 O canal analítico, ponta a ponta



Onde há perda possível, e só ali: entre o `track()` e o *flush*. Se o processo do navegador morrer com o *buffer* cheio — aba fechada à força, queda de energia — aqueles eventos somem. O `sendBeacon` do `pagehide` cobre o fechamento normal. A partir do Pub/Sub, a entrega é **durável**: é exatamente por isso que um *buffer* em Redis não substituiria o Pub/Sub — ele reintroduziria a perda que o Pub/Sub eliminou.

Por que o *beacon* responde 204 sempre: o navegador já descartou a página e ninguém lê o corpo. Erro aqui só serviria para poluir o log do cliente. Falha de verificação de *token* vira descarte silencioso — com aviso no servidor, não no cliente. O *token* vai no **corpo** porque `sendBeacon` não envia cabeçalho, e isso é limitação da API do navegador, não escolha.

Figura 10.1 – O canal analítico ponta a ponta. Os únicos eventos que podem se perder são os que estão no *buffer* em memória do navegador no instante em que o processo morre; do Pub/Sub em diante a entrega é durável.

10.3 A rota de ingestão e sua tranca

`POST /telemetry/ingest` é a rota mais delicada da API: ela grava dado de aluno, o `escolaId` vem no corpo (já enriquecido pelo `/batch`) e ela **não passa pelo `OidcGuard`** — porque quem chama é o Pub/Sub, não um usuário, e não existe *token* do hub nessa chamada.

REGRA INVIOLÁVEL

Sem `PubSubPushGuard`, a rota seria um *endpoint* público que escreve telemetria falsa em qualquer escola. O *guard* verifica **três** coisas no ID token que o Pub/Sub assina:

1. **emissor Google** (`accounts.google.com`), pelo JWKS público;
2. `aud` igual a `TELEMETRIA_PUSH_AUDIENCE`;
3. `email` igual a `TELEMETRIA_PUSH_SA`.

As três juntas é que fazem a prova. A assinatura sozinha só diz “algum projeto GCP”, que não é ninguém em particular.
Falha fechada: sem as duas variáveis, a rota rejeita tudo com 401 — nunca fica aberta. O Terraform casa os dois lados: a *subscription* é criada com `oidc_token` dessa mesma conta de serviço e audiência.

DECISÃO DE PROJETO

***Push subscription* para a própria API, e não um *worker* dedicado.** O desenho original previa `{contrato}-telemetry-worker` como serviço próprio. A *push subscription* entrega o mesmo resultado com uma peça a menos para operar — um serviço a menos com *deploy*, log, escala e *alerting* próprios. O *worker* separado volta se o volume justificar isolar o consumo do tráfego de aluno.

10.4 Destino: PostgreSQL, não BigQuery

DECISÃO DE PROJETO

O canal analítico grava no mesmo PostgreSQL (decisão de 2026-08-27, **ADR-14**).
Avaliou-se mandar o canal analítico para o BigQuery por *subscription* nativa do Pub/Sub. Não foi, por dois motivos concretos: (a) os relatórios do professor **não leem** `telemetria_eventos` — eles leem `exp_01_events`, o canal síncrono do motor —, então o BigQuery não resolvia problema nenhum *hoje*; e (b) criava uma segunda plataforma de dados para operar, com IAM, custo e ferramentas próprios.
Ficando no mesmo banco, se um relatório precisar desse dado um dia, é um `JOIN` local com `turmas` e `users` — e é uma plataforma de dados a menos.

O que sustenta essa escolha na escala alvo são duas medidas, e nenhuma delas é um serviço novo:

10.4.1 `INSERT` multi-linha

Listing 10.1 – O gargalo real, e a correção (`src/telemetry/telemetry.controller.ts`)

```
// UM INSERT multi-linha, nao um por evento. E o gargalo real deste caminho:
// um lote de 50 eventos custava 50 round-trips ao banco, e no pico da manha
// (~4k alunos simultaneos) isso e ~800 INSERTs/s numa instancia pequena.
// Em lote vira ~16/s, que e trivial.
//
// Nao e caso para Redis: quem amortece o pico ja e o Pub/Sub, que ainda por
// cima e DURAVEL — um buffer em memoria perderia eventos no processo que
// morre antes do flush.
const linhas = doGrupo.map(
  (e) => Prisma.sql`(
    ${e.eventId}::uuid, ${new Date(e.ts)}, ${e.escolaId}, ${e.alunoId}, ${e.expId},
    ${e.expVersion}, ${e.sessaoId}::uuid, ${e.verbo}, ${e.objeto},
```

```
    ${JSON.stringify(e.resultado ?? null)}::jsonb,
    ${JSON.stringify(e.contexto  ?? null)}::jsonb)`,
);
await tx.$executeRaw(Prisma.sql`
  INSERT INTO telemetria_eventos
    (event_id, ts, escola_id, aluno_id, exp_id, exp_version, sessao_id,
     verbo, objeto, resultado, contexto)
  VALUES ${Prisma.join(linhas)}
  ON CONFLICT (event_id, ts) DO NOTHING`);
```

NOTA

O lote é **agrupado por escola** antes da gravação, e cada grupo grava na sua própria *tenancy* (`comTenancy(fn, escolaId)`). Um lote do Pub/Sub pode conter eventos de escolas diferentes, e uma única transação não poderia ter duas políticas de RLS armadas ao mesmo tempo.

10.4.2 Partição mensal

Detalhada na Seção 8.3. Em resumo: `PARTITION BY RANGE (ts)`, partições no *schema* `particoes` (fora do alcance do Prisma), chave primária composta `(event_id, ts)` porque o PostgreSQL exige a chave de partição em todo índice único, partição `DEFAULT` como rede de segurança, e faixas declaradas até 2028-12.

DECISÃO DE PROJETO

Nada de Redis como *buffer* (**ADR-14**). Quem amortece o pico já é o Pub/Sub — e ele é *durável*. Um *buffer* em memória perde os eventos do processo que morre antes do *flush*, ou seja, reintroduz exatamente a perda que o Pub/Sub eliminou. Redis só entra com gargalo medido, e o gargalo medido aqui era o laço de `INSERT`.

10.5 Privacidade da telemetria

REGRA INVIOLÁVEL

- O evento carrega `aluno_id` **opaco** (UUID), nunca nome ou *e-mail*.
- A identidade é **derivada do *token*** no servidor. O que o cliente enviou nesses campos é descartado — então o cliente não *pode* colocar PII ali.
- Nenhuma PII em log em texto claro.
- Texto livre sensível (o *check-in* de bem-estar) **nunca** vai à telemetria; só contagens.
- Em `exp_70_sessoes.payload`, `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**. O campo de texto livre que existia era o único canal de texto arbitrário do *payload* — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração. Trava estrutural, não lista de proibidos.

LIMITAÇÃO CONHECIDA

Retenção sem prazo declarado também aqui. `telemetria_eventos` guarda evento por `aluno_id` de *todas* as experiências e não tem prazo — pela mesma razão registrada na Seção 8.3 e em **ADR-13**: prazo só existe junto com a rotina que o cumpre. A diferença é que, particionada, essa rotina passou a ser trivial de escrever (`DROP TABLE particoes.<mês>`), o que remove o último argumento técnico para não ter prazo. A pergunta que sobra é de produto e de contrato, e é da plataforma inteira.

10.6 Comportamento em desenvolvimento

Configuração	Comportamento
`TELEMETRIA_TOPIC` vazio	Os eventos são apenas *logados* no console (`[telemetria/dev] N eventos`). Não há Pub/Sub em desenvolvimento e nada chega a `/ingest`.
`TELEMETRIA_PUSH_SA` / `..._AUDIENCE` vazias	`/telemetry/ingest` responde 401 a tudo. É o comportamento correto: em desenvolvimento ninguém deveria chamá-la.

NOTA

Essa combinação é o que permite rodar a plataforma inteira, com telemetria funcionando de ponta a ponta do lado do cliente, sem nenhuma credencial de GCP — e é o que faz **RNF-14** (testabilidade sem nuvem) ser verdade e não aspiração.

11

Camada de inteligência artificial

A IA generativa é usada em quatro lugares da plataforma, com propósitos e riscos muito diferentes. Este capítulo descreve a camada única por onde todos passam, a economia de *tokens* que a sustenta e os *guardrails* que impedem cada uso de produzir dano.

11.1 Onde a IA é usada

Uso	Onde	O que gera, e o que NÃO gera
Geração de *feed*	exp-01, `src/geracao/`	Personas fictícias, *captions*, *hashtags* e arte dos posts. O *mix* (quantos posts de cada lado, perfil e intensidade) é decidido pelo **servidor**; a IA só preenche.
Reação ao vivo	exp-01, `src/motor/`	Comentários de persona reagindo ao que o aluno publicou. Desligável pelo professor no meio da aula, sem *deploy* (`ia_ao_vivo`).
Plano de aula	exp-90, `src/planos/`	Um plano anual completo em Markdown, a partir do currículo da rede e das unidades do ano. **Não** decide nota nem avalia aluno.
Entorno do *check-in*	exp-91, `src/bemestar/`	Abertura do dia, pergunta aberta e acolhimento. **Nunca** reescreve um item do DASS-21, nunca vê pontuação, nome ou escola.

REGRA INVIOLÁVEL

Correção, pontuação e gabarito nunca são gerados por IA. A IA produz *conteúdo* e *texto de entorno*. Tudo que vale nota, classifica ou mede é código determinístico e testável — e roda no servidor (Capítulo 12).

11.2 A camada única

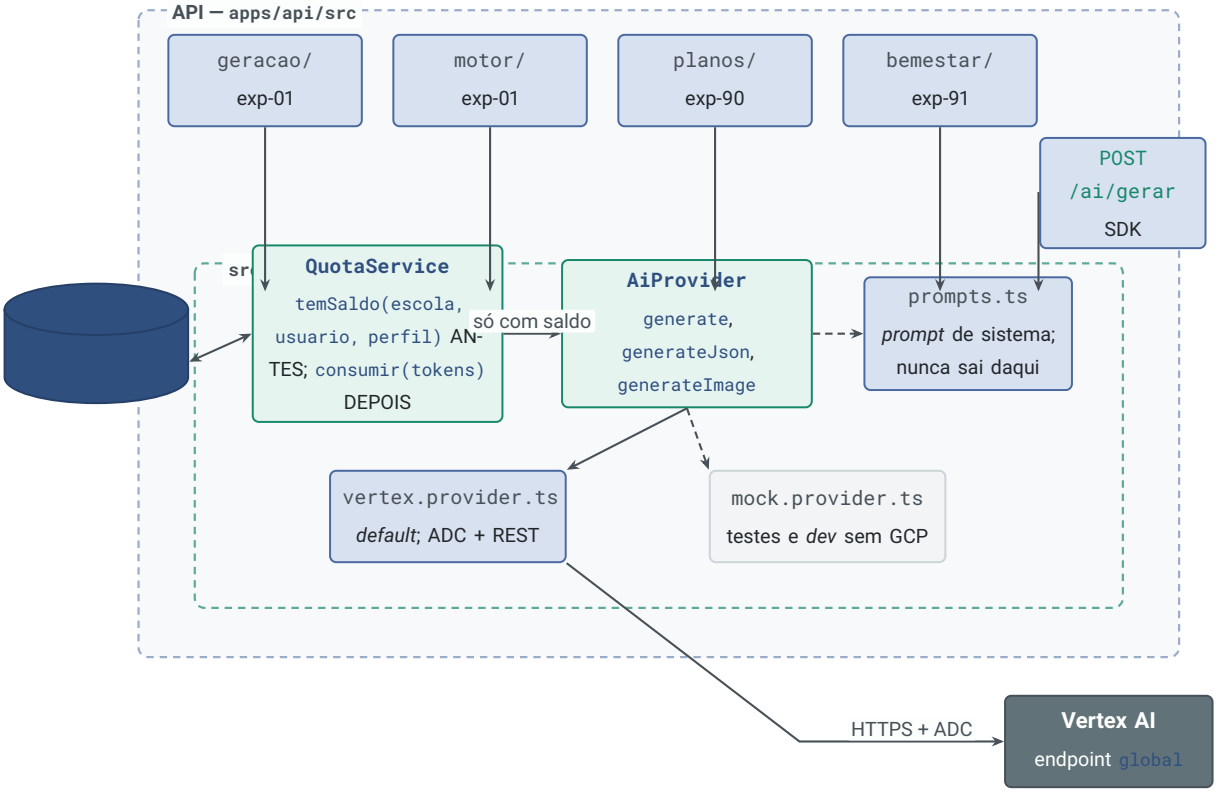


Figura 11.1 – A camada de IA. Quatro módulos de domínio e uma rota de SDK entram; todos passam por `QuotaService` antes do *provider*. Nenhum módulo fora da fronteira verde conhece o *vendor* do modelo.

11.3 Quota por perfil

> **REGRA INVIOLÁVEL**
>
> **Nunca exponha chamada de IA sem passar pelo `QuotaService`: `temSaldo(escola, usuario, perfil)` antes, `consumir` depois.**

Listing 11.1 – Incremento atômico da quota (`src/ai/quota.service.ts`)

```ts
/** Incremento atomico; devolve o total do dia depois do consumo. */
async consumir(escolaId: string, usuarioId: string, tokens: number): Promise<number> {
  const dia = this.hoje();
  return this.prisma.comTenancy(async (tx) => {
    const linhas = await tx.$queryRaw<Array<{ tokens_usados: number }>>`
      INSERT INTO ia_uso_diario (escola_id, usuario_id, dia, tokens_usados)
      VALUES (${escolaId}, ${usuarioId}, ${dia}, ${tokens})
      ON CONFLICT (escola_id, usuario_id, dia)
        DO UPDATE SET tokens_usados = ia_uso_diario.tokens_usados + ${tokens}
      RETURNING tokens_usados`;
    return Number(linhas[0]?.tokens_usados ?? tokens);
  }, escolaId);
}
```

DECISÃO DE PROJETO

A quota é persistida, não um `Map` em memória. O `Map` que existia antes tinha dois furos: zerava a cada *restart* e era *por instância* — com `min-instances: 3` no Cloud Run, a mesma criança tinha três quotas. Uma linha por (escola, usuário, dia), com incremento atômico, resolve os dois.

DECISÃO DE PROJETO

O limite depende do PERFIL. O aluno conversa com o tutor, e 20.000 *tokens*/dia bastam. O professor gera planos anuais em que só o *prompt* — currículo da rede mais material didático — passa de 200.000 *tokens*: o teto dele é outro (`AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR`, *default* 2.000.000/dia, o que dá seis a nove planos). Um teto único obrigaria a escolher entre "aluno com quota absurda" e "professor que não consegue gerar um plano".

NOTA

A verificação acontece **antes** de chamar o *provider*. Gastar *token* e depois recusar seria pagar pelo que não se entrega. Há também *rate limit* por usuário (`src/comum/rate-limit.ts`), que é uma proteção diferente: a quota limita custo diário, o *rate limit* limita rajada.

11.4 Credencial: ADC, nunca chave de API

REGRA INVIOLÁVEL

O *provider* obtém o *token* com `google-auth-library`, que resolve na mesma ordem nos três ambientes:

1. `GOOGLE_APPLICATION_CREDENTIALS`;
2. o arquivo de `gcloud auth application-default login`;
3. o *metadata server* (Cloud Run).

Não existe variável de ambiente para chave de API em nenhum lugar do projeto — nem em `.env.example`.

ATENÇÃO

Antes dessa mudança havia **só** o *metadata server*, e por isso *nenhuma* chamada de IA funcionava fora do Cloud Run — nem no *host*, nem no contêiner, onde `metadata.google.internal` não resolve. Em desenvolvimento a API ficava presa em `AI_PROVIDER=mock` e a exp-90 devolvia um plano **falso**, o que é pior que falhar: parecia funcionar. Os dois *compose* montam o ADC do *host* em `/adc` somente lei-

> tura; em Cloud Run a conta de serviço do serviço já *é* a credencial, e ela precisa de `roles/aiplatform.user`. Sem o papel, toda geração falha com 403 do Vertex.

11.5 Modelo, região e o custo de escolher

> **DECISÃO DE PROJETO**
>
> **Modelo `gemini-3.6-flash`, localização `global`** — para texto *e* para o plano de aula (decisão de 2026-08, **ADR-16**).
> Medido no projeto do contrato: o `3.6-flash` dá **404** em `southamerica-east1`, `us-east1` e `us-central1`. Só o endpoint `global` (sem *host* regional) o serve.
> A consequência é explícita e foi aceita: **o tráfego de IA sai de São Paulo, inclusive o do aluno**, e isso precisa constar do contrato e do DPA da rede. Voltar para `VERTEX_LOCATION=southamerica-east1` é possível a qualquer momento, mas exige trocar **também** o modelo pelo que a região publica — a família 2.5, já que a 2.0 saiu do catálogo e a 3.x nunca esteve lá. As duas variáveis mudam juntas, ou a geração para de funcionar.

11.5.1 Imagem: proporção pedida ao modelo, não recortada depois

Fato	Consequência
O *default* anterior era `gemini-2.0-flash-exp`	Esse modelo **não existe mais** em lugar nenhum (404 na região *e* em `global`), e por isso *toda* geração de imagem caía no *fallback* procedural — sem erro visível para o professor.
Modelo atual: `gemini-3.1-flash-image`	Responde em `global`. Alternativas verificadas: `gemini-2.5-flash-image` e `gemini-3-pro-image`.
A proporção `4:5` é PEDIDA no `imageConfig`	Sem isso, o 3.1 devolve 1408×768, e o `cover` do `AssetsService` (1080×1350) descarta mais da metade da largura.
O modelo aceita 1:1, 2:3, 3:2, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9 e 21:9 pelo MESMO custo em *tokens*	Experiência com outro formato deve **pedir** o formato, não recortar depois.

11.6 Raciocínio conta como saída

ATENÇÃO

Nos modelos das famílias 2.5 e 3.x, os *tokens de pensamento* saem do `maxOutputTokens`. Com pensamento livre, o plano anual voltava **vazio**, com `finishReason: MAX_TOKENS` — o modelo gastava todo o orçamento pensando e não sobrava saída. `VERTEX_THINKING_BUDGET` (*default* `0`, desligado) mantém a paridade com o aplicativo de origem, que usava um modelo sem essa etapa; `-1` volta ao dinâmico.

REGRA INVIOLÁVEL

Quem gera texto longo checa `AiResult.finishReason`. `MAX_TOKENS` significa *conteúdo cortado*, e isso é **erro** — não resultado parcial aceitável. Entregar meio plano de aula a um professor é pior que dizer que falhou.

Parâmetro	Valor e motivo
`MAX_TOKENS_SAIDA` do plano	**64k**, não os 32k do aplicativo de origem (que eram o limite do modelo usado lá). Medido: com o material real (currículo da rede + as 4 unidades do ano) o plano é cortado em 32k e o professor recebe "o plano ficou grande demais" sem plano nenhum. A família 2.5 entrega até 65k; 64k dá a folga sem chegar ao limite duro.
`AiRequest.modelo`	`"padrao"` ou `"longo"`. O `longo` mapeia para `VERTEX_MODEL_LONGO`. Quem chama **não sabe** qual modelo é — a família é neutra de *vendor*.

11.7 *Guardrails* do *check-in* de bem-estar

Este é o uso de IA com maior risco de dano na plataforma, e por isso o mais cercado. O instrumento é o DASS-21 (LOVIBOND; LOVIBOND, 1995), na versão brasileira validada (VIGNOLA; TUCCI, 2014), em domínio público.

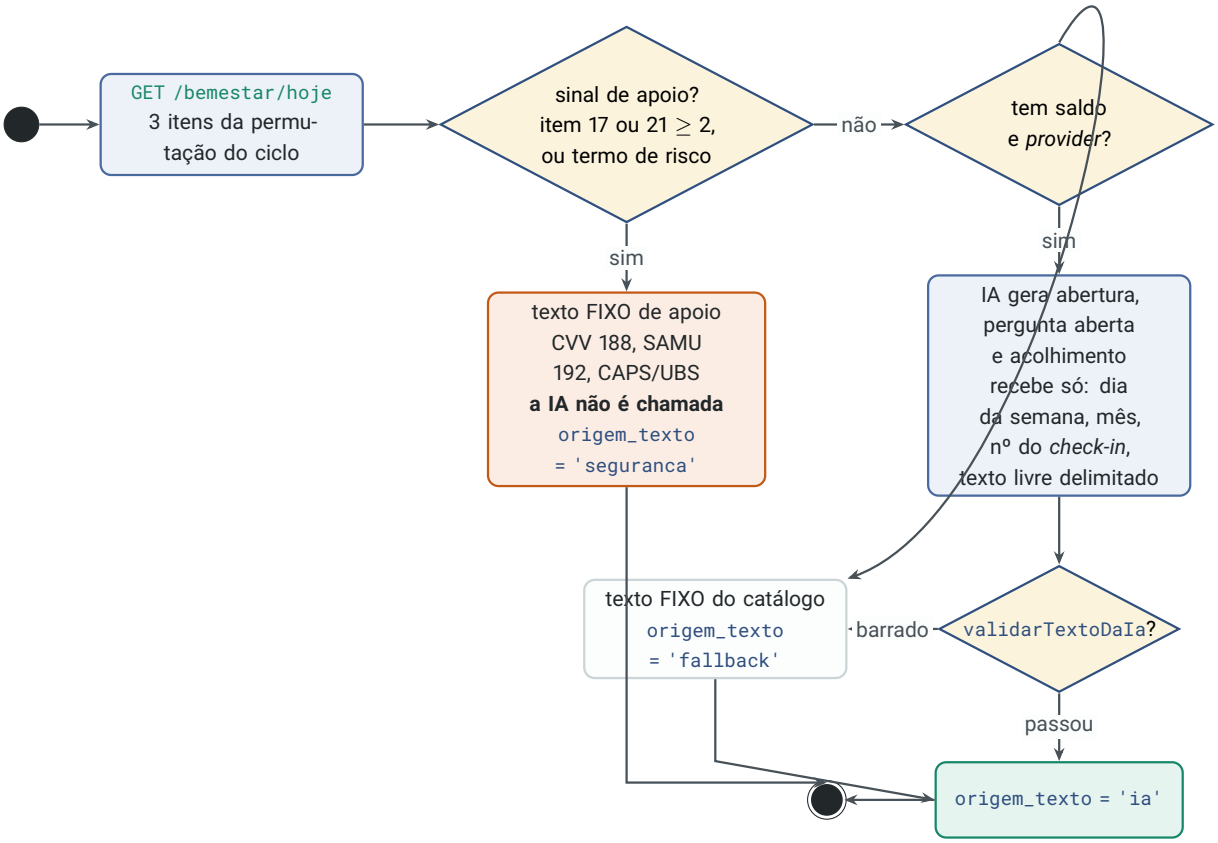


Figura 11.2 – Os *guardrails* do *check-in*. Três caminhos levam a texto seguro, e em nenhum deles o *check-in* deixa de abrir: se a IA falha, é barrada ou não há saldo, entra o texto fixo do catálogo; se há sinal de apoio, a IA nem é chamada.

REGRA INVIOLÁVEL

1. **Os 21 itens são fixos e *verbatim*** (`catalogo.ts`). A IA **nunca** reescreve um item: reformular quebra a validação psicométrica e invalida as faixas de corte publicadas.
2. **O que a IA gera é o entorno**, e cada texto passa por `regras.validarTextoDaIa`: sem vocabulário clínico, sem ordens, sem promessas, sem linguagem de gestão, sem *links*, com limite de tamanho.
3. **Barrado, sem saldo ou sem *provider* ⇒ texto fixo do catálogo.** O *check-in* nunca deixa de abrir.
4. **A IA nunca recebe** nome, escola, pontuação ou faixa — só dia da semana, mês, número do *check-in* e o texto livre, *delimitado como conteúdo*.
5. **Sinal de apoio bypassa a IA**: item 17 ou 21 ≥ 2, ou termo de risco no texto ⇒ mensagem **fixa** de apoio, e a IA não é chamada.
6. **A correção é *server-side***: ao sétimo *check-in* a API soma por subescala ($\times 2$) e classifica pelas faixas publicadas (`pontuar` e `faixaDe` em `regras.ts`).

NOTA

O texto livre do professor *vai* ao Vertex (endpoint `global`), delimitado como conteúdo e sem nenhum outro contexto identificável. Isso consta do DPA junto com o resto do uso de IA — e é a razão de a Seção 7 tratar localidade de processamento como item de conformidade, não de arquitetura.

11.8 Variáveis de configuração da IA

Variável	Default	Papel
`AI_PROVIDER`	`vertex`	`vertex` ou `mock`.
`VERTEX_PROJECT_ID`	—	**Obrigatória** com `AI_PROVIDER=vertex`: o *boot* recusa sem ela. Sem projeto, a URL viraria `.../projects/undefined/...` e o erro só apareceria na primeira geração.
`VERTEX_LOCATION`	`global`	Ver **ADR-16**.
`VERTEX_MODEL`	`gemini-3.6-fl`	Modelo do dia a dia.
`VERTEX_MODEL_LONGO`	`gemini-3.6-flash`	Modelo de saída longa. A variável separada existe para trocar um sem tocar no outro.
`VERTEX_IMAGE_MODEL`	`gemini-3.1-fl`	Quem trocar precisa conferir a proporção devolvida.
`VERTEX_THINKING_BUDGET`	`0`	`0` desligado, `-1` dinâmico, > 0 teto.
`AI_DAILY_TOKEN_QUOTA_PER_`	20.000	Quota diária do aluno.
`AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR`	2.000.000	Quota diária de professor, gestor e admin.

DECISÃO DE PROJETO

Validação de *env* no *boot*, com Zod. `VERTEX_PROJECT_ID` ausente com `AI_PROVIDER=vertex` derruba o processo na subida, com mensagem que diz o que fazer ("em *dev*, use `AI_PROVIDER=mock` para subir sem GCP"). A alternativa — descobrir na primeira geração de um professor real — é um incidente em vez de um erro de configuração.

12

Catálogo de experiências

O catálogo alvo é de 48 experiências. Cinco existem, e são elas que definiram o gabarito que as próximas seguem. Este capítulo documenta cada uma até onde a mecânica impõe exigência ao *software* — e não como material pedagógico.

12.1 Visão do catálogo

Experiência	Séries	Público	Capabilities	Módulo na API
`exp-01-instanooria`	3EF	aluno	telemetria, progresso, ia	`motor`, `geracao`, `atividades`, `perfis`, `dossie`
`exp-02-unootube`	4EF	aluno	telemetria, progresso	`unootube`
`exp-70-vila-do-lume`	**transversal**	aluno	telemetria	`vila`
`exp-90-plano-de-aula`	—	**professo**	telemetria, ia	`planos`
`exp-91-bem-estar`	—	**professor**	telemetria, ia	`bemestar`
`_template`	—	—	telemetria, progresso	—

Experiência	Porta em *dev*	Onde aparece no hub
`exp-01-instanooria`	5173	`/exp/exp-01-instanooria`
`exp-02-unootube`	5174	`/exp/exp-02-unootube`
`exp-90-plano-de-aula`	5175	`/professor/plano-de-aula`
`exp-91-bem-estar`	5176	`/professor/bem-estar`
`exp-70-vila-do-lume`	5177	`/exp/exp-70-vila-do-lume`

> **NOTA**
>
> As portas vivem em `apps/hub/lib/experiencias-dev.ts` — uma por experiência. É esse mapa que o hub usa quando não há manifesto, e é por isso que `EXP_INTERNAL_HOST` tem de *preservar a porta* (Seção 9.2).

12.2 exp-01 — Instanooria

Clone de rede social de imagens. O aluno navega um *feed* curado por um motor adaptativo que o empurra progressivamente para um dos dois lados de uma controvérsia; na virada da missão, o dossiê mostra o que ele não viu.

Exigência ao *software*	Como é atendida
Feed adaptativo por blocos, determinístico	`motor/curadoria.ts`: função pura que recebe um gerador de aleatoriedade. É isso que permite ao teste afirmar "só engaja A ⇒ domínio > 0,75 até o bloco 4".
Serialização por sessão	`pg_advisory_xact_lock(runId)` dentro da transação, com `timeout` de 20 s.
Gabarito de curadoria fora do cliente	`motor/sanitizador.ts` remove `lado`, `perfil` e `intensidade` antes de o *feed* sair.
Geração de conteúdo em lote, assíncrona	`geracao/`: fila em `exp_01_generation_jobs`, *worker in-process*, reciclagem de órfão por `updated_at`.
Momento coletivo controlado pelo professor	`activities.etapa` é um relógio **coletivo**: o professor abre e fecha, e a tela do aluno trava conforme.
Reação ao vivo, desligável sem *deploy*	`activities.ia_ao_vivo`, alterável por `PATCH /atividades/:id/ia`.
Largada por perfil, sorteio ou redistribuição	`perfis/largada.ts`, com precedência explícita e `lado_origem` gravado para auditoria.
Casca de aparelho de 375×812	`viewport` no manifesto ⇒ `DispositivoFrame`.

Listing 12.1 – Precedência da largada (`src/perfis/largada.ts`)

```
// 1. `lado_forcado` — o professor redistribuiu antes da aula. Essa decisao NAO
//    pode ser desfeita por um recalculo de afinidade no login.
// 2. Afinidade do perfil — o mecanismo normal.
// 3. Sorteio balanceado — sem perfil, ou perfil `misto`. Escolhe o lado com
//    menos runs, para a turma ainda sair perto de 50/50.
//
// Existe como modulo proprio porque DUAS rotas precisam da MESMA regra: o
// primeiro inicio e o reset da sessao. Quando o reset tinha regra propria, ele
// mantinha o lado antigo — e reaproveitar o aparelho repetia a semente sem
// ninguem perceber.
```

Verbo	Significado documentado
`attempted`	entrou no aparelho
`progressed`	mudou de fase
`completed`	concluiu a missão (dossiê visto)

12.3 exp-02 — UnooTube

Clone de plataforma de vídeo, para o 4º ano. O aluno salva até três vídeos; ao terceiro, o ranking desbloqueia; "adicionar credibilidade" reordena o ranking por fontes e é a virada pedagógica.

Decisão	Motivo
Gabarito (`factual`) fora do *bundle*	Vive em `src/unootube/catalogo.ts`. A API corrige e devolve inscritos e fontes — nunca a resposta (regra 3).
Ranking com colegas **fictícios**	Distribuição desenhada para a lição. Não há pontuação de menor real exposta a outro menor (LGPD).
URLs das fontes como **texto**	O *sandbox* do `iframe` bloqueia *popups*, e o aluno não sai da plataforma.
Salvos **congelados** depois de "adicionar credibilidade"	O *score* da experiência já foi reportado — nota e estado não podem divergir. Curtir segue livre; reiniciar descongela.
`ranking_desbloqueado` é *latch*	Remover um salvo depois não tranca o ranking de novo. Estado que anda para trás confunde a criança e o professor.
Sem `viewport`	Foi desenhada para *desktop*: não entra em casca de aparelho.
Tema escolhido pelo aluno	Perfil → APARÊNCIA, lembrado em `localStorage` — que a regra 4 proíbe para *token*, não para preferência de tela.

> **NOTA**
>
> A exp-02 é o caso canônico da **linha da revelação** (Capítulo 14): *feed*, cromo, *playlists* e canal usam a seção do clone; ranking e os dois diálogos da missão usam a seção da plataforma. O *toast* não cruza a linha — ele fala de dentro da ficção ("Vídeo salvo!").

12.4 exp-70 — Vila do Lume

Jogo 3D em primeira pessoa, **transversal** (`series: []`): não pertence a ano nenhum. É o "Ato 1" que alimenta a largada da exp-01.

12.4.1 Duas leituras de uma partida, e elas não se tocam

REGRA INVIOLÁVEL

O retrato Schwartz (`mapeamento.ts`: 10 valores + 2 eixos) lê *escolhas com custo* dentro do jogo. A missão "A Noite Longa" (`mensagens.ts`) lê *julgamento sobre 5 mensagens*. A separação é **de medida**: um golpe que custasse fagulha faria a criança que caiu aparecer como *menos generosa* no retrato, e nada no relatório mostraria isso.

12.4.2 O servidor recalcula; nunca aceita o perfil do cliente

REGRA INVIOLÁVEL

O jogo sabe computar o retrato e **deliberadamente não o envia**: `CHAVES_DE_PERFIL` em `src/vila/contrato.ts` *recusa payload* que traga o perfil. É isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir (hoje 3, herdada do piloto para os dados serem comparáveis).

12.4.3 `grau` é invariante de API, não regra de tela

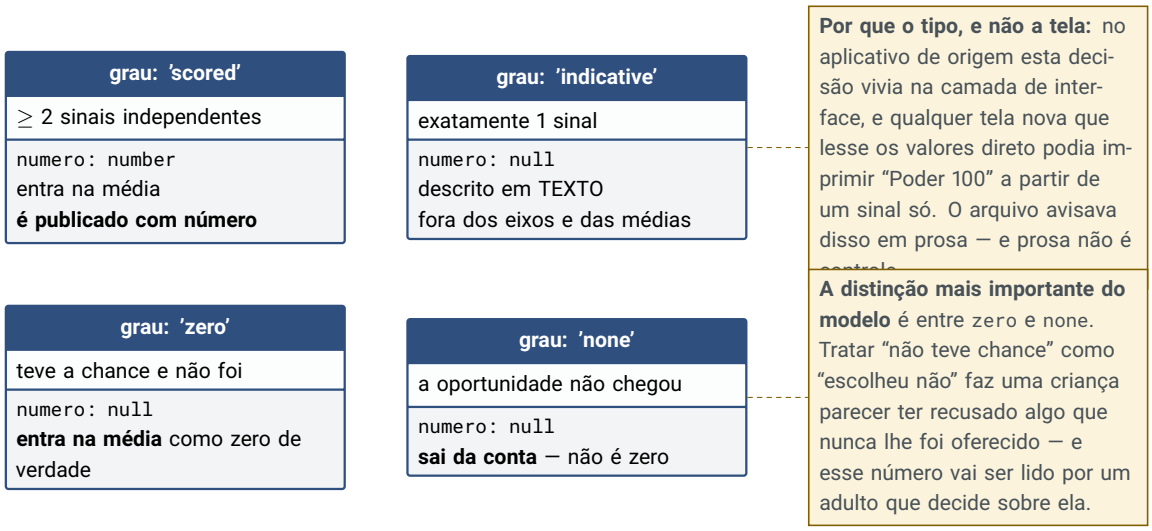


Figura 12.1 – Os quatro graus de medida da exp-70. A API devolve uma união discriminada em que **só** a variante `scored` tem `numero: number`; as outras têm `null` *por tipo*. O hub fica estruturalmente incapaz de imprimir número onde não há medida.

12.4.4 A bolha é do servidor

REGRA INVIOLÁVEL

`POST /vila/sessoes` atribui a bolha A/B/C **balanceada dentro da turma**, persiste em `exp_70_bolhas` (estável entre partidas) e devolve os cartões — **só os desta criança e sem gabarito**.
Antes era `alunoSlot % 3` no navegador, o que punha no *bundle* a condição do próprio instrumento e deixava as três versões legíveis antes da aula.
Nos anos iniciais (1EF–5EF) a bolha B **não é servida** ("suporte" exige entender o que é suporte) — regra que só a API pode aplicar, porque só ela sabe a série.
O *lock* consultivo por turma não é zelo: a sala clica junto, e sem ele os 25 caem todos na bolha A.

ATENÇÃO

`cartoesServidos` ≠ "apareceram". O primeiro é quantos a API entregou; o segundo, quantos chegaram a aparecer na partida. Zero aparecidos ⇒ a tela diz "as mensagens não chegaram nesta partida", **nunca** "ela ignorou tudo" — é a mesma distinção entre `zero` e `none`.

12.4.5 Telas do professor

Rota	Conteúdo e regra
`.../vila-do-lume`	Ficha: para que serve, quem consome o perfil, passo a passo.
`.../vila-do-lume/guia`	O debate. Projeta as mensagens das **três** bolhas lado a lado — é a revelação da missão, e a primeira vez que a turma vê as outras versões.
`.../resultados?turmaId=`	Listagem da turma, com o botão de zerar.
`.../alunos/:id`	A leitura individual, que abre pelo **nível**.

REGRA INVIOLÁVEL

1. **O `/guia` precisa saber a TURMA.** Nos anos iniciais a API não serve a bolha B, e uma tela que projetasse as três mandaria o professor esperar a reação de um grupo que a sala dele não tem. `GET /vila/professor/cartoes?turmaId=` filtra pela *mesma* regra de `abrir` (`bolhasElegiveis`).
2. **Os textos vêm da API, nunca de cópia no hub.** Os textos *são* o instrumento, e uma vírgula divergente entre o que o jogo serviu e o que o professor projeta põe a sala discutindo uma mensagem que ninguém recebeu.
3. **A listagem da turma não mostra valor de Schwartz nenhum.** Listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez, com o grau ao lado.

4. **O agregado abre para gestor; a linha de uma criança, não.** São *controllers* separados, e não um `@Roles` misto com exceção por método.
5. **Zerar a turma apaga as BOLHAS junto.** Rodar de novo com a mesma bolha serviria à criança o mesmo golpe: ela reconhece, e a segunda rodada não mede nada. O que **não** é tocado é `exp_01_exp_70_perfis_importados` — aquilo pertence à atividade da exp-01, e apagar de lá trocaria os lados de uma aula em andamento.

12.4.6 A exceção de ADR: o jogo em JavaScript clássico

DECISÃO DE PROJETO

`apps/experiencias/exp-70-vila-do-lume/public/jogo/` — cerca de dez mil linhas de JavaScript clássico com variáveis globais compartilhadas e `three.min.js` vendorizado — **fica como está**, e está nos `ignores` do ESLint (**ADR-18**).
É um instrumento com distâncias e tempos calibrados, e o compilador não tem como saber que uma refatoração mudou a *medida*. O *shim* (`src/`, TS *strict*) é a única porta: `window.VILA`. A rede que substitui o `tsc` ali é o *smoke* visual.

12.5 exp-90 — Gerador de plano de aula

Ferramenta do professor. Detalhada nas Seções 11 e 4; o que resta registrar é a divisão de responsabilidade de armazenamento.

Artefato	Onde vive, e por quê
`gerado.md`	*Bucket* privado, **imutável**. É o original da IA: restaurar é voltar a ele.
`atual.html`	*Bucket* privado. O que o editor salvou, **sanitizado por *allowlist***. Versionamento de objetos dá histórico de cada "Salvar".
Referências curriculares	PostgreSQL (`exp_90_referencias`), como **texto**. Catálogo do contrato, sem `escola_id`, fora da RLS. Importadas por `scripts/importar-referencias-plano.mjs` com `PLANOS_IMPORT_TOKEN`.
Metadados e estado	PostgreSQL (`exp_90_planos`).

ATENÇÃO

`PLANOS_IMPORT_TOKEN` ausente ⇒ a rota de importação responde **503**, não fica aberta. O mesmo vale para `PERFIL_IMPORT_TOKEN`.

12.6 exp-91 — Check-in de bem-estar

Ferramenta do professor, detalhada na Seção 11. O que resta registrar é a mecânica de micro-doses e a decisão de escopo de leitura.

Mecânica	Implementação
21 itens em 7 *check-ins*	`ordem_itens` é a permutação de 1..21 sorteada na abertura do ciclo; cada *check-in* leva 3 itens, um de cada subescala.
Um *check-in* por dia	`UNIQUE (escola_id, professor_id, dia)`, com `dia` no fuso `America/Sao_Paulo`.
Ciclo expira em 21 dias	`expira_em`; ciclo não fechado vira `expirado`, sem pontuação.
Resumo leve para o *card* do hub	`GET /bemestar/resumo`, sem IA. É o que a home do professor lê — e a home não pode depender de uma chamada de modelo.
Apagar tudo	`DELETE /bemestar/dados`.

DECISÃO DE PROJETO

Nenhuma rota de gestor nem agregado (**ADR-10**). Dado de saúde (LGPD art. 11) com RLS e posse por `professor_id` em toda *query*; só o professor lê. Uma visão agregada e anonimizada para a rede é decisão *futura*, com DPA e *k*-anonimato — e não sai de um `GROUP BY`.

12.7 Como criar uma experiência nova

Listing 12.2 – O caminho completo

```
# 0. Uma branch por experiencia, com o nome do slug
git switch -c exp-07-fracoes

# 1. Scaffolding do _template (copia e ajusta nomes)
pnpm new:exp exp-07-fracoes

# 2. Implementar em src/App.tsx+
#    - auth, api e telemetria: SOMENTE pela SDK
#    - interface: SOMENTE o @unooria/design-system, pela ponte packages/ui,
#      na secao do clone que a experiencia imita
#    Falta componente? Ele nasce no repo storybook-unooria e chega por versao
#    nova — nunca como arquivo local.

# 3. Declarar capabilities e verbos no public/experiencia.json

# 4. Documentar verbo/objeto de telemetria no README da experiencia

# 5. Build gera dist/ 100% estatico. Testar embedado:
pnpm --filter @edu/exp-07-fracoes dev     # porta nova em lib/experiencias-dev.ts
# e abrir a bancada do hub:
#   http://localhost:3000/dev-embed?src=http://localhost:5178
```

Checklist	Verificação
Módulo de domínio na API	Necessário sempre que houver gabarito, estado de missão ou visão de professor. Prefixo minúsculo igual ao slug (`src/<slug>/`), com `catalogo.ts` + `regras.ts` + *service* com `comTenancy` + *controllers* por papel.
Tabelas	Prefixo `exp_NN_`, `escola_id NOT NULL` + RLS.
Porta de *dev*	Registrada em `apps/hub/lib/experiencias-dev.ts`.
`allowedHosts` do Vite	Deve incluir `host.docker.internal`, senão o hub em contêiner não lê o manifesto.
`viewport`	Só se a experiência exigir casca de aparelho.
`permissoes`	Só o que o `iframe` realmente precisa.
Mídia $>$ 2 MB	Vai para o *bucket* de *assets*, referenciada por variável de *build*. Nunca no *git*, nunca no *bundle*.
Smoke visual	Um *walkthrough* em `scripts/` se a experiência tem tela nova no hub.

> **NOTA**
>
> O `_template` continua **cru de propósito**: o que ele não traz, o *design system* traz. Um *template* rico em componentes locais seria a primeira violação da regra de interface, replicada 48 vezes.

13

Especificação da API

A API é um serviço NestJS (NestJS, 2026) com 22 módulos. Este capítulo especifica a superfície HTTP: as convenções transversais, os grupos de rotas e a autorização de cada um. O catálogo completo, rota por rota, está no Apêndice C.

13.1 Convenções transversais

Aspecto	Convenção
Autenticação	`Authorization: Bearer <JWT>`, validado por `OidcGuard` contra o JWKS de `OIDC_ISSUER`. Três rotas fogem disso, e cada uma tem substituto explícito (abaixo).
Autorização	`@Roles(...)` + `RolesGuard`, *sempre* depois do `OidcGuard` na lista de *guards*. Sem `@Roles`, a rota aceita qualquer papel autenticado.
Tenancy	`TenancyInterceptor` global abre o contexto com a escola do *token*; toda *query* passa por `comTenancy` e filtra `escola_id` explicitamente.
Validação de entrada	Zod em todo corpo e *query*. Não há `class-validator` nem *pipes* de DTO: o esquema Zod é a fonte única do tipo e da validação.
Corpo	JSON com teto de 2 MB; `text/csv` cru também com 2 MB, para a importação de usuários.
CORS	Lista explícita em `CORS_ORIGINS` — hub mais os *dev servers* das experiências. Nunca `*`.
Erros	*Exceptions* do Nest (`BadRequestException`, `ForbiddenException`, `NotFoundException`). Domínio com vocabulário próprio de erro usa um `CodigoErro` tipado (ex.: `src/unootube/erros.ts`).
Assincronia longa	**202 + *polling***. Nenhuma rota fica aberta esperando modelo de linguagem.

13.1.1 As três rotas sem `OidcGuard`

Rota	Substituto	Por quê
`GET /healthz`, `GET /readyz`	Nenhum — públicas	Sonda de saúde. Exigir *token* tornaria a sonda dependente do IdP.
`POST /telemetry/beacon`	Verificação manual do *token*, pelo `TokenService`	`sendBeacon` não envia cabeçalho, então o *token* vem no corpo. O rigor da verificação é o mesmo; só o transporte muda.
`POST /telemetry/ingest`	`PubSubPushGuard`	Quem chama é o Pub/Sub, não um usuário. Verifica emissor Google, `aud` e `email` da conta de serviço. **Falha fechada** sem configuração.

> **ATENÇÃO**
>
> Há ainda duas rotas de *serviço* protegidas por chave em cabeçalho, e não por JWT: `POST /perfis/importar` (`x-perfil-token`) e `POST /planos/referencias` (`x-import-token`). Ambas são chamadas por *script* local ou por integração externa, e ambas respondem **503** quando a variável correspondente está ausente — desligadas, não abertas.

13.2 Mapa de rotas por área

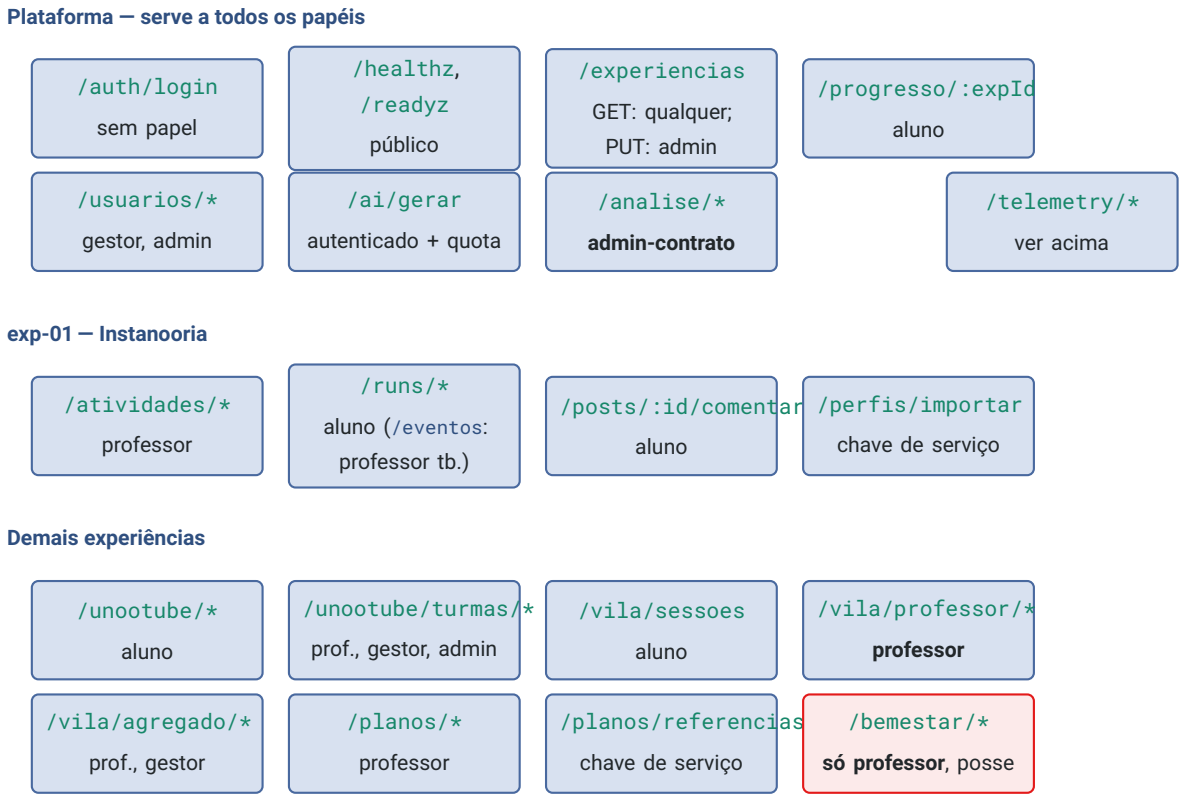


Figura 13.1 – Mapa das rotas por área e papel autorizado. A área de bem-estar está destacada porque é a única sem *nenhuma* rota de gestor ou de agregado — ausência deliberada, não lacuna.

13.3 Superfície que a SDK consome

Uma experiência nunca chama a API "à mão": chama pela SDK, e a SDK oferece exatamente quatro capacidades.

Listing 13.1 – `ExperienciaHandle` — tudo que a experiência pode fazer

```ts
export interface ExperienciaHandle {
  aluno: Aluno;
  config: Record<string, unknown>;

  /** Cliente HTTP autenticado. Sem retry fora do GET — repetir um POST
   *  de dominio pode duplicar efeito. */
  api: {
    get<T>(path: string): Promise<T>;
    post<T>(path: string, body?: unknown): Promise<T>;
    put<T>(path: string, body?: unknown): Promise<T>;
    patch<T>(path: string, body?: unknown): Promise<T>;
    delete<T>(path: string): Promise<T>;
  };

  /** Unica porta para IA. A experiencia NAO conhece modelo nem prompt. */
  ai: { gerar(input: { atividade: string; mensagem: string }): Promise<{ texto: string }> };

```

```
  /** Telemetria em lote. A experiencia informa SO verbo, objeto e dados. */
  track(verbo: TelemetryEvent["verbo"], objeto: string,
        dados?: Omit<TelemetryEvent, "verbo" | "objeto">): void;

  progresso: {
    salvar(dados: Record<string, unknown>): Promise<void>;
    carregar<T>(): Promise<T | null>;
  };

  concluir(payload?: { score?: number; dados?: Record<string, unknown> }): void;
  on(evento: "pause" | "resume", cb: () => void): void;

  /** Estado de alto nivel para o chrome opcional do hub (exp:estado). */
  enviarEstado(estado: Record<string, unknown>): void;
}
```

> **NOTA**
>
> Não há método de “corrigir” nem de “pontuar” na SDK, e isso é intencional: correção é rota de domínio da experiência na API (`/unootube/*`, `/vila/*`, `/runs/*`), não capacidade genérica. Uma API genérica de avaliação convidaria a mandar a resposta certa junto.

13.4 Grupos de rotas em detalhe

13.4.1 Motor de *feed* (exp-01)

O grupo com mais rotas da plataforma, todas escopadas por `run` e todas de papel `aluno`, com duas exceções marcadas.

Rota	Papel	Semântica
`POST /runs`	aluno	Cria ou recupera a sessão; resolve a largada (perfil, sorteio ou redistribuição).
`GET /runs/boot`	aluno	Estado inicial que a experiência precisa para montar a primeira tela.
`POST /runs/:id/entrar`	aluno	Nome no aplicativo e avatar escolhidos.
`GET /runs/:id/feed`	aluno	**Próximo bloco curado.** Abre transação, toma o *lock* da `run`, aplica eventos pendentes, cura, persiste e sanitiza.
`POST /runs/:id/eventos`	**aluno, professor**	Canal operacional de telemetria (*dwell*, curtidas, salvos). Síncrono e transacional.
`GET /runs/:id/fase`	aluno	Fase atual, para travar a tela conforme o relógio coletivo da turma.
`POST /runs/:id/conclusao`	aluno	Inclinação escolhida (A, B ou dúvida).

`POST /runs/:id/reflexao`, `GET .../reflexao`	aluno	Texto livre de fechamento, sem limite de caracteres.
`POST /runs/:id/student-post`	aluno	Post publicado pelo aluno.
`POST /runs/:id/reacoes`	aluno	Dispara reações de persona ao vivo (se `ia_ao_vivo`).
`GET /runs/:id/dossie`	**aluno, professor**	O dossiê de fechamento — a única rota do grupo que o professor também lê.
`GET /runs/:id/perdidos`	aluno	Os posts que o aluno *não* viu. É a virada pedagógica da missão.
`POST /runs/:id/reset`	**professor**	Zera a sessão. Recalcula a largada pela *mesma* regra do primeiro início.

13.4.2 Planos de aula (exp-90)

Rota	Código	Semântica
`POST /planos`	**202**	Enfileira a geração e devolve o id. *Não* espera o modelo.
`GET /planos/:id`	200	Estado (`pendente`, `gerando`, `pronto`, `erro`) e, quando pronto, o conteúdo. É o alvo do *polling*.
`PUT /planos/:id/conteudo`	200	Salva o HTML do editor, sanitizado por *allowlist*.
`POST /planos/:id/restaurar`	200	Volta ao `gerado.md`, que nunca foi sobrescrito.
`PATCH /planos/:id`	200	Renomeia.
`DELETE /planos/:id`	200	Exclui.
`GET /planos/referencias/disponiveis`	200	Que disciplinas e séries têm material importado — é o que popula o formulário.

13.4.3 Bem-estar (exp-91)

Rota	Semântica
`GET /bemestar/resumo`	**Leve, sem IA.** É o que o *card* do hub lê — a home do professor não pode depender de uma chamada de modelo.
`GET /bemestar/hoje`	Abre o *check-in* do dia: 3 itens *verbatim* + textos de entorno (IA ou *fallback*).
`POST /bemestar/checkins/:id/respostas`	Grava. No sétimo, pontua e classifica *server-side*.

`GET /bemestar/historico`	Série do próprio professor.
`DELETE /bemestar/dados`	Apaga tudo do próprio professor.

REGRA INVIOLÁVEL

Todas as cinco são `@Roles("professor")` e filtram `professor_id = req.auth.usuarioId` em toda *query*. RLS por escola não basta aqui: dois professores da mesma escola não podem se ler.

13.4.4 Análise

Rota	Papel	Escopo
`GET /atividades/:id/analise`	professor	Uma atividade da própria escola.
`GET /atividades/:id/analise/c`	professor	Exportação da mesma visão.
`GET /analise/resumo`	**admin-contrato**	Toda a rede.
`GET /analise/atividades/:id`	**admin-contrato**	Uma atividade, cross-escola.
`GET /analise/csv`	**admin-contrato**	Exportação da rede.

DECISÃO DE PROJETO

São **dois *controllers* no mesmo arquivo**, com `@Roles` diferentes: um em `/atividades` para o professor, outro em `/analise` para o `admin-contrato`. A separação física é o que garante que a próxima rota adicionada herde a permissão certa — e é a mesma decisão tomada em `src/vila/`.

13.5 Sanidade e prontidão

Rota	Semântica	Uso
`GET /healthz`	O processo está vivo	*Liveness* do Cloud Run.
`GET /readyz`	As dependências estão prontas	*Readiness*: verifica o banco. É o que impede uma revisão nova de receber tráfego antes de conseguir consultar.

> **NOTA**
>
> As duas são públicas de propósito. Uma sonda de saúde que precise de *token* depende do provedor de identidade para dizer se a API está viva — o que inverte a relação de diagnóstico exatamente quando ela mais importa.

14

Design system e camada de interface

> **REGRA INVIOLÁVEL**
>
> **Todo elemento de interface do hub e das experiências vem do `@unooria/design-system`, sempre na última versão publicada.** Componente de interface novo se escreve *no repositório do design system* e chega aqui por versão — não em `packages/ui`, não dentro de uma experiência. Antes de criar um botão, um *card* ou um campo: procure na seção correspondente do pacote, ela provavelmente já tem.

O pacote é npm **público** no escopo `@unooria` — sem *registry* privado, sem `.npmrc` no repositório. A fonte é o repositório `storybook-unooria` (Storybook em `:6007`, publicado por *tag* `v*` no GitHub Actions).

14.1 Duas famílias no mesmo pacote

O pacote publica **seções** independentes, cada uma com *namespace* de token e prefixo de classe próprios. Elas nunca se cruzam.

Seção	Import	Tokens	Onde vale
`unooria` (39)	raiz do pacote	`--unooria-*`, classe `.unooria-theme-light|dark`	Hub: *login*, `/professor`, `/admin`, `/analise`; *header* de todos os papéis; telas **pós-revelação** do aluno
`instanooria` (21)	`/instanooria`	`--insta-*`, `insta-`	exp-01
`unitube` (30)	`/unitube`	`--uni-*`, `uni-` — **só escuro**	exp-02
`whatsappmobile` (29)	`/whatsappmobi`	`--wa-*` — só escuro	—
`chatgpt` (17)	`/chatgpt`	`--gpt-*`, `data-gpt-theme`	—

`tiktooria` (35)	`/tiktooria`	`--tik-*`	— (dois *contextos* de superfície: cromo claro de lista e perfil, imersivo preto de *feed* e câmera — não dois temas)

ATENÇÃO

A seção `unitube` é **só escura**: o Figma não publica variante clara, e **derivar uma é proibido**. Uma paleta clara inventada não é o clone — é uma aproximação, e a experiência mede reconhecimento.

14.2 A linha que separa as seções é a REVELAÇÃO

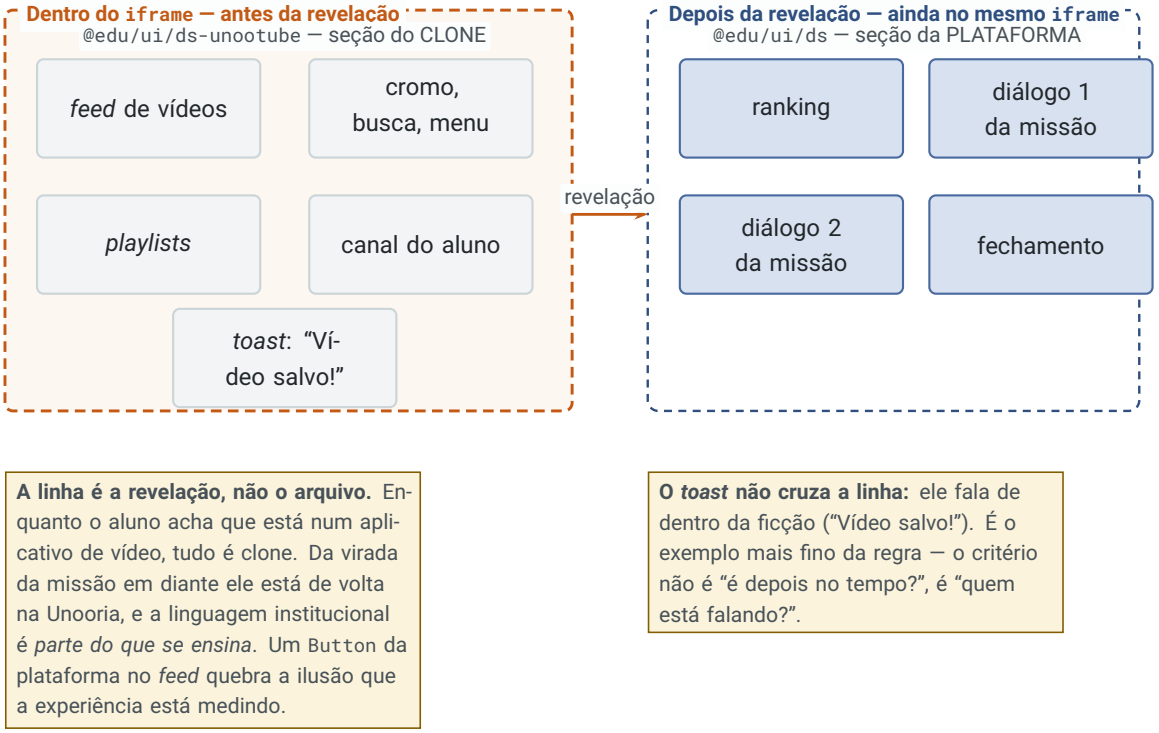


Figura 14.1 – A linha da revelação, na exp-02. As duas seções do *design system* coexistem no mesmo *bundle* e a fronteira entre elas é pedagógica, não técnica.

14.3 Consumo: sempre pela ponte

REGRA INVIOLÁVEL

Consumir sempre pela ponte em `packages/ui`, nunca importando o pacote direto num componente:

`@edu/ui/ds`	`unooria`	plataforma
`@edu/ui/ds-insta`	`instanooria`	exp-01

`@edu/ui/ds-unootubeunitube`	exp-02

A ponte existe por dois motivos, e os dois estão registrados nos arquivos:

1. **Marcar `'use client'` de uma vez**, para o App Router do Next.js.
2. **Desfazer a colisão de nomes.** `Icon`, `Avatar`, `Badge`, `Button`, `Logo` e `TextField` existem nas *duas* famílias. Os clones só saem pelo *namespace* (`InstanooriaUI.Icon`), e **destruturar na ponte** é o que mantém o *tree-shaking* e evita o erro `TS2742` causado pelo `@types/react` interno do pacote.

> **NOTA**
>
> Clone novo em uso = arquivo novo `packages/ui/src/ds-<clone>.ts` + `exports` no `packages/ui/package.json`. Mesmo padrão, sempre.

14.4 CSS: uma vez, no ponto de entrada

Aplicação	Onde o CSS entra
Hub	`components/casca-plataforma.tsx` — e é *por isso* que `/exp/*` e `/dev-embed`, que estão fora do grupo de rotas `(plataforma)`, também têm o *design system*.
Experiência	O próprio `src/main.tsx`, com o CSS da *sua* seção.

> **REGRA INVIOLÁVEL**
>
> **Nunca importe o consolidado `/styles.css` numa experiência**: ele arrasta as seis seções e a tipografia da plataforma para dentro do clone.

14.4.1 No hub, o CSS não basta: as fontes também

> **ATENÇÃO**
>
> O `unooria.css` declara `Open Sans` e `Material Symbols Rounded` por `@import` de URL remota — e o **Next remove `@import` externo** de CSS importado por JS, *silenciosamente*: a regra continua lá, o `getComputedStyle` responde "Open Sans" e o navegador desenha com a fonte do sistema.
> Como `Material Symbols Rounded` é fonte de **ligadura**, o sintoma é o **ícone virando texto**: onde deveria haver um ícone de adicionar, aparece "addInvestigação".
> Por isso `components/casca-plataforma.tsx` também monta `FontesDaPlataforma` (`@edu/ui/fontes`). Numa experiência Vite o `@import` sobrevive e isso não é necessário.

REGRA INVIOLÁVEL

Quem importa, declara. Todo aplicativo que escreve `@unooria/design-system` no código tem a dependência no seu `package.json` — hub incluído, porque ele importa o `unooria.css`. Depender do *hoisting* do pnpm via `packages/ui` funcionava *por acaso*: bastava a ponte trocar de versão (ou o `node_modules` ser reinstalado com outro *layout*) para o CSS do hub sumir sem ninguém ter mexido nele.

14.5 Frames: dois níveis, não empilhar

Componente	Onde vale
`PhoneFrame`, `DesktopFrame`	Cascas de aparelho **do clone**, publicadas pelo pacote. Servem ao Storybook e a telas soltas.
`DispositivoFrame` (hub)	**Dentro da plataforma, quem desenha o aparelho é o hub**, dirigido pelo `viewport` do manifesto. A experiência embedada preenche o `iframe` e não traz casca própria.

14.6 Tema

Seção	Como recebe o tema
Clone (`unitube`)	Lê `data-uni-theme` na raiz do documento.
Plataforma (`unooria`)	Recebe `theme` por **prop**.

ATENÇÃO

Quem troca o tema escreve nos **dois** (`exp-02/src/lib/tema.ts`), senão o aluno escolhe escuro no *feed* e o ranking volta branco — porque o ranking é da seção da plataforma, que não lê o atributo da raiz. Na exp-02 a escolha é do aluno (Perfil → APARÊNCIA) e é lembrada em `localStorage`, que a regra 4 proíbe para *token*, não para preferência de tela.

14.7 Versão: atualizar é ato deliberado

ATENÇÃO

`^0.1.11` **não** pega `0.2.0`: *caret* em `0.x` só sobe *patch*. "Última versão" é, portanto, ativo:

```
npm view @unooria/design-system version     # qual e a ultima publicada
pnpm up -r @unooria/design-system --latest  # sobe em ui/hub/exps de uma vez
# commitando o lockfile E os package.json juntos
```

14.8 Compensações locais

REGRA INVIOLÁVEL

Componente que falta no pacote não nasce na experiência. Vira *compensação local* marcada no CSS (comentário `(b) COMPENSAÇÃO`) e um pedido ao `storybook-unooria`. A compensação é **exceção controlada e decrescente**: cada bloco compensa uma lacuna real, anotada no comentário, e sai quando o pacote publicar a correção.

14.8.1 Lacunas abertas, verificadas na 0.1.14

Lacuna	Consequência
`unitube` não publica *toast*, diálogo, tabela nem estado vazio	Quatro compensações locais na exp-02.
`ChannelHeader` não tem `showSubscribe`	O botão de inscrever-se é *sempre* desenhado, e o canal da exp-02 é o do **próprio aluno**.
`.uni-channel-header__cover` usa a primitiva `--uni-color-neutral-50`, não um token semântico	No escuro vira uma laje branca.
`styles-unootube.css` repete o `@import` da Roboto no **meio** do arquivo	O Vite descarta com aviso; o do topo continua valendo, então a fonte carrega. É ruído, não bug.
Não existe componente de seleção (só `TextField`, `TextArea`, `PasswordInput`, `SearchInput`)	`<select>` do hub usa `.unooria-select`, com os tokens do pacote — mesma altura, raio e cores do `TextField`.

14.8.2 Avisos que ficaram obsoletos

NOTA

Dois avisos herdados do repositório de origem estão **obsoletos na 0.1.11**, medidos no pacote:

- o `MetricCard` **não inventa mais tendência** — a prop é `tagText?` e a documentação diz "sem ela a etiqueta não é renderizada";
- o `PositiveNegativeBarChart` tem escala **fixa** em ± 100, que é exatamente o que os eixos do circumplexo pedem — então a barra divergente *não* é mais escrita à mão.

O terceiro segue valendo: `AlertBox` é **diálogo** de 436 px; para faixa de contexto, o componente é `FeedbackAlert` (e a prop é `type`, não `variant`).

ATENÇÃO

Sobrescrever classe do pacote exige DOIS seletores (`.app-shell > .x`): o CSS do pacote é injetado pelo módulo do componente, ou seja, *depois* do CSS da experiência no grafo de importação. Com peso igual, quem ganha é sempre o pacote.

14.9 Cor da plataforma: classe escrita à mão

ATENÇÃO

A cor das superfícies da plataforma vem de classes escritas à mão em `packages/ui/styles/plataforma.css`, e o tema do Tailwind está **vazio** (`theme.extend: {}`) — de propósito, para que nada da marca institucional vaze para a superfície do aluno, nem por nome de classe.

A consequência é uma armadilha real: **classe de cor que não existe ali não é erro em lugar nenhum**. Em texto, ela herda a cor e passa desapercebida; em **fundo**, o elemento fica branco no branco. Foi o que aconteceu com `bg-unooria-neutral-900`: a letra da bolha ficou invisível. O *smoke* da exp-70 checa o `backgroundColor` **computado** do círculo por isso — `toBeVisible()` não pega.

Existem hoje apenas: `bg-unooria-{navy-800, neutral-50, neutral-100, neutral-200}`, `border-unooria-neutral-200` e `text-unooria-{danger, navy-900, neutral-500, neutral-700, neutral-800, neutral-950}`.

NOTA

O *preflight* do Tailwind fica **ligado**. Desligá-lo parecia inofensivo — o pacote traz o próprio CSS — mas o pacote estiliza **componentes**, não elementos: sem o *reset*, todo `<a>` volta a ser sublinhado e todo `<ul>` volta a ter marcador. Foi o que o *smoke* visual pegou nos *cards* de produto.

14.10 Exceção registrada

DECISÃO DE PROJETO

A exp-70 está fora da regra do *design system*, deliberadamente: é jogo em *canvas* 3D, sem DOM de interface (**ADR-18**). A pendência da exp-02 — que tinha CSS próprio em `src/estilos` — foi **resolvida**: ela consome a seção `unitube` pela ponte `@edu/ui/ds-unootube`, com a aparência escolhida pelo aluno.

15

Engenharia de entrega

15.1 O *monorepo* e o *build* incremental

DECISÃO DE PROJETO

***Monorepo* único, com pnpm *workspaces* + Turborepo** (ADR-02). A SDK é consumida por 48 aplicações: uma mudança nela precisa testar tudo no *mesmo pull request*, ou a incompatibilidade aparece num *bundle* publicado semanas depois. Repositórios separados só se houver estúdio externo produzindo experiência.

Listing 15.1 – `turbo.json` — o *pipeline* inteiro

```
{
  "tasks": {
    "build":     { "dependsOn": ["^build"],
                   "outputs": ["dist/**", ".next/**", "!.next/cache/**"] },
    "dev":       { "cache": false, "persistent": true },
    "lint":      { },
    "typecheck": { "dependsOn": ["^build"] },
    "test":      { "dependsOn": ["^build"] }
  }
}
```

O `dependsOn: ["^build"]` é o que faz o grafo funcionar: `typecheck` de uma experiência exige o `build` da SDK, então o *filtro* por mudança arrasta as dependências corretas.

Comando	Uso
`pnpm turbo build --filter=@edu/exp-01`	Builda uma experiência mais as dependências dela.
`pnpm turbo build --filter='...[HEAD^]'`	Só o que mudou. É o que o CI usa.
`pnpm turbo typecheck lint test`	O portão local da *Definition of Done*.

15.2 Integração contínua

Listing 15.2 – `.github/workflows/ci.yml`

```yaml
name: ci
on:
  pull_request:
  # Chamado pelo deploy.yml no push para main. NAO tem `push: main` proprio:
  # rodaria duas vezes no mesmo commit, e o deploy precisa ESPERAR por ele —
  # dois workflows em paralelo deixariam um teste vermelho subir para escola.
  workflow_call:

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 2 }
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm turbo typecheck lint test build --filter='...[HEAD^]'
```

DECISÃO DE PROJETO

O CI é chamado pelo *deploy* por `workflow_call`, e **não** tem gatilho próprio em `push: main`. Com gatilho próprio, ele rodaria duas vezes no mesmo *commit* — e, pior, em *paralelo* com o *deploy*, o que deixaria um teste vermelho subir para uma escola. Assim o *deploy* **espera** pelo portão.

15.3 Entrega contínua

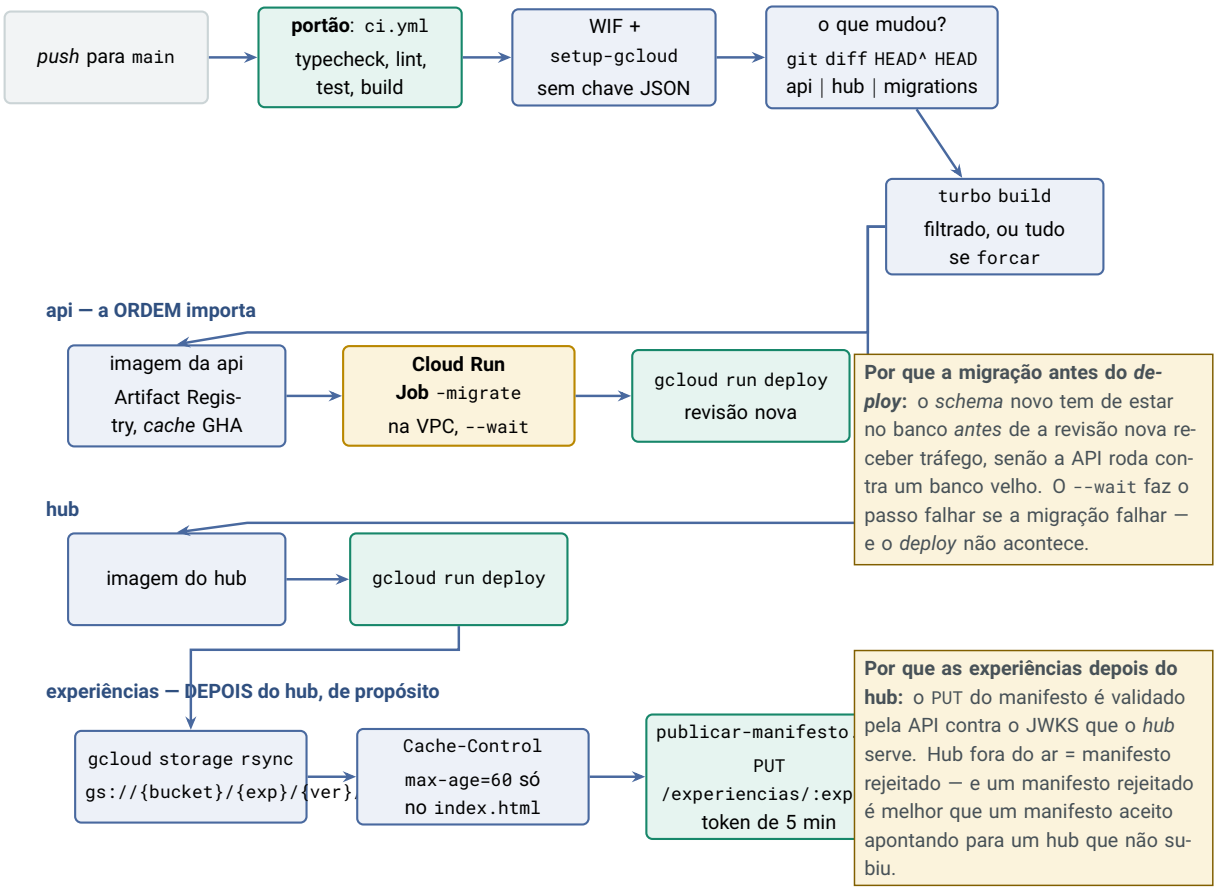


Figura 15.1 – O *pipeline* de entrega. Três trilhas independentes com uma ordem obrigatória entre elas, e um portão de qualidade antes de tudo.

15.3.1 Nada de nome de recurso no *workflow*

> **DECISÃO DE PROJETO**
>
> Tudo que muda de um contrato para outro entra por `vars` do repositório — o *workflow* **não sabe** o nome de nenhum recurso. Os valores saem de `terraform output`, e há um *script* que os configura (`scripts/configurar-github-ci.sh`).
> É isso que faz o mesmo *workflow* servir a `dev` e a produção, e ao próximo contrato, sem *fork*.

	Conteúdo
`GCP_PROJECT`, `REGION`	Projeto e região.
`AR_REPO`	Artifact Registry.
`API_SERVICE`, `HUB_SERVICE`, `MIGRATE_JOB`	Nomes dos serviços e do *job*.

`EXP_BUCKET`, `ASSETS_BASE_URL`	*Buckets* e base pública de *assets*.
`API_URL`, `HUB_ORIGIN`	Endereços públicos.
`WIF_PROVIDER`, `DEPLOY_SA`	Identidade federada do CI.

15.3.2 Concorrência e reexecução

Listing 15.3 – A política de concorrência do *deploy*

```yaml
# Dois deploys ao mesmo tempo brigam pela mesma revisao do Cloud Run e pelo
# mesmo banco. `cancel-in-progress: false` de proposito: abortar no meio de
# uma migration deixa schema e imagem fora de sincronia — melhor enfileirar.
concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false
```

> **NOTA**
>
> Há um `workflow_dispatch` com *input* `forcar`, para publicar tudo ignorando o *diff* — primeiro *deploy* de um contrato, ou reexecução depois de falha. Com `forcar`, o *build* também tem de ser de **todas** as experiências, senão o laço de publicação encontra pasta sem `dist/`.

> **NOTA**
>
> O laço de publicação é tolerante por desenho: pasta de experiência sem `package.json`, sem `public/experiencia.json` ou sem `dist/` é **pulada com aviso**, não é erro. Experiência incompleta é trabalho em andamento, e trabalho em andamento não deve quebrar o *deploy* do resto.

15.4 Ambientes locais

Modo	Comando	Quando usar
Banco em contêiner, resto no *host*	`docker compose up -d pg` + `pnpm dev`	Iteração rápida com *debugger* do editor. O caminho mais direto.
Tudo em contêiner, sincronizado	`pnpm docker:dev`	**Recomendado.** Banco, API, hub e as cinco experiências, cada um com o próprio *watcher*, com o código do *host* por *bind mount*: salvar um arquivo recompila lá dentro (API ∼1 s, hub e Vite na hora), sem `build` e sem `up` de novo.

Imagens congeladas	`docker compose up -d --build`	Aproximar-se da produção. Migrações e *seed* rodam sozinhos no primeiro `up`.

> **ATENÇÃO**
>
> As portas são as mesmas nos três modos (hub `:3000`, API `:3001`, experiências `:5173`–`:5177`), então **não dá para rodar `pnpm docker:dev` e `pnpm dev` ao mesmo tempo** — é um *ou* o outro.

Situação	O que fazer
Mexeu em dependência (`package.json` ou *lockfile*)	`pnpm docker:dev:deps` e reiniciar. O `node_modules` de cada contêiner é **volume próprio** — binário de macOS não roda no Linux da imagem — então instalar no *host* não basta.
Salvou um arquivo e nada recompilou	`DEV_POLLING=1 pnpm docker:dev`. O *inotify* pode não atravessar o *bind mount* em alguns *setups*.

15.5 O provedor de identidade sobe à parte

Listing 15.4 – O WSO2 tem ciclo de vida próprio

```
docker compose -f wso2/docker-compose.yml up -d     # IS :9443, banco dele :5433
docker compose -f wso2/docker-compose.yml watch     # olha conf/ e bootstrap/
```

> **NOTA**
>
> O `watch` é o equivalente do *hot reload* para um servidor Java: `conf/` já entra por volume (nada de *rebuild*), e mudar o `deployment.toml` **reinicia** o servidor — que é o ciclo mais curto possível ali, porque o *entrypoint* só copia o volume de configuração para o *server home* no *boot* (∼2 min). Mudar `bootstrap/` reexecuta o *bootstrap* (idempotente); só mudar o `Dockerfile` (o *driver* JDBC) refaz imagem.

15.6 Convenções de *git*

Convenção	Regra
Tronco	`main`. A migração para o *monorepo* foi mergeada nela em 2026-08-25 por *fast-forward*: 26 *commits*, histórico linear, sem *merge commit*.
Branches	**Uma por experiência**, com o nome do slug (`exp-07-fracoes`). Trabalho que não é experiência usa o prefixo do tipo do *commit*: `fix/quota-por-perfil`, `chore/deps`, `ci/deploy`.

Commits	`feat(exp-12): ...`, `fix(api): ...`, `chore(sdk): ...` — o escopo é o slug da experiência ou o aplicativo tocado.

ATENÇÃO

Nada de *build* no *git*. O `.gitignore` cobre `dist/`, `.next/`, `.next-prod/`, `.turbo/`, `*.tsbuildinfo`, `storage/`, `storage-privado/` e os *prints* dos *walkthroughs*. `apps/hub/.next-prod/` já entrou uma vez por um `git add -A` distraído: 358 arquivos e 250 MB de *cache* de *webpack* no histórico, que só saíram reconstruindo a *branch* inteira. **Stage por caminho, nunca `-A` sem olhar o `git status`.**

NOTA

`apps/hub/next-env.d.ts` **é** versionado (o Next espera isso), mas alterna entre `./.next` e `./.next-prod` conforme o último *build*. Modificado depois de um `build:isolado` = descartar, não *commitar*.

16

Qualidade

16.1 A *Definition of Done*

REGRA INVIOLÁVEL

Nenhuma tarefa está concluída sem estes seis itens. É o portão que substitui a revisão em par que uma pessoa não tem.

1. ***Typecheck* + *lint* + testes passando** (`pnpm turbo typecheck lint test`).
2. **Se mexeu em tela: *smoke* VISUAL** (`node scripts/walkthrough-aluno.mjs <pasta>` e `walkthrough-professor.mjs`) — *typecheck* verde não pega ícone virando texto, *link* sublinhado nem `iframe` vazio.
3. ***Input* externo validado com Zod.**
4. ***Queries* novas escopadas por *tenancy*** (revisar mentalmente a regra 1).
5. **Sem segredo nem URL interna em código de cliente.**
6. **Se tocou contrato** (ponte, telemetria, *provider*, manifesto): **documentação atualizada** — em `CLAUDE.md` e neste documento.

16.2 Estratégia de testes

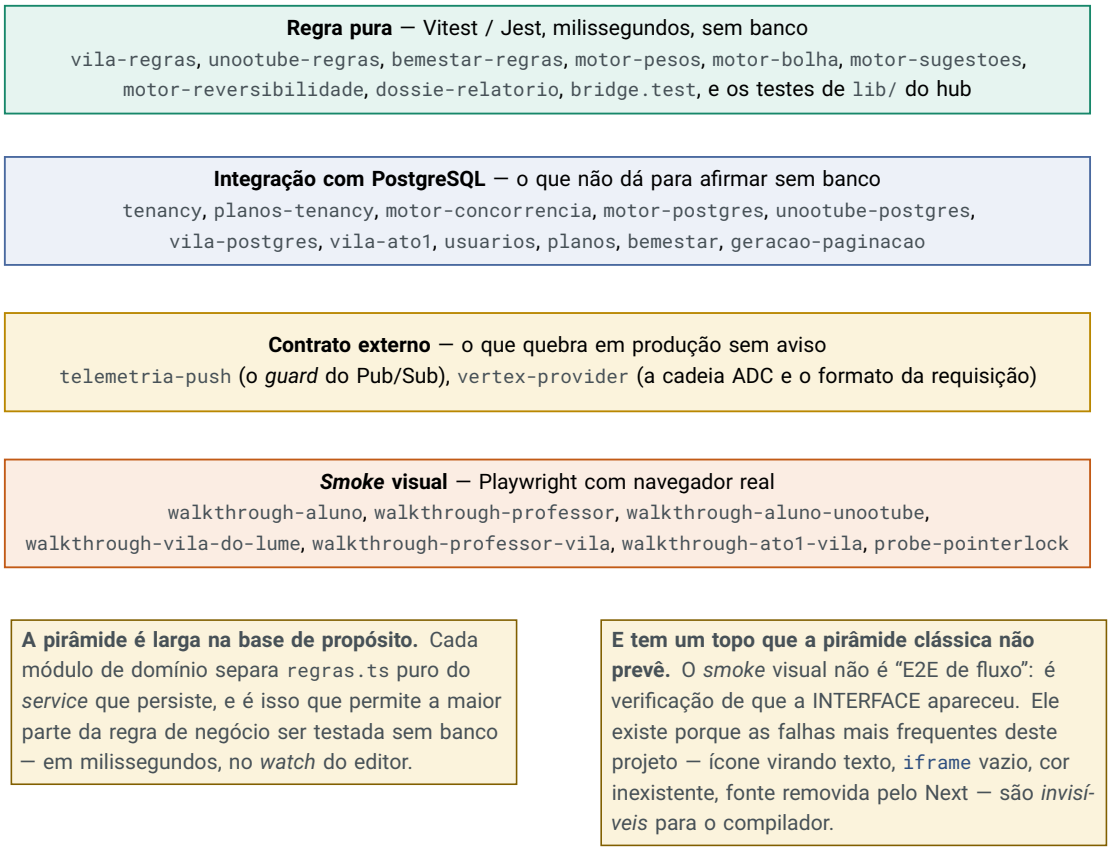


Figura 16.1 – As quatro camadas de verificação. A quarta é incomum e é a mais importante deste projeto: ela pega a classe de falha que o *typecheck* estruturalmente não vê.

Camada	Ferramenta	Mínimo obrigatório
Regra pura	Vitest (SDK, experiências) / Jest (NestJS)	Contratos: ponte, telemetria, *guards* de *tenancy*. Toda função de `regras.ts`.
Integração	Jest + PostgreSQL de teste	*Tenancy*, *lock*, concorrência, idempotência.
Contrato externo	Jest com *fake* de rede	*Guard* do Pub/Sub e formato da requisição ao Vertex.
Smoke visual	Playwright, navegador real	Toda mudança de tela.

16.3 O *smoke* visual, em detalhe

Listing 16.1 – Os *walkthroughs* disponíveis

```
node scripts/walkthrough-aluno.mjs      testes/wt-aluno
node scripts/walkthrough-professor.mjs  testes/wt-professor   # cobre a exp-90 embedada
node scripts/walkthrough-aluno-unootube.mjs   testes/walkthrough-unootube   # aluno4a01
node scripts/walkthrough-vila-do-lume.mjs     testes/walkthrough-vila       # usa o CHROME do sistema
node scripts/walkthrough-professor-vila.mjs   testes/wt-prof-vila 8A
node scripts/walkthrough-ato1-vila.mjs        testes/wt-ato1
node scripts/probe-pointerlock.mjs
```

ATENÇÃO

Dois *walkthroughs* exigem o **Chrome do sistema**, não o Chromium do Playwright: o da exp-70 e o *probe* de *pointer lock*. O Chromium empacotado **não concede *pointer lock* nem no topo da página**, então usá-lo produziria um falso negativo — o teste diria que a permissão de *sandbox* não funciona quando o problema é o navegador.

NOTA

Os *walkthroughs* precisam da plataforma no ar (contêineres ou `pnpm dev`) e gravam *prints* na pasta indicada — que está no `.gitignore`. A senha de todos os usuários de demonstração é `unooria123`; os *logins* do *seed* são `gestor.demo`, `prof.demo` e `aluno01`–`aluno40`.

16.3.1 O que só o *smoke* visual pega

Falha	Por que o *typecheck* não vê
Ícone virando texto	O Next remove `@import` externo de CSS importado por JS, *silenciosamente*. `Material Symbols Rounded` é fonte de ligadura, então o sintoma é "addInvestigação" no lugar do ícone. O `getComputedStyle` *responde* "Open Sans" — a regra continua no CSS.
Cor inexistente em fundo	Classe de cor que não existe em `plataforma.css` não é erro em lugar nenhum. Em texto herda a cor; em **fundo** o elemento fica branco no branco. O *smoke* da exp-70 checa o `backgroundColor` **computado** — `toBeVisible()` não pega.
`iframe` vazio	*Handshake* que expira, *dev server* fora do ar, manifesto ausente: o componente monta, o `iframe` existe, e não há nada dentro.
Experiência esticada na janela	`viewport` não lido (manifesto vazio + `EXP_INTERNAL_HOST` errado). Nenhum tipo muda.
Link sublinhado, marcador de lista de volta	*Preflight* do Tailwind desligado. O pacote estiliza *componentes*, não elementos.
Câmera do jogo que não anda	`allow-pointer-lock` ausente: `requestPointerLock()` morre com `SecurityError` e nada no console diz por quê.

> **ATENÇÃO**
>
> **Testes de navegador precisam de portão de hidratação.** Em desenvolvimento, o SSR chega *antes* do *bundle*, e o `fill` do Playwright não move o estado do React — o campo mostra o texto e o `onChange` nunca rodou. Todo *walkthrough* espera um sinal de hidratação antes de interagir.

16.4 Análise estática

Regra	Aplicação
TypeScript *strict* em tudo	`tsconfig.base.json`, herdado por todos os *workspaces*. Inclui `noUncheckedIndexedAccess` — é por isso que o código de curadoria usa `!` em índices provadamente válidos, com comentário.
Sem `any`	Use `unknown` + *narrow*. Uma exceção, registrada em ADR: `exp-70/public/jogo/**` está nos `ignores` do ESLint.
Zod em todo *input* externo	Corpo, *query*, `postMessage` recebido, variáveis de ambiente.
ESLint	`eslint.config.mjs` na raiz, com `typescript-eslint`, `eslint-plugin-react-hooks` e `@next/eslint-plugin-next`.

> **ATENÇÃO**
>
> O ESLint **ignora a pasta `storage`**. Um módulo com esse nome passaria no *lint* sem ser lido — foi por isso que o módulo de armazenamento de objetos se chama `object-storage`, e não `storage`.

16.5 Armadilhas de ferramenta registradas

Cada linha abaixo é um erro efetivamente cometido, com o sintoma que ele produz. Estão aqui porque o sintoma nunca aponta para a causa.

Armadilha	Sintoma, e o que fazer
Build isolado apagando o `.next` do *dev*	Sintoma: "tela branca" ou "CSS bugado" depois de validar um *build* de produção. Validar sempre com `pnpm build:isolado` (`HUB_DIST_DIR=.next-prod`).
Standalone do Next não copia `static` nem `public`	Sintoma: CSS 404 e página sem estilo. Distinguir do caso acima pelo **código** do erro, não pela aparência.
Substituição de texto sem asserção	Edição por *replace* que não casa passa em **silêncio**, e o `tsc` não acusa. Sempre afirmar o casamento.

Arquivo em `public/` escrito em *runtime*	Dá 404: `public/` é do *build*. E `Cache-Control` de rota dinâmica só cola via `next.config`.
Probe de *hot reload* como comentário	O *esbuild* remove comentários — o falso negativo faz parecer que o *inotify* não atravessa o *bind mount*.
`sanitize-html` recente é ESM-only	Fixar `~2.13.1` na API (CJS, Node 20).
Design system quebrando o *dev server* do Vite	ESM importando diretório: o *build* passa, o *dev* morre. Sintoma: `iframe` vazio ou `ERR_CONNECTION_REFUSED`.
Vite pulando de porta	Uma segunda instância da mesma experiência ocupa a porta seguinte — e o hub passa a ler o manifesto da experiência errada.

16.6 Cobertura: onde está e onde não está

Área	Cobertura	Observação
Tenancy e RLS	✓ Forte	Teste com *role* sem *ownership*; o caso mais crítico do sistema.
Regra de domínio	✓ Forte	`regras.ts` puro em todos os módulos.
Concorrência do motor	✓ Forte	*Lock* consultivo, reversibilidade, paginação da geração.
Contrato da ponte	✓ Forte	*Origin*, esquema, versão maior.
Guard do Pub/Sub	✓ Forte	Inclui o caso *fail closed*.
Provider de IA	✓ Parcial	Cadeia ADC e formato da requisição. Não há teste contra o Vertex real — e não deveria haver.
Telas	✓ *Smoke*	Sete *walkthroughs*; não é cobertura de interação, é verificação de que apareceu.
Carga	**– Ausente**	O teste de 4.000 *logins* em dez minutos é obrigatório antes de novembro de 2026 e não foi executado. Risco `R-01`.
Segurança externa	– Ausente	Sem revisão externa nem teste de intrusão. Risco `R-06`.
Acessibilidade	– Ausente	Nenhuma verificação automatizada de contraste, foco ou leitor de tela. Risco `R-07`.

LIMITAÇÃO CONHECIDA

Não há métrica de cobertura de linha configurada, e não há meta numérica. A escolha é deliberada: com uma pessoa desenvolvendo, uma meta de percentual incentiva testar o que é fácil em vez do que é caro se falhar. O critério em uso é o da *Definition of Done* — contratos, *tenancy* e regra pura — e ele é verificável por inspeção da lista de arquivos de teste.

17

Registro de decisões arquiteturais

Cada decisão abaixo segue a forma de um ADR (NYGARD, 2011): contexto, decisão, alternativas descartadas, consequências — inclusive as ruins — e o gatilho que faria reavaliá-la. Decisão sem alternativa descartada e sem gatilho de reavaliação não é decisão registrada: é justificativa.

17.1 Índice

ID	Decisão	Data	Estado
ADR-01	Experiências como *bundles* estáticos, não serviços	2026-08	Aceita
ADR-02	*Monorepo* único (pnpm + Turborepo)	2026-08	Aceita
ADR-03	Autenticação por OIDC genérico	2026-08	Aceita
ADR-04	Uma *stack* por contrato; escolas por RLS	2026-08	Aceita
ADR-05	Cloud SQL HA, não AlloyDB	2026-08	Aceita
ADR-06	Pub/Sub para telemetria desde o início	2026-08	Aceita
ADR-07	IA atrás de um *gateway* na própria API	2026-08	Aceita
ADR-08	RLS forçada e uma porta única de banco	2026-08-24	Aceita
ADR-09	Domínio por experiência na API; gabarito *server-side*	2026-08-24	Aceita
ADR-10	Bem-estar: DASS-21 fixo, IA só no entorno, sem visão de gestor	2026-08-25	Aceita
ADR-11	Telemetria sem PII e vocabulário fechado no *payload*	2026-08-26	Aceita
ADR-12	`grau` como invariante de API, não regra de tela	2026-08-26	Aceita
ADR-13	Retenção sem prazo declarado, em troca de reprocessamento	2026-08-26	Aceita
ADR-14	Telemetria no PostgreSQL particionado; sem BigQuery, sem Redis	2026-08-27	Aceita

ADR-15	O hub assina os *tokens* enquanto o IdP central não entra	2026-08	Aceita, transitória
ADR-16	Credencial por ADC; modelo `3.6-flash` em `global`	2026-08	Aceita, com custo
ADR-17	*Claim* atômico em vez de fila externa	2026-08	Aceita
ADR-18	Exceções registradas: jogo em JS clássico, exp-70 fora do DS	2026-08-26	Aceita
ADR-19	*Design system* externo como fonte única de interface	2026-08	Aceita
ADR-20	Um banco, um *schema*; prefixo `exp_NN_`	2026-08-27	Aceita
ADR-21	*Push subscription* para a própria API, sem *worker*	2026-08	Aceita

17.2 ADR-01 — Experiências como *bundles* estáticos

> **Contexto**
>
> O catálogo alvo é de 48 experiências, cada uma uma aplicação React independente. O pico de carga é uma parede às oito da manhã, e o grosso dos *bytes* é o código e a mídia dessas aplicações. Uma pessoa opera tudo.

Decisão. Cada experiência é um *bundle* Vite 100% estático, publicado em `gs://{contrato}-exp/{expId}/{v` e servido pela CDN. Não há processo, porta, nem *scaling* por experiência. O hub monta a URL do `iframe` com `versaoAtiva` do manifesto.

Alternativas descartadas.

- *Microsserviço por experiência*: 48 serviços com *deploy*, log, escala e *alerting* próprios — para servir arquivos estáticos.
- *Rotas do hub monolítico*: cada experiência entraria no *bundle* do hub, e o hub cresceria sem limite. Uma mudança em qualquer experiência exigiria *deploy* do hub inteiro.

Consequências.

- Custo marginal de publicação ≈ armazenamento + tráfego de CDN.
- O caminho carrega a versão ⇒ o *bundle* é imutável e pode ficar um ano na borda.
- ***Rollback* é um `UPDATE`**: trocar `versaoAtiva`. Sem *build*, sem imagem, sem *deploy*. É a operação de emergência mais rápida da plataforma.
- *Ruim*: a experiência não tem *backend* próprio, então toda lógica de domínio dela vive na API — o que concentra responsabilidade num serviço só (mitigado por **ADR-09**).
- *Ruim*: um *bundle* publicado não é reconstruído quando a API muda, o que torna os contratos do Capítulo 9 rígidos.

Gatilho de reavaliação. Uma experiência que precise de execução no servidor por natureza — multiplayer em tempo real com autoridade de servidor, ou processamento pesado por sessão.

17.3 ADR-02 — *Monorepo* único

Contexto. A SDK é consumida por 48 aplicações. O *design system* é externo, mas a ponte é interna.

Decisão. Um *monorepo* com pnpm *workspaces* e Turborepo (Vercel, 2026). `apps/*`, `apps/experiencias/*`, `packages/*`.

Alternativa descartada. Repositório por experiência, com a SDK publicada em npm. Uma mudança na SDK só seria testada contra as 48 aplicações depois de publicada — e a incompatibilidade apareceria num *bundle* servido a alunos semanas depois.

Consequências. Mudança na SDK testa tudo no mesmo *pull request*; o CI builda só o que mudou (`--filter='...[HEAD^]'`). *Ruim*: o repositório cresce, e `pnpm install` completo é lento.

Gatilho. Estúdio externo produzindo experiência — aí a experiência sai para repositório próprio e passa a consumir a SDK publicada.

17.4 ADR-03 — Autenticação por OIDC genérico

Contexto. O WSO2 Identity Server é exigência do cliente e da estratégia multi-contrato: é único para várias redes.

Decisão. A aplicação **não conhece o WSO2**: conhece OIDC. A API valida *token* por `OIDC_ISSUER`, `OIDC_JWKS_URI` e `OIDC_AUDIENCE`. Trocar de provedor de identidade é trocar essas variáveis.

Alternativa descartada. SDK do WSO2 na aplicação. Amarraria o código a um produto que pode mudar por decisão comercial, e tornaria o teste local dependente de um servidor Java em *bootstrap*.

Consequências. `OIDC_ISSUER` e `OIDC_JWKS_URI` vão como *env* em texto plano (não são segredo: são URLs públicas), derivadas de `hub_domain` — e é isso que garante que o `iss` do *token* e o esperado pela API **não podem** divergir.

Gatilho. Nenhum previsto. É a decisão mais estável do conjunto.

17.5 ADR-04 — Uma *stack* por contrato; escolas por RLS

Contexto. Cada contrato é uma relação jurídica com um ente federativo; cada contrato tem 100+ escolas.

Decisão. Isolamento **físico** entre contratos (*stack* GCP inteira, por *module* Terraform) e **lógico** entre escolas (`escola_id` + RLS).

Alternativas descartadas.

- *Uma stack por escola*: cem infraestruturas por contrato, operadas por uma pessoa. E o custo fixo de uma escola pequena passaria a ser o de uma rede.
- *Uma stack para todos os contratos*: um incidente num município seria um incidente em todos, e o desenho não sobreviveria à primeira análise jurídica de um edital.

Consequências. `escola_id` em toda tabela de dado de aluno, RLS forçada, e `admin-contrato` como única exceção cross-escola. *Ruim*: cada contrato novo é uma *stack* nova para provisionar, monitorar e faturar.

Gatilho. Exigência contratual de isolamento físico *por escola* — o que mudaria o modelo comercial, não só a arquitetura.

17.6 ADR-05 — Cloud SQL HA, não AlloyDB

Decisão. PostgreSQL gerenciado (Cloud SQL) com HA, PITR e *pooler* de conexões obrigatório.

Alternativa descartada. AlloyDB. **Motivo:** a robustez que este sistema precisa vem de HA + PITR + *pooler*, não do *engine*. Ambos são PostgreSQL, então migrar depois é trivial se a carga justificar.

Consequências. O *pooler* é **obrigatório**: o *autoscaling* do Cloud Run multiplica *pools* do Prisma e estouraria `max_connections`. E o banco só tem IP privado — o que obriga a migração a rodar como Cloud Run Job dentro da VPC.

Gatilho. Carga medida que sature uma instância de Cloud SQL com *pooler* bem dimensionado.

17.7 ADR-06 — Pub/Sub para telemetria desde o início

Contexto. No pico, quatro mil alunos simultâneos gerando evento a cada poucos segundos.

Decisão. O canal *analítico* de telemetria vai para o Pub/Sub. **O resto da aplicação fala PostgreSQL direto** — inclusive o canal *operacional* do motor.

Alternativa descartada. Tudo síncrono no PostgreSQL. Colocaria um `INSERT` sincronizado por interação de aluno dentro da mesma transação que serve a tela.

Consequências. Uma peça de infraestrutura a mais — e é a única exceção declarada à regra "nada de fila sem gargalo medido", junto com **ADR-07**. Em troca: o pico é amortecido por um *buffer* **durável** (ver **ADR-14**).

Gatilho. Nenhum. É decisão tomada de antemão e mantida.

17.8 ADR-07 — IA atrás de um *gateway* na própria API

Decisão. Toda chamada de IA passa por `apps/api/src/ai`, com interface neutra de *vendor*, *prompt* de sistema no servidor, quota diária persistida por perfil e *rate limit*. Pré-gerar em lote sempre que a pedagogia permitir.

Alternativas descartadas.

- *Chamada direta do bundle*: exporia chave e *prompt* de sistema, e tornaria a quota incontrolável.
- *Serviço de IA separado*: um Cloud Run a mais para operar, sem ganho — a API já tem CPU sempre alocada.

Consequências. Nenhum módulo fora de `src/ai` conhece o *vendor*; trocar de provedor é criar um arquivo. *Ruim*: a API carrega o custo de latência das gerações longas, o que obriga o padrão 202 + *polling*.

Gatilho. Volume de IA que justifique isolar o consumo, ou requisito de GPU própria.

17.9 ADR-08 — RLS forçada e uma porta única de banco

Decisão. `ENABLE` e `FORCE ROW LEVEL SECURITY` em toda tabela de dado de aluno; acesso ao banco **exclusivamente** por `PrismaService.comTenancy`, que abre transação e executa `set_config('app.es` `..., true)` antes do trabalho.

Alternativas descartadas.

- *Apenas filtro na aplicação*: uma *query* nova que esqueça o filtro devolve dado de outra escola, e ninguém percebe.
- *Apenas RLS*: um erro de migração desliga a proteção inteira.
- *Duas roles* (dona para migração, sem *ownership* para a aplicação): desnecessário com `FORCE`, que sujeita o próprio dono às políticas.

Consequências. Duas trancas independentes. O terceiro argumento `true` do `set_config` é **crítico**: local à transação, então a conexão volta limpa ao *pool*. `comTenancy` sem escola **lança** — não consulta sem filtro. *Ruim*: toda leitura, inclusive trivial, abre transação.

Gatilho. Nenhum. É a decisão que sustenta o produto.

17.10 ADR-09 — Domínio por experiência na API; gabarito *server-side*

Contexto. A exp-02 (2026-08-24) foi a primeira experiência com gabarito.

Decisão. Não há módulo genérico de avaliação. Cada experiência com lógica de domínio ganha um módulo próprio na API (`src/<slug>/`), com `catalogo.ts` + `regras.ts` + *service* + *controllers* por papel. O gabarito nunca vai ao *bundle*; a API corrige.

Alternativa descartada. Um módulo genérico de avaliação com "itens" e "respostas corretas". Uma API genérica de correção convida a mandar a resposta certa junto — e cada experiência tem uma mecânica que não cabe num formato comum.

Consequências. Decisões concretas herdadas desta ADR: ranking com colegas **fictícios** (sem pontuação de menor real exposta a outro menor); URLs das fontes como **texto** (o *sandbox* bloqueia *popups*, e o aluno não sai da plataforma); salvos **congelados** depois da conclusão (nota e estado não podem divergir); aluno roteado pela **série**. *Ruim*: um módulo novo por experiência — 48 módulos no limite.

Gatilho. Se três experiências consecutivas tiverem mecânica idêntica, extrair o comum. Nunca antes.

17.11 ADR-10 — Bem-estar: instrumento fixo, IA só no entorno, sem gestor

Contexto. 2026-08-25. Um *check-in* de bem-estar *do professor*, com o DASS-21.

Decisão. O instrumento entra ***verbatim*** (versão brasileira de Vignola & Tucci (VIGNOLA; TUCCI, 2014)) em microdoses de três itens por dia. A IA gera abertura, pergunta aberta e acolhimento, atrás de `regras.validarTextoDaIa`, com *fallback* fixo, e **nunca vê** pontuação, nome ou escola. Correção *server-side* ao sétimo *check-in*. Item sentinela (17 ou 21 ≥ 2) ou termo de risco $\Rightarrow$ mensagem fixa de apoio, sem IA.

Por ser dado de saúde (LGPD art. 11): não há visão de gestor nem agregado por escola. RLS + posse por `professor_id`, e o próprio professor apaga tudo.

Alternativas descartadas.

- *IA reescrevendo os itens* para "soar mais natural": quebra a validação psicométrica e invalida as faixas de corte.
- *Questionário completo de uma vez*: engajamento vem de custar menos de um minuto, não de ser curto uma vez.
- *Painel agregado para a rede*: sai de um `GROUP BY` e é exatamente o que não se deve fazer com dado de saúde de uma equipe pequena, onde *n* baixo reidentifica.

Consequências. O *check-in* nunca deixa de abrir. *Ruim*: a rede não tem nenhuma visão de bem-estar dos professores — o que é uma perda real de informação de gestão, aceita.

Gatilho. Uma visão agregada e anonimizada é decisão futura, com DPA e *k*-anonimato explícito.

17.12 ADR-11 — Telemetria sem PII e vocabulário fechado

Decisão. A telemetria carrega `aluno_id` **opaco**, e a identidade é derivada do *token* no servidor — o que o cliente enviou nesses campos é descartado. Em `exp_70_sessoes.payload`, `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**.

Alternativa descartada. Campo `texto` livre no *payload* (que existia). Era o único canal de texto arbitrário — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração.

Consequências. Trava **estrutural**, não lista de proibidos: não há como enviar texto arbitrário, então não há como revisar mal. *Ruim*: um evento novo que precise de rótulo textual exige acrescentar o rótulo ao vocabulário — ou seja, exige uma decisão explícita.

Gatilho. Nenhum.

17.13 ADR-12 — `grau` como invariante de API

Contexto. O mapeamento comportamento → valor de Schwartz é **hipótese de *design*, não instrumento validado**: Schwartz se valida por questionário (PVQ para adultos, PBVS-C para crianças (SCHWARTZ, 2012; DöRING et al., 2010)), não por telemetria de jogo.

Decisão. A API devolve uma **união discriminada** em que só a variante `scored` tem `numero: number`; as outras têm `null` **por tipo**. E a distinção entre `zero` ("teve a chance e não foi", entra na média) e `none` ("a oportunidade não chegou", sai da conta) é preservada em todo o caminho.

Alternativa descartada. Manter a decisão na camada de interface, com um comentário avisando. Foi assim no aplicativo de origem, e qualquer tela nova podia imprimir "Poder 100" a partir de um sinal só. **Prosa não é controle; um tipo é.**

Consequências. O hub fica estruturalmente incapaz de desenhar o número errado — ele nunca o recebe, e o compilador nem deixa procurar. Decisões herdadas: o relatório abre pelo **nível** do instrumento antes do desempenho; a listagem de turma **não** mostra valor psicométrico (listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças); `cartoesServidos` $\neq$ "apareceram".

Gatilho. Se um instrumento validado por questionário for adotado, a união ganha uma variante — não perde a distinção.

17.14 ADR-13 — Retenção sem prazo declarado

Contexto. 2026-08-26. Havia `expires_at` de 90 dias no *schema* da exp-70 e **nenhuma rotina que o cumprisse**.

Decisão. A coluna **saiu** (`20260826170000_vila_sem_prazo`). Não há prazo de retenção declarado para `exp_70_sessoes` nem para `telemetria_eventos`.

Alternativa descartada. Manter a coluna. **Coluna de prazo que ninguém lê é pior que prazo nenhum**, porque quem lê o *schema* acredita nela — é a mesma armadilha de um campo `consentimento_em` que existia no aplicativo de origem: a regra saiu do código, a coluna ficou, e todo mundo continuou acreditando.

Consequências. Ganha-se o **reprocessamento**, que é requisito real do instrumento: `payload` é a única fonte para recomputar o perfil quando `VERSAO_MAPEAMENTO` subir, e 90 dias matavam exatamente a comparação entre duas gerações do mapeamento. *Ruim, e é explícito*: dado comportamental de menor fica armazenado indefinidamente, e isso é uma exposição de conformidade em aberto.

Gatilho. Exigência contratual de prazo. Quando vier, ele volta **junto com a rotina que o cumpre** — e, com a tabela particionada (**ADR-14**), essa rotina é `DROP TABLE particoes.<mês>`. A pergunta é da plataforma inteira, não de um módulo.

17.15 ADR-14 — Telemetria no PostgreSQL particionado

Contexto. 2026-08-27. Avaliou-se mandar o canal analítico para o BigQuery por *subscription* nativa do Pub/Sub. Mediu-se também o custo real do caminho de gravação.

Decisão. O canal analítico grava no **mesmo PostgreSQL**, com `INSERT` multi-linha e `telemetria_eventos` particionada por mês. **Sem BigQuery. Sem Redis.**

Alternativas descartadas.

- *BigQuery*: os relatórios do professor **não leem** `telemetria_eventos` (leem `exp_01_events`), então não resolvia problema nenhum hoje — e criava uma segunda plataforma de dados para operar.
- *Redis como buffer*: quem amortece o pico já é o Pub/Sub, e ele é **durável**. Um *buffer* em memória perde os eventos do processo que morre antes do *flush*, ou seja, reintroduz a perda que o Pub/Sub eliminou. **O gargalo medido era o laço de `INSERT`** — um lote de 50 custava 50 *round-trips*, o que no pico dava ∼800 `INSERT`/s; em lote, ∼16/s.

Consequências. A chave primária é composta `(event_id, ts)`, porque o PostgreSQL exige a chave de partição em todo índice único — e a *dedupe* segue correta, porque a reentrega traz o mesmo `ts`. As partições vivem no *schema* `particoes`, fora do alcance do Prisma. Há partição `DEFAULT` como rede de segurança, porque evento recusado é telemetria perdida em silêncio. *Ruim*: as faixas vão até 2028-12 e **não há *job* agendado** — criar partição é item de operação.

Gatilho. Volume que sature o PostgreSQL, ou um relatório que efetivamente precise de análise sobre `telemetria_eventos` em escala.

17.16 ADR-15 — O hub assina os *tokens* de experiência

Contexto. O IdP central (WSO2) tem ciclo de vida próprio e não estava pronto quando as experiências precisaram de *token*.

Decisão. O **hub** assina os *tokens* de experiência (RS256, TTL 15 min, *claims* `{sub, escola_id, turma_id?, perfil}`, `aud plataforma-api`) e serve o JWKS em `/.well-known/jwks.json`. A API valida por `OIDC_ISSUER` e `OIDC_JWKS_URI`.

Alternativa descartada. Esperar o WSO2. Faria toda a plataforma depender de um servidor Java em *bootstrap* para que qualquer experiência pudesse ser testada.

Consequências. Trocar para o WSO2 é trocar duas variáveis — o `OidcGuard` não muda uma linha. **Efeito colateral útil:** o CI usa a mesma chave para assinar o *token* de cinco minutos do *upsert* de manifesto, o que é o motivo de as experiências serem publicadas *depois* do hub. *Ruim*: o hub acumula responsabilidade de emissor, e `TOKEN_SIGNING_KEY_PEM` passa a ser obrigatória em produção.

Gatilho. WSO2 pronto e configurado por contrato. A ADR é **transitória por desenho**.

17.17 ADR-16 — ADC como credencial; modelo *versus* região

Decisão, parte 1 — credencial. O *provider* de IA obtém o *token* com `google-auth-library`, que resolve `GOOGLE_APPLICATION_CREDENTIALS` → arquivo de `gcloud auth application-default login` → *metadata server*. **Nunca chave de API** — não existe variável para ela no projeto.

Contexto. Antes havia *só* o *metadata server*, e por isso *nenhuma* chamada de IA funcionava fora do Cloud Run. Em desenvolvimento a API ficava presa em `mock` e a exp-90 devolvia um plano **falso** — o que é pior que falhar, porque parecia funcionar.

Decisão, parte 2 — modelo e região. Modelo `gemini-3.6-flash`, localização `global`, para texto *e* para plano de aula.

Medição. O `3.6-flash` dá **404** em `southamerica-east1`, `us-east1` e `us-central1`. Só o endpoint `global` o serve.

Consequência aceita, e ela é séria. O tráfego de IA sai de São Paulo, inclusive o do aluno, e isso precisa constar do contrato e do DPA da rede.

Gatilho. Exigência de residência de dados. Voltar à região é trocar `VERTEX_LOCATION` **e** o modelo, juntos — a região publica a família 2.5, não a 3.x.

Decisões relacionadas. `VERTEX_THINKING_BUDGET = 0`: nos modelos 2.5 e 3.x os *tokens* de pensamento saem do `maxOutputTokens`, e com pensamento livre o plano anual voltava **vazio** com `finishReason: MAX_TOKENS`. Teto de saída do plano em **64k**, não 32k. Modelo de imagem `gemini-3.1-flash-image`, com proporção `4:5` **pedida** ao modelo — o *default* anterior não existe mais, e sem pedir a proporção o `cover` descarta metade da largura.

17.18 ADR-17 — *Claim* atômico em vez de fila externa

Contexto. Duas rotinas longas e assíncronas: geração de *feed* (exp-01) e geração de plano de aula (exp-90). Duas ou três instâncias do Cloud Run rodando ao mesmo tempo.

Decisão. *Claim* atômico na própria linha (`UPDATE ... WHERE status = 'pendente'`, com contagem de linhas afetadas) e *worker in-process*. Para o motor de *feed*, *lock* consultivo por `run` (`pg_advisory_xact_lock`).

Alternativas descartadas.

- *Fila externa* (Pub/Sub, Cloud Tasks): peça a mais para operar, para um problema que um `UPDATE` condicionado resolve.
- *Sem lock no motor*: dois pedidos simultâneos de bloco (duas abas, *reload* no meio do *fetch*, *retry*) se sobrescreviam e um bloco inteiro desaparecia do estado — posts entregues voltavam a ser servidos e a série por bloco do dossiê ficava corrompida.

Consequências. A API precisa de **CPU sempre alocada** e `min-instances` ≥ 1, porque há trabalho fora do ciclo de requisição. Retomada de órfã é disparada pelo *polling* do painel, não por *scheduler*. *Ruim*: se todas as instâncias morrerem e ninguém abrir o painel, uma geração órfã fica parada até que alguém abra.

Gatilho. Primeira reclamação real de geração parada.

17.19 ADR-18 — Exceções registradas

Contexto. Duas regras da plataforma têm exceção, e exceção não registrada vira precedente.

Exceção 1 — `any` e ESLint. `apps/experiencias/exp-70-vila-do-lume/public/jogo/` — cerca de dez mil linhas de JavaScript clássico com globais compartilhadas e `three.min.js` vendorizado — fica como está, e está nos `ignores` do ESLint. **Motivo:** é um instrumento com distâncias e tempos calibrados, e o compilador não tem como saber que uma refatoração mudou a *medida*. O *shim* (`src/`, TS *strict*) é a única porta: `window.VILA`. A rede que substitui o `tsc` ali é o *smoke* visual.

Exceção 2 — *design system*. A exp-70 não usa o `@unooria/design-system`. **Motivo:** é jogo em *canvas* 3D, sem DOM de interface.

Consequências. As duas exceções são **nominais** — valem para uma pasta e uma experiência, não para uma categoria. Qualquer outra experiência que peça a mesma exceção precisa de uma ADR nova.

17.20 ADR-19 — *Design system* externo como fonte única de interface

Decisão. Todo elemento de interface vem do `@unooria/design-system`, consumido pela ponte `packages/ui`. Componente que falta **nasce no repositório do *design system*** e chega por versão nova — não como arquivo local.

Alternativa descartada. Componentes locais em `packages/ui`. Com 48 experiências, um componente local viraria 48 divergências — e a fidelidade ao clone é o que a experiência *mede*.

Consequências. Duas famílias no mesmo pacote que **nunca se cruzam**; a linha entre elas é a **revelação**, não o arquivo. Compensação local só como exceção marcada no CSS, com pedido correspondente ao repositório do *design system*. *Ruim*: uma lacuna do pacote bloqueia a experiência até a próxima publicação — e `^0.1.x` não sobe *minor*, então atualizar é ato deliberado.

Gatilho. Nenhum. É decisão de produto e de marca, não de engenharia.

17.21 ADR-20 — Um banco, um *schema*; prefixo `exp_NN_`

Decisão. Um banco, um *schema* lógico. Tabela de experiência leva `exp_NN_`; tabela de plataforma fica sem prefixo; tabela que é contrato entre duas experiências carrega **os dois números** (`exp_01_exp_70_perfis_importados`).

Alternativas descartadas.

- *Banco por experiência*: elas **não** são isoladas — o perfil da Vila alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos. E um *pool* do Prisma por experiência estouraria `max_connections`.
- *Schema por experiência*: a única coisa que daria além do prefixo é fronteira de `GRANT`, e existe **uma** *role* de aplicação.

Consequências. Migração de referência `20260827090000_tabelas_por_experiencia`: `RENAME`, nunca `DROP`/`CREATE`, renomeando *constraints*, índices e políticas junto. Duas armadilhas registradas lá: `FOR ... IN SELECT` sobre o catálogo **relê o que o próprio laço alterou**, e `RENAME CONSTRAINT` renomeia **também** o índice que a sustenta — as duas produzem `exp_01_exp_01_..._pkey`.

Gatilho. Necessidade de fronteira de `GRANT` por experiência — o que só faria sentido com mais de uma *role* de aplicação.

17.22 ADR-21 — *Push subscription* para a própria API

Decisão. O canal analítico é consumido por *push subscription* direto num *endpoint* da própria API (`POST /telemetry/ingest`), e não por um `{contrato}-telemetry-worker` separado.

Alternativa descartada. *Worker* dedicado, como previa o desenho original de infraestrutura. Seria um Cloud Run a mais com *deploy*, log, escala e *alerting* próprios.

Consequências. A rota grava dado de aluno e **não** passa pelo `OidcGuard`, então precisa de tranca própria: `PubSubPushGuard` verifica emissor Google, `aud` e `email` da conta de serviço de *push*, e **falha fechada** sem configuração. O Terraform casa os dois lados. Gravação idempotente por `ON CONFLICT`, porque a entrega é *at least once*.

Gatilho. Volume que justifique isolar o consumo do tráfego de aluno.

18

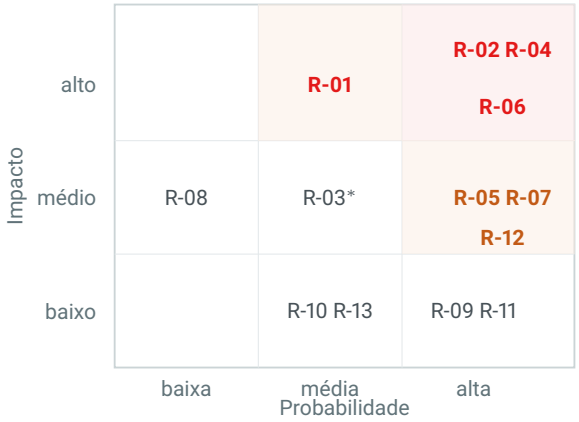
Riscos e evolução

18.1 Registro de riscos

Probabilidade e impacto em escala de três (baixo, médio, alto). A ordem é por severidade, e a severidade considera o prazo de novembro de 2026.

ID	Risco	Prob	Imp	Mitigação e estado
R-01	**Carga não verificada.** O comportamento no pico de 4.000 simultâneos é projetado, não medido.	Média	**Alto**	Teste de carga com k6 — 4.000 *logins* e aberturas de experiência em dez minutos — **obrigatório antes de novembro de 2026**. **Não executado.** É o maior risco técnico aberto.
R-02	**Retenção indefinida de dado de menor.** Não há prazo declarado para `exp_70_sessoes` nem para `telemetria_eventos`.	Alta	**Alto**	Decisão consciente (ADR-13) em troca de reprocessamento. Com a tabela particionada, a rotina de expurgo passou a ser trivial — o que remove o último argumento técnico. **Pendente de decisão de produto e contrato**, e é da plataforma inteira.
R-03	**Localidade de processamento de IA.** O tráfego de IA sai de São Paulo, inclusive o do aluno.	Certa	Médio	Consequência aceita e documentada (ADR-16). **Precisa constar do contrato e do DPA de cada rede.** Voltar à região exige trocar modelo e localização juntos.
R-04	**Fator humano único.** Uma pessoa desenvolve e opera.	Certa	**Alto**	Mitigado *por arquitetura*: menos peças, reversão por ponteiro, decisões registradas em ADR e este documento. Não é mitigável por engenharia sozinha.
R-05	**Sem *alerting*.** Há log e métrica nativa; não há alerta nem painel.	Alta	Médio	Configurar alerta de taxa de 5xx e de saturação de conexões do Cloud SQL. **Obrigatório antes de novembro de 2026.**

ID	Risco	Prob	Imp	Mitigação e estado
R-06	**Sem revisão de segurança externa.** Nenhum teste de intrusão.	Alta	**Alto**	Item de contratação, não de engenharia. Para plataforma que trata dado de menor em rede pública, é exigência prática de edital.
R-07	**Acessibilidade não verificada.** Nenhuma checagem de contraste, foco ou leitor de tela.	Alta	Médio	Rede pública tem exigência legal e prática. O *design system* é o lugar certo para resolver de uma vez — o que torna isto um pedido ao `storybook-unooria`, não 48 correções.
R-08	**Partição de telemetria sem *job*.** As faixas vão até 2028-12 e não há rotina que crie novas.	Baixa	Médio	A partição `DEFAULT` é rede de segurança (evento não é recusado). Verificar a contagem dela é item do *checklist* de operação. Encher = faltou criar partição.
R-09	**Mídia pesada publicada à mão.** O CI publica `dist/`; os 52 MB de MP4 da exp-02 vão por `gsutil` manual.	Alta	Baixo	Aceitável para uma experiência; **não escala para 48**. O passo `gsutil rsync` de mídia precisa entrar no CI.
R-10	**Geração órfã sem retomada no *boot*.** Se todas as instâncias morrerem e ninguém abrir o painel, a geração fica parada.	Média	Baixo	Aceito (**ADR-17**): o *polling* do painel é justamente quando alguém se importa com aquela geração. Gatilho: primeira reclamação real.
R-11	**Dependência do *design system* externo.** Lacuna do pacote bloqueia a experiência até a próxima publicação.	Média	Baixo	Compensação local marcada no CSS, decrescente por política, com pedido correspondente. Cinco lacunas abertas, verificadas na 0.1.14.
R-12	**Catálogo de modelos de IA instável.** Dois modelos *default* já saíram do catálogo sem aviso.	Alta	Médio	*Provider* plugável (**ADR-07**), modelo em variável de ambiente, e `finishReason` tratado como erro. O sintoma da última vez foi silencioso: toda geração de imagem caindo no *fallback* procedural.
R-13	**WSO2 ainda não é o emissor.** A plataforma roda com o hub assinando.	Média	Baixo	**ADR-15** é transitória por desenho: trocar são duas variáveis. O risco real é o `iss` *tenant-qualified* do WSO2 ser montado por concatenação em vez de copiado do *discovery*.



* R-03 tem probabilidade *certa* — é consequência aceita, não eventualidade. Está posicionado em "média" apenas para caber na matriz.

A zona vermelha concentra o que **tem de ser resolvido antes de novembro de 2026**: carga verificada, retenção decidida e revisão de segurança contratada.

Figura 18.1 – Matriz de riscos. Três dos quatro riscos da zona crítica não são resolvidos por código: retenção é decisão de produto, segurança externa é contratação, e o fator humano único é estrutural.

18.2 Estado de implementação

Fase	Escopo	Estado	Observação
M0	Contratos congelados (*claims*, ponte, telemetria)	✓ Concluído	`packages/sdk`, `src/auth/claims.ts`
M1	*Tenancy*, RLS, papéis, importação de usuários	✓ Concluído	Teste com *role* sem *ownership*
M2	exp-01 completa: motor, geração, dossiê	✓ Concluído	Migração do aplicativo de origem
M3	exp-02, domínio por experiência	✓ Concluído	**ADR-09**
M4	Ferramentas de professor: exp-90, exp-91	✓ Concluído	Armazenamento privado, *guardrails*
M5	exp-70 e o Ato 1 integrado	✓ Concluído	`exp_01_exp_70_perfis_importados`
M6	Hub do aluno, *header* único, casca compartilhada	✓ Concluído	`/aluno`, `CascaPlataforma`
—	*Stack* `dev` em GCP operacional	✓ Concluído	CI/CD completo com WIF
—	**Teste de carga**	– Pendente	`R-01`
—	***Alerting***	– Pendente	`R-05`
—	**WSO2 como emissor**	– Pendente	**ADR-15**

—	43 experiências restantes	– Em aberto	Gabarito estabelecido

18.3 Evolução planejada

18.3.1 Antes de novembro de 2026 (bloqueante)

1. **Teste de carga com k6**: 4.000 *logins* e aberturas de experiência em dez minutos, contra a *stack* `dev`. Verificar saturação de conexões, latência do `/runs/:id/feed` sob *lock*, e o caminho de ingestão de telemetria.
2. ***Alerting* mínimo**: taxa de 5xx do Cloud Run, saturação de conexões do Cloud SQL, falha do Cloud Run Job de migração, e crescimento da partição `DEFAULT` de telemetria.
3. **Decisão de retenção**, com a rotina que a cumpre. Se a resposta for "sem prazo", ela precisa estar no contrato — não só neste documento.
4. **Revisão de segurança externa** e teste de intrusão.
5. **DPA por rede** contemplando a localidade de processamento de IA.
6. **WSO2 como emissor**, com o `iss` *tenant-qualified* copiado literalmente do *discovery*.

18.3.2 Depois, por ordem de retorno

Evolução	Por quê, e o que ela exige
Rsync de mídia no CI	`R-09`. Com 48 experiências, publicação manual de mídia deixa de ser aceitável.
Filtro por série vindo do manifesto	Hoje o hub do aluno lista missões de um mapa estático. Quando o manifesto virar a fonte, `series: []` tem de **continuar** querendo dizer "todas" — o contrário faz a experiência transversal desaparecer da tela sozinha.
Acessibilidade no *design system*	`R-07`. Resolver no pacote é resolver uma vez; resolver nas experiências é resolver 48 vezes.
Telemetria offline real (IndexedDB)	Hoje o *buffer* é em memória, e há um `TODO` explícito na SDK. Rede pública tem escola com conectividade ruim, e é lá que o dado se perde.
Métricas de aplicação	Hoje só métrica nativa de infraestrutura. Nada mede "quantos alunos concluíram a missão hoje".
Reprocessamento do mapeamento	`VERSAO_MAPEAMENTO` vai subir, e o *payload* está guardado justamente para isso (**ADR-13**). Falta a rotina que reprocessa e o relatório que compara duas gerações.
Visão agregada de bem-estar	Só com DPA e *k*-anonimato explícitos (**ADR-10**). **Não sai de um `GROUP BY`.**

Worker dedicado de telemetria	Se o volume justificar isolar o consumo (**ADR-21**).
BigQuery	Só se um relatório efetivamente precisar de análise em escala sobre `telemetria_eventos` (**ADR-14**).

18.4 O que continuará fora de escopo

Descartado	Condição para reavaliar
Kubernetes (GKE)	*Workload* de longa duração ou topologia de rede que o Cloud Run não expresse.
Microsserviço por experiência	Nenhuma. Contraria **ADR-01** na raiz.
Multi-região	Exigência contratual de continuidade regional.
Redis	Gargalo medido que o Pub/Sub e o `INSERT` em lote não resolvam.
Módulo genérico de avaliação	Três experiências consecutivas com mecânica idêntica.
Banco ou *schema* por experiência	Necessidade de fronteira de `GRANT` por experiência.

> **NOTA**
>
> A lista acima é tão importante quanto a de evolução. Num projeto com uma pessoa e um prazo, **o que se recusa a construir é o que permite terminar o resto** — e recusar sem registrar a condição de reavaliação transforma a recusa em teimosia.

19

Conclusão

19.1 O que foi construído

A Plataforma Unooria é um *monorepo* com dois serviços em execução — um hub Next.js e uma API NestJS — servindo um catálogo crescente de experiências imersivas entregues como *bundles* estáticos versionados em CDN. Em 27 de agosto de 2026, o estado é:

Dimensão	Estado
Serviços em execução	2 (hub, API) + 2 gerenciados (PostgreSQL, Pub/Sub)
Experiências publicadas	5 de 48, cobrindo os quatro casos estruturais: aluno com IA, aluno sem IA, jogo transversal e ferramenta de professor
Módulos da API	22, com um módulo de domínio por experiência
Tabelas	27 em `public` + partições mensais em `particoes`
Migrações	9, todas com RLS preservada
Arquivos de teste	34 (regra pura, integração, contrato, *smoke*)
Walkthroughs visuais	7
Decisões registradas	21 ADRs
Milestones concluídos	M0–M6

19.2 As decisões que mais definiram o resultado

Se este documento tivesse de ser reduzido a cinco frases, seriam estas.

1. **Experiência é artefato, não serviço** (ADR-01). É a decisão que torna 48 experiências viáveis para uma pessoa: o custo marginal de publicar a próxima é copiar arquivos, e a reversão de emergência é um `UPDATE` de uma coluna.
2. **Isolamento físico entre contratos, lógico entre escolas** (ADR-04, ADR-08). Uma *stack* por rede satisfaz o edital; RLS forçada com uma porta única de banco satisfaz a engenharia. E as duas trancas — política no banco e filtro na aplicação — são independentes de propósito.

3. **Regra que pode ser expressa em tipo é expressa em tipo** (ADR-12). `comTenancy` *lança* sem escola em vez de consultar sem filtro; a leitura de valores não *tem* campo numérico onde não há medida. Comentário avisa; tipo impede. É a diferença entre uma regra documentada e uma regra que vale.
4. **O que se recusa a construir é o que permite terminar** (Capítulo 18). Sem GKE, sem microsserviço por experiência, sem Redis, sem BigQuery, sem *worker* de telemetria, sem módulo genérico de avaliação — cada um com a condição de reavaliação escrita, para que a recusa não vire teimosia.
5. **Medir antes de otimizar, e medir antes de estimar** (ADR-14, ADR-16). O gargalo da telemetria não era o transporte, era um laço de `INSERT`. O armazenamento da exp-70 não era 2,4 GB, era 30 MB. O modelo de IA não existia na região que parecia óbvia. Nenhuma dessas três coisas era previsível — e todas as três mudaram uma decisão.

19.3 O que este projeto ensinou sobre documentação

Vale registrar, porque muda como este documento deve ser mantido.

> **NOTA**
>
> As decisões mais finas deste projeto não estavam em documento nenhum: estavam em **comentários no código**, ao lado da linha que elas explicam — por que o terceiro argumento do `set_config` é `true`, por que `exp:ready` é reenviado, por que a tabela de contrato carrega dois números, por que `cartoesServidos` não é "apareceram".
> Isso é um bom padrão e deve continuar. O que este documento acrescenta não é substituir aqueles comentários — é dar-lhes um *lugar*: a estrutura em que uma pessoa que nunca viu o repositório consegue entender por que ele é assim, e a matriz que liga cada decisão ao requisito que a exige e ao teste que a verifica.

> **ATENÇÃO**
>
> O risco desta documentação é o mesmo do `expires_at` que foi removido do *schema* (ADR-13): **documento que ninguém atualiza é pior que documento nenhum**, porque quem o lê acredita nele.
> Por isso a atualização deste documento é item explícito da *Definition of Done* (Capítulo 16) e está registrada em `CLAUDE.md`, não apenas aqui. Uma implementação nova que mude arquitetura, contrato, modelo de dados, decisão ou risco **tem de** atualizar o capítulo correspondente e regerar o PDF no mesmo *commit*.

19.4 Estado de prontidão para produção

Dimensão	Estado	Falta

Funcionalidade	✓ Pronta	As 43 experiências restantes são *conteúdo*, não plataforma.
Isolamento e segurança de *tenancy*	✓ Pronta e verificada	Revisão externa (`R-06`).
Entrega e reversão	✓ Pronta	*Rsync* de mídia no CI (`R-09`).
Identidade	✓ Funcional	WSO2 como emissor (**ADR-15**).
Desempenho sob pico	**– Não verificado**	**Teste de carga** (`R-01`). É o item que separa "funciona" de "funciona às oito da manhã".
Operabilidade	– Parcial	***Alerting*** (`R-05`).
Conformidade	– Parcial	**Retenção decidida** (`R-02`) e **DPA com a localidade de IA** (`R-03`).
Acessibilidade	– Não verificada	Checagem no *design system* (`R-07`).

LIMITAÇÃO CONHECIDA

A plataforma está **funcionalmente pronta** e **operacionalmente incompleta**. As quatro lacunas em negrito acima são bloqueantes para novembro de 2026, e três delas não são resolvidas por código: retenção é decisão de produto, DPA é jurídico, revisão de segurança é contratação. Reconhecer isso explicitamente é mais útil que declarar prontidão.

19.5 Encerramento

O desenho descrito aqui é conservador por escolha. Não há tecnologia interessante nele: há PostgreSQL com RLS, contêineres sem orquestrador, arquivos estáticos numa CDN e uma fila onde a fila era inevitável. A parte difícil não foi escolher as peças — foi recusar as que não cabiam, e escrever por que.

O que este documento tenta preservar não é o *estado* do sistema, que o código descreve melhor. É o *porquê*: as vinte e uma decisões, as medições que as sustentam, as armadilhas que já custaram tempo e as consequências ruins que foram aceitas de olhos abertos. Um sistema pode ser lido a partir do código. Um projeto, não.

Apêndice A

Fontes Mermaid dos diagramas

Todo diagrama deste documento é vetorial, desenhado nativamente em TikZ — o que garante tipografia única, nitidez em qualquer *zoom* e ausência de dependência externa de renderização. Este apêndice traz a fonte **Mermaid equivalente** de cada figura, para quem precisa editar, revisar ou versionar o diagrama fora do LaTeX (num *pull request*, num Storybook, num quadro).

> **REGRA INVIOLÁVEL**
>
> **Ao acrescentar um diagrama TikZ, inclua aqui a fonte Mermaid equivalente**, referenciando o mesmo número de figura. Diagrama sem par Mermaid é diagrama que só uma pessoa consegue alterar.

> **NOTA**
>
> As fontes abaixo usam `C4Context`, `C4Container` e `C4Component` onde o diagrama é C4, `sequenceDiagram` onde é sequência, `erDiagram` onde é entidade-relacionamento, `stateDiagram-v2` onde é máquina de estados e `flowchart` nos demais. Não são idênticas ao TikZ: o Mermaid não expressa posicionamento fino nem notas ancoradas, e as notas longas dos diagramas TikZ aparecem aqui como `note` ou comentário.

A.1 Figura 1.1 — Visão geral da plataforma

```
C4Container
  title Visao geral da Plataforma Unooria

  Person(aluno, "Aluno", "1o EF ao 3o EM")

  Container_Boundary(plat, "Plataforma Unooria") {
    Container(hub, "Hub", "Next.js", "Autentica, roteia por papel, embeda")
    Container(exp, "Experiencia", "Vite estatico + CDN", "Roda no iframe do navegador")
    Container(api, "API", "NestJS", "Regra, gabarito, tenancy")
    ContainerDb(pg, "PostgreSQL", "Cloud SQL", "RLS por escola")
    ContainerQueue(ps, "Pub/Sub", "", "Telemetria analitica")
  }

  System_Ext(idp, "WSO2 Identity Server", "Identidade, futuro")
  System_Ext(vertex, "Vertex AI", "Gemini")
```

```
16
17    Rel(aluno, hub, "HTTPS")
18    BiRel(hub, exp, "postMessage v1", "token, contexto")
19    Rel_L(hub, idp, "OIDC")
20    Rel(hub, api, "BFF")
21    Rel(exp, api, "SDK: /telemetry, /ai, dominio")
22    BiRel(api, pg, "SQL + RLS")
23    Rel(api, ps, "publica")
24    Rel(ps, api, "push POST /telemetry/ingest")
25    Rel(api, vertex, "quota por perfil")
```

A.2 Figura 4.1 — Casos de uso

```
1  flowchart LR
2    aluno([Aluno]); prof([Professor]); gestor([Gestor escolar]); admin([Admin do contrato])
3    idp[[WSO2 IS]]; vertex[[Vertex AI]]; pubsub[[Pub/Sub]]; ci[[CI/CD]]
4
5    subgraph SIS["Plataforma Unooria"]
6      UC01(("UC-01 Autenticar-se"))
7      UC02(("UC-02 Escolher missao no hub"))
8      UC03(("UC-03 Realizar experiencia"))
9      UC04(("UC-04 Registrar telemetria"))
10     UC05(("UC-05 Renovar token da sessao"))
11     UC06(("UC-06 Ver o proprio resultado"))
12     UC10(("UC-10 Preparar atividade"))
13     UC11(("UC-11 Gerar conteudo com IA"))
14     UC12(("UC-12 Conduzir o momento coletivo"))
15     UC13(("UC-13 Ler resultados da turma"))
16     UC14(("UC-14 Gerar plano de aula"))
17     UC15(("UC-15 Check-in de bem-estar"))
18     UC16(("UC-16 Zerar dados da turma"))
19     UC20(("UC-20 Importar usuarios por CSV"))
20     UC21(("UC-21 Redefinir senha"))
21     UC22(("UC-22 Ler agregado da rede"))
22     UC23(("UC-23 Publicar manifesto"))
23     UC24(("UC-24 Reverter versao"))
24   end
25
26   aluno --- UC01 & UC02 & UC03 & UC06
27   prof  --- UC01 & UC10 & UC12 & UC13 & UC14 & UC15 & UC16
28   gestor --- UC20 & UC21
29   admin --- UC22 & UC23
30
31   UC03 -.->|include| UC04
32   UC03 -.->|include| UC05
33   UC10 -.->|include| UC11
34   UC23 -.->|extend| UC24
35
36   UC01 --- idp
37   UC11 --- vertex
38   UC14 --- vertex
39   UC15 --- vertex
40   UC04 --- pubsub
41   UC23 --- ci
```

A.3 Figura 5.1 — C4 nível 1: contexto

```
C4Context
  title C4 nivel 1 - Contexto do sistema

  Person(aluno, "Aluno", "1o EF ao 3o EM")
  Person(prof, "Professor", "Conduz a aula")
  Person(gestor, "Gestor escolar", "Mantem usuarios")
  Person(admin, "Admin do contrato", "Rede")

  System(sis, "Plataforma Unooria", "Hub multi-tenant que autentica por papel, embeda experiencias imersivas versionadas, corrige e pontua no servidor, coleta telemetria e produz leitura de turma - com isolamento logico por escola")

  System_Ext(idp, "WSO2 Identity Server", "Identidade central, varios contratos")
  System_Ext(vertex, "Vertex AI - Gemini", "Texto e imagem generativos")
  System_Ext(ci, "GitHub Actions", "Publica bundles e manifestos")
  System_Ext(tf, "Terraform (repo separado)", "Provisiona a stack do contrato")

  Rel(aluno, sis, "realiza missoes")
  Rel(prof, sis, "prepara, conduz, le")
  Rel(gestor, sis, "administra")
  Rel(admin, sis, "le o agregado")
  Rel(idp, sis, "OIDC")
  BiRel(sis, vertex, "HTTPS + ADC")
  Rel(ci, sis, "publica")
  Rel(tf, sis, "provisiona")
```

A.4 Figura 5.2 — C4 nível 2: contêineres

```
C4Container
  title C4 nivel 2 - Conteineres (stack de um contrato)

  Person(aluno, "Aluno")
  Person(prof, "Professor")
  Person(adm, "Gestor / Admin")

  Container_Boundary(stack, "Plataforma Unooria - stack de um contrato") {
    Container(hub, "Hub", "Next.js App Router", "Sessao, roteamento por papel, chrome institucional, montagem do embed, BFF, JWKS e assinatura de token")
    Container(exp, "Experiencia", "Vite + React, ESTATICO", "Uma por missao. Roda no navegador; nunca e um processo do servidor")
    Component(sdk, "@edu/sdk", "TypeScript", "Ponte, telemetria, API, progresso")
    Container(api, "API", "NestJS + Prisma", "Autorizacao por papel, tenancy, regra de dominio, gabarito, correcao, gateway de IA, ingestao de telemetria")
    ContainerDb(pg, "PostgreSQL", "Cloud SQL HA + PITR", "Unico banco, unico schema; RLS por escola")
    ContainerQueue(ps, "Pub/Sub", "", "Canal analitico de telemetria")
    Container(buckets, "Object Storage", "Cloud Storage", "bundles, assets publicos, planos privados ")
  }

  System_Ext(vertex, "Vertex AI")
  Container_Ext(idp, "WSO2 IS", "", "Externo a stack")
```

```
  Rel(aluno, hub, "HTTPS")
  Rel(prof, hub, "HTTPS")
  Rel(adm, hub, "HTTPS")
  BiRel(hub, exp, "postMessage v1 - targetOrigin explicito")
  Rel(exp, sdk, "usa")
  Rel(sdk, api, "HTTPS + Bearer")
  Rel(hub, api, "BFF")
  BiRel(api, pg, "SQL + RLS")
  Rel(api, ps, "publica")
  Rel(ps, api, "push /telemetry/ingest")
  Rel(api, buckets, "assets, planos")
  Rel(api, vertex, "HTTPS + ADC")
  Rel(hub, idp, "OIDC")
  Rel(buckets, exp, "CDN serve o bundle")
```

A.5 Figura 5.3 — C4 nível 3: componentes da API

```
C4Component
  title C4 nivel 3 - Componentes da API (NestJS)

  Container_Boundary(api, "API") {
    %% --- transversal: roda em toda rota
    Component(guard,  "OidcGuard", "", "Valida JWT por JWKS")
    Component(roles,  "RolesGuard", "", "Restringe por papel")
    Component(tenint, "TenancyInterceptor", "", "Escola no AsyncLocalStorage")
    Component(env,    "validarEnv", "Zod", "Valida env no boot")

    %% --- infraestrutura: as portas unicas
    Component(prisma, "PrismaService", "Prisma", "comTenancy: transacao + RLS")
    Component(ai,     "ai/", "", "AiProvider, QuotaService")
    Component(store,  "object-storage/", "", "local | GCS")
    Component(assets, "assets/", "", "Imagem gerada")

    %% --- plataforma: serve a todas as experiencias
    Component(auth,   "auth/", "", "TokenService")
    Component(usu,    "usuarios/", "", "CSV, senha")
    Component(expmod, "experiencias/", "", "Manifesto")
    Component(tel,    "telemetry/", "", "batch, beacon, ingest")
    Component(prog,   "progress/", "", "Progresso generico")
    Component(health, "health/", "", "healthz, readyz")
    Component(analise,"analise/", "", "Turma e rede, CSV")
    Component(comum,  "comum/", "", "Series, rate limit")

    %% --- dominio por experiencia (nenhum conhece outro)
    Component(motor,  "motor/ geracao/ dossie/ atividades/ perfis/", "", "exp-01")
    Component(utube,  "unootube/", "", "exp-02")
    Component(vila,   "vila/", "", "exp-70")
    Component(planos, "planos/", "", "exp-90")
    Component(bem,    "bemestar/", "", "exp-91")
  }

  Rel(guard, roles, "antes de")
  Rel(tenint, prisma, "escola do token")
  Rel(motor, prisma, "comTenancy")
  Rel(vila, prisma, "comTenancy")
```

```
  Rel(planos, ai, "IA + quota")
  Rel(bem, ai, "IA + quota")
  Rel(planos, store, "bucket privado")
```

A.6 Figura 5.4 — C4 nível 3: componentes do hub

```
C4Component
  title C4 nivel 3 - Componentes do Hub (Next.js)

  Container_Boundary(hub, "Hub") {
    Component(mw,      "middleware.ts", "", "Sessao + area do papel")
    Component(papeis,  "lib/papeis.ts", "", "edge-safe: papel -> area")
    Component(authlib, "lib/auth.ts", "NextAuth", "Sessao")
    Component(jwks,    "/.well-known/jwks.json", "", "Chave publica")

    Component(embed,  "lib/embed-experiencia.ts", "", "Token + manifesto + origem, UMA vez")
    Component(chave,  "lib/chave-token.ts", "", "Assina RS256, TTL 15 min")
    Component(devsrv, "lib/experiencias-dev.ts", "", "Porta do dev server")
    Component(apiserv,"lib/api-servidor.ts", "", "BFF")

    Component(casca,  "CascaPlataforma", "", "CSS do DS + fontes")
    Component(header, "HeaderPlataforma", "", "Um so para todos os papeis")
    Component(disp,   "DispositivoFrame", "", "Casca de aparelho")
    Component(frame,  "ExpFrame", "", "Sandbox + protocolo v1")
  }

  Rel(mw, papeis, "usa")
  Rel(embed, chave, "assina")
  Rel(embed, devsrv, "porta em dev")
  Rel(disp, frame, "envolve")
```

A.7 Figura 5.5 — C4 nível 4: tipos que carregam a regra

```
classDiagram
  class connectToHub {
    +hubOrigin: string
    +timeoutMs = 8000
    +READY_RETRY_MS = 400
    valida ev.origin, senao descarta
    reenvia exp:ready ate responder
    compara protocolVersion MAIOR
    valida pos-init com Zod
  }

  class SdkTypes {
    +PROTOCOL_VERSION = 1
    +HubInitSchema
    +HubParaExpSchema
    +ExpParaHubSchema
    unioes discriminadas por type
  }
```

```
  class PrismaService {
    +comTenancy(fn, escolaId?, opts?)
    abre $transaction
    set_config('app.escola_id', ..., true)
    sem escola: LANCA
  }

  class TenancyContext {
    +AsyncLocalStorage~string~
    +comEscolaNoContexto(id, fn)
    +escolaDoContexto(): string?
  }

  class LinhaDeLeitura {
    <<union>>
    scored:      numero: number
    indicative:  numero: null
    zero:        numero: null
    none:        numero: null
  }

  connectToHub ..> SdkTypes : usa
  PrismaService ..> TenancyContext : le
  note for LinhaDeLeitura "Comentario avisa; TIPO impede.\nO hub nunca recebe numero onde nao ha medida."
```

A.8 Figura 5.6 — Grafo de dependências do *monorepo*

```
flowchart TB
  ds[["@unooria/design-system (npm publico)"]]
  sdk["packages/sdk"]
  ui["packages/ui"]
  hub["apps/hub"]; api["apps/api"]
  e01["exp-01"]; e02["exp-02"]; e70["exp-70"]; e90["exp-90"]; e91["exp-91"]
  tpl["_template"]

  ds -->|versao publicada| ui
  sdk --> hub & api & e01 & e02 & e70 & e90 & e91 & tpl
  ui --> hub & e01 & e02 & e90 & e91
  %% exp-70 NAO depende de packages/ui: canvas 3D, sem DOM de UI (ADR-18)
```

A.9 Figura 5.7 — Curadoria de um bloco de *feed*

```
sequenceDiagram
  autonumber
  participant E as Experiencia (exp-01 no iframe)
  participant C as runs.controller (NestJS)
  participant S as motor/servico
  participant P as PostgreSQL

```

```
  E->>C: POST /runs/:id/eventos (dwell, curtidas)
  C->>S: aplicarEventos
  S->>P: comTenancy: set_config('app.escola_id')
  S->>P: pg_advisory_xact_lock(runId)
  Note over S,P: Sem o lock, dois pedidos simultaneos de bloco<br/>se sobrescrevem e um bloco
↪ inteiro desaparece.<br/>Transacao com timeout de 20s.
  P-->>S: estado da run
  S->>S: curarBloco (puro e deterministico)
  S->>P: UPDATE engine_state + posts entregues
  P-->>S: COMMIT (libera o lock)
  S-->>C: bloco + modulos + diagnostico
  C->>C: paraFeedPost: remove lado, perfil, intensidade
  Note over C: Esses campos sao a variavel independente<br/>do experimento. No cliente, o aluno
↪ saberia<br/>de que lado o algoritmo o colocou.
  C-->>E: GET /runs/:id/feed (sanitizado)
```

A.10 Figura 5.8 — Estados de uma geração assíncrona

```
stateDiagram-v2
  [*] --> pendente : POST (202)
  pendente --> gerando : claim atomico (updateMany por status)
  gerando --> pronto : gravou o artefato
  gerando --> erro : falha do provider OU finishReason=MAX_TOKENS
  gerando --> pendente : orfa (>12 min sem worker), ate 2 retomadas
  pronto --> [*]

  note right of pendente
    O claim atomico substitui uma fila externa:
    duas instancias podem tentar, so uma ganha a linha.
    Menor peca possivel que resolve o problema (ADR-17).
  end note
```

A.11 Figura 6.1 — Implantação de um contrato

```
C4Deployment
  title Implantacao de um contrato

  Deployment_Node(internet, "Internet", "escola") {
    Container(browser, "Navegador do aluno", "", "")
  }

  Deployment_Node(gcp, "Google Cloud - projeto do contrato", "GCP") {
    Deployment_Node(borda, "Borda global", "") {
      Container(lb,  "Cloud Load Balancing", "", "1 IP, host rules")
      Container(cdn, "Cloud CDN", "", "")
    }
    Deployment_Node(regiao, "Regiao de execucao (us-east1)", "") {
      Container(hub, "Cloud Run {contrato}-frontend", "hub", "min>=1")
      Container(api, "Cloud Run {contrato}-api", "", "CPU sempre alocada, conc. 80")
      Deployment_Node(vpc, "VPC - rede privada", "") {
        ContainerDb(sql, "Cloud SQL {contrato}-pg", "PostgreSQL HA, PITR, pooler", "")
```

```
      }
    }
    Deployment_Node(gcs, "Cloud Storage", "") {
      Container(bexp, "{contrato}-exp", "publico, imutavel, CDN", "")
      Container(bass, "{contrato}-assets", "publico, CDN", "")
      Container(bpla, "{contrato}-planos", "PRIVADO, versionado", "")
    }
    ContainerQueue(ps, "Pub/Sub {contrato}-telemetria", "", "")
  }

  System_Ext(idp, "WSO2 IS", "projeto proprio")
  System_Ext(vertex, "Vertex AI", "endpoint global")

  Rel(browser, lb, "443")
  Rel(lb, cdn, "bundles e assets")
  Rel(cdn, bexp, "serve")
  Rel(lb, hub, "HTTPS")
  Rel(lb, api, "HTTPS")
  Rel(api, sql, "5432 - IP PRIVADO")
  Rel(api, bpla, "so pela API")
  BiRel(api, ps, "publica / recebe push")
  Rel(hub, idp, "OIDC")
  Rel(api, vertex, "HTTPS + ADC")
```

A.12 Figura 6.2 — Implantação detalhada

```
flowchart TB
  browser([Navegador do aluno]); gh[[GitHub Actions]]; wso2[[WSO2 IS]]; vertex[[Vertex AI - global]]

  subgraph GCP["Google Cloud - projeto do contrato"]
    subgraph BORDA["Borda global"]
      lb["Load Balancer (1 IP, host rules)"]; cdn["Cloud CDN"]; cert["Certificados gerenciados (um por dominio)"]
    end
    subgraph EXEC["Regiao de execucao (us-east1)"]
      hub["Cloud Run -frontend (min>=1)"]
      api["Cloud Run -api (CPU sempre alocada, conc. 80)"]
      job["Cloud Run Job -migrate (na VPC)"]
      subgraph VPC["VPC - rede privada"]
        sql[("Cloud SQL -pg (HA, PITR, pooler)")]
        conn["Serverless VPC Access (conector)"]
      end
    end
    subgraph STORAGE["Cloud Storage"]
      bexp["{contrato}-exp - publico, imutavel, CDN"]
      bass["{contrato}-assets - publico, CDN"]
      bpla["{contrato}-planos - PRIVADO, sem CDN, versionado"]
    end
    subgraph MSG["Mensageria"]
      topic["topic -telemetria"]; sub["push sub -telemetria-ingest (oidc_token)"]
    end
    subgraph GOV["Identidade, segredos e artefatos"]
      sm["Secret Manager (6 segredos)"]; sa["Service Accounts (-api, -run, -push)"]
      wif["WIF pool github (sem chave JSON)"]; ar["Artifact Registry"]
      log["Cloud Logging / Monitoring"]
    end
```

```
  end

  browser -->|443| lb
  lb --> cdn --> bexp & bass
  lb --> hub & api
  hub -->|BFF| api
  api -->|5432 privado| conn --> sql
  job --> conn
  api -->|so pela API| bpla
  api --> bass
  api --> topic --> sub -->|POST /telemetry/ingest| api
  gh -->|OIDC do GitHub| wif
  gh -->|push da imagem| ar
  wso2 -->|OIDC| hub
  api -->|ADC| vertex
  sm --> api & hub
  sa --> api & hub
```

A.13 Figura 6.3 — Identidade de carga de trabalho

```
flowchart LR
  subgraph CI["CI: Workload Identity Federation"]
    gh["GitHub Actions (id-token: write)"] --> tok["token OIDC do GitHub (repo, ref, workflow)"]
    tok --> wif["WIF pool github (valida e troca)"]
    wif --> sadeploy["access token da DEPLOY_SA (curta duracao)"]
  end

  subgraph APP["Aplicacao: Application Default Credentials"]
    run["Cloud Run (API) - sem env de credencial"] --> md["metadata server (ADC nivel 3)"]
    md --> saapi["SA {contrato}-api (roles/aiplatform.user)"]
    saapi --> vx["Vertex AI - autorizado"]
  end

  %% A mesma cadeia nos tres ambientes:
  %%   1. GOOGLE_APPLICATION_CREDENTIALS
  %%   2. arquivo de `gcloud auth application-default login`
  %%   3. metadata server (Cloud Run)
  %% Antes havia SO o nivel 3 — e por isso nenhuma chamada de IA funcionava
  %% fora do Cloud Run. NUNCA chave de API.
```

A.14 Figura 7.1 — Os três níveis de isolamento

```
flowchart TB
  subgraph A["Contrato A - stack GCP"]
    apiA["API do contrato A"] --> pgA[("PostgreSQL do contrato A")]
    subgraph ESC["Dentro de UMA stack: isolamento LOGICO"]
      e1["escola sp-001"]; e2["escola sp-002"]; e3["... 100+"]
    end
    tab["Toda tabela de dado de aluno:<br/>escola_id TEXT NOT NULL<br/>ENABLE + FORCE ROW LEVEL SECURITY<br/>POLICY USING (escola_id = current_setting('app.escola_id', true))"]
    e1 -.-> tab; e2 -.-> tab; e3 -.-> tab
```

```
  end
  subgraph B["Contrato B - OUTRA stack"]
    apiB["API do contrato B"] --> pgB[("PostgreSQL do contrato B")]
  end
```

A.15 Figura 7.2 — Propagação da escola

```
flowchart LR
  req["Requisicao HTTP (Authorization: Bearer)"] --> guard["OidcGuard (JWKS; injeta req.auth)"]
  guard --> roles["RolesGuard (le req.auth.perfil)"]
  roles --> int["TenancyInterceptor (comEscolaNoContexto)"]
  int --> ctrl["controller"] --> svc["service: comTenancy(fn)"]
  svc --> tx["Transacao: set_config + query"]
  svc -->|sem escola no contexto| fail["LANCA"]

  %% Guards rodam ANTES de interceptors, entao req.auth ja existe.
  %% Rota sem OidcGuard nao abre contexto — e comTenancy sem escola falha alto.
  %% A RLS e rede de seguranca, nao a unica tranca: os services continuam
  %% filtrando escola_id explicitamente em toda query.
```

A.16 Figura 8.1 — Modelo de dados da plataforma

```
erDiagram
  escolas ||--o{ turmas : tem
  escolas ||--o{ users  : tem
  turmas  ||--o{ users  : agrupa
  experiencias ||..o{ progresso : "exp_id"
  telemetria_eventos ||..o{ particoes : "ATTACH por mes"

  escolas {
    text id PK "MESMO valor do claim escola_id e do set_config"
    text nome
    timestamptz created_at
  }
  turmas {
    uuid id PK
    text escola_id FK
    text nome "UNIQUE (escola_id, nome)"
    text serie "1EF..9EF | 1EM..3EM"
  }
  users {
    uuid id PK
    text role "aluno|professor|gestor|admin-contrato"
    text escola_id FK "NULL SO p/ admin-contrato (CHECK)"
    uuid turma_id FK
    text username "UNIQUE (escola_id, username)"
    text display_name
    text password_hash
  }
  experiencias {
    text exp_id PK
```

```
    text nome
    text versao_ativa "rollback = UPDATE deste campo"
    text_array series "VAZIO = TODAS as series"
    text_array capabilities
    jsonb viewport "presente => casca de aparelho"
    text publico "aluno|professor"
    text_array permissoes "hoje so pointer-lock"
  }
  progresso {
    text escola_id PK
    text aluno_id PK
    text exp_id PK
    jsonb dados
  }
  ia_uso_diario {
    text escola_id PK
    text usuario_id PK
    text dia PK "YYYY-MM-DD"
    int tokens_usados
  }
  telemetria_eventos {
    uuid event_id PK "PK COMPOSTA: o Postgres exige a chave de particao"
    timestamptz ts PK
    text escola_id
    text aluno_id
    text exp_id
    text exp_version
    uuid sessao_id
    text verbo
    text objeto
    jsonb resultado
    jsonb contexto
  }
```

A.17 Figuras 8.2 e 8.3 — Modelo da exp-01

```
erDiagram
  exp_01_activities ||--o{ exp_01_authors : tem
  exp_01_authors    ||--o{ exp_01_posts   : escreve
  exp_01_posts      ||--o{ exp_01_post_comments : tem
  exp_01_activities ||--o{ exp_01_generation_jobs : enfileira
  exp_01_activities ||--o{ exp_01_runs : hospeda
  exp_01_runs ||--o{ exp_01_events : registra
  exp_01_runs ||--o{ exp_01_student_posts : publica
  exp_01_runs ||--|| exp_01_conclusoes : conclui
  exp_01_runs ||--|| exp_01_reflexoes : reflete
  exp_01_runs ||--|| exp_01_dossiers : gera
  exp_01_runs ||--o{ exp_01_comentarios_da_sessao : ancora
  exp_01_activities ||--o{ exp_01_exp_70_perfis_importados : "largada"

  exp_01_activities {
    uuid id PK
    text escola_id FK
    uuid professor_id FK
    text pergunta_chave
    text pergunta_origem "ia|professor - auditoria de autoria"
```

```
    text etapa "relogio COLETIVO da turma"
    boolean ia_ao_vivo "desligavel sem deploy"
    jsonb antagonismo
    jsonb engine_config
    text status
  }
  exp_01_posts {
    uuid id PK
    text lado "VARIAVEL INDEPENDENTE - sanitizada antes de sair"
    text perfil "idem"
    smallint intensidade "idem"
    text image_status "pendente|ok|fallback"
    boolean aprovado
  }
  exp_01_runs {
    uuid id PK
    text lado_inicial
    text lado_origem "perfil|sorteio|redistribuido - auditoria"
    jsonb engine_state "estado do motor; unidade de LOCK"
    text nome_no_app "a conta e do APARELHO, nao da crianca"
  }
  exp_01_events {
    bigint id PK
    text tipo "canal OPERACIONAL: sincrono e transacional"
    jsonb payload "{dwellMs, dwellNorm}"
  }
  exp_01_comentarios_da_sessao {
    uuid post_id FK "XOR com student_post_id (CHECK na migration)"
    uuid student_post_id FK
    boolean de_ia "para o professor distinguir na auditoria"
  }
  exp_01_exp_70_perfis_importados {
    jsonb escores "4 quadrantes, 0-1"
    text dominante
    text origem "jogo|professor"
    text lado_forcado "PRESERVADO no upsert"
  }
```

A.18 Figuras 8.4 e 8.5 — Demais experiências

```
erDiagram
  exp_70_sessoes ||--o{ exp_70_perfis : "recalculado pelo servidor"
  exp_70_perfis  ||..|| exp_70_bolhas : "mesmo aluno"
  exp_90_planos  ||..o{ exp_90_referencias : "monta o prompt"
  exp_91_ciclos  ||--o{ exp_91_checkins : "7 check-ins"

  exp_02_sessoes {
    text escola_id PK
    uuid aluno_id PK
    text_array salvos "CHECK cardinality <= 3"
    boolean ranking_desbloqueado "LATCH: nao volta"
    boolean credibilidade_adicionada "congela os salvos"
  }
  exp_70_sessoes {
    uuid id PK "EMITIDO pela API - o cliente colidia"
    uuid turma_id FK "desnormalizado: turma NO MOMENTO da partida"
```

```
    text bolha "A|B|C - do SERVIDOR"
    smallint cartoes_servidos "!= apareceram"
    timestamptz aberta_em "relogio do SERVIDOR"
    jsonb payload "bruto - unica fonte do reprocessamento"
    jsonb leitura_mensagens
  }
  exp_70_perfis {
    smallint versao_mapeamento "sobe => nasce linha nova"
    jsonb valores "null NAO e zero"
    jsonb graus "scored|indicative|zero|none"
    float abertura_vs_conservacao "NULL se polo nao observado"
    boolean amostra_suficiente "falso => o painel avisa que NAO SABE"
  }
  exp_70_bolhas {
    text escola_id PK
    uuid aluno_id PK "por ALUNO, nao por partida"
    text bolha
  }
  exp_90_planos {
    text status "pendente|gerando|pronto|erro"
    text caminho_gerado_md "IMUTAVEL, bucket privado"
    text caminho_html "o que o editor salvou, sanitizado"
  }
  exp_90_referencias {
    text disciplina PK
    text serie PK "NULL = todas"
    text nome PK
    text texto "catalogo do CONTRATO - sem escola_id, FORA da RLS"
  }
  exp_91_ciclos {
    uuid id PK
    int_array ordem_itens "permutacao de 1..21"
    int depressao "somas x2, ao concluir - DADO SENSIVEL"
  }
  exp_91_checkins {
    smallint numero "1..7"
    text dia "UNIQUE (escola_id, professor_id, dia)"
    text texto_livre "NUNCA em log nem telemetria"
    text origem_texto "ia|fallback|seguranca"
    boolean sinal_de_apoio
  }
```

A.19 Figura 9.1 — *Handshake* hub-experiência

```
sequenceDiagram
  autonumber
  participant E as Experiencia (@edu/sdk no iframe)
  participant F as ExpFrame (React, no hub)
  participant R as Rota do hub (servidor)
  participant A as API (NestJS)

  R->>F: monta o embed: assina token RS256 (15 min)
  R->>A: GET /experiencias/:expId (manifesto)
  A-->>R: { versaoAtiva, viewport, permissoes }
  F->>E: renderiza iframe src = URL versionada
  E->>E: connectToHub({ hubOrigin })
```

```
  loop a cada 400ms, ate o hub responder
    E->>F: exp:ready { protocolVersion: 1 }
  end
  F->>F: valida ev.origin e a versao MAIOR
  F-->>E: hub:init { token, aluno, apiUrl, config }
  E->>E: valida com Zod; resolve a Promise
  E->>A: chamadas de dominio, /telemetry/batch, /ai/gerar
  E->>F: exp:token-expirando
  F->>R: GET /api/exp-token
  R-->>F: { token }
  F-->>E: hub:token { token }

  Note over E,F: Com assets em cache (F5), a experiencia fica pronta ANTES<br/>de o ExpFrame hidratar . Um disparo unico se perderia.<br/>hub:init repetido e evento normal, nao ruido.
```

A.20 Figura 9.2 — Resolução da URL do *embed*

```
flowchart TB
  ini([inicio]) --> q{tem linha em experiencias?}
  q -->|sim| com["{cdn}/{expId}/{versaoAtiva}/index.html?hubOrigin=..."]
  q -->|nao| sem["{baseDev}/?hubOrigin=... (porta de lib/experiencias-dev.ts<br/>ou o src explicito de /dev-embed)"]
  com --> vp1["viewport e permissoes vem do MANIFESTO"]
  sem --> vp2["le {baseInterna}/experiencia.json<br/>baseInterna = EXP_INTERNAL_HOST aplicado a base "]
  vp1 --> frame["ExpFrame: sandbox + allow,<br/>dentro de DispositivoFrame se houver viewport"]
  vp2 --> frame
  frame --> fim([fim])

  %% A leitura do experiencia.json NAO e so de dev: stack com a tabela vazia
  %% servia a exp-01 esticada na janela em vez de dentro do aparelho.
```

A.21 Figura 9.3 — Contrato entre duas experiências

```
flowchart LR
  v["exp-70 (Vila do Lume)"] -->|payload bruto| mv["src/vila/ - quadrantes.ts: colapso 10 -> 4"]
  mv -->|"POST /atividades/:id/perfis/da-vila (professor)"| t["exp_01_exp_70_perfis_importados"]
  t -->|le| mi["src/perfis/largada.ts - decide o lado do feed"]
  mi --> i["exp-01 (Instanooria)"]

  %% Duas travas que CUSTAM alunos, de proposito:
  %%  1. valor nao observado (indicative|none) fica FORA da media — nunca zero
  %%  2. quadrante sem nenhum membro observado INVALIDA o perfil inteiro
  %% Sem perfil, o Instanooria cai no sorteio balanceado, que e honesto sobre
  %% nao saber. E `lado_forcado` e PRESERVADO no upsert.
```

A.22 Figura 10.1 — Canal analítico

```
flowchart LR
  exp["Experiencia: track(verbo, objeto, dados)"] --> sdk["TelemetryClient<br/>buffer em memoria; lote de 50 ou 10s"]
  sdk -->|"<= 50 eventos"| batch["POST /telemetry/batch<br/>OidcGuard; identidade vem do TOKEN"]
  batch --> topic[["Pub/Sub topic -telemetria"]]
  sdk -->|sendBeacon no pagehide| beacon["POST /telemetry/beacon<br/>token NO CORPO; responde 204 SEMPRE"]
  beacon --> topic
  topic -->|push autenticado| ingest["POST /telemetry/ingest<br/>PubSubPushGuard: ID token do Google, aud, email"]
  ingest --> pg[("telemetria_eventos<br/>INSERT multi-linha<br/>ON CONFLICT (event_id, ts) DO NOTHING<br/>particionada por mes")]

  %% Onde ha perda possivel, e SO ali: entre o track() e o flush.
  %% Do Pub/Sub em diante a entrega e DURAVEL — e por isso um buffer em Redis
  %% nao o substituiria: reintroduziria a perda que ele eliminou.
```

A.23 Figura 11.1 — Camada de IA

```
C4Component
  title Camada de IA - src/ai e a UNICA porta

  Container_Boundary(api, "API") {
    Component(g, "geracao/", "", "exp-01")
    Component(m, "motor/", "", "exp-01")
    Component(p, "planos/", "", "exp-90")
    Component(b, "bemestar/", "", "exp-91")
    Component(a, "POST /ai/gerar", "", "SDK")

    Container_Boundary(ai, "src/ai/ - a UNICA porta") {
      Component(q,  "QuotaService", "", "temSaldo(escola, usuario, perfil) ANTES; consumir DEPOIS")
      Component(pr, "AiProvider", "", "generate, generateJson, generateImage")
      Component(pp, "prompts.ts", "", "prompt de sistema; NUNCA sai daqui")
      Component(vx, "vertex.provider.ts", "", "default; ADC + REST")
      Component(mk, "mock.provider.ts", "", "testes e dev sem GCP")
    }
    ComponentDb(ia, "ia_uso_diario", "", "quota persistida por (escola, usuario, dia)")
  }
  System_Ext(vertex, "Vertex AI", "endpoint global")

  Rel(g, q, "1")
  Rel(m, q, "1")
  Rel(p, q, "1")
  Rel(b, q, "1")
  Rel(a, q, "1")
  BiRel(q, ia, "incremento atomico")
  Rel(q, pr, "SO com saldo")
  Rel(pr, pp, "usa")
  Rel(pr, vx, "AI_PROVIDER=vertex")
  Rel(pr, mk, "AI_PROVIDER=mock")
  Rel(vx, vertex, "HTTPS + ADC")
```

A.24 Figura 11.2 — *Guardrails* do bem-estar

```
flowchart TB
  ini([inicio]) --> abre["GET /bemestar/hoje - 3 itens da permutacao do ciclo"]
  abre --> sinal{"sinal de apoio?<br/>item 17 ou 21 >= 2, ou termo de risco"}
  sinal -->|sim| fixo["texto FIXO de apoio (CVV 188, SAMU 192, CAPS/UBS)<br/>a IA NAO e chamada<br/>origem_texto = 'seguranca'"]
  sinal -->|nao| saldo{tem saldo e provider?}
  saldo -->|nao| fall["texto FIXO do catalogo<br/>origem_texto = 'fallback'"]
  saldo -->|sim| ia["IA gera abertura, pergunta aberta e acolhimento<br/>recebe SO: dia da semana, mes, no do check-in,<br/>texto livre delimitado"]
  ia --> val{validarTextoDaIa?}
  val -->|barrado| fall
  val -->|passou| ok["origem_texto = 'ia'"]
  fixo --> fim([fim]); fall --> fim; ok --> fim

  %% Os 21 itens do DASS-21 sao FIXOS e VERBATIM. A IA nunca reescreve um item:
  %% reformular quebra a validacao psicometrica e invalida as faixas de corte.
  %% O check-in NUNCA deixa de abrir.
```

A.25 Figura 12.1 — Os quatro graus de medida

```
classDiagram
  class scored {
    >= 2 sinais independentes
    numero: number
    entra na media
    E PUBLICADO com numero
  }
  class indicative {
    exatamente 1 sinal
    numero: null
    descrito em TEXTO
    fora dos eixos e das medias
  }
  class zero {
    teve a chance e NAO foi
    numero: null
    ENTRA na media, como zero de verdade
  }
  class none {
    a oportunidade NAO chegou
    numero: null
    SAI da conta - nao e zero
  }
  note "A distincao mais importante do modelo e entre zero e none.\nTratar 'nao teve chance' como 'escolheu nao' faz uma crianca\nparecer ter recusado algo que nunca lhe foi oferecido."
```

A.26 Figura 13.1 — Mapa de rotas da API

```
flowchart TB
  subgraph PLAT["Plataforma - serve a todos os papeis"]
    au["/auth/login - sem papel"]; he["/healthz, /readyz - PUBLICO"]
    ex["/experiencias - GET qualquer; PUT admin-contrato"]
    pg["/progresso/:expId - aluno"]; te["/telemetry/* - ver capitulo 10"]
    us["/usuarios/* - gestor, admin"]; ai["/ai/gerar - autenticado + quota"]
    an["/analise/* - ADMIN-CONTRATO"]
  end
  subgraph E01["exp-01 - Instanooria"]
    atv["/atividades/* - professor"]; rn["/runs/* - aluno (/eventos: professor tb.)"]
    po["/posts/:id/comentarios - aluno"]; pf["/perfis/importar - chave de servico"]
  end
  subgraph OUT["Demais experiencias"]
    ut["/unootube/* - aluno"]; utt["/unootube/turmas/* - prof, gestor, admin"]
    vi["/vila/sessoes - aluno"]; vp["/vila/professor/* - PROFESSOR"]
    va["/vila/agregado/* - prof, gestor"]; pl["/planos/* - professor"]
    pref["/planos/referencias - chave de servico"]
    be["/bemestar/* - SO PROFESSOR, com posse"]
  end
```

A.27 Figura 14.1 — A linha da revelação

```
flowchart LR
  subgraph CLONE["Dentro do iframe, ANTES da revelacao - @edu/ui/ds-unootube"]
    feed["feed de videos"]; cromo["cromo, busca, menu"]
    play["playlists"]; canal["canal do aluno"]; toast["toast: 'Video salvo!'"]
  end
  subgraph PLATF["DEPOIS da revelacao, no MESMO iframe - @edu/ui/ds"]
    rank["ranking"]; d1["dialogo 1 da missao"]; d2["dialogo 2 da missao"]; fim["fechamento"]
  end
  CLONE ==>|REVELACAO| PLATF

  %% A linha e a REVELACAO, nao o arquivo. O toast NAO cruza: ele fala de
  %% dentro da ficcao. O criterio nao e "e depois no tempo?", e "quem fala?".
```

A.28 Figura 15.1 — *Pipeline* de entrega

```
flowchart TB
  push["push para main"] --> ver["PORTAO: ci.yml (typecheck, lint, test, build)"]
  ver --> auth["WIF + setup-gcloud (sem chave JSON)"]
  auth --> diff["o que mudou? git diff HEAD^ HEAD: api | hub | migrations"]
  diff --> build["turbo build (filtrado, ou tudo se 'forcar')"]

  build --> img["imagem da api -> Artifact Registry"]
  img --> mig["Cloud Run Job -migrate (na VPC, --wait)"]
  mig --> dep["gcloud run deploy api (revisao nova)"]

  build --> imgh["imagem do hub"] --> deph["gcloud run deploy hub"]

  deph --> rsync["gcloud storage rsync -> gs://{bucket}/{exp}/{ver}/"]
  rsync --> cc["Cache-Control: max-age=60 SO no index.html"]
```

```
  cc --> man["publicar-manifesto.mjs -> PUT /experiencias/:expId (token de 5 min)"]

  %% ORDEM 1: migracao ANTES do deploy — o schema novo tem de estar no banco
  %%          antes de a revisao nova receber trafego. --wait faz falhar.
  %% ORDEM 2: experiencias DEPOIS do hub — o PUT do manifesto e validado contra
  %%          o JWKS que o HUB serve.
```

A.29 Figura 16.1 — Estratégia de testes

```
flowchart TB
  v["SMOKE VISUAL - Playwright, navegador real<br/>walkthrough-aluno, -professor, -unootube, -vila-do-lume,<br/>-professor-vila, -ato1-vila, probe-pointerlock"]
  c["CONTRATO EXTERNO<br/>telemetria-push (guard do Pub/Sub), vertex-provider (cadeia ADC)"]
  i["INTEGRACAO COM POSTGRES<br/>tenancy, planos-tenancy, motor-concorrencia, motor-postgres,<br/>unootube-postgres, vila-postgres, vila-ato1, usuarios, planos, bemestar"]
  u["REGRA PURA - Vitest/Jest, milissegundos, sem banco<br/>vila-regras, unootube-regras, bemestar-regras, motor-pesos,<br/>motor-bolha, motor-sugestoes, motor-reversibilidade, bridge.test"]
  u --- i --- c --- v

  %% A base e larga de proposito: regras.ts puro em todo modulo de dominio.
  %% E o TOPO nao e "E2E de fluxo": e verificacao de que a INTERFACE apareceu.
  %% Icone virando texto, iframe vazio, cor inexistente, fonte removida pelo
  %% Next — tudo INVISIVEL para o compilador.
```

A.30 Figura 18.1 — Matriz de riscos

```
quadrantChart
  title Matriz de riscos - probabilidade x impacto
  x-axis "Probabilidade baixa" --> "Probabilidade alta"
  y-axis "Impacto baixo" --> "Impacto alto"
  quadrant-1 "CRITICO - resolver antes de nov/2026"
  quadrant-2 "Impacto alto, prob. baixa"
  quadrant-3 "Monitorar"
  quadrant-4 "Prob. alta, impacto baixo"

  "R-01 carga nao verificada": [0.55, 0.92]
  "R-02 retencao indefinida":  [0.90, 0.90]
  "R-04 fator humano unico":   [0.98, 0.88]
  "R-06 sem revisao externa":  [0.85, 0.86]
  "R-05 sem alerting":         [0.85, 0.58]
  "R-07 acessibilidade":       [0.85, 0.55]
  "R-12 catalogo de modelos":  [0.82, 0.50]
  "R-03 localidade de IA":     [0.98, 0.55]
  "R-08 particao sem job":     [0.20, 0.50]
  "R-09 midia manual":         [0.85, 0.22]
  "R-10 orfa sem retomada":    [0.50, 0.20]
  "R-11 dependencia do DS":    [0.50, 0.22]
  "R-13 WSO2 nao e o emissor": [0.50, 0.18]
```

Apêndice B

Variáveis de ambiente

Catálogo completo, derivado de `.env.example` e de `apps/api/src/config/env.ts`. Toda variável da API é validada com Zod **no *boot***: valor errado derruba o processo na subida, não no meio de uma requisição de aluno.

> **REGRA INVIOLÁVEL**
>
> Variável marcada **fecha** significa: *ausente* ⇒ a funcionalidade responde 401 ou 503, **nunca** fica aberta. Essa é a convenção de *fail closed* da plataforma (**RNF-09**).

B.1 API — `apps/api/.env`

B.1.1 Banco e identidade

Variável	Default	Papel
`DATABASE_URL`	— (obrigatória)	URL do PostgreSQL. Validada com `startsWith("postgresql://")`. Em produção vem do Secret Manager.
`OIDC_ISSUER`	— (obrigatória)	Emissor aceito. Hoje é o hub; amanhã o WSO2. **Não é segredo** — é URL pública, e vai em texto plano derivada de `hub_domain`, o que garante que `iss` e esperado não possam divergir.
`OIDC_JWKS_URI`	— (obrigatória)	JWKS do emissor.
`OIDC_AUDIENCE`	`plataforma-ap`	Audiência exigida no *token*.
`CORS_ORIGINS`	hub + *dev servers*	Lista explícita, separada por vírgula. **Nunca** `*`.
`PORT`	`3001`	Porta HTTP.

> **ATENÇÃO**
>
> O `issuer` do WSO2 é *tenant-qualified* (`.../t/{contrato}/oauth2/token`) e deve ser copiado **literalmente** do *discovery*, nunca montado por concatenação. Um `iss` montado à mão que difere por uma barra faz toda validação falhar com "*issuer* inválido" — e a mensagem não diz qual dos dois lados está errado.

B.1.2 Inteligência artificial

Variável	Default	Papel
`AI_PROVIDER`	`vertex`	`vertex` ou `mock`. Com `mock` a plataforma sobe sem nenhuma credencial de GCP.
`VERTEX_PROJECT_ID`	—	**Obrigatória com `AI_PROVIDER=vertex`**: o *boot* recusa sem ela. Sem projeto, a URL viraria `.../projects/undefined/...` e o erro só apareceria na primeira geração real.
`VERTEX_LOCATION`	`global`	O `3.6-flash` **só** existe em `global` (medido: 404 em `southamerica-east1`, `us-east1`, `us-central1`). Trocar por uma região exige trocar *também* o modelo.
`VERTEX_MODEL`	`gemini-3.6-fla`	Modelo do dia a dia.
`VERTEX_MODEL_LONGO`	`gemini-3.6-flas`	Modelo de saída longa (planos anuais). A variável separada existe para trocar um sem tocar no outro.
`VERTEX_IMAGE_MODEL`	`gemini-3.1-fla`	Arte da exp-01. O antigo `gemini-2.0-flash-exp` **não existe mais** — e por isso toda geração de imagem caía no *fallback* procedural. Quem trocar precisa conferir a proporção devolvida.
`VERTEX_THINKING_BUDGET`	`0`	Tokens de *raciocínio* saem do `maxOutputTokens`. `0` desligado (paridade com o aplicativo de origem); `-1` dinâmico; > 0 teto. Com pensamento livre, o plano anual voltava **vazio**.
`AI_DAILY_TOKEN_QUOTA_PER_`	20.000	Quota diária do aluno.
`AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR`	2.000.000	Quota diária de professor, gestor e admin. Um plano anual gasta ~220.000 *tokens* só de *prompt*.

> **NOTA**
>
> **Não existe variável para chave de API de IA** em lugar nenhum do projeto — nem aqui, nem em `.env.example`. A credencial é sempre ADC (**ADR-16**).

B.1.3 Armazenamento

Variável	Default	Papel
`ASSETS_DRIVER`	`local`	`local` (a API serve em `/assets`) ou `gcs` (*bucket* público atrás da CDN).
`ASSETS_BUCKET`	—	`{contrato}-assets`.
`ASSETS_PUBLIC_URL`	—	Base pública da CDN de *assets*.
`ASSETS_LOCAL_DIR`	—	Diretório local em *dev*.
`PLANOS_DRIVER`	`local`	`local` grava em `storage-privado/`; `gcs` usa o *bucket* **privado**.
`PLANOS_BUCKET`	—	`{contrato}-planos`. **Privado**, sem CDN, com versionamento de objetos.
`PLANOS_LOCAL_DIR`	—	Diretório local em *dev*.

B.1.4 Telemetria e chaves de serviço

Variável	Ausente	Papel
`TELEMETRIA_TOPIC`	console	Vazio em *dev* = eventos apenas *logados*. Não há Pub/Sub local.
`TELEMETRIA_PUSH_SA`	**fecha (401)**	*E-mail* da conta de serviço que assina o *push* do Pub/Sub.
`TELEMETRIA_PUSH_AUDIENCE`	**fecha (401)**	Audiência configurada na *subscription*. Sem as duas, `/telemetry/ingest` rejeita tudo.
`PERFIL_IMPORT_TOKEN`	**fecha (503)**	Cabeçalho `x-perfil-token` da ponte externa do Ato 1.
`PLANOS_IMPORT_TOKEN`	**fecha (503)**	Cabeçalho `x-import-token` do *script* de importação de referências curriculares.

B.2 Hub — `apps/hub/.env.local`

Variável	Escopo	Papel
`NEXT_PUBLIC_API_URL`	**cliente**	Endereço da API que o **navegador** usa. Vai ao `hub:init` como `apiUrl`.
`API_INTERNAL_URL`	servidor	Endereço da API que o *hub* usa (*authorize*, BFF, leitura de manifesto).
`NEXT_PUBLIC_CDN_EXP_URL`	**cliente**	Base dos *bundles*. Em produção, a CDN; em *dev* sem manifesto, cada experiência usa a porta de `lib/experiencias-dev.ts`.
`EXP_INTERNAL_URL`	servidor	De onde o hub **lê** o `experiencia.json`. Endereço **fixo** — serve quando há *uma* origem de *bundles* (a CDN, em produção).

EXP_INTERNAL_HOST	servidor	Troca **só o host** e **preserva a porta**. É o que serve em *dev* e em contêiner, onde cada experiência tem um *dev server* numa porta diferente.
HUB_ORIGIN	servidor	*Origin* do hub, passada à experiência por *query string* e usada como `targetOrigin`.
AUTH_SECRET	servidor	Segredo de sessão do NextAuth.
TOKEN_SIGNING_KEY_PEM	servidor	Par RS256 que assina os *tokens* de experiência. Gerar com `node apps/hub/scripts/gerar-chave-token.mjs`. **Obrigatória em produção**; vem do Secret Manager.
AUTH_WSO2_ISSUER	servidor	A **presença** desta variável registra o *provider* OIDC no NextAuth.
AUTH_WSO2_CLIENT_ID	servidor	Cliente OIDC do hub.
AUTH_WSO2_CLIENT_SECRET	servidor	Segredo do cliente OIDC.

> **ATENÇÃO**
>
> EXP_INTERNAL_URL *versus* EXP_INTERNAL_HOST é a confusão de configuração mais custosa do projeto, porque o sintoma não aponta para a causa: com um EXP_INTERNAL_URL fixo em *dev*, as cinco experiências leem o manifesto de *uma só* — e o UnooTube, que não declara `viewport`, aparece **dentro da casca de celular da Instanooria**.

B.3 Experiências — variáveis de *build* do Vite

Variável	Papel
VITE_VIDEOS_BASE_URL	exp-02: base dos 21 arquivos MP4 (52 MB), que ficam fora do *git* e fora do *bundle*. Vazio = `./videos` (`public/videos/`, *gitignorado*, em *dev*). Em produção: `https://<cdn-assets>/exp-02-unootube/videos`.

> **NOTA**
>
> **A exp-70 não tem nenhuma variável de ambiente.** Ela não usa IA, não usa *bucket* e não tem prazo de retenção declarado — o que é registrado explicitamente em `.env.example`, para que a ausência seja lida como decisão e não como esquecimento.

B.4 `vars` do repositório (CI/CD)

O *workflow* de *deploy* não conhece o nome de nenhum recurso: tudo entra por `vars` do repositório, cujos valores saem de `terraform output`. Há um *script* que os configura (`scripts/configurar-github-`

	Conteúdo
`GCP_PROJECT`	Projeto do contrato.
`REGION`	Região do Cloud Run e do Artifact Registry.
`AR_REPO`	Repositório do Artifact Registry.
`API_SERVICE`, `HUB_SERVICE`	Nomes dos serviços do Cloud Run.
`MIGRATE_JOB`	Nome do Cloud Run Job de migração.
`EXP_BUCKET`	*Bucket* dos *bundles*.
`ASSETS_BASE_URL`	Base pública de *assets* (entra no *build* da exp-02).
`API_URL`, `HUB_ORIGIN`	Endereços públicos, usados pelo *script* de manifesto.
`WIF_PROVIDER`, `DEPLOY_SA`	Identidade federada do CI.

Secret do CI	Conteúdo
`TOKEN_SIGNING_KEY_PEM`	A mesma chave RS256 do hub (`{contrato}-token-signing-key` no Secret Manager). O CI a usa para assinar o *token* de cinco minutos do *upsert* de manifesto.

B.5 Configuração mínima para subir sem GCP

Listing B.1 – A combinação que faz **RNF-14** ser verdade

```
# API
DATABASE_URL=postgresql://postgres:pass@localhost:5432/unooria
OIDC_ISSUER=http://localhost:3000
OIDC_JWKS_URI=http://localhost:3000/.well-known/jwks.json
AI_PROVIDER=mock          # sem GCP nenhum; devolve texto de exemplo
ASSETS_DRIVER=local       # a API serve /assets
PLANOS_DRIVER=local       # grava em storage-privado/
TELEMETRIA_TOPIC=         # vazio: eventos so no console
# TELEMETRIA_PUSH_* vazias  => /telemetry/ingest responde 401 (correto em dev)

# HUB
NEXT_PUBLIC_API_URL=http://localhost:3001
API_INTERNAL_URL=http://localhost:3001
NEXT_PUBLIC_CDN_EXP_URL=http://localhost:5173
HUB_ORIGIN=http://localhost:3000
AUTH_SECRET=troque-me
TOKEN_SIGNING_KEY_PEM=    # node apps/hub/scripts/gerar-chave-token.mjs --env
```

Apêndice C

Catálogo de *endpoints*

Todas as rotas da API, com papel autorizado. Convenções transversais e justificativas estão no Capítulo 13.

Marca	Significado
(sem)	`OidcGuard` apenas: qualquer papel autenticado.
`papel`	`@Roles(...)` restringe aos papéis listados.
público	Sem *guard*. Apenas `/healthz` e `/readyz`.
serviço	Chave em cabeçalho; **503** se a variável estiver ausente.
push	`PubSubPushGuard`; **401** se as variáveis estiverem ausentes.

C.1 Sanidade

Rota	Autorização	Semântica
`GET /healthz`	**público**	O processo está vivo.
`GET /readyz`	**público**	As dependências estão prontas (banco).

C.2 Autenticação

Rota	Autorização	Semântica
`POST /auth/login`	—	Autentica credencial da rede. Usada pelo *authorize* do hub.

C.3 Usuários

Rota	Autorização	Semântica
`GET /usuarios`	gestor, admin-contrato	Lista os usuários da escola.
`POST /usuarios/importar-csv`	gestor, admin-contrato	Corpo `text/csv` cru. Colunas: `nome,usuario,senha,papel,turma,serie`.
`POST /usuarios/:id/reset-senha`	gestor, admin-contrato	Redefine senha.

C.4 Manifesto de experiências

Rota	Autorização	Semântica
`GET /experiencias`	*(sem)*	Catálogo de manifestos.
`GET /experiencias/:expId`	*(sem)*	Um manifesto. É o que o hub lê para montar a URL do `iframe`.
`PUT /experiencias/:expId`	**admin-contrato**	*Upsert* do manifesto. Chamada pelo CI com *token* de cinco minutos.

C.5 Progresso genérico (SDK)

Rota	Autorização	Semântica
`GET /progresso/:expId`	*(sem)*	Carrega o progresso salvo.
`POST /progresso/:expId`	*(sem)*	Salva o progresso.
`POST /progresso/:expId/concluir`	*(sem)*	Marca conclusão.

C.6 Telemetria

Rota	Autorização	Semântica
`POST /telemetry/batch`	*(sem)*	Lote de até 50 eventos. A identidade é derivada do *token*; o que o cliente enviou nesses campos é descartado.
`POST /telemetry/beacon`	verificação manual	`204 sempre`. O *token* vem **no corpo**, porque `sendBeacon` não envia cabeçalho.
`POST /telemetry/ingest`	**push**	`204`. Consumo da *push subscription*. `ON CONFLICT (event_id, ts) DO NOTHING`.

C.7 IA

Rota	Autorização	Semântica
`POST /ai/gerar`	*(sem)*	Geração de texto pela SDK. Passa por `QuotaService` **antes** do *provider*; *prompt* de sistema fica no servidor.

C.8 exp-01 — atividades (professor)

Rota	Autorização	Semântica
`GET /atividades`	professor	Lista as atividades do professor.
`GET /atividades/:id`	professor	Uma atividade.
`POST /atividades`	professor	Cria.
`DELETE /atividades/:id`	professor	Exclui.
`POST /atividades/:id/limpar`	professor	Limpa as sessões da atividade.
`PATCH /atividades/:id/ia`	professor	Liga ou desliga a reação ao vivo, sem *deploy*.
`POST /atividades/:id/ativar`	professor	Ativa a atividade.
`POST /atividades/:id/etapa`	professor	Avança o relógio **coletivo** da turma.
`GET /atividades/:id/etapa`	professor	Etapa atual.
`GET /atividades/:id/monitor`	professor	Monitor da turma em tempo quase real.
`GET /atividades/:id/alunos/:runId`	professor	Uma sessão de aluno.

C.9 exp-01 — geração de conteúdo (professor)

Rota	Autorização	Semântica
`POST /atividades/:id/destilar`	professor	Destila o antagonismo e propõe a pergunta-chave. O professor aprova ou reescreve.
`PATCH /atividades/:id/antagonism`	professor	Aprova ou edita a destilação.

`POST /atividades/:id/gerar`	professor	Enfileira a geração. O *mix* é decidido pelo servidor.
`GET /atividades/:id/geracao/pr`	professor	Alvo do *polling*; também dispara a retomada de fila órfã.
`GET /atividades/:id/posts`	professor	Posts gerados, paginados.
`PATCH /posts/:id`	professor	Aprova, reprova ou edita.
`DELETE /posts/:id`	professor	Exclui.
`POST /posts/:id/regenerar`	professor	Regenera um post.

C.10 exp-01 — perfis e largada

Rota	Autorização	Semântica
`POST /atividades/:id/perfis`	professor	Importa perfis manualmente.
`POST /atividades/:id/perfis/da-`	professor	**O Ato 1 integrado**: colapsa o perfil da exp-70 nos quatro quadrantes. `lado_forcado` é **preservado** no *upsert*.
`GET /atividades/:id/perfis`	professor	Perfis importados.
`GET /atividades/:id/distribuic`	professor	Distribuição prevista dos lados.
`POST /atividades/:id/redistribuir`	professor	Redistribui a turma (grava `lado_forcado`).
`POST /perfis/importar`	**serviço**	Cabeçalho `x-perfil-token`. Compatibilidade com a ponte externa original.

C.11 exp-01 — motor de *feed* (aluno)

Rota	Autorização	Semântica
`POST /runs`	*(sem)*	Cria ou recupera a sessão; resolve a largada.
`GET /runs/boot`	aluno	Estado inicial da primeira tela.
`POST /runs/:id/entrar`	aluno	Nome no aplicativo e avatar.
`GET /runs/:id/feed`	aluno	**Próximo bloco curado.** *Lock* consultivo por `run`; saída sanitizada.

`POST /runs/:id/eventos`	**aluno, professor**	Canal **operacional** de telemetria. Síncrono e transacional.
`GET /runs/:id/fase`	aluno	Fase atual (trava a tela conforme a etapa da turma).
`POST /runs/:id/finish`	aluno	Encerra a sessão.
`POST /runs/:id/reset`	**professor**	Zera a sessão; recalcula a largada pela *mesma* regra.
`GET /runs/:id/busca`	aluno	Busca no *feed*.
`GET /runs/:id/salvos`	aluno	Itens salvos.
`GET /runs/:id/perdidos`	aluno	**Os posts que o aluno não viu** — a virada pedagógica.
`POST /runs/:id/conclusao`	aluno	Inclinação (A, B ou dúvida).
`POST /runs/:id/reflexao`	aluno	Texto de fechamento, sem limite.
`GET /runs/:id/reflexao`	aluno	Lê a reflexão.
`POST /runs/:id/student-post`	aluno	Post publicado pelo aluno.
`POST /runs/:id/reacoes`	aluno	Dispara reações de persona ao vivo.
`GET /runs/:id/dossie`	**aluno, professor**	Dossiê de fechamento.
`GET /posts/:id/comentarios`	aluno	Comentários de um post.

C.12 exp-02 — UnooTube

Rota	Autorização	Semântica
`GET /unootube/sessao`	aluno	Estado da sessão. **Nunca** devolve o gabarito.
`POST /unootube/salvos`	aluno	Salva um vídeo (máx. 3).
`POST /unootube/salvos/:videoId/remover`	aluno	Remove um salvo.
`POST /unootube/curtidos/:vide`	aluno	Curte. Segue livre mesmo depois da conclusão.
`GET /unootube/ranking`	aluno	Ranking com colegas **fictícios**.
`POST /unootube/credibilidade`	aluno	A virada: reordena por fontes e **congela** os salvos.
`POST /unootube/reiniciar`	aluno	Descongela e reinicia.

`GET /unootube/turmas`	professor, gestor, admin-contrato	Turmas com resultado.
`GET /unootube/turmas/:turmaId/resultados`	professor, gestor, admin-contrato	Resultados da turma.

C.13 exp-70 — Vila do Lume

Rota	Autorização	Semântica
`POST /vila/sessoes`	aluno	Abre a partida. **Atribui a bolha** (do servidor, balanceada na turma, com *lock*) e devolve os cartões — **só os desta criança e sem gabarito**.
`POST /vila/sessoes/:id/concluir`	aluno	Fecha a partida. O servidor **recalcula** o perfil; *payload* que traga chave de perfil é **recusado**.
`GET /vila/agregado/turmas`	professor, gestor	Turmas com partidas.
`GET /vila/agregado/turmas/:turm`	professor, gestor	Agregado da turma. **Sem valor psicométrico na listagem.**
`GET /vila/professor/turmas/:turmaId`	**professor**	Listagem da turma.
`GET /vila/professor/cartoes?tur`	**professor**	Textos das três bolhas **com** gabarito, para o guia do debate. Filtra pela *mesma* regra de elegibilidade.
`GET /vila/professor/alunos/:alunoId`	**professor**	A leitura individual. Abre pelo **nível**.
`DELETE /vila/professor/turmas/:tur`	**professor**	Zera a turma — **apagando as bolhas junto**. Não toca `exp_01_exp_70_perfis_importados`.

C.14 exp-90 — planos de aula

Rota	Autorização	Semântica
`GET /planos`	professor	Lista os planos do professor.
`POST /planos`	professor	**202.** Enfileira a geração.
`GET /planos/:id`	professor	Estado e, quando pronto, conteúdo. Alvo do *polling*.
`PUT /planos/:id/conteudo`	professor	Salva o HTML do editor, sanitizado por *allowlist*.
`POST /planos/:id/restaurar`	professor	Volta ao `gerado.md`.
`PATCH /planos/:id`	professor	Renomeia.

`DELETE /planos/:id`	professor	Exclui.
`GET /planos/referencias/dispo`	professor	Disciplinas e séries com material importado.
`GET /planos/referencias`	**serviço**	Lista as referências.
`POST /planos/referencias`	**serviço**	Importa. Cabeçalho `x-import-token`.
`DELETE /planos/referencias/:id`	**serviço**	Remove.

C.15 exp-91 — bem-estar

Rota	Autorização	Semântica
`GET /bemestar/resumo`	**professor**	**Leve, sem IA.** É o que o *card* do hub lê.
`GET /bemestar/hoje`	**professor**	Abre o *check-in*: 3 itens *verbatim* + textos de entorno.
`POST /bemestar/checkins/:id/respostas`	**professor**	Grava. No sétimo, pontua e classifica *server-side*.
`GET /bemestar/historico`	**professor**	Série do próprio professor.
`DELETE /bemestar/dados`	**professor**	Apaga tudo do próprio professor.

> **REGRA INVIOLÁVEL**
>
> Todas as cinco filtram `professor_id = req.auth.usuarioId` **além** da RLS por escola: dois professores da mesma escola não podem se ler. **Não existe rota de gestor nem agregado** nesta área — ausência deliberada (**ADR-10**).

C.16 Análise

Rota	Autorização	Escopo
`GET /atividades/:id/analise`	professor	Uma atividade da própria escola.
`GET /atividades/:id/analise/cs`	professor	Exportação.
`GET /analise/resumo`	**admin-contrato**	Toda a rede.
`GET /analise/atividades/:id`	**admin-contrato**	Uma atividade, cross-escola.
`GET /analise/csv`	**admin-contrato**	Exportação da rede.

C.17 Rotas do hub (Next.js)

Rota	Escopo	Semântica
`GET /.well-known/jwks.json`	**público**	JWKS da chave que assina os *tokens* de experiência. É o que a API valida — e o que o CI usa indiretamente ao publicar manifesto.
`GET /api/exp-token`	sessão	Emite um *token* novo de experiência. Alvo do `exp:token-expirando`.
`ALL /api/plataforma/[...path]`	sessão	BFF: *proxy* autenticado para a API.
`ALL /api/auth/[...nextauth]`	—	NextAuth.

Apêndice D

Payloads e esquemas de validação

Exemplos reais dos *payloads* que atravessam as fronteiras da plataforma, com os esquemas Zod que os validam. Todo *input* externo é validado; nenhum esquema aqui é decorativo.

D.1 Protocolo `postMessage` (v1)

Listing D.1 – `packages/sdk/src/types.ts` — os esquemas do protocolo

```ts
export const PROTOCOL_VERSION = 1;

export const AlunoSchema = z.object({
  id: z.string(), nome: z.string(),
  serie: z.string(),          // "6EF", "2EM"
  turmaId: z.string(), escolaId: z.string(),
});

export const HubInitSchema = z.object({
  type: z.literal("hub:init"),
  protocolVersion: z.number(),
  token: z.string(),
  aluno: AlunoSchema,
  apiUrl: z.string().url(),
  config: z.record(z.unknown()).default({}),
});

/** Mensagens POS-init, validadas com Zod nos DOIS lados. Unioes
 *  discriminadas — mensagem que nao casa e DESCARTADA com aviso, nunca
 *  despachada crua. Tudo aqui e ADITIVO ao v1: hub antigo ignora mensagem
 *  que nao conhece; experiencia antiga nunca recebe as novas sem pedir. */
export const HubParaExpSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("hub:token"), token: z.string() }),
  z.object({ type: z.literal("hub:pause") }),
  z.object({ type: z.literal("hub:resume") }),
]);

export const ExpParaHubSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("exp:ready"), protocolVersion: z.number() }),
  z.object({ type: z.literal("exp:concluir"),
             payload: ExpConcluirPayloadSchema.default({}) }),
  z.object({ type: z.literal("exp:token-expirando") }),
  z.object({ type: z.literal("exp:estado"), estado: z.record(z.unknown()) }),
]);
```

Listing D.2 – `hub:init` — o que a experiência recebe

```
{
  "type": "hub:init",
  "protocolVersion": 1,
  "token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "aluno": {
    "id": "8f2c1a44-...", "nome": "Ana",
    "serie": "8EF",
    "turmaId": "b17e...", "escolaId": "sp-001"
  },
  "apiUrl": "https://api-entreredes.unooria.com",
  "config": {}
}
```

D.2 *Token* de experiência

Listing D.3 – *Claims* do JWT — contrato congelado

```
{
  "sub":       "8f2c1a44-...",       // usuarioId (nao "alunoId": professor
                                     // passa pelo MESMO guard)
  "escola_id": "sp-001",             // vazio APENAS para admin-contrato
  "turma_id":  "b17e...",            // opcional
  "perfil":    "aluno",              // aluno | professor | gestor | admin-contrato
  "aud":       "plataforma-api",
  "iss":       "https://entreredes.unooria.com",
  "exp":       1756290000            // TTL de 15 minutos
}
```

> **NOTA**
>
> `sub` vira `usuarioId` nas *claims* tipadas da API (`src/auth/claims.ts`) — e não `alunoId`, porque professor, gestor e admin passam pelo mesmo *guard*. O nome do campo é documentação: um `alunoId` que às vezes contém um professor é uma armadilha para quem lê a *query* seis meses depois.

D.3 Telemetria — canal analítico

Listing D.4 – `src/telemetry/telemetry.controller.ts` — os esquemas

```
const EventoSchema = z.object({
  eventId:    z.string().uuid(),
  ts:         z.string(),
  expId:      z.string(),
  expVersion: z.string(),
  sessaoId:   z.string(),
```

```
  verbo:      z.string(),
  objeto:     z.string(),
  resultado:  z.record(z.unknown()).optional(),
  contexto:   z.record(z.unknown()).optional(),
});
// alunoId e escolaId NAO estao no esquema de entrada: sao SOBRESCRITOS a
// partir do token. O cliente nao PODE colocar PII no lugar do id.

const BatchSchema  = z.object({ eventos: z.array(EventoSchema).max(50) });

/** O beacon do pagehide NAO envia headers, entao o token vem no CORPO. */
const BeaconSchema = BatchSchema.extend({
  beacon: z.literal(true).optional(),
  token:  z.string().min(1),
});

/** Envelope de push subscription do Pub/Sub. */
const PushSchema = z.object({
  message: z.object({ data: z.string().optional(), messageId: z.string().optional() }),
  subscription: z.string().optional(),
});
```

Listing D.5 – `POST /telemetry/batch` — corpo

```
{
  "eventos": [
    {
      "eventId":    "3f9a2c10-8b41-4d2e-9c77-1a5e6b0d4f22",
      "ts":         "2026-08-27T11:14:03.512Z",
      "expId":      "exp-02-unootube",
      "expVersion": "1.0.0",
      "sessaoId":   "c81b5e77-2d94-4a10-b3f6-77c2e9a4d001",
      "verbo":      "interacted",
      "objeto":     "video/v07",
      "resultado":  { "acao": "salvou", "salvos": 2 },
      "contexto":   { "tela": "feed", "ordenacao": "relevancia" }
    }
  ]
}
```

D.4 exp-70 — o *payload* de uma partida

Este é o *payload* mais elaborado da plataforma, e o que carrega mais decisões de projeto por campo.

Listing D.6 – `POST /vila/sessoes/:id/concluir` — corpo (abreviado)

```
{
  "v": 1,
  "bundleVersion": "1.0.0",
  "duracaoSeg": 1042,          // do "ENTRAR NA VILA" ao fim: tempo de JOGO
  "duracaoTotalSeg": 1387,     // desde a pagina abrir, incluindo a intro
  "status": "completa",

```

```
  "resumo": {
    "fogueirasAcesas": 8, "pedrasQuebradas": 3, "aldeoesAjudados": 2,
    "lampioesAcesos": 4, "tochasAcesas": 6,
    "dilemasVistos": 3, "dilemasResolvidos": 2, "distanciaM": 1840
  },

  "medidas": {
    "cobertura": { "celulas": 62, "total": 144, "pisoObrigatorio": 31 },
    "movimento": { "movendoSeg": 712, "escuroSeg": 128, "luzSeg": 584,
                   "atalhos": 2, "desvioM": 340, "raioMax": 58 },
    "lume":      { "conversas": 4, "perguntasCaminho": 1, "lore": 2,
                   "ajuda": 1, "sugestaoSeguida": null },
    "tocha":     { "aldeao": 2, "lampiao": 3, "pedra": 1,
                   "passarela": 0, "retomadas": 1 },
    "pedra":     { "borda": true, "destaque": false,
                   "nomesApagados": 0, "nomesLidos": 5 },
    "fumaca":    { "avistadas": 3, "seguidas": 2 },
    "brinquedos": 4,
    "resgates":  [ { "at": 512, "firesLit": 5 } ]
  },

  "oportunidades": { "lampiao": 6, "aldeao": 4, "pedra": 1 },

  "dilemas": [ { "pairId": 0, "escolha": "aldeao", "emMs": 214000 } ],

  "eventos": [
    { "tipo": "chama_acesa",  "emMs": 61000,  "dados": { "zona": "praca" } },
    { "tipo": "tocha_gasta",  "emMs": 188000, "dados": { "destino": "aldeao" } },
    { "tipo": "pedra_gravou", "emMs": 402000, "dados": { "acao": "borda" } }
  ],

  "seguranca": {
    "apareceram": 4, "respondidas": 3, "ignoradas": 1,
    "cartoes": [
      { "id": "c1", "emMs": 240000, "viuOpcoes": true,
        "escolha": 2, "decisaoMs": 9400 },
      { "id": "c2", "emMs": 431000, "viuOpcoes": true,
        "escolha": 1, "decisaoMs": 1800 },
      { "id": "c3", "emMs": 655000, "viuOpcoes": false,
        "escolha": null, "decisaoMs": null }
    ]
  }
}
```

D.4.1 As decisões que cada campo carrega

Campo	Decisão registrada
Ausência de `sessaoLocalId`, `destino`, `iniciadaEm`	Identidade e turma vêm da *sessão autenticada*; repeti-las no *payload* criaria uma segunda fonte de verdade para o servidor conferir contra a primeira. O *id* da partida é emitido pela API (é a chave de idempotência) e o instante de início é cravado por ela.

`seguranca.cartoes[]` sem classificação	**Regra 3.** O aplicativo de origem recebia `seguras`, `risco`, `golpeAceito`, `legitimaRecusada` e `bolha` por cartão — o *cliente* classificando as próprias respostas. Agora ele manda o que *fez*, e a API corrige contra `catalogo.ts`. Um *bundle* adulterado já não pode se reportar como tendo recusado tudo.
`viuOpcoes`	Separa "deixei passar" de "nem vi". As duas caíam em `ignoradas`, e são crianças diferentes.
`decisaoMs`	O campo mais informativo do conjunto: cair em dois segundos e cair depois de dez são duas crianças diferentes — a primeira não avaliou, a segunda avaliou e foi convencida.
`oportunidades` — **a ausência de uma chave**	Significa "não observado", que é *diferente* de zero. Zero é "teve a chance e não foi" e entra na média do polo; não observado **sai da conta**.
`pedra.borda` *versus* `pedra.destaque`	Gravou onde ainda cabia, sem apagar ninguém (Realização) *versus* apagou um nome tendo a borda livre (Poder). **A distinção é o ponto.**
`lume.sugestaoSeguida`: `null`	`null` = a sugestão não chegou a ser testada. **Não** é "não seguiu".
`cobertura.pisoObriga`	Quantas células a rota obrigatória já toca. Sem isso, cobertura mediria *cumprimento do objetivo* em vez de iniciativa.
`eventos[].emMs`	Tempo **relativo** ao início. Nunca relógio de parede.
`eventos[].dados`	Número, booleano ou *string* de **vocabulário fechado** (ids das 8 zonas + Fogo Sagrado, destinos da Tocha, as duas ações da Pedra, ids dos cartões). Nome de criança não cabe num vocabulário fechado.
`bundleVersion`	Fica na linha para o *skew* de *publish* ser **visível**: quem estava com a aba aberta durante uma publicação termina a partida com o *bundle* anterior, e uma partida dura 15–25 minutos.

D.4.2 As duas travas estruturais

Listing D.7 – Por que `.strict()` e por que a inspeção acontece no BRUTO

```
/**
 * ATENCAO AO `.strict()` E A ORDEM DE `validarPayload`.
 *
 * A primeira versao disto no app de origem tinha as recusas mais importantes
 * escritas como `.refine()`, e elas NAO FUNCIONAVAM: o zod remove chaves
 * desconhecidas ANTES de rodar refine, entao `{ nome: "Lara" }` era descartado
 * em silencio e o refine recebia um objeto ja limpo. O payload passava e o nome
 * simplesmente sumia sem ninguem saber que tinha chegado. Por isso:
 *
 *   1. `.strict()` — chave desconhecida e ERRO, nunca descarte silencioso;
 *   2. `validarPayload()` inspeciona o dado BRUTO antes do parse, porque so o
 *      bruto ainda contem o que nao deveria ter vindo.
 */
```

Trava	Como funciona
`CHAVES_PROIBIDAS` (por *substring*)	`nome`, `name`, `apelido`, `nick`, `aluno`, `crianca`, `jogador`, `player`, `cpf`, `matricula`, `email`, `telefone`, `responsavel`, `professor`, `endereco`, `nascimento`. Por *substring*, porque `nomeCompleto`, `nome_do_aluno` e `playerName` são a mesma coisa para quem for reidentificar a criança.
`CHAVES_PROIBIDAS_EXATA`	`rg`, `ra`, `id_aluno`, `doc` — **só por igualdade**. `rg` casaria dentro de `perguntasCaminho`, e um falso positivo recusa a partida inteira de uma criança.
`CHAVES_DO_CONTRATO`	`nomesApagados` e `nomesLidos` são chaves legítimas que contêm um termo proibido como *substring* (contam nomes na Pedra — números, nunca texto). A lista é **fechada**: campo novo que colida tem de ser adicionado aqui *de forma consciente*.
`CHAVES_DE_PERFIL`	Recusa *payload* que traga o perfil calculado. O jogo sabe computá-lo e deliberadamente não o envia — é isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir.

> **NOTA**
>
> A varredura por nome de criança importa *mais* aqui do que no aplicativo de origem: lá o *bundle* não tinha nome nenhum em memória; aqui o *shim* entrega o primeiro nome da criança ao jogo, para o Lume chamá-la e para a Pedra dos Nomes. **Existe um nome a vazar agora** — e rejeitar é melhor que ignorar em silêncio, porque um campo de nome aparecendo é *bug do jogo*.

D.4.3 O cap de eventos, e por que ele caiu de 2000 para 200

```
.max(200)
/**
 * 200, e o teto REAL do jogo e ~22.
 *
 * MEDIDO no bundle: o jogo tem 7 pontos que gravam evento, e todos sao atos
 * discretos com guarda propria — 8 Chamas Sagradas, 4 aldeoes, 1 Borda + 3
 * Destaques, 4 cartoes da missao, 1 primeira conversa com o Lume. Nada por
 * quadro, nada por metro andado. `restart()` zera o log, entao replay nao
 * acumula.
 *
 * Herdamos 2000 do app de origem. Cabe baixar: o cap e a UNICA coisa que
 * limita o tamanho da linha se um cliente adulterado mandar lixo, e
 * 2000 x ~80 bytes = 160KB por partida (1,6GB num contrato de 10 mil alunos)
 * contra ~2KB reais. Com 200 ha 9x de folga para eventos novos.
 */
```

D.5 Manifesto de experiência

Listing D.8 – `public/experiencia.json` — exp-01, com `viewport`

```json
{
  "id": "exp-01-instanooria",
  "nome": "Instanooria — O Conteúdo que não chegou",
  "versao": "1.0.0",
  "series": ["8EF"],
  "capabilities": ["telemetria", "progresso", "ia"],
  "viewport": { "largura": 375, "altura": 812 },
  "verbos": {
    "attempted":  "entrou no aparelho",
    "completed":  "concluiu a missão (dossiê visto)",
    "progressed": "mudou de fase"
  }
}
```

Listing D.9 – `public/experiencia.json` — exp-90, ferramenta de professor

```json
{
  "id": "exp-90-plano-de-aula",
  "nome": "Gerador de Plano de Aula",
  "versao": "1.0.0",
  "series": [],
  "capabilities": ["telemetria", "ia"],
  "publico": "professor",
  "verbos": {
    "attempted":  "pediu a geração de um plano anual (disciplina + série)",
    "completed":  "salvou o plano pela primeira vez",
    "interacted": "exportou .doc, restaurou o original ou excluiu um plano"
  }
}
```

NOTA

`series: []` na exp-90 significa "não se aplica" porque `publico: "professor"`; na exp-70 significa **todas as séries**, porque é experiência de aluno *transversal*. O mesmo valor, duas leituras — e o que as distingue é `publico`. Quando o filtro por série voltar a valer, vazio tem de continuar querendo dizer "todas" para o público `aluno`.

D.6 Validação de variáveis de ambiente

Listing D.10 – `src/config/env.ts` — o *refine* que evita um incidente

```
}).superRefine((env, ctx) => {
  if (env.AI_PROVIDER === "vertex" && !env.VERTEX_PROJECT_ID) {
    ctx.addIssue({
      code: z.ZodIssueCode.custom,
      path: ["VERTEX_PROJECT_ID"],
      message:
        "obrigatorio com AI_PROVIDER=vertex (em dev, use AI_PROVIDER=mock " +
        "para subir sem GCP)",
    });
  }
});
```

```
12
13 export function validarEnv(config: Record<string, unknown>): Env {
14   const parsed = EnvSchema.safeParse(config);
15   if (!parsed.success) {
16     const erros = parsed.error.issues
17       .map((i) => `  ${i.path.join(".")}: ${i.message}`)
18       .join("\n");
19     throw new Error(`Variaveis de ambiente invalidas:\n${erros}`);
20   }
21   return parsed.data;
22 }
```

NOTA

Sem esse `superRefine`, a URL do Vertex viraria `.../projects/undefined/...` e o erro só apareceria na *primeira geração real* — provavelmente de um professor, em aula. A mensagem diz o que fazer, não só o que está errado.

D.7 Importação de usuários por CSV

Listing D.11 – `POST /usuarios/importar-csv` — corpo `text/csv` cru

```
1 nome,usuario,senha,papel,turma,serie
2 Ana Souza,ana.souza,trocar123,aluno,8A,8EF
3 Bruno Lima,bruno.lima,trocar123,aluno,8A,8EF
4 Carla Dias,carla.dias,trocar123,professor,,
```

NOTA

O corpo é `text/csv` **cru**, com teto de 2 MB — por isso `src/main.ts` registra um *body parser* explícito de texto além do JSON. `username` é único **por escola**: uma rede tem muitos `joao.silva`.

Apêndice E

Dicionário de dados

Catálogo das 27 tabelas de `public`, mais o *schema* `particoes`. Os diagramas entidade-relacionamento estão no Capítulo 8.

Regime	Significado
Tenant	Tem `escola_id NOT NULL` + `ENABLE`/`FORCE ROW LEVEL SECURITY` + política por escola. Acesso só por `comTenancy`.
Tenant+posse	O acima, *mais* filtro por `professor_id` em toda *query*: dois professores da mesma escola não se leem.
Catálogo	Sem `escola_id`, **fora** da RLS. Pertence ao contrato, não à escola.

E.1 Índice de tabelas

Tabela	Regime	Conteúdo
`escolas`	Tenant raiz	Unidades escolares do contrato
`turmas`	Tenant	Turmas por escola e série
`users`	Tenant	Usuários e papéis
`experiencias`	**Catálogo**	Manifesto de versões
`progresso`	Tenant	Progresso genérico da SDK
`ia_uso_diario`	Tenant	Contador de quota de IA
`telemetria_eventos`	Tenant, particionada	Canal analítico
`exp_01_activities`	Tenant	Atividade preparada pelo professor
`exp_01_authors`	Tenant	Personas fictícias
`exp_01_posts`	Tenant	Posts gerados
`exp_01_post_comments`	Tenant	Comentários dos posts
`exp_01_generation_jobs`	Tenant	Fila de geração

Tabela	Regime	Conteúdo
exp_01_runs	Tenant	Sessão de um aluno
exp_01_events	Tenant	Canal **operacional** de telemetria
exp_01_student_posts	Tenant	Posts publicados pelo aluno
exp_01_conclusoes	Tenant	Inclinação escolhida
exp_01_reflexoes	Tenant	Texto de fechamento
exp_01_dossiers	Tenant	Dossiê calculado
exp_01_comentarios_da_sessao	Tenant	Comentários da sessão
exp_01_exp_70_perfis_imp	Tenant	**Contrato entre duas experiências**
exp_02_sessoes	Tenant	Estado da missão do UnooTube
exp_70_sessoes	Tenant	Uma partida da Vila do Lume
exp_70_perfis	Tenant	Retrato Schwartz recalculado
exp_70_bolhas	Tenant	Bolha por aluno
exp_90_planos	Tenant+posse	Metadados do plano de aula
exp_90_referencias	**Catálogo**	Referências curriculares
exp_91_ciclos	Tenant+posse	Ciclo do DASS-21 (**sensível**)
exp_91_checkins	Tenant+posse	*Check-in* diário (**sensível**)

E.2 Tabelas da plataforma

E.2.1 escolas

Coluna	Tipo	Nota
id	TEXT PK	**Texto, não UUID.** É o *mesmo* valor que circula no *claim* escola_id do JWT e no set_config da RLS — um id legível (sp-001) vale mais que um UUID quando o valor aparece em três lugares de um diagnóstico.
nome	TEXT	
created_at	TIMESTAMPTZ(6)	

E.2.2 turmas

Coluna	Tipo	Nota
id	UUID PK	
escola_id	TEXT FK	

`nome`	`TEXT`	`UNIQUE (escola_id, nome)`: duas escolas podem ter “8A”.
`serie`	`TEXT`	`1EF–9EF` | `1EM–3EM`, validado com Zod na aplicação.

E.2.3 `users`

Coluna	Tipo	Nota
id	`UUID` PK	
`role`	`TEXT`	`aluno` | `professor` | `gestor` | `admin-contrato`.
escola_id	`TEXT?` FK	***CHECK* na migração:** `NOT NULL` *exceto* quando `role = 'admin-contrato'`. Nenhuma outra combinação é aceita pelo banco.
turma_id	`UUID?` FK	
`username`	`TEXT`	`UNIQUE (escola_id, username)` — único **por escola**, não globalmente: uma rede tem muitos `joao.silva`.
`display_name`	`TEXT`	
`password_hash`	`TEXT`	
`avatar`	`TEXT?`	

E.2.4 `experiencias` (catálogo)

Coluna	Tipo	Nota
exp_id	`TEXT` PK	Slug da experiência.
`nome`	`TEXT`	
`versao_ativa`	`TEXT`	***Rollback* = `UPDATE` deste campo.** O hub monta a URL do `iframe` com ele.
`series`	`TEXT[]`	**Vazio = TODAS as séries**, não nenhuma. É assim que uma experiência transversal se declara.
`capabilities`	`TEXT[]`	`telemetria`, `progresso`, `ia`, `avaliacao`, `tempo-real`.
`viewport`	`JSONB?`	`{largura, altura}` em pixels lógicos. Presente ⇒ o hub embeda dentro do `DispositivoFrame`.
`publico`	`TEXT`	`aluno` (default) | `professor`, com *CHECK*. `professor` ⇒ `/exp/*` responde 404.
`permissoes`	`TEXT[]`	Permissões de `iframe`; hoje só `pointer-lock`, com *CHECK* do vocabulário. **`fullscreen` NÃO entra aqui**: o `allow` do *frame* já o delega a todas, e campo de manifesto que não muda nada é pior que campo nenhum.

E.2.5 `progresso`

PK composta `(escola_id, aluno_id, exp_id)`; `dados JSONB DEFAULT '{}'`; `atualizado_em`. É o progresso *genérico* da SDK — estado rico de experiência vai para a tabela da experiência.

E.2.6 `ia_uso_diario`

PK composta `(escola_id, usuario_id, dia)`, com `dia` no formato `YYYY-MM-DD`; `tokens_usados INT`. Incremento atômico por `ON CONFLICT DO UPDATE ... RETURNING`, para que requisições simultâneas não se sobrescrevam.

E.2.7 `telemetria_eventos` (particionada)

Coluna	Tipo	Nota
event_id	`UUID` PK_1	Chave de *dedupe*, gerada pela SDK.
ts	`TIMESTAMPTZ(6)` PK_2	**Chave de partição.** A PK é composta porque o PostgreSQL exige a chave de partição em todo índice único — e a *dedupe* segue correta, porque a reentrega da mesma mensagem traz o *mesmo* `ts`.
`escola_id`, `aluno_id`	`TEXT`	`aluno_id` é **opaco**: nunca nome nem *e-mail*. Derivado do *token* no servidor.
`exp_id`, `exp_version`	`TEXT`	
`sessao_id`	`UUID`	
`verbo`, `objeto`	`TEXT`	Vocabulário xAPI simplificado.
`resultado`, `contexto`	`JSONB?`	Livres, por experiência.

Objeto	Nota
`idx_tel_escola_exp_ts`	Índice `(escola_id, exp_id, ts)`.
`particoes.telemetria_eventos_YYYY_MM`	Uma por mês, faixas declaradas até 2028-12. Vivem no *schema* `particoes` — em `public` cada uma viraria *drift* de *schema*.
`particoes.…_default`	**Rede de segurança.** Sem ela, evento com `ts` fora das faixas seria **recusado** — e telemetria perdida em silêncio é pior que telemetria mal guardada. Encher = faltou criar partição.

E.3 Tabelas da exp-01

E.3.1 `exp_01_activities`

Coluna	Tipo	Nota
id	`UUID` PK	
professor_id	`UUID` FK	
`titulo`, `disciplina`, `tema_geral`, `recorte`	`TEXT`	
`contexto_aula`	`TEXT?`	
`pergunta_chave`	`TEXT`	Gerada pela IA a partir do tema, validada pelo professor.
`pergunta_origem`	`TEXT`	`ia` | `professor` — **auditoria de autoria pedagógica**, e custa uma coluna.
`etapa`	`TEXT`	`investigacao` | `conclusao` | `debate` | `explicabilidade` | `perdidos` | `reflexao` | `fim`. **Relógio COLETIVO da turma:** o professor abre e fecha, e a tela do aluno trava conforme.
`ia_ao_vivo`	`BOOLEAN`	Personas reagem ao vivo. Desligável no meio da aula, **sem *deploy***.
`antagonismo`	`JSONB?`	Destilação **aprovada** pelo professor.
`categorias_neutras` `neutros_no_tema`	`TEXT[]`	Categorias dos neutros fora do tema; rótulos dos neutros dentro dele.
`estilo_visual`	`TEXT`	
`qtd_posts_tese`	`INT`	Posts que carregam o antagonismo; divisível por 8 (2 lados × 4 perfis).
`proporcoes`, `engine_config`	`JSONB`	Pesos e mixagem do motor.
`status`	`TEXT`	`rascunho` | `antagonismo` | `gerando` | `revisao` | `ativa` | `encerrada`.

E.3.2 `exp_01_posts`

Coluna	Tipo	Nota
`lado`	`TEXT`	`A` | `B` | `neutro`.
`perfil`	`TEXT`	`abertura` | `autotranscendencia` | `autopromocao` | `conservacao`.
`intensidade`	`SMALLINT`	1, 2 ou 3.

> **REGRA INVIOLÁVEL**
>
> `lado`, `perfil` e `intensidade` são **a variável independente do experimento**. Eles existem no banco e são **removidos pelo sanitizador** antes de o *feed* sair da API. Se chegassem ao cliente, a experiência poderia exibi-los — e o aluno saberia de que lado o algoritmo o colocou antes da revelação.

Coluna	Tipo	Nota
`formato`	`TEXT`	`feed` | `story` | `enquete`.
`caption`, `hashtags`, `emotional_hook`	`TEXT`/`TEXT[]`	Gerados pela IA.
`image_brief`	`TEXT`	*Prompt* da arte.
`image_url`	`TEXT?`	URL absoluta da CDN de *assets*.
`image_status`	`TEXT`	`pendente` | `ok` | `fallback`. `fallback` = arte procedural, porque a geração falhou — e a atividade não trava.
`likes_count`, `timestamp_label`	`INT`/`TEXT`	Ficção visual do clone.
`aprovado`	`BOOLEAN`	O professor revisa antes de ativar.

Índice de curadoria: `(activity_id, lado, perfil, intensidade, aprovado)` — é exatamente a consulta que o motor faz a cada bloco.

E.3.3 `exp_01_runs`

Coluna	Tipo	Nota
`fase`	`TEXT`	`onboarding` | `feed` | `encerramento` | `conclusao` | `revelacao` | `dossie` | `comparacao`.
`lado_inicial`	`TEXT`	`A` | `B`. Vem da afinidade do perfil; **nunca muda**.
`lado_origem`	`TEXT`	`perfil` | `sorteio` | `redistribuido` — **auditoria de por que** aquele aluno caiu naquele lado. Sem isso, uma reclamação de professor não tem resposta.
`perfil_jogo`	`JSONB?`	Perfil do Ato 1, **congelado** na criação da `run`.
`perfil_origem`	`TEXT`	`jogo` | `professor` | `ausente`.
`engine_state`	`JSONB`	Estado do motor. Lido e escrito *inteiro*, sempre dentro do mesmo *lock* — normalizá-lo daria *joins* sem ganho de consulta.
`nome_no_app`	`TEXT?`	O nome que a criança digitou. **Fica na `run`, não em `users`**, porque a conta é **do aparelho**: outra turma reusa o mesmo *login*.
`avatar_escolhido`	`TEXT?`	Mesma razão.

`UNIQUE (activity_id, aluno_id)`: uma sessão por aluno por atividade.

E.3.4 `exp_01_events` — o canal operacional

Coluna	Tipo	Nota
id	`BIGSERIAL` PK	Ordenação por inserção é significativa aqui — diferente do canal analítico.
`tipo`	`TEXT`	`visto` | `curtiu` | `descurtiu` | `salvou` | `abriu_comentarios` | `comentou` | `compartilhou` | `seguiu` | `deixou_seguir` | `nao_interessa` | `denunciou` | `enquete` | `rever` | `abriu_perfil`.
`payload`	`JSONB?`	`{dwellMs, dwellNorm}`, `{texto}`, `{opcao}`, `{authorId}`…
`client_ts`	`BIGINT`	Relógio do cliente, guardado como *dado*, não usado como verdade.

REGRA INVIOLÁVEL

***Dwell* nunca passa pelo Pub/Sub.** A latência e a falta de ordenação quebrariam a adaptação do *feed* e o *lock* por `run`. Esta tabela é o canal *operacional*; `telemetria_eventos` é o analítico (Capítulo 10).

E.3.5 `exp_01_exp_70_perfis_importados`

Coluna	Tipo	Nota
`escores`	`JSONB`	Os quatro quadrantes normalizados 0–1.
`dominante`	`TEXT`	Um dos quatro, ou `misto`.
`origem`	`TEXT`	`jogo` | `professor`.
`lado_forcado`	`TEXT?`	`A` | `B` quando o professor redistribuiu a turma; `NULL` = segue a afinidade. **PRESERVADO no *upsert*** — importar sozinho, ou levar o objeto inteiro no `update`, desfaria em silêncio uma redistribuição feita antes da aula.
`medido_em`	`TIMESTAMPTZ`	

E.4 Tabelas das demais experiências

E.4.1 `exp_02_sessoes`

Coluna	Tipo	Nota
(escola_id, aluno_id)	PK composta	Uma linha por aluno.

`salvos`	`TEXT[]`	Ids do catálogo. *CHECK* `cardinality(salvos) <= 3` na migração.
`curtidos`	`TEXT[]`	Segue livre mesmo após a conclusão.
`ranking_desbloqueado`	`BOOLEAN`	***Latch***: remover um salvo depois *não* tranca o ranking de novo. Estado que anda para trás confunde a criança e o professor.
`credibilidade_adicionada`	`BOOLEAN`	A conclusão pedagógica. **Congela os salvos**: o *score* já foi reportado, e nota e estado não podem divergir.
`concluido_em`	`TIMESTAMPTZ?`	*CHECK*: conclusão implica `concluido_em`.
`reinicios`	`SMALLINT`	Reiniciar descongela.

REGRA INVIOLÁVEL

O gabarito (`factual`) **não está aqui** e não é devolvido em nenhuma rota de aluno: vive em `src/unootube/catalogo.ts`.

E.4.2 `exp_70_sessoes`

Coluna	Tipo	Nota
id	`UUID` PK	**Emitido pela API.** No aplicativo de origem o cliente escolhia a própria chave de idempotência, e duas crianças na mesma máquina colidiam — a segunda partida voltava "duplicada" e desaparecia.
turma_id	`UUID?` FK	**Desnormalizado:** turma *no momento da partida*. O aluno troca de turma entre anos letivos, e o resultado precisa continuar na turma em que foi medido.
`exp_version`	`TEXT`	Versão do *bundle* que jogou — torna o *skew* de *publish* visível em vez de silencioso.
`schema_version`	`SMALLINT`	Versão do contrato de *payload*.
`bolha`	`TEXT`	`A` | `B` | `C`. Atribuída pela **API**, nunca pelo cliente, e copiada aqui porque **o relatório abre pelo NÍVEL**, antes do que ela fez.
`cartoes_servidos`	`SMALLINT`	Quantos a API *entregou* — $\neq$ **quantos apareceram**. Zero $\Rightarrow$ a missão não aconteceu, e o relatório precisa dizer isso em vez de "ela ignorou tudo".
`aberta_em`, `concluida_em`	`TIMESTAMPTZ`	Cravados pelo **servidor**. O aplicativo de origem derivava do relógio do cliente, e ele mesmo documentava que a hora numa máquina de escola costuma estar errada.
`duracao_seg`	`INT?`	Tempo de **jogo** — denominador dos ritmos.
`duracao_total_seg`	`INT?`	Desde a página abrir, incluindo a intro. Uma criança de 6 anos leva vários minutos ali, e sem o total o professor cronometra a atividade inteira e vê um número bem menor no painel.

`status`	`TEXT`	`aberta` | `completa` | `abandonada`. `aberta` que nunca fecha é partida perdida — e saber quantas são é *um dado sobre o jogo*.
`fogueiras_acesas` … `distancia_m`	`SMALLINT`/`INT`	Resumo **tipado**: o que o painel consulta e ordena. O detalhe fica no *payload*.
`payload`	`JSONB?`	O corpo bruto, íntegro. **Única fonte para recomputar** o perfil quando `VERSAO_MAPEAMENTO` subir.
`leitura_mensagens`	`JSONB?`	A leitura da missão, **persistida**: o relatório sobrevive a qualquer expurgo futuro do *payload* e não precisa recomputar a cada visita.

> **DECISÃO DE PROJETO**
>
> **Não há `expires_at`** (**ADR-13**). Houve, de 90 dias, e nenhuma rotina que o cumprisse. Coluna de prazo que ninguém lê é pior que prazo nenhum.

E.4.3 `exp_70_perfis`

Coluna	Tipo	Nota
`versao_mapeamento`	`SMALLINT`	Hoje **3**, herdada do piloto para os dados serem comparáveis. Sobe a cada mudança de peso, *probe* ou fórmula; **nunca** reescreve uma versão publicada.
`valores`	`JSONB`	Os dez valores em 0–100, e **`null` NÃO É ZERO**: é valor que não recebeu número, porque tem uma fonte de medida só ou porque a oportunidade não chegou.
`graus`	`JSONB`	`scored` | `indicative` | `zero` | `none` por valor. **A distinção mais importante do modelo.**
`abertura_vs_conservac`	`FLOAT?`	Eixo do circumplexo. **`NULL` quando um dos polos não foi observado** — eixo com polo não observado não é zero, é indefinido.
`transcendencia_vs_autopromocao`	`FLOAT?`	Idem.
`amostra_suficiente`	`BOOLEAN`	Falso ⇒ o painel abre avisando que **não sabe**. Não é rigor estatístico: é impedir que uma partida sem escolha vire uma frase sobre quem a criança é.

`UNIQUE (sessao_id, versao_mapeamento)`: quando a hipótese mudar, nasce uma linha nova e as antigas ficam — porque comparar as duas é o único jeito de saber o que a mudança fez.

E.4.4 `exp_70_bolhas`

PK composta `(escola_id, aluno_id)`; `bolha`, `atribuida_em`.

> **DECISÃO DE PROJETO**
>
> **Por aluno, não por partida.** Repetir a partida não pode trocar o nível: a folha do professor diz "esse estava na C", e o relatório abre pelo nível. A atribuição é balanceada **dentro da turma** — o aplicativo de origem fazia `nº do aluno mod 3`, que dava terços de graça porque os números eram contíguos de 1 a 40; *hash* de UUID dá terços na média da rede e pode dar 14/6/5 numa sala de 25 — e aí o debate da missão não acontece, porque ele depende de as três bolhas estarem na mesma sala.

E.4.5 `exp_90_planos` e `exp_90_referencias`

Coluna	Tipo	Nota
`status`	`TEXT`	`pendente` | `gerando` | `pronto` | `erro`, com *CHECK*.
`tentativas`	`SMALLINT`	Retomadas de geração *interrompida* (*restart*, *deploy*), não de falha da IA. Máximo 2.
`iniciado_em`	`TIMESTAMPTZ?`	`gerando` há mais de 12 minutos sem *worker* nesta instância = órfã.
`tokens_usados`	`INT`	
`caminho_gerado_md`	`TEXT?`	`planos/{escolaId}/{id}/gerado.md` — Markdown da IA, **imutável**.
`caminho_html`	`TEXT?`	`atual.html` — o que o editor salvou, sanitizado por *allowlist*.

`exp_90_referencias` é **catálogo do contrato**: sem `escola_id`, fora da RLS, com `serie NULL` significando "vale para todas as séries da disciplina" (ex.: o currículo da rede). Escrita só pelo *script* de importação.

E.4.6 `exp_91_ciclos` e `exp_91_checkins`

Coluna	Tipo	Nota
`ordem_itens`	`INT[]`	Permutação dos itens 1..21 sorteada na abertura do ciclo. Cada *check-in* leva 3 itens, um de cada subescala.
`expira_em`	`TIMESTAMPTZ`	21 dias. Ciclo não fechado vira `expirado`, sem pontuação.
`depressao`, `ansiedade`, `estresse`	`INT?`	Já $\times 2$ (escala do DASS-42), preenchidas **ao concluir** o sétimo *check-in*, no servidor.
`numero`	`SMALLINT`	1..7 dentro do ciclo.
`dia`	`TEXT`	`YYYY-MM-DD` no fuso `America/Sao_Paulo`. `UNIQUE (escola_id, professor_id, dia)`: um por dia.
`itens`	`INT[]`	Os 3 itens deste *check-in*.

`respostas`	`JSONB?`	`[{ item, valor }]` ×3, valor 0..3.
`texto_livre`	`TEXT?`	**Nunca vai a log nem à telemetria.** Só contagens.
`abertura`, `pergunta`, `acolhimento`	`TEXT`	Os textos de entorno que o professor *viu* — guardados, não regerados.
`origem_texto`	`TEXT`	`ia` | `fallback` (IA indisponível ou barrada pelo *guardrail*) | `seguranca` (sinal de apoio — texto fixo, a IA nem é chamada).
`sinal_de_apoio`	`BOOLEAN`	Item 17 ou 21 ≥ 2, ou termo de risco no texto.
`tokens_ia`	`INT`	

REGRA INVIOLÁVEL

Dado sensível de saúde (LGPD art. 11). RLS por `escola_id` **mais** posse por `professor_id` em toda *query*; **nenhuma** rota de gestor ou agregado; e o próprio titular apaga tudo com `DELETE /bemestar/dados`.

E.5 O que o Prisma não modela

ATENÇÃO

Ler apenas `schema.prisma` dá uma visão **incompleta** do banco. Vivem *só* na migração:

- políticas de **RLS** (`ENABLE`, `FORCE`, `CREATE POLICY`);
- ***CHECK* constraints** (papel *versus* `escola_id`, cardinalidade dos salvos, vocabulário de `permissoes`, coerência de conclusão, XOR de ancoragem de comentário);
- **partições** e o *schema* `particoes`;
- **`GRANT`** para a *role* de aplicação.

Os campos correspondentes são `TEXT`/`SMALLINT` no *schema*, e a validação de aplicação é feita com Zod.

Apêndice F

Operação

F.1 Comandos de desenvolvimento

Listing F.1 – Subir a plataforma

```
# --- Tudo em container, sincronizado com o editor (RECOMENDADO)
pnpm docker:dev          # pg + api + hub + as 5 experiencias, cada um com watcher
pnpm docker:dev:logs     # acompanha a recompilacao
pnpm docker:dev:deps     # DEPOIS de mexer em dependencia; e reiniciar
pnpm docker:dev:down     # desce

# --- So o banco em container, o resto no host
docker compose up -d pg
cp .env.example apps/api/.env && cp .env.example apps/hub/.env.local   # e editar
node scripts/gerar-chave-token.mjs --env      # chave RS256 do hub
pnpm --filter @edu/api migrate:dev            # migrations (Prisma + RLS)
pnpm --filter @edu/api seed                   # escola-demo, prof/gestor/aluno01-40
pnpm --filter @edu/api seed:demo              # atividade DEMO com 92 posts, sem IA
pnpm dev                                      # hub :3000, api :3001, exp :5173

# --- Imagens congeladas (mais proximo da producao)
docker compose up -d --build                  # migrations e seed no 1o up
```

> **ATENÇÃO**
>
> As portas são as mesmas nos três modos, então **não dá para rodar `pnpm docker:dev` e `pnpm dev` ao mesmo tempo**. Se salvar um arquivo e nada recompilar: `DEV_POLLING=1 pnpm docker:dev`.

Listing F.2 – Experiências, uma a uma

```
pnpm --filter @edu/exp-01-instanooria    dev   # :5173
pnpm --filter @edu/exp-02-unootube       dev   # :5174 (videos em public/videos)
pnpm --filter @edu/exp-90-plano-de-aula  dev   # :5175 -> /professor/plano-de-aula
pnpm --filter @edu/exp-91-bem-estar      dev   # :5176 -> /professor/bem-estar
pnpm --filter @edu/exp-70-vila-do-lume   dev   # :5177 -> /exp/exp-70-vila-do-lume

pnpm new:exp exp-07-fracoes                    # scaffolda do _template
```

Listing F.3 – O IdP sobe à parte

```
docker compose -f wso2/docker-compose.yml up -d    # IS :9443, pg dele :5433
docker compose -f wso2/docker-compose.yml watch    # olha conf/ e bootstrap/
docker compose -f wso2/docker-compose.yml logs bootstrap   # imprime as envs OIDC
```

Listing F.4 – Verificação e build

```
pnpm turbo typecheck lint test                # o portao da Definition of Done
pnpm turbo build --filter=@edu/exp-01         # 1 experiencia (+ deps)
pnpm turbo build --filter='...[HEAD^]'        # so o que mudou (o CI usa isso)
pnpm build:isolado                            # build de PRODUCAO do hub
                                              # (HUB_DIST_DIR=.next-prod)
pnpm --filter @edu/api test
```

ATENÇÃO

Validar *build* de produção **sempre** com `pnpm build:isolado`. Um `next build` comum apaga o `.next` do *dev*, e o sintoma é "tela branca" ou "CSS bugado" depois — que se confunde com bug de aplicação. E `apps/hub/next-env.d.ts` alterna entre `./.next` e `./.next-prod`: modificado depois de um `build:isolado` = **descartar, não *commitar***.

F.2 *Smoke* visual

```
node scripts/walkthrough-aluno.mjs            testes/wt-aluno
node scripts/walkthrough-professor.mjs        testes/wt-professor
node scripts/walkthrough-aluno-unootube.mjs   testes/walkthrough-unootube
node scripts/walkthrough-vila-do-lume.mjs     testes/walkthrough-vila
node scripts/walkthrough-professor-vila.mjs   testes/wt-prof-vila 8A
node scripts/walkthrough-ato1-vila.mjs        testes/wt-ato1
node scripts/probe-pointerlock.mjs
```

Pré-requisito	Detalhe
Plataforma no ar	Contêineres ou `pnpm dev`.
Chrome do **sistema**	Obrigatório para o *walkthrough* da exp-70 e para o *probe* de *pointer lock*: o Chromium do Playwright não concede *pointer lock* nem no topo da página.
Logins do *seed*	`gestor.demo` → `/admin/usuarios`; `prof.demo` → `/professor`; `aluno01`–`aluno40` → `/aluno`; `aluno4a01` para a exp-02. Senha de todos: `unooria123`.
Bancada de *embed*	`/dev-embed?src=http://localhost:5174`. Sanidade do *design system*: `/ds-smoke` (só em *dev*).

F.3 *Design system*

```
npm view @unooria/design-system version         # qual e a ultima publicada
pnpm up -r @unooria/design-system --latest      # sobe em ui/hub/exps de uma vez
                                                # (^0.1.x NAO sobe minor)
cd ../storybook-unooria && npm run storybook     # :6007, o catalogo das 6 secoes
```

ATENÇÃO

Commitar o *lockfile* **e** os `package.json` juntos. E lembrar: **quem importa, declara** — todo aplicativo que escreve `@unooria/design-system` no código tem a dependência no próprio `package.json`, hub incluído.

F.4 Publicação e reversão

Listing F.5 – Reversão — as três formas, por velocidade

```
# 1. EXPERIENCIA (o mais rapido: sem build, sem imagem, sem deploy)
node apps/hub/scripts/publicar-manifesto.mjs exp-02-unootube --versao 0.9.0
#    Requer: TOKEN_SIGNING_KEY_PEM, API_URL, HUB_ORIGIN

# 2. HUB ou API (revisao do Cloud Run)
gcloud run services update-traffic "$HUB_SERVICE" \
  --region "$REGION" --project "$GCP_PROJECT" --to-revisions "<revisao>=100"

# 3. CONTEUDO DE PROFESSOR (versionamento de objetos do bucket de planos)
gcloud storage ls --all-versions \
  "gs://$PLANOS_BUCKET/planos/$ESCOLA/$PLANO/atual.html"
```

Listing F.6 – Publicar tudo (primeiro *deploy* de um contrato)

```
# GitHub Actions -> workflow "deploy" -> Run workflow -> forcar: true
# Ignora o diff e publica hub, api e TODAS as experiencias.
```

F.5 Provisionamento de um contrato novo

1. No repositório de Terraform: instanciar o *module* `contrato` com o novo prefixo. **Nada é criado por *script* deste repositório.**
2. `terraform output` → configurar as `vars` do repositório (`scripts/configurar-github-ci.sh`).
3. Gerar o par RS256 e gravar em `{contrato}-token-signing-key` no Secret Manager; espelhar no *secret* `TOKEN_SIGNING_KEY_PEM` do GitHub.
4. Conceder `roles/aiplatform.user` à conta de serviço `{contrato}-api`. **Sem esse papel, toda geração falha com 403 do Vertex** — e o 403 vem do Vertex, o que se confunde com quota esgotada.

5. Verificar que a *role* de banco **não** tem `BYPASSRLS` nem `SUPERUSER` (SQL de verificação no *runbook* do repositório de infra).
6. Executar o *deploy* com `forcar: true`.
7. Importar usuários por CSV (`POST /usuarios/importar-csv`).
8. Importar as referências curriculares: `node scripts/importar-referencias-plano.mjs` com `PLANOS_IMPORT_TOKEN`.
9. **Manual, e pendente de automação:** `gsutil rsync` da mídia pesada da exp-02 para o *bucket* de *assets* (`R-09`).
10. Rodar os *walkthroughs* contra o ambiente novo.

F.6 Diagnóstico: sintoma → causa

Esta tabela existe porque, nesta plataforma, o sintoma quase nunca aponta para a causa.

Sintoma	Causa provável
Ícone virando texto (“addInvestigação”)	O Next removeu o `@import` externo do CSS do *design system*, e `Material Symbols Rounded` é fonte de ligadura. Verificar `FontesDaPlataforma` em `casca-plataforma.tsx`. O `getComputedStyle` *responde* “Open Sans”, então não confie nele.
Elemento branco no branco	Classe de cor que não existe em `plataforma.css`. Não é erro em lugar nenhum. Checar o `backgroundColor` **computado**.
Experiência esticada na janela em vez de dentro do aparelho	`viewport` não lido: tabela `experiencias` vazia *e* `EXP_INTERNAL_HOST` ausente ou errado.
O UnooTube aparece dentro da casca de celular da Instanooria	`EXP_INTERNAL_URL` fixo em *dev*: as experiências estão lendo o manifesto de *uma só*. Usar `EXP_INTERNAL_HOST`, que preserva a porta.
`iframe` vazio	*Handshake* expirado, *dev server* fora do ar, ou *origin* divergente. Ver o console: a SDK avisa o descarte por esquema.
Câmera do jogo não anda	`allow-pointer-lock` ausente: `permissoes` não chegou (manifesto vazio + `experiencia.json` não lido). `requestPointerLock()` morre com `SecurityError` e nada diz por quê.
Vídeo não toca (`play()` rejeitado em silêncio)	`autoplay` não delegado no `allow` do *frame*: a *Permissions-Policy* tem *default* `'self'`.
`confirm()` da experiência não aparece	`allow-modals` ausente no *sandbox*: o navegador **ignora** e devolve `false`.
Toda geração de imagem sai como *fallback* procedural	`VERTEX_IMAGE_MODEL` aponta para um modelo que saiu do catálogo. Verificar no Vertex.
Arte cortada pela metade	Proporção não pedida no `imageConfig`: o modelo devolve 1408×768 e o `cover` de 1080×1350 descarta a largura. Pedir `4:5`.

Sintoma	Causa provável
Plano de aula volta vazio com `finishReason: MAX_TOKENS`	`VERTEX_THINKING_BUDGET` diferente de `0`: os *tokens* de pensamento saem do `maxOutputTokens`.
Toda chamada de IA falha com 403	A conta de serviço não tem `roles/aiplatform.user`. O 403 vem do *Vertex*, não da API — e se confunde com quota esgotada.
IA não funciona no *host* nem no contêiner	ADC ausente. `gcloud auth application-default login`; os *compose* montam o arquivo em `/adc`.
API não sobe, reclamando de `VERTEX_PROJECT_ID`	Correto: com `AI_PROVIDER=vertex` a variável é obrigatória. Em *dev*, use `AI_PROVIDER=mock`.
`/telemetry/ingest` responde 401	Em *dev*, correto (`TELEMETRIA_PUSH_*` vazias = rota fechada). Em produção, conferir SA e *audience* contra a *subscription*.
Validação de *token* falha com "*issuer* inválido"	`iss` montado por concatenação. O do WSO2 é *tenant-qualified* e deve ser copiado **literalmente** do *discovery*.
Um bloco de *feed* desapareceu; posts repetidos	*Lock* consultivo não tomado. Verificar `pg_advisory_xact_lock(runId)` e o `timeout` de 20 s.
`comTenancy sem escola`	Rota sem `OidcGuard`/`TenancyInterceptor`, ou chamada fora de requisição sem `escolaId` explícito. **É o comportamento correto** — a alternativa era consultar sem filtro.
A sala inteira caiu na bolha A	*Lock* consultivo por turma não tomado em `POST /vila/sessoes`. A sala clica junto.
Partida da exp-70 recusada	`.strict()` pegou chave desconhecida, ou a varredura achou chave de nome, ou o *payload* trouxe perfil. **Recusa é o comportamento correto**; chave nova legítima precisa entrar no contrato *de forma consciente*.
Dependência nova não aparece no contêiner	`node_modules` do contêiner é volume próprio. `pnpm docker:dev:deps` + reiniciar.
Vite recusa a conexão do hub em contêiner	`allowedHosts` sem `host.docker.internal`. O Vite 5.4+ recusa `Host` que não conheça.
Vite subiu numa porta diferente	Segunda instância da mesma experiência. O hub passa a ler o manifesto da experiência errada.
***Link* sublinhado, marcador de lista de volta**	*Preflight* do Tailwind desligado. O pacote estiliza *componentes*, não elementos.
Sobrescrita de classe do *design system* não pega	Peso igual: o CSS do pacote é injetado *depois*. Usar **dois** seletores (`.app-shell > .x`).
`exp_01_exp_01_..._pkey` depois de uma migração de renomeação	`FOR ... IN SELECT` sobre o catálogo releu o que o laço alterou, ou `RENAME CONSTRAINT` renomeou também o índice. Ver a migração de referência.

F.7 *Checklist* de operação

Frequência	Item
Após cada *deploy*	Conferir que a migração rodou (`--wait` no *job*); abrir `/readyz`; abrir uma experiência de aluno e uma ferramenta de professor.
Semanal	Taxa de 5xx do Cloud Run; conexões ativas do Cloud SQL; contagem de `status = 'aberta'` em `exp_70_sessoes` (partidas perdidas); planos em `status = 'erro'`.
Mensal	**Contagem da partição `DEFAULT` de telemetria** — encher significa que faltou criar partição (`R-08`); volume de `telemetria_eventos` por mês; consumo de quota de IA por professor.
Trimestral	`pnpm up -r @unooria/design-system --latest` e rodar os *walkthroughs*; revisar as lacunas do *design system* ainda compensadas localmente; conferir se os modelos de IA em uso continuam no catálogo (`R-12`).
Anual, ou por *release*	Revisar as ADRs cujo gatilho de reavaliação pode ter sido acionado (Capítulo 17).

Listing F.7 – As consultas do *checklist*

```sql
-- A particao DEFAULT esta enchendo? (deveria ser sempre 0)
SELECT count(*) FROM particoes.telemetria_eventos_default;

-- Partidas perdidas (aba fechada no meio) por turma
SELECT turma_id, count(*) FROM exp_70_sessoes
 WHERE status = 'aberta' AND aberta_em < now() - interval '1 day'
 GROUP BY turma_id ORDER BY 2 DESC;

-- Planos que falharam
SELECT id, serie, disciplina, tentativas, erro FROM exp_90_planos
 WHERE status = 'erro' ORDER BY updated_at DESC;

-- Consumo de IA no dia
SELECT usuario_id, tokens_usados FROM ia_uso_diario
 WHERE dia = to_char(now(), 'YYYY-MM-DD') ORDER BY 2 DESC LIMIT 20;

-- Volume de telemetria por mes (nome da particao ja diz o mes)
SELECT relname, n_live_tup FROM pg_stat_user_tables
 WHERE schemaname = 'particoes' ORDER BY relname;
```

> **ATENÇÃO**
>
> Toda consulta manual acima roda **fora** de `comTenancy`, ou seja, com a *role* de aplicação a RLS a filtraria para nada. Diagnóstico em produção usa conexão administrativa — e é por isso que a *role* de aplicação **não pode** ter `BYPASSRLS`: a separação entre "a aplicação" e "um humano diagnosticando" tem de ser real.

F.8 Compilar esta documentação

```
cd documentacao-latex
./compilar.sh              # build completo (Tectonic: baixa o que falta e roda o BibTeX)
./compilar.sh --rapido     # uma passagem, para iterar na escrita
./compilar.sh --limpar     # remove os auxiliares

# Com uma distribuicao de TeX completa instalada, alternativamente:
latexmk -pdf main.tex
```

Requisito	Detalhe
Motor	**Tectonic** (recomendado: baixa os pacotes sob demanda e chama o BibTeX sozinho) *ou* qualquer distribuição com `pdflatex` + `bibtex`.
Fontes	`roboto` e `roboto-mono`.
Diagramas	`tikz` e `pgfplots` — todos vetoriais, sem renderização externa.
Bibliografia	`abntex2cite` (estilo `alf`) processado por **BibTeX** — *não* biblatex/biber, para que o Tectonic feche o ciclo sem ferramenta externa.

REGRA INVIOLÁVEL

Atualizar esta documentação faz parte da *Definition of Done* (Capítulo 16). Implementação que mude arquitetura, contrato, modelo de dados, decisão ou risco atualiza o capítulo correspondente **e regera o `main.pdf` no mesmo *commit***. A regra está registrada em `CLAUDE.md`, não apenas aqui.

Referências Bibliográficas

Advanced Distributed Learning Initiative. *Experience API (xAPI) Specification, version 2.0*. 2023. <https://github.com/adlnet/xAPI-Spec>. Acesso em: 27 ago. 2026.

Autoridade Nacional de Proteção de Dados. *Guia orientativo: tratamento de dados pessoais de crianças e adolescentes*. 2024. ANPD, Brasília. Acesso em: 27 ago. 2026.

Brasil. *Lei n. 13.709, de 14 de agosto de 2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Diário Oficial da União, Brasília.

BRIKMAN, Y. *Terraform: Up & Running — Writing Infrastructure as Code*. 3. ed. Sebastopol: O'Reilly Media, 2022.

BROWN, S. *Software Architecture for Developers: Volume 2 — Visualise, document and explore your software architecture*. Victoria: Leanpub, 2024.

BROWN, S. *The C4 model for visualising software architecture*. 2026. <https://c4model.com>. Acesso em: 27 ago. 2026.

DöRING, A. K. et al. Assessing values at an early age: The picture-based value survey for children (pbvs-c). *Journal of Personality Assessment*, v. 92, n. 5, p. 439–448, 2010.

International Organization for Standardization. *ISO/IEC/IEEE 42010: Systems and software engineering — Architecture description*. Genebra, 2022.

International Organization for Standardization. *ISO/IEC 25010: Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. Genebra, 2023.

KRUCHTEN, P. The 4+1 view model of architecture. *IEEE Software*, v. 12, n. 6, p. 42–50, 1995.

LOVIBOND, P. F.; LOVIBOND, S. H. The structure of negative emotional states: Comparison of the depression anxiety stress scales (dass) with the beck depression and anxiety inventories. *Behaviour Research and Therapy*, v. 33, n. 3, p. 335–343, 1995.

MCDONNELL, C. *Zod: TypeScript-first schema validation with static type inference*. 2026. <https://zod.dev>. Acesso em: 27 ago. 2026.

Mozilla Developer Network. *The iframe sandbox attribute — HTML*. 2026. <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe>. Acesso em: 27 ago. 2026.

Mozilla Developer Network. *Window.postMessage() — Web APIs*. 2026. <https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage>. Acesso em: 27 ago. 2026.

NestJS. *NestJS Documentation: Guards, Interceptors and Modules*. 2026. <https://docs.nestjs.com>. Acesso em: 27 ago. 2026.

NYGARD, M. *Documenting Architecture Decisions*. 2011. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 27 ago. 2026.

OWASP Foundation. *OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks*. 2021. <https://owasp.org/Top10/>. Acesso em: 27 ago. 2026.

SCHWARTZ, S. H. An overview of the schwartz theory of basic values. *Online Readings in Psychology and Culture*, v. 2, n. 1, 2012.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies*. 2026. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>. Acesso em: 27 ago. 2026.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: Table Partitioning*. 2026. <https://www.postgresql.org/docs/current/ddl-partitioning.html>. Acesso em: 27 ago. 2026.

Vercel. *Turborepo Documentation: task pipelines, filtering and remote caching*. 2026. <https://turborepo.com/docs>. Acesso em: 27 ago. 2026.

VIGNOLA, R. C. B.; TUCCI, A. M. Adaptation and validation of the depression, anxiety and stress scale (dass) to brazilian portuguese. *Journal of Affective Disorders*, v. 155, p. 104–109, 2014.